py-mathx-lab Documentation

Release 1.6.0

Walter Weinmann

Dec 31, 2025

CONTENTS

Small, reproducible math experiments implemented in Python.

- **Audience:** curious engineers, students, and researchers
- **Idea:** each experiment is a self-contained runnable module with a short write-up
- **Goal:** a growing, searchable "lab notebook" of experiments

CHAPTER

ONE

START HERE

- *Mathematical experimentation* - what "experiments" in mathematics mean and how to read this repo
- *Experiments Gallery* - experiment gallery (IDs, tags, how to run)
- *Valid Tags* - central directory of valid tags for experiments
- *Background* - mathematical background for experiments
- *Getting started* - install, setup, and run your first experiment
- *Development* - Makefile workflow, CI, coding conventions
- *References* - bibliography and reading list
- *PDF download* - download the PDF version of these docs

CHAPTER

TWO

RUN ONE EXPERIMENT

```
make uv-check
make venv
make install-dev
make run EXP=e001 ARGS="--out out/e001 --seed 1"
```

CHAPTER

THREE

LATEST

- – **E123** - experiments/e123

3.1 Mathematical experimentation

Mathematical experimentation is the practice of using examples, computation, and visualization to discover structure, to generate conjectures, find counterexamples, estimate quantities, and build intuition that later supports (or refutes) formal arguments.

It is *not* "proof by computer output". Instead, experiments are a disciplined way to ask better questions and to stress-test ideas-especially now that modern computers make it easy to explore large search spaces, high-precision numerics, and rich visualizations.

This repository, **py-mathx-lab**, is a small "lab notebook" of such experiments: compact, reproducible, and readable.

3.1.1 What counts as an "experiment" in mathematics?

An experiment is a finite procedure that produces evidence about a mathematical claim or object. Typical outcomes:

- **Conjecture generation**: patterns suggest statements that might be true (or false).
- **Counterexample search**: systematic exploration tries to break a hypothesis early.
- **Quantitative exploration**: estimate constants, rates, limits, or distributions.
- **Model checking**: validate (or invalidate) approximations and heuristics.
- **Visualization**: reveal structure that is hard to see symbolically.

The modern viewpoint-where computation is a genuine part of mathematical discovery-is widely discussed under the name *experimental mathematics* ([BB05, BBC04, Bor05]). Some authors emphasize "plausible reasoning" supported by computation, paired with careful verification and eventual proof ([Bor09, BB08]). Other texts take a more problem-driven, exploratory style aimed at students and engineers ([LCD+03]), or present computation as a tool to formulate concrete research problems ([Arn15]).

3.1.2 Why computers change the game

Computers do not replace mathematical thinking-but they expand what is feasible to *inspect*:

- **Scale**: enumerate millions of cases (to find the first failure or build confidence).
- **Precision**: compute with high-precision floats or exact rationals/integers to avoid roundoff illusions.
- **Multiple lenses**: combine numerics, exact arithmetic, symbolic manipulation, and plotting.
- **Search**: automate discovery (parameter sweeps, optimization, random sampling, heuristics).
- **Reproducibility**: re-run the same pipeline with fixed seeds and pinned dependencies.

Used well, computation turns "I wonder if…" into "here is evidence, here are edge cases, and here is what we should prove next".

3.1.3 Experiments vs. proofs

A proof is the end of the story; an experiment is often the beginning.

Experiments are excellent for:

- **disproving** statements (one counterexample ends it),
- identifying **what is actually true** (after a naive conjecture fails),
- suggesting **lemmas** and **invariants** that make a proof possible.

But experiments can also mislead. Common failure modes:

- floating point error and catastrophic cancellation,
- plotting artifacts,
- "pattern matching" based on too few samples,
- unintentional selection bias ("I tried the cases that worked").

The goal is to use experiments to *reduce uncertainty*, not to hide it.

3.1.4 Targets for py-mathx-lab

py-mathx-lab aims to be a practical, long-lived collection of experiments with shared conventions:

1. **Reproducible runs**
   Each experiment is runnable as a module, writes results to a single output directory, and (when relevant) uses a fixed seed.
2. **Readable code**
   The code should be short, well-typed, and structured so readers can modify it.
3. **Useful artifacts**
   Each experiment should generate at least one of:
   - a figure
   - a table / summary statistics
   - a counterexample / witness object
   - a short narrative explaining what was learned
4. **Clear boundaries**
   An experiment write-up should state:
   - what is being tested,
   - what counts as "success" or "failure",
   - what might invalidate the result (precision limits, domain constraints, runtime limits).
5. **Traceability**
   Each page includes references to books/papers that motivated the work or explain the background.

3.1.5 Repository conventions for experiments

An experiment page should usually include:

- **Goal** (one paragraph)
- **How to run** (a command that works from the repo root)
- **Parameters** (including defaults and ranges, if swept)

- **Results** (figures/tables and a short interpretation)
- **Notes / pitfalls** (numerical caveats, surprising behavior)
- **References** (bibliography keys)

In code, prefer:

- deterministic outputs (fixed seeds, stable sorting),
- explicit configuration objects / CLI arguments,
- sanity checks (dimension checks, bounds checks, invariants),
- cross-checks (e.g., float vs. exact, two independent formulas).

3.1.6 Examples of good "experiment themes"

The experiment format supports many domains, for example:

- **Numerical analysis**: error landscapes, stability regions, conditioning, Monte Carlo integration.
- **Number theory**: continued fractions and convergents, integer sequences, modular patterns, primality heuristics.
- **Geometry/topology**: random point clouds, curvature approximations, combinatorial invariants.
- **Optimization/probability**: stochastic search behavior, concentration phenomena, empirical distributions.

A concrete example of a rich, experiment-friendly topic is continued fractions, where it is natural to compute and plot convergents, partial quotient statistics, and periodicity phenomena ([BvdPSZ14]).

Next steps: start with *Getting started*, browse the *Valid Tags* directory, and then browse the *Experiments Gallery* gallery.

3.2 Valid Tags

This page defines the allowed tags for experiments in **py-mathx-lab**. Tags are used in the *Experiments Gallery* and individual experiment pages to categorize content.

3.2.1 Primary Tags (Domains)

These represent the broad mathematical area of the experiment.

Tag	Description
`analysis`	Calculus, real/complex analysis, limits, and approximation.
`aps`	Arithmetic Progressions and related theorems.
`conjecture-generation`	Patterns suggest statements that might be true (or false).
`counterexample-search`	Systematic exploration tries to break a hypothesis early.
`dirichlet-characters`	Dirichlet characters modulo q, orthogonality, and primitive characters.
`l-functions`	Dirichlet L-functions and related analytic objects controlling prime distribution.
`model-checking`	Validate (or invalidate) approximations and heuristics.
`number-theory`	Properties of integers, divisibility, and prime numbers.
`prime-races`	Prime number races (e.g., Chebyshev bias) comparing counts in residue classes.
`quantitative-exploratic`	Estimate constants, rates, limits, or distributions.
`visualization`	Reveal structure that is hard to see symbolically.

3.2.2 Secondary Tags (Topics & Methods)

These provide more specific detail about the techniques or subtopics involved.

Tag	Description
`arithmetic-functions`	Classical functions on integers (e.g., ϕ, μ, σ, τ) and their relations.
`bounds`	Explicit mathematical bounds (e.g., on prime-counting functions).
`carmichael-lambda`	Carmichael's lambda function (n), the exponent of (Z/nZ)*.
`carmichael`	Carmichael numbers (absolute Fermat pseudoprimes).
`classification`	Grouping objects into classes based on shared properties.
`critical-line`	Zeta/L-function values on Re(s)=1/2 and related numerics.
`dirichlet-convolution`	Dirichlet convolution of arithmetic functions and identity checks.
`dirichlet-series`	Dirichlet generating functions and related analytic tools.
`divisor-function`	Divisor functions such as (n)=d(n) and (n), including record behavior.
`explicit-formula`	Explicit formulas connecting primes and zeros ((x), (x), etc.).
`exploration`	Open-ended search for patterns or properties.
`factorization`	Integer factorization methods and hardness.
`fermat`	Fermat numbers and Fermat primes.
`gaps`	Studies of the distribution of gaps between primes.
`generating-functions`	Ordinary/exponential generating functions (esp. partitions).
`gram-points`	Gram points and Gram's law heuristics for zeta zeros.
`hardy-z`	Hardy's Z-function and Riemann–Siegel theta function.
`heuristics`	Probabilistic or empirical models (e.g., Cramér's model for primes).
`li-x`	Logarithmic integral li(x) and its relation to $\pi(x)$.
`liouville`	Liouville function (n) and related parity questions for Ω(n).
`lucas-lehmer`	Lucas–Lehmer primality test for Mersenne numbers.
`mangoldt`	von Mangoldt function Λ(n) and related Chebyshev functions (x), (x).
`mersenne`	Mersenne numbers $M_p = 2^p - 1$ and Mersenne primes.
`miller-rabin`	Miller–Rabin primality test and its counterexamples.
`mobius`	Möbius function $\mu(n)$, Möbius inversion, squarefree indicators.
`multiplicative`	Multiplicative arithmetic functions; Dirichlet convolution viewpoints.
`numerics`	Heavy use of floating-point or high-precision computation.
`omega`	Prime-factor counting functions (n) and Ω(n) (distinct vs with multiplicity).
`open-problems`	Related to famous unproven conjectures.
`optimization`	Finding maxima, minima, or best-fit parameters.
`partition`	Partition function p(n), identities, and asymptotics.
`perfect`	Related specifically to perfect, abundant, or deficient numbers.
`pnt`	Prime Number Theorem and related asymptotics.
`primes`	Prime number distribution, density, and related theorems.
`primorial`	Primorials, Euclid numbers, primorial primes.
`pseudoprime`	Pseudoprimes, primality-test failures.
`pseudoprimes`	Collective study of various types of pseudoprimes.
`riemann-zeta`	Riemann zeta function (s): series, Euler product, analytic continuation, zeros.
`search`	Systematic search through a large state space.
`semiprime`	Semiprimes, RSA-type composites.
`sieving`	Sieve methods (Eratosthenes, Sundaram, Atkin, etc.).
`sigma`	Related to the sum-of-divisors function $\sigma(n)$.
`summatory`	Summatory functions (e.g., Mertens $M(x)$, summatory totient $\Phi(x)$).
`taylor`	Related to Taylor series and their approximations.
`totient`	Euler's totient function $\phi(n)$, totient equations, summatory behavior.
`twin`	Twin primes and the twin prime conjecture.
`ulam`	Ulam spiral and related prime patterns in 2D.
`wieferich`	Wieferich primes and related congruences.
`wilson`	Wilson's theorem and Wilson primes.
`zeta-zeros`	Nontrivial zeros, zero-counting $N(T)$, root bracketing.

3.2.3 Usage

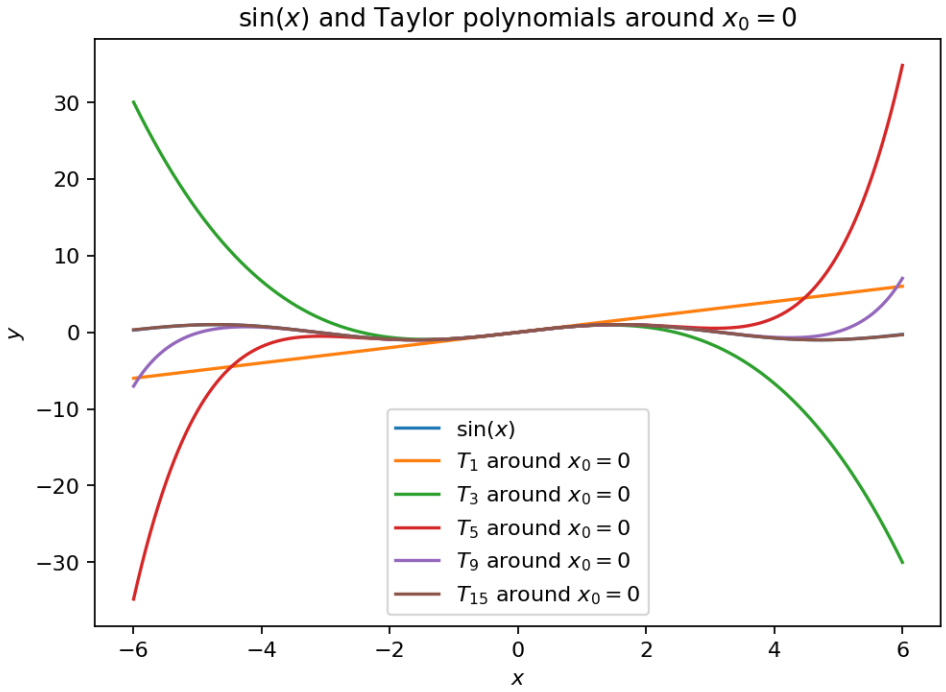
When adding a new experiment:

1. Choose at least one **Primary Tag** (Domain or Type).
2. Choose one or more **Secondary Tags** (Topics & Methods).
3. Add them to the `**Tags:**` line in your `.md` file.
4. Update the *Experiments Gallery* using the corresponding CSS classes (`tag-primary` for primary tags, `tag-secondary` for secondary tags).

3.3 Experiments Gallery

A compact, image-first overview of the experiments in **py-mathx-lab**.

3.3.1 E001 — Taylor Error Landscapes



Tags: `analysis`, `quantitative-exploration`, `visualization`, `numerics`, `taylor` See: *Valid Tags*.

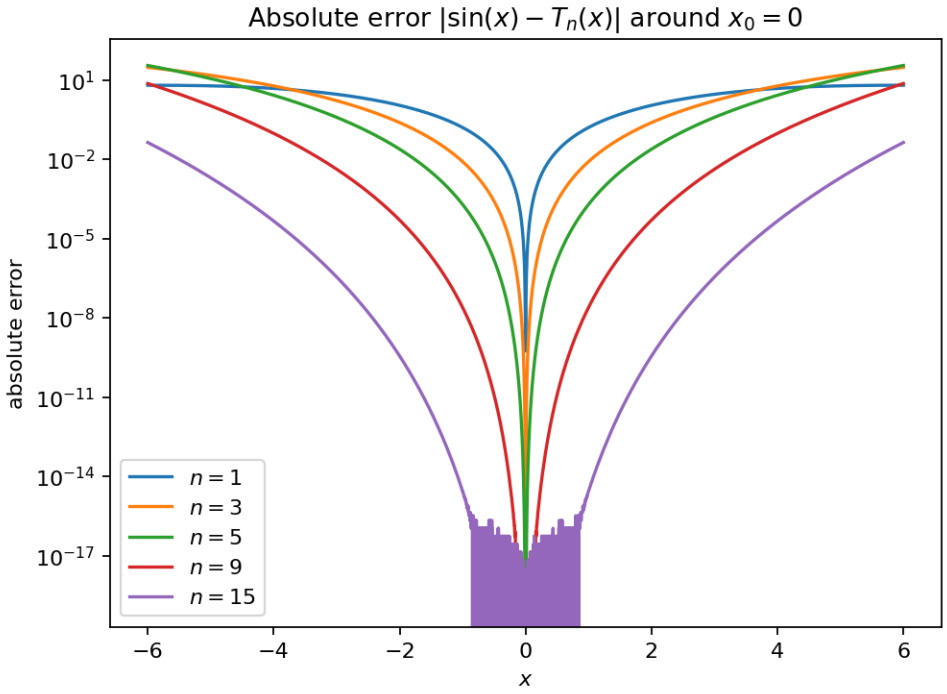
Goal

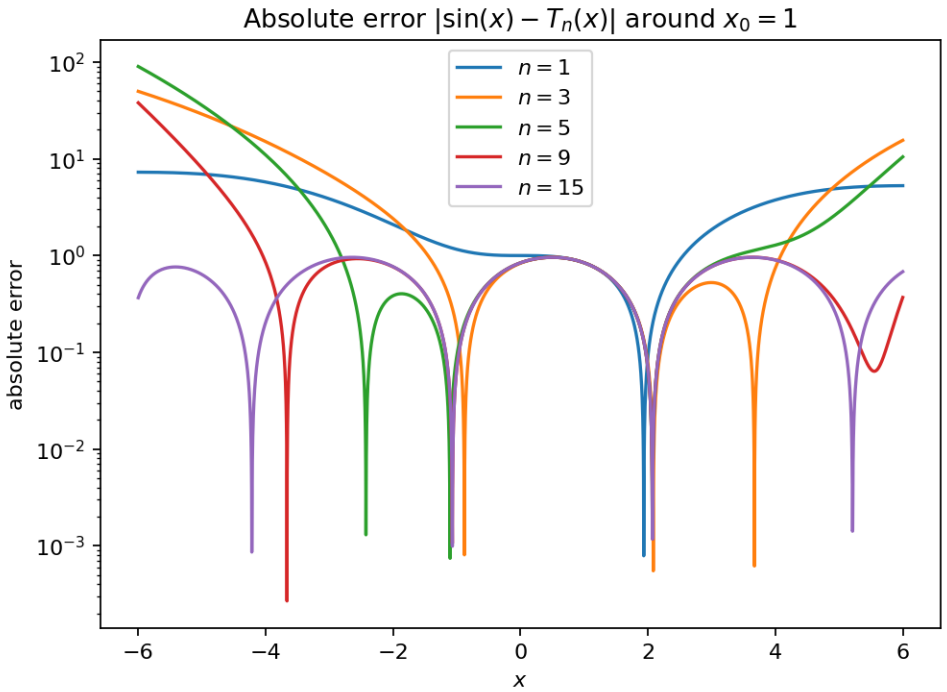
Build intuition for Taylor truncation error by visualizing the absolute error landscape $|\sin(x) - T_n(x; x_0)|$ over a domain while varying:

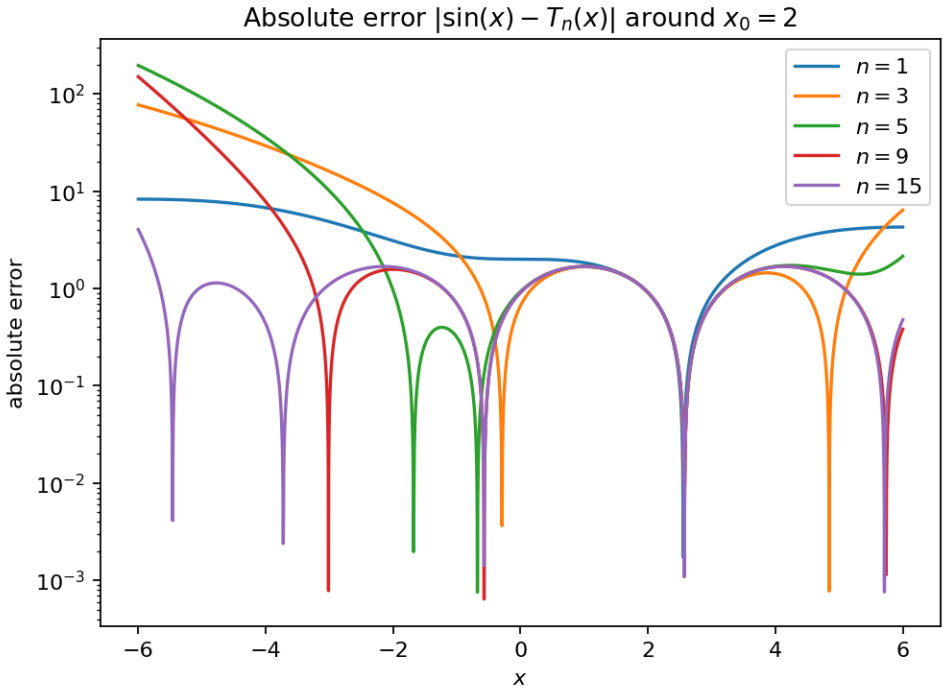
- the polynomial degree n,
- the expansion center x_0.

Background (quick refresher)

If you want a short mathematical recap first, read: *Taylor series refresher*.


Absolute error |sin(x) − T_n(x)| around x_0 = 0
absolute error
x
n = 1
n = 3
n = 5
n = 9
n = 15


Absolute error |sin(x) − T_n(x)| around x_0 = 1
absolute error
x
n = 1
n = 3
n = 5
n = 9
n = 15



Research question

How does the approximation error of Taylor polynomials for $\sin(x)$ depend on the polynomial degree n and the expansion center x_0, over a fixed domain of x?

Concretely: for a chosen grid of (n, x_0), what does the error landscape $E_{n,x_0}(x) = |\sin(x) - T_n(x; x_0)|$ look like, and where do numerical artifacts start to dominate the truncation error?

Why this qualifies as a mathematical experiment

This page is not just a worked example or a derivation — it is an *experiment* in the sense of **experimental mathematics**: a finite, reproducible procedure that produces **evidence** about how a mathematical object behaves.

For E001, the object is the family of Taylor polynomials $T_n(x; x_0)$ for $\sin(x)$, and the observable is the error function $E_{n,x_0}(x) = |\sin(x) - T_n(x; x_0)|$. The experiment qualifies because it:

- **Explores a parameter space:** it varies degree n and center x_0 and inspects how the entire error landscape changes.
- **Generates testable conjectures:** e.g. "there is a widening low-error region around x_0 as n increases" and "improvement is not uniform across a fixed interval".
- **Searches for failure modes / edge cases:** large $|x - x_0|$, high degrees, and wide domains can expose numerical artifacts that *look* like truncation error but are actually floating-point limitations.
- **Produces artifacts that can be checked independently:** plots and parameter snapshots make it easy to compare runs, reproduce the same conditions, and verify that an observed pattern is not accidental.

The goal is to turn "Taylor series should be good near x_0" into *structured evidence* about **where** and **how** the approximation is good (or bad), which then informs what one would try to prove or bound formally.

- How does the truncation error $|\sin(x) - T_n(x; x_0)|$ behave as we move away from the center x_0?
- How many terms are needed to achieve a specific precision (e.g., 10^{-6}) over a fixed interval?
- Does increasing the degree n always improve the result everywhere in the domain?

Experiment design

- **Target function:** $\sin(x)$
- **Evaluation:** $x \in [-2\pi, 2\pi]$ (default)
- **Parameters:**
  - `degrees`: $\{1, 3, 5, 7, 9\}$
  - `centers`: $\{0, \pi/2, \pi\}$
- **Outputs:**
  - Plot of $f(x)$ vs $T_n(x)$
  - Semi-log plot of $|f(x) - T_n(x)|$

How to run

```
make run EXP=e001 ARGS="--seed 1"
```

Artifacts are written under `out/e001/` (figures, parameters, and a short `report.md`).

What to expect

Qualitatively (and this is what the plots should confirm):

- near x_0, the higher degree reduces the error quickly,
- away from x_0, the approximation can degrade even for higher degrees,
- changing x_0 shifts the "low-error region".

Results

After running the experiment, include (or regenerate) the figures in the documentation. The canonical output location is `out/e001/`. For publishing, copy one representative "hero" image into `docs/_static/experiments/` (see "Gallery images" below).

Notes / pitfalls

- Use **log-scale** for absolute error plots to see the full dynamic range.
- Be careful interpreting relative error near zeros of $\sin(x)$.
- Huge domains and high degrees can expose floating-point artifacts that are not truncation error.

Extensions

- **Alternative functions:** Repeat the experiment for $exp(x)$ or $1/(1-x)$.
- **Relative error:** Plot $|(f - T_n)/f|$ instead of absolute error.
- **Automatic degree selection:** Find the minimal n such that error $< \epsilon$ on a given interval.

Gallery images (recommended)

To keep the experiment gallery attractive and stable:

1. run the experiment locally,
2. pick one representative output figure,
3. copy it into the repo under:

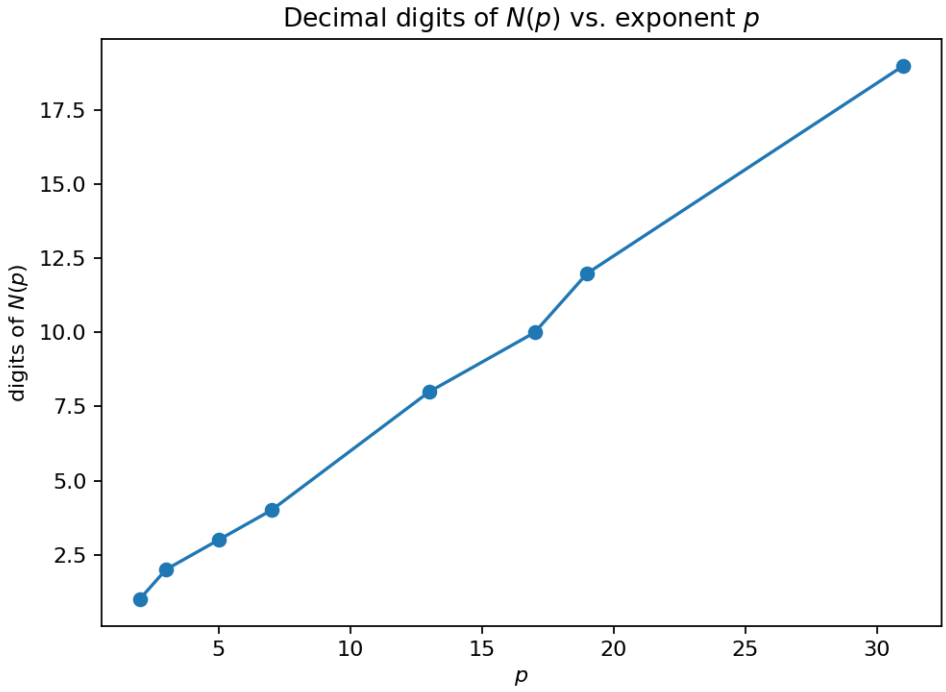
   `docs/_static/experiments/e001_hero.png`

This allows the docs to show thumbnails without depending on generated `out/` artifacts.

References

See *References*. [BB05, Bor05, BB08]

3.3.2 E002: Even Perfect Numbers — Generator and Growth



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `numerics` See: *Valid Tags*.

Highlights

- Generate even perfect numbers from known Mersenne exponents p.
- Plot digits and bit-length growth vs. p.
- Test the approximation $\log_{10}(N(p)) \approx 2p \log_{10}(2)$.

Goal

Generate **even perfect numbers** from known Mersenne prime exponents p and visualize how fast the numbers grow. Measure growth via **digit count**, **bit length**, and simple logarithmic approximations.

Research question

For

$$N(p) = 2^{p-1}(2^p - 1),$$

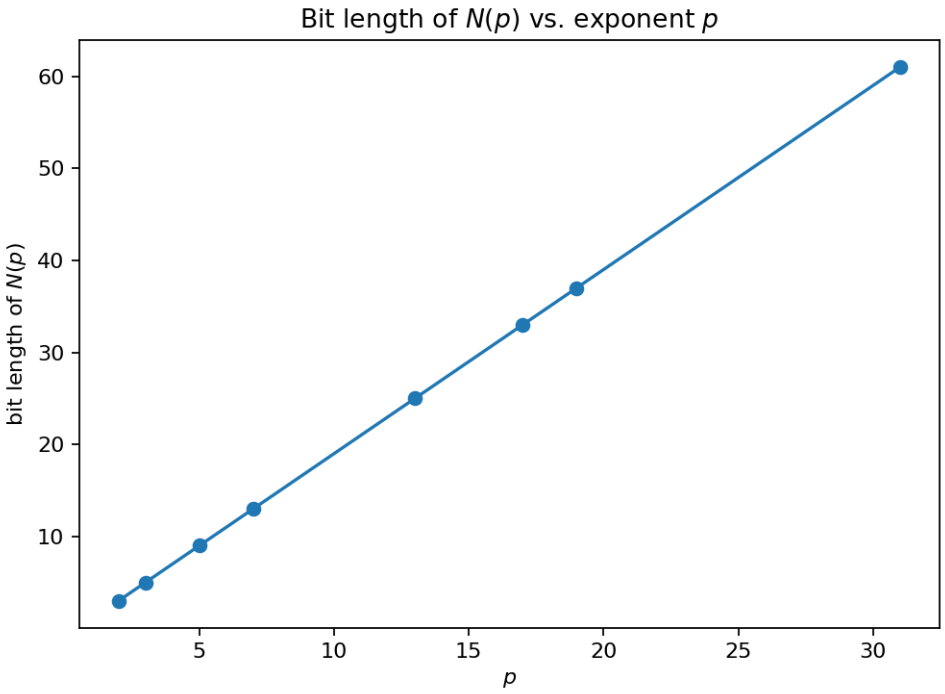
how do:

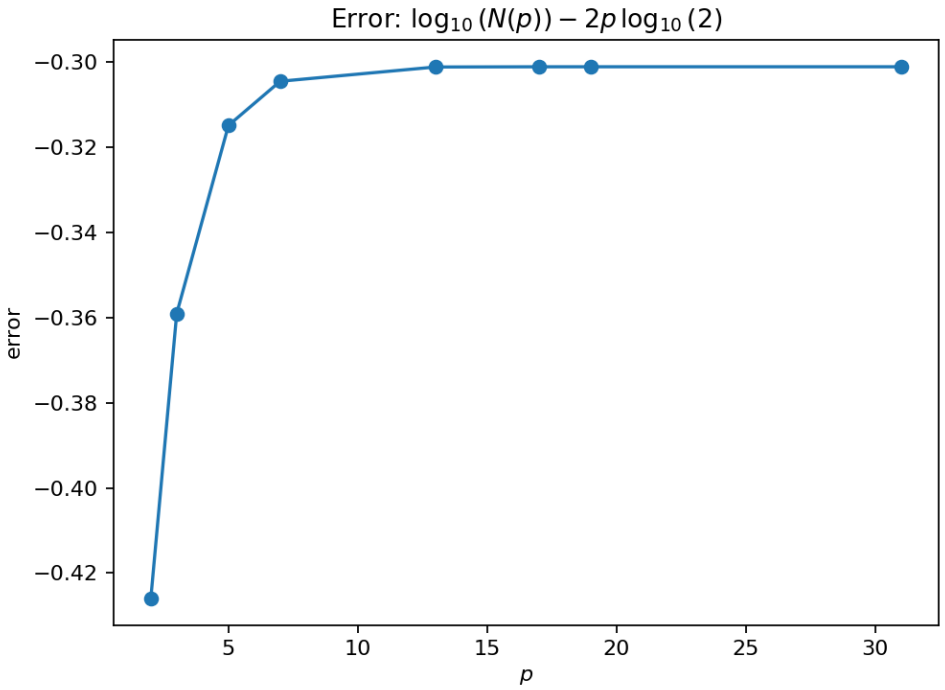
- digits(N),
- bit_length(N),
- $\log_{10}(N)$

scale with the exponent p?

How accurate is the approximation

$$\log_{10}(N(p)) \approx 2p \log_{10}(2)?$$


Bit length of $N(p)$ vs. exponent p
bit length of $N(p)$
p
10
20
30
40
50
60
5
10
15
20
25
30


Error: $\log_{10}(N(p)) - 2p\log_{10}(2)$
error
p
−0.30
−0.32
−0.34
−0.36
−0.38
−0.40
−0.42
5
10
15
20
25
30

Why this qualifies as a mathematical experiment

The Euclid–Euler theorem tells us exactly what even perfect numbers look like, but it does not directly convey how quickly the objects become astronomically large. This experiment uses computation and visualization to build quantitative intuition and test simple asymptotic approximations.

Experiment design

Inputs

- A curated list of known Mersenne prime exponents, e.g. $p \in \{2, 3, 5, 7, 13, 17, 19, 31, \dots\}$.

Observables

For each exponent p:

- $N(p)$ as a Python integer
- digits$(N(p))$
- bit_length$(N(p))$
- approximation error:

$$\Delta(p) = \log_{10}(N(p)) - 2p \log_{10}(2)$$

Plots

- p vs. digits
- p vs. bit length
- p vs. $\Delta(p)$

How to run

From the repo root:

```
make run EXP=e002
```

or:

```
uv run python -m mathxlab.experiments.e002
```

Notes / pitfalls

- Avoid converting huge integers to decimal strings repeatedly in tight loops; compute digits using logs where possible.
- Use `int.bit_length()` for stable bit length (fast and exact).
- Keep the exponent list modest for fast docs builds; this is a growth experiment, not a search for new Mersenne primes.

Extensions

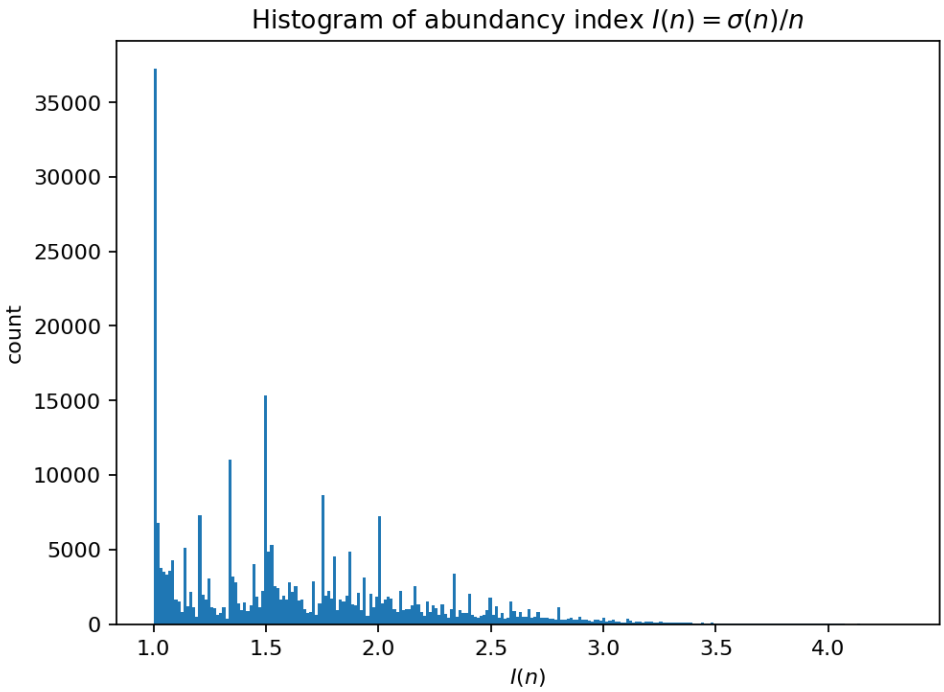
- Compare growth to 2^{2p} and quantify the relative gap.
- Add a "human scale" axis: compare digits$(N(p))$ to common benchmarks (atoms in the observable universe, etc.).
- Pull the exponent list from a small data file so the experiment can be updated without code changes.

References

See *References*.

[Cald.a, Voi98, OEISFInc25a]

3.3.3 E003: Abundancy Index Landscape



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `numerics` See: *Valid Tags*.

Highlights

- Compute $\sigma(1..N)$ via a divisor-sum sieve.
- Visualize the distribution of $I(n) = \sigma(n)/n$.
- Highlight perfect numbers as the razor-thin level set $I(n) = 2$.

Goal

Visualize how **rare** perfect numbers are by plotting the distribution of the **abundancy index**

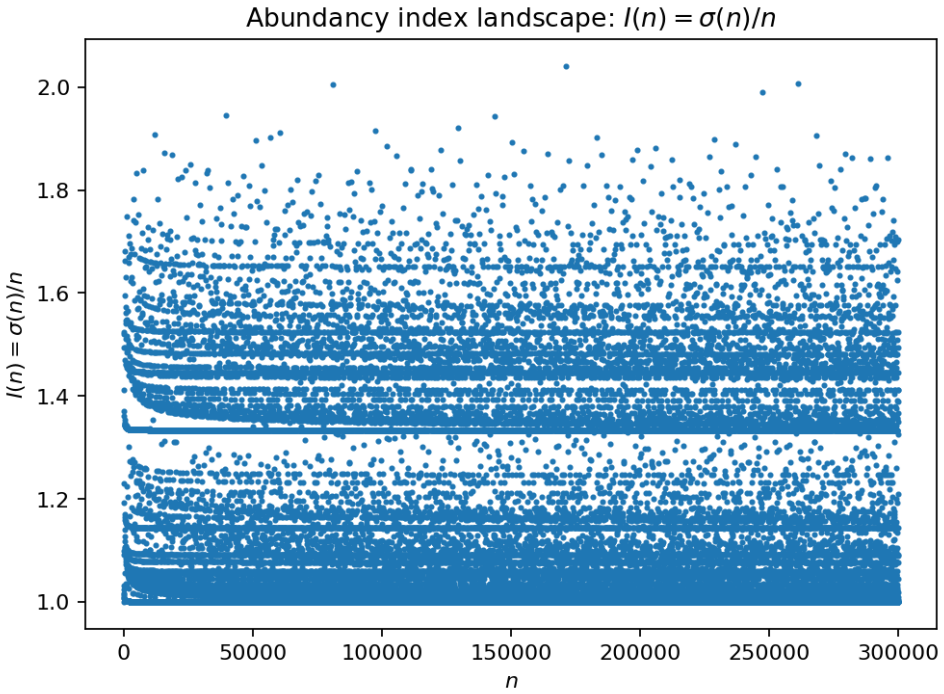
$$I(n) = \frac{\sigma(n)}{n}$$

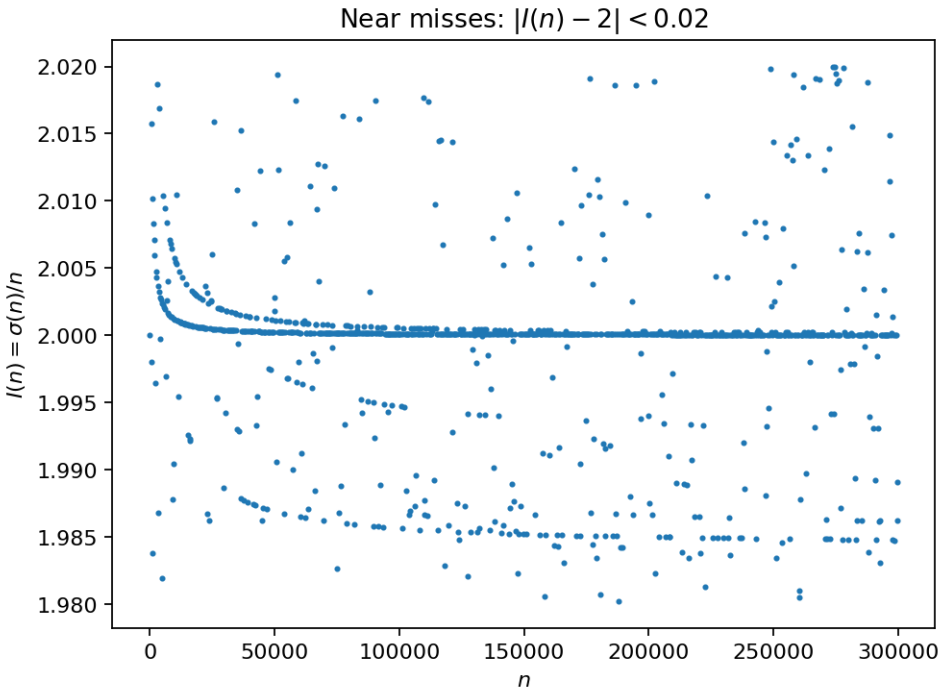
for integers $n \leq N$. Perfect numbers satisfy $I(n) = 2$.

Research question

For a given bound N:

- What does the empirical distribution of $I(n)$ look like?
- How frequently do we see values near 2?
- Where do the perfect numbers appear in the landscape?


Abundancy index landscape: $I(n) = \sigma(n)/n$
$I(n) = \sigma(n)/n$
2.0
1.8
1.6
1.4
1.2
1.0
0
50000
100000
150000
200000
250000
300000
n


Near misses: $|I(n) - 2| < 0.02$
$I(n) = \sigma(n)/n$
2.020
2.015
2.010
2.005
2.000
1.995
1.990
1.985
1.980
0
50000
100000
150000
200000
250000
300000
n

Why this qualifies as a mathematical experiment

The divisor-sum function $\sigma(n)$ is highly structured but "spiky" and hard to intuit symbolically. Computational sweeps reveal qualitative structure (clusters, gaps, tails) and put perfect numbers into context.

Experiment design

Method: compute $\sigma(1)$, ..., $\sigma(N)$ via a divisor-sum sieve

Use the identity "each divisor contributes to its multiples":

- initialize an array `sigma[0..N]` with zeros
- for each `d = 1..N`:
  - add `d` to `sigma[k]` for all multiples `k = d, 2d, 3d, ...`

This runs in about $O(N \log N)$ time and avoids per-number factorization.

Observables

- $I(n) = \sigma(n)/n$
- distance to perfection: $|I(n) - 2|$
- classification:
  - deficient: $I(n) < 2$
  - perfect: $I(n) = 2$
  - abundant: $I(n) > 2$

Plots

- histogram of $I(n)$ (or of $I(n) - 1$)
- scatter plot of n vs. $I(n)$ (optionally with log-scale on n)
- highlight perfect numbers

How to run

```
make run EXP=e003
```

or:

```
uv run python -m mathxlab.experiments.e003
```

Notes / pitfalls

- $I(n)$ is rational. For classification, compare integers using $\sigma(n)$ and $2n$ rather than floats.
- For plots, floats are fine, but compute the "perfect" condition as `sigma[n] == 2*n`.
- Start with $N \leq 1\,000\,000$ to keep runtime and memory reasonable.

Extensions

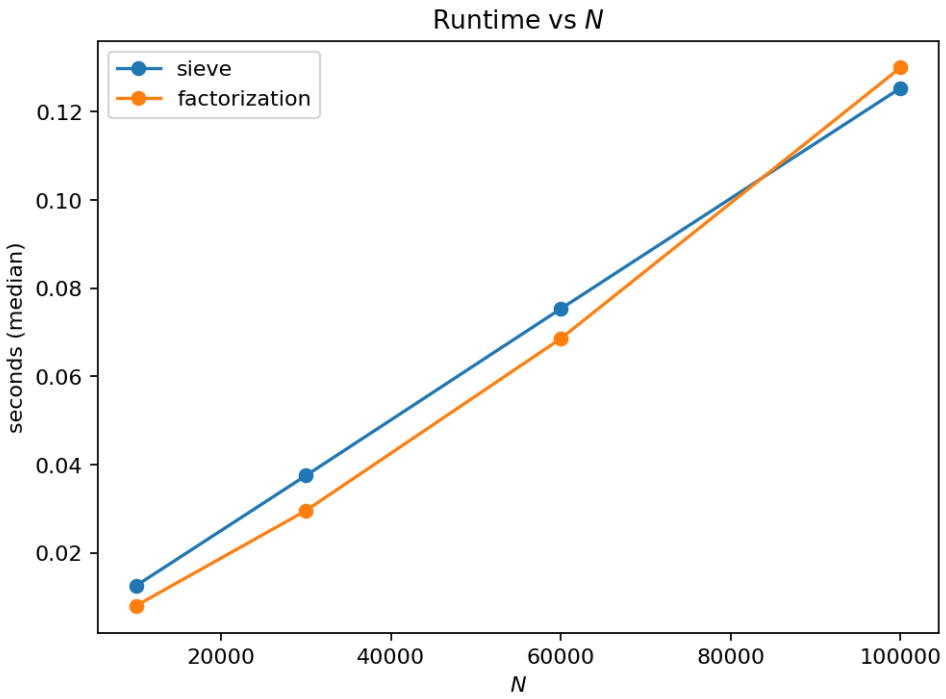
- Repeat for different ranges and overlay histograms.
- Plot the top-k values of $I(n)$ and compare to known extremal families.
- Explore the "near misses" set (feeds directly into E006).

References

See *References*.

[Voi98, Wei03, OEISFInc25a]

3.3.4 E004: Computing $\sigma(n)$ at Scale — Sieve vs. Factorization



Tags: `number-theory`, `quantitative-exploration`, `numerics`, `optimization` See: *Valid Tags*.

Highlights

- Benchmark bulk sieve vs. per-number factorization.
- Find runtime crossover points as N grows.
- Validate correctness on sampled inputs.

Goal

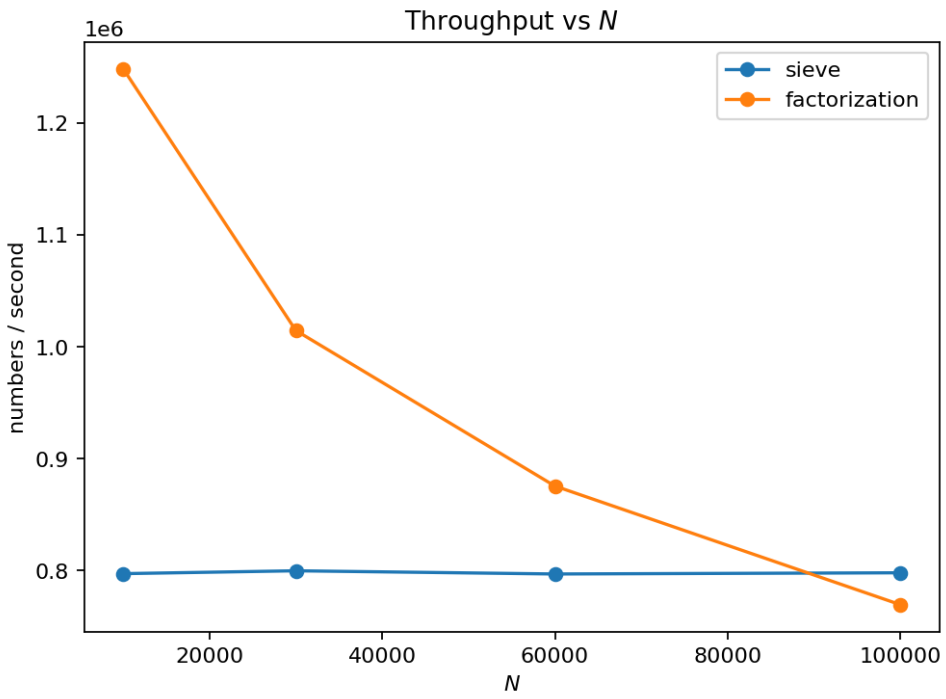
Benchmark two practical ways to compute the sum-of-divisors function:

1. **Sieve method:** compute all $\sigma(1..N)$ in one pass.
2. **Factorization method:** compute $\sigma(n)$ per-number from the prime factorization.

Research question

For a range of bounds N:

- which approach is faster?
- where is the crossover point?
- what are the memory tradeoffs?



Why this qualifies as a mathematical experiment

Both methods are mathematically equivalent, but performance depends on constants, caching, and implementation details. This is a quantitative exploration of algorithmic behavior grounded in number-theory structure.

Experiment design

Method A: divisor-sum sieve (bulk computation)

Compute `sigma[1..N]` by adding each divisor to its multiples (as in E003).

Method B: per-number factorization

Factor each n (e.g. using a precomputed prime list up to $\sqrt{N}$) and compute:

If

$$n = \prod_{i=1}^{k} p_i^{a_i},$$

then

$$\sigma(n) = \prod_{i=1}^{k} \frac{p_i^{a_i+1} - 1}{p_i - 1}.$$

Measurements

- wall-clock runtime vs. N (multiple trials, median)
- peak memory (rough estimate acceptable for v1)
- correctness cross-check: random sample where both methods agree

Outputs

- plot: runtime vs. N for both methods
- short table: N, runtime(A), runtime(B), speedup

How to run

```
make run EXP=e004
```

or:

```
uv run python -m mathxlab.experiments.e004
```

Notes / pitfalls

- Factorization becomes expensive quickly; keep the factorization method limited to moderate N.
- Use integer arithmetic for correctness checks (`sigma[n] == 2*n` etc.).
- Report both runtime and *effective throughput* (numbers processed per second).

Extensions

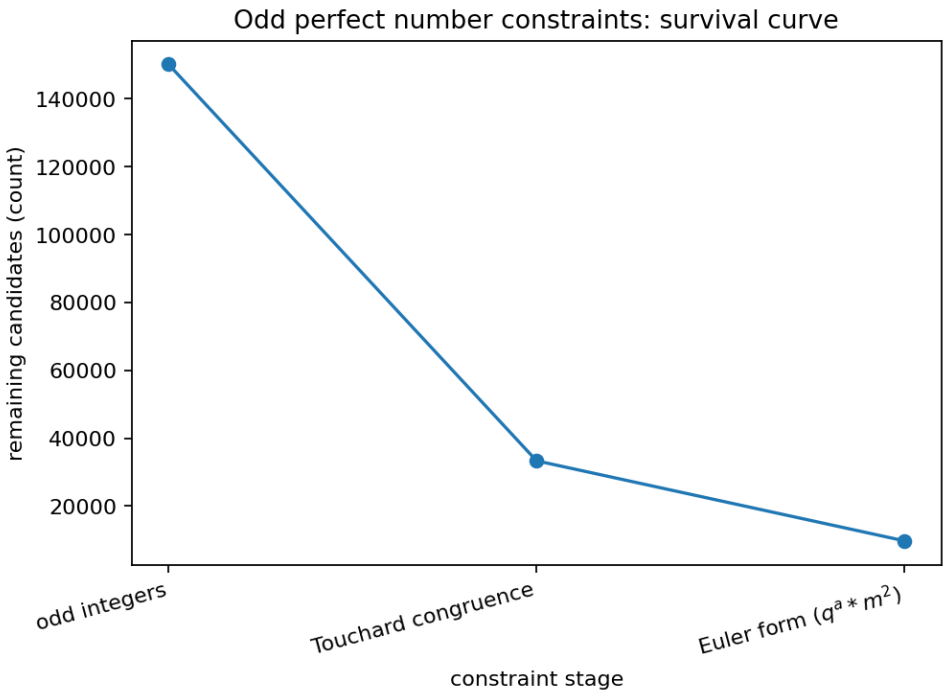
- Add a third method using smallest-prime-factor (SPF) sieve for fast factorization.
- Compare pure Python vs. `numpy` arrays for Method A.
- Turn the benchmark into a reusable utility for later experiments.

References

See *References*.

[Voi98, Wei03]

3.3.5 E005: Odd Perfect Numbers — Constraint Filter Pipeline



Tags: `number-theory`, `counterexample-search`, `visualization`, `search` See: *Valid Tags*.

Highlights

- Apply necessary constraints as staged filters on odd candidates.
- Plot survival curves (remaining candidates per constraint stage).
- Make the "open problem" constraints tangible at finite scales.

Goal

Demonstrate *why brute-force search for odd perfect numbers is unrealistic* by applying known **necessary conditions** as a step-by-step filter pipeline and visualizing how the candidate pool collapses.

Research question

Given odd integers up to a bound N:

- how many survive each constraint stage?
- which constraints are most "eliminating" in practice at small scales?
- what does a survival curve suggest about scalability?

Why this qualifies as a mathematical experiment

Odd perfect numbers are an open problem. Theorems provide constraints rather than a classification. Computation makes these constraints tangible by measuring their elimination power on finite candidate sets.

Experiment design

Candidate set

Start with odd integers $1 < n \leq N$.

Constraint stages (v1)

Use a conservative, well-known set of *necessary* conditions that can be checked mechanically:

1. **Euler form (shape constraint):** any odd perfect number must be of the form

   $$n = q^{\alpha} m^2$$

   where q is prime, $\gcd(q, m) = 1$, and

   $$q \equiv \alpha \equiv 1 \pmod 4.$$

2. **Congruence filter (Touchard-type):** odd perfect numbers satisfy strong congruence restrictions (implemented as one or two simple congruence checks supported by the references).
3. **Small-prime structure filters:** apply a few lightweight necessary conditions (e.g., reject candidates divisible by some specific small patterns if justified by the reference set).

For each stage, record "remaining candidates".

Output

- table: stage name, remaining count, elimination percentage
- plot: survival curve (stage index vs. remaining candidates)

How to run

```
make run EXP=e005
```

or:

```
uv run python -m mathxlab.experiments.e005
```

Notes / pitfalls

- Be explicit about what is *proved* vs. what is a heuristic. Only implement conditions that are truly necessary.
- At small N, some deep constraints won't show their full strength; the goal is the *pipeline idea*, not a record bound.
- Euler-form testing requires factorization; keep N small enough for trial division (v1).

Extensions

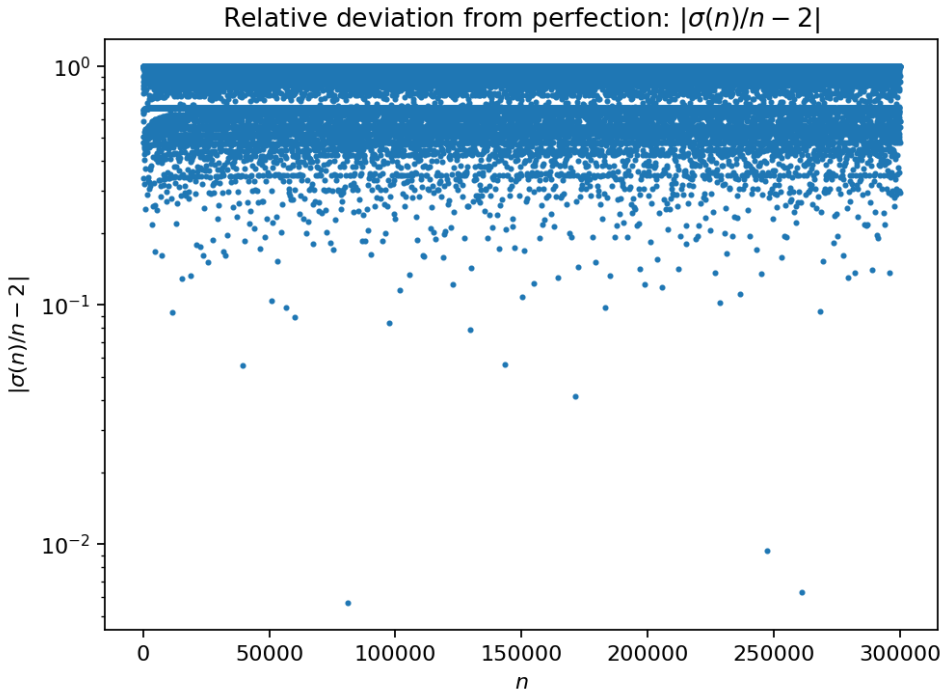
- Add stronger bounds/constraints from the modern literature and compare elimination power.
- Replace trial division with an SPF sieve to scale the Euler-form test.
- Report not just counts but also the distribution of remaining prime-factor patterns.

References

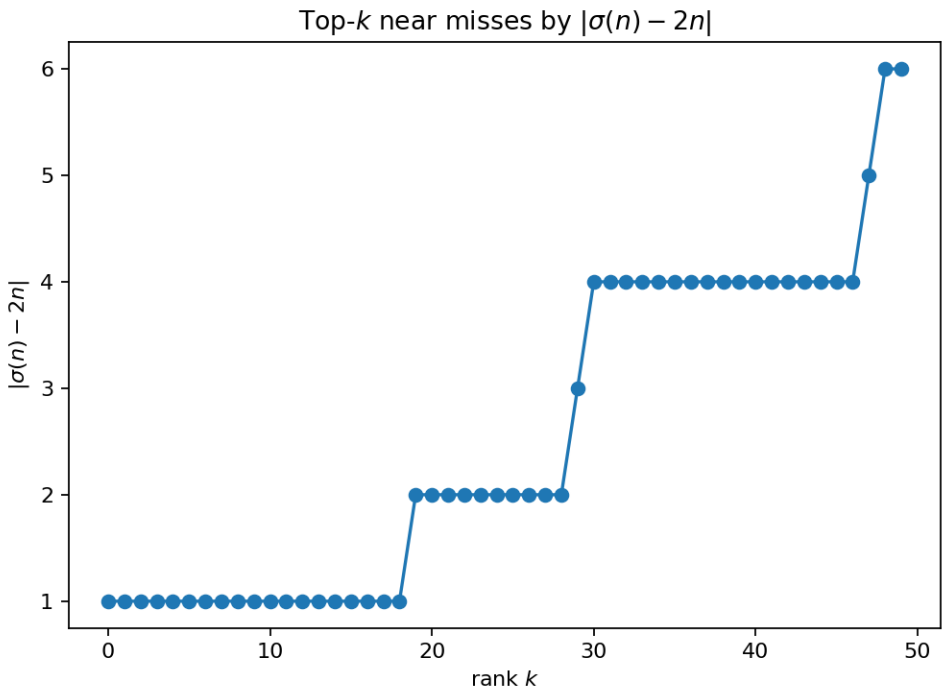
See *References*.

[Guy04, OR14, Sto24, Voi98]

3.3.6 E006: Near Misses to Perfection



Tags: `number-theory`, `conjecture-generation`, `visualization`, `numerics` See: *Valid Tags*.



Highlights

- Search for n with $\sigma(n)$ unusually close to $2n$.
- Build leaderboards for absolute and relative deviation.
- Visualize how "near perfection" clusters by structure.

Goal

Find and visualize integers whose divisor sum is **unusually close** to the perfect condition $\sigma(n) = 2n$, without being perfect.

Research question

For integers $n \leq N$, which numbers minimize:

- absolute deviation:

$$D_1(n) = |\sigma(n) - 2n|$$

- relative deviation:

$$D_2(n) = \left|\frac{\sigma(n)}{n} - 2\right|?$$

Do "near misses" cluster in recognizable families (highly composite, abundant, etc.)?

Why this qualifies as a mathematical experiment

The perfect condition is a sharp equality. Studying the closest failures often reveals structure and suggests new questions (e.g., which multiplicative patterns drive $\sigma(n)$ toward $2n$).

Experiment design

Computation

- Compute $\sigma(1..N)$ via the divisor-sum sieve (as in E003).

- For each n, compute $D_1(n)$ and $D_2(n)$.
- Keep the top-k smallest deviations (excluding actual perfect numbers).

Outputs

- table: top-k near misses (with n, $\sigma(n)$, D_1, D_2)
- plot: n vs. $D_2(n)$ (log-scale on D_2 often helps)
- mark perfect numbers for reference

How to run

```
make run EXP=e006
```

or:

```
uv run python -m mathxlab.experiments.e006
```

Notes / pitfalls

- Use integer comparisons to identify perfect numbers (`sigma[n] == 2*n`).
- For D_2, floats are fine for plotting, but store exact rational values for ranking when possible (e.g., compare $|\sigma(n) - 2n|$ first, then normalize for reporting).
- Choose k small (e.g. 50 or 200) so the report stays readable.

Extensions

- Repeat for different N and compare stability of the "near miss" leaderboard.
- Add a second leaderboard restricted to odd n only.
- Compare near misses to known abundant/deficient classifications and prime factorizations.

References

See *References*.

[Voi98, Wei03, OEISFInc25a]

3.3.7 E007: Mersenne growth (bits and digits)

Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

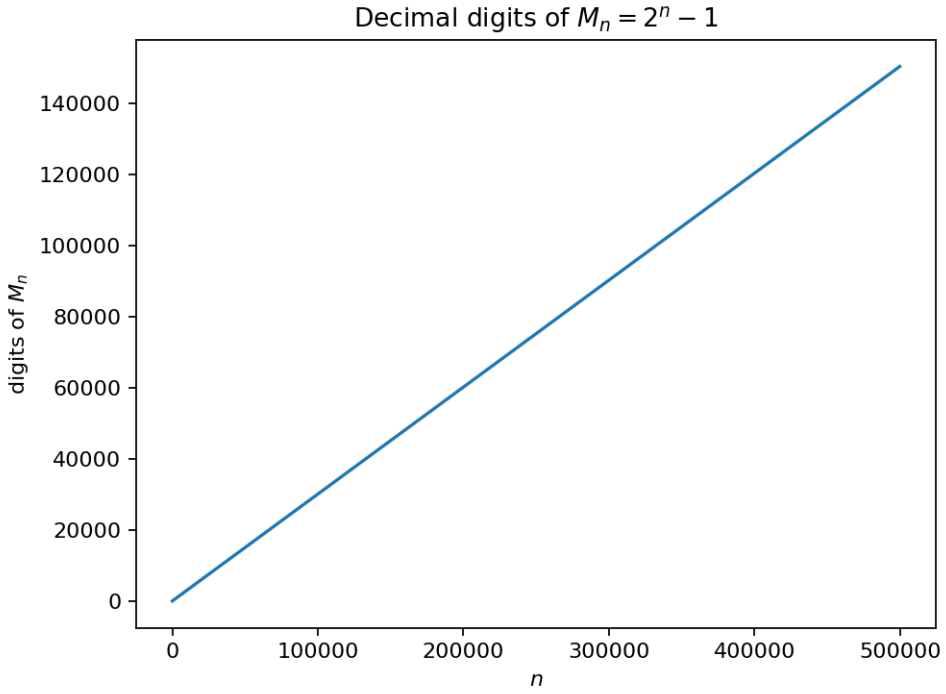
Highlights

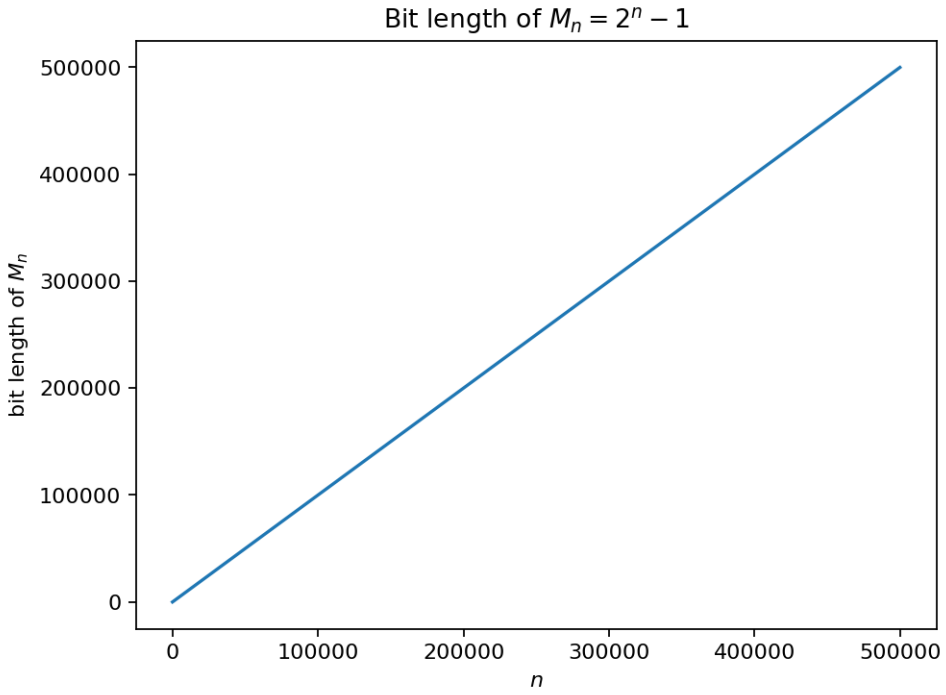
- Visualize how quickly $M_n = 2^n - 1$ grows in **bits** and **decimal digits**.
- Compute size metrics *without* constructing huge integers.
- Provide simple rules-of-thumb for choosing feasible bounds in later experiments.

Goal

Quantify and visualize the growth of Mersenne numbers $M_n = 2^n - 1$ as n increases.

This experiment focuses on **size** (bit-length and decimal digits), because it determines what later algorithms (Lucas–Lehmer, sieving) can realistically handle on a laptop.


Decimal digits of $M_n = 2^n - 1$
digits of M_n
0
20000
40000
60000
80000
100000
120000
140000
0
100000
200000
300000
400000
500000
n


Bit length of $M_n = 2^n - 1$
bit length of M_n
0
100000
200000
300000
400000
500000
0
100000
200000
300000
400000
500000
n

Background (quick refresher)

- *Mersenne numbers and primes refresher*

Research question

How fast does the size of M_n grow with n, and what simple formulas approximate:

- the number of bits of M_n
- the number of decimal digits of M_n

Why this qualifies as a mathematical experiment

- **Finite procedure:** evaluate size formulas on a range $n = 1..N$.
- **Observable(s):** bit-length and digit count as functions of n.
- **Parameter space:** vary N (and optional sampling density).
- **Outcome:** visual evidence (curves) and tables usable as planning guidance.
- **Reproducibility:** parameters written to `params.json`; figures written to `figures/`.

Experiment design

Computation

Key facts:

- $\text{bits}(2^n - 1) = n$ for $n \geq 1$.
- $\text{digits}(2^n - 1) = \lfloor n \log_{10}(2) \rfloor + 1$ for $n \geq 1$.

Compute both for a grid of n values.

Outputs

- plot: n vs. bits
- plot: n vs. digits
- table: selected n with digits (e.g., 10, 100, 1k, 10k, …)

How to run

```
make run EXP=e007
```

or:

```
uv run python -m mathxlab.experiments.e007
```

Notes / pitfalls

- Avoid converting huge M_n to decimal just to count digits. The logarithm formula is exact for the digit count.
- For very large n, use stable floating-point evaluation for $n \log_{10}(2)$ (double precision is fine up to very large n).
- This experiment is intentionally lightweight; it should run quickly.

Extensions

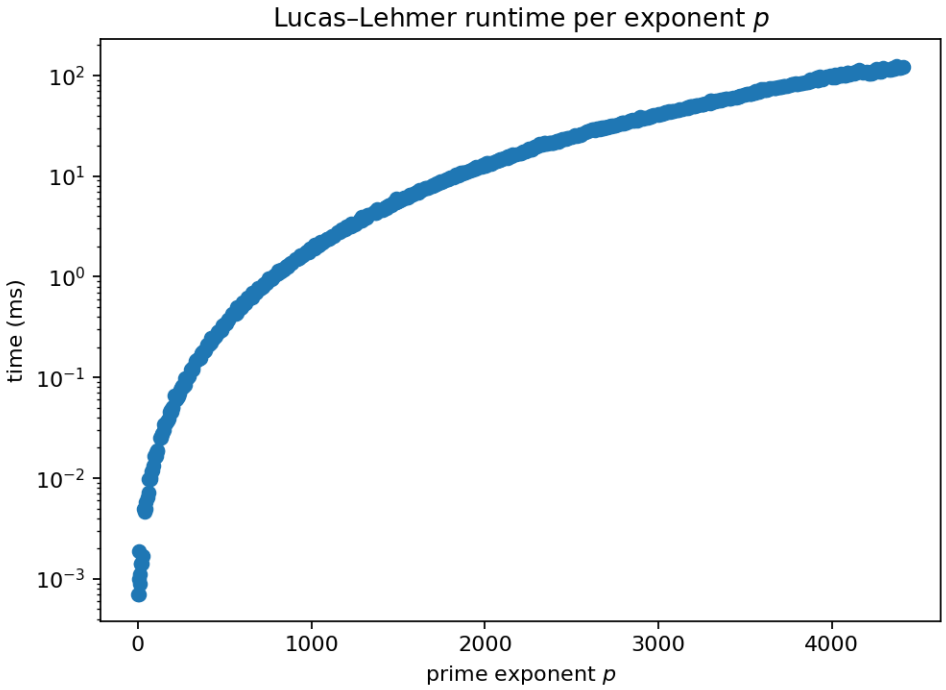
- Overlay "feasibility bands" (e.g., digits thresholds) for what your other experiments can handle.
- Compare M_n to other fast-growing families (factorials, primorials) on a log scale.

References

See *References*.

[con25b]

3.3.8 E008: Lucas–Lehmer scan (prime exponents)



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- Run the Lucas–Lehmer test for many prime exponents p.
- Record timing and residue behavior for composite vs. prime outcomes.
- Produce a reproducible "scan report" (which p were tested, which passed).

Goal

Empirically explore Mersenne primality testing by scanning prime exponents p and applying the Lucas–Lehmer test to $M_p = 2^p - 1$.

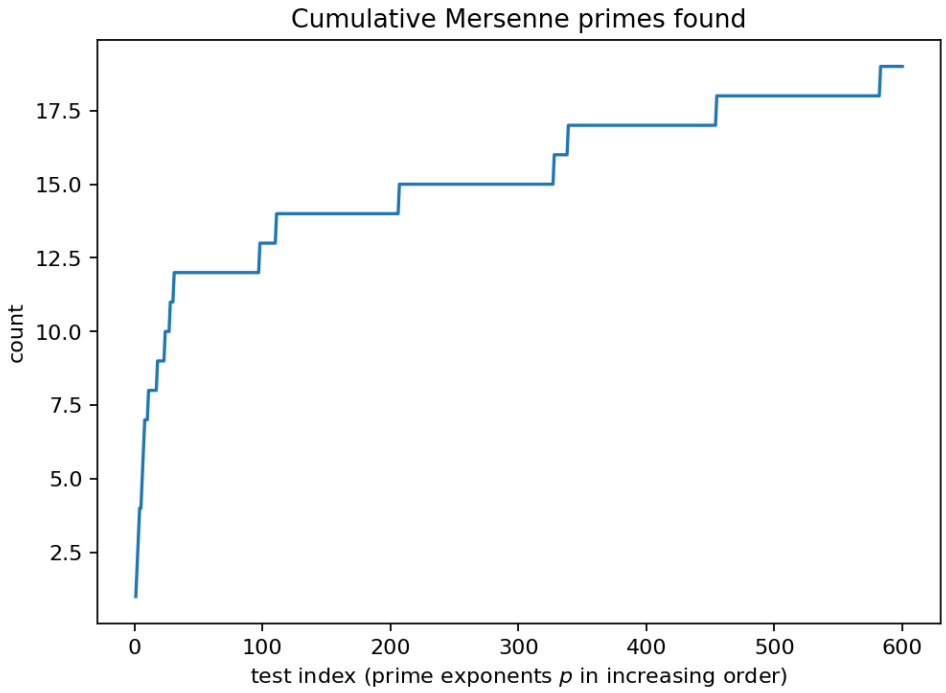
Background (quick refresher)

- *Mersenne numbers and primes refresher*

Research question

For prime exponents $p \leq P$, how does:

- runtime of the Lucas–Lehmer test scale with p?
- the distribution of final residues (mod M_p) differ between prime and composite outcomes?



Why this qualifies as a mathematical experiment

- **Finite procedure:** enumerate prime exponents up to a bound and test them.
- **Observable(s):** pass/fail, final residue, and runtime.
- **Parameter space:** vary P and optional "max tests" cap.
- **Outcome:** a dataset (tested p values) and plots/tables that suggest scaling behavior.
- **Failure modes:** naive implementations can be slow; parameters bound the search to keep runs feasible.

Experiment design

Computation

- Enumerate prime exponents p up to a bound P.
- For each p, compute $M_p = 2^p - 1$ and run Lucas–Lehmer:
  - $s_0 = 4$
  - $s_{k+1} = s_k^2 - 2 \pmod{M_p}$
  - M_p is prime iff $s_{p-2} \equiv 0 \pmod{M_p}$.
- Record (at minimum): p, pass/fail, final residue, and wall time.

Outputs

- table: scanned exponents with result and time
- plot: p vs runtime (often log-scale is helpful)
- plot: residue magnitude / patterns (optional)

How to run

```
make run EXP=e008
```

or:

```
uv run python -m mathxlab.experiments.e008
```

Notes / pitfalls

- Only run Lucas–Lehmer for **prime** p; if p is composite then M_p is composite.
- The test is defined for odd prime p; treat $p = 2$ separately ($M_2 = 3$ is prime).
- Keep bounds modest at first; the cost grows quickly with p.

Extensions

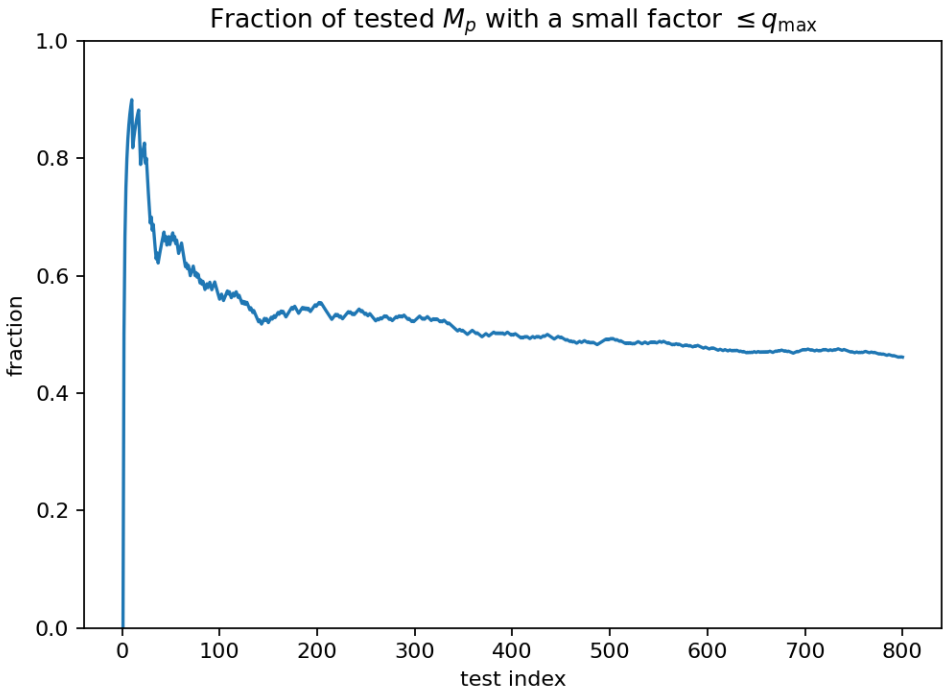
- Add a "pre-sieve" that checks for small factors before running Lucas–Lehmer.
- Compare your timings to published implementations (as a sanity check, not a competition).

References

See *References*.

[Cal21, con25a, MR24]

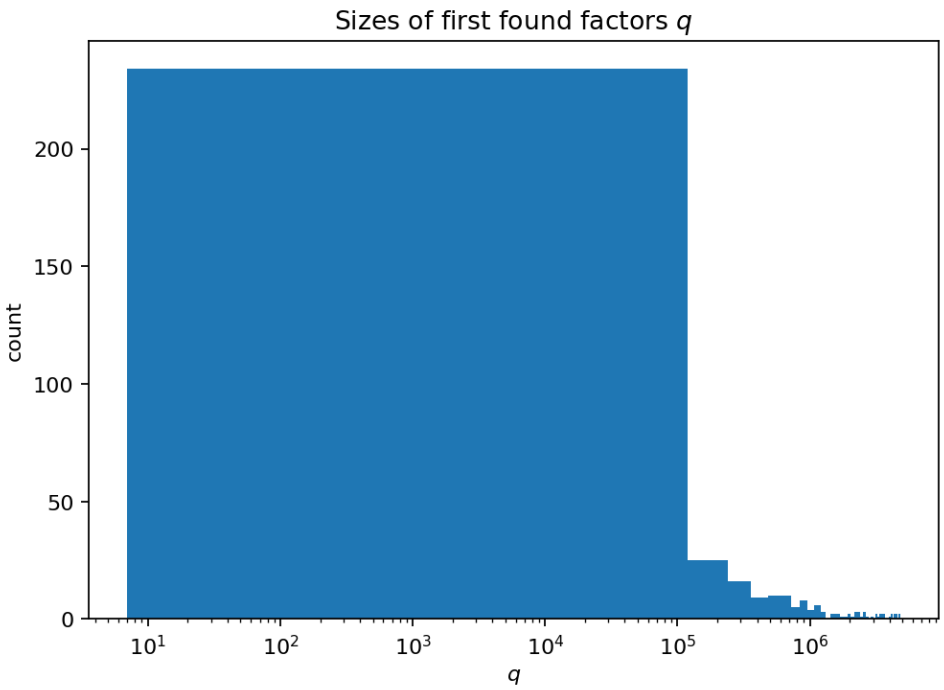
3.3.9 E009: Small-factor scan for Mersenne numbers



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- Quickly eliminate many M_p candidates by finding small prime factors.
- Use the structure of possible factors of M_p to reduce wasted work.
- Show how often "trivial" factors appear in a range of exponents.



Goal

Explore how often Mersenne numbers $M_p = 2^p - 1$ (with prime p) have **small factors**, and how a lightweight sieve can filter candidates before running Lucas–Lehmer.

Background (quick refresher)

- *Mersenne numbers and primes refresher*

Research question

For prime exponents $p \leq P$ and a factor bound $q \leq Q$:

- how many M_p have a factor below Q?
- how much compute can be saved by sieving before Lucas–Lehmer?

Why this qualifies as a mathematical experiment

- **Finite procedure:** scan a finite range of p and candidate factors.
- **Observable(s):** first small factor found (if any), counts by p and by factor size.
- **Parameter space:** vary P and Q.
- **Outcome:** tables/plots that show factor frequency and sieve effectiveness.
- **Failure modes:** incomplete sieve bounds can mislead; clearly report Q as a limitation.

Experiment design

Computation

For fixed prime exponent p, any prime factor q of M_p satisfies:

- $q \equiv 1 \pmod{2p}$ (a common necessary condition used to generate candidates)

A practical sieve approach:

- Generate candidate primes q of the form $q = 2pk + 1$ up to Q.

- For each candidate prime q, check:
  - $2^p \equiv 1 \pmod{q}$

  If true, then q divides M_p.

Outputs

- table: per p, smallest factor found (or "none up to Q")
- plot: p vs. smallest factor (when present)
- summary: fraction of p eliminated by the sieve

How to run

```
make run EXP=e009
```

or:

```
uv run python -m mathxlab.experiments.e009
```

Notes / pitfalls

- This is a **bounded** sieve: "no factor found" only means "none found up to Q".
- Prime testing for q should be fast; for small Q, a simple deterministic method is fine.
- Report both P and Q prominently in the report.

Extensions

- Combine with E008: run Lucas–Lehmer only on exponents not eliminated by the sieve.
- Track how much wall time is saved by the combined pipeline.

References

See *References*.

[Cal21, con25b, Inc25b]

3.3.10 E010: Even perfect numbers from Mersenne primes

Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- Generate even perfect numbers via the Euclid–Euler theorem.
- Connect Mersenne prime exponents p to perfect numbers $N = 2^{p-1}(2^p - 1)$.
- Visualize growth and verify the defining property $\sigma(N) = 2N$ for sampled cases.

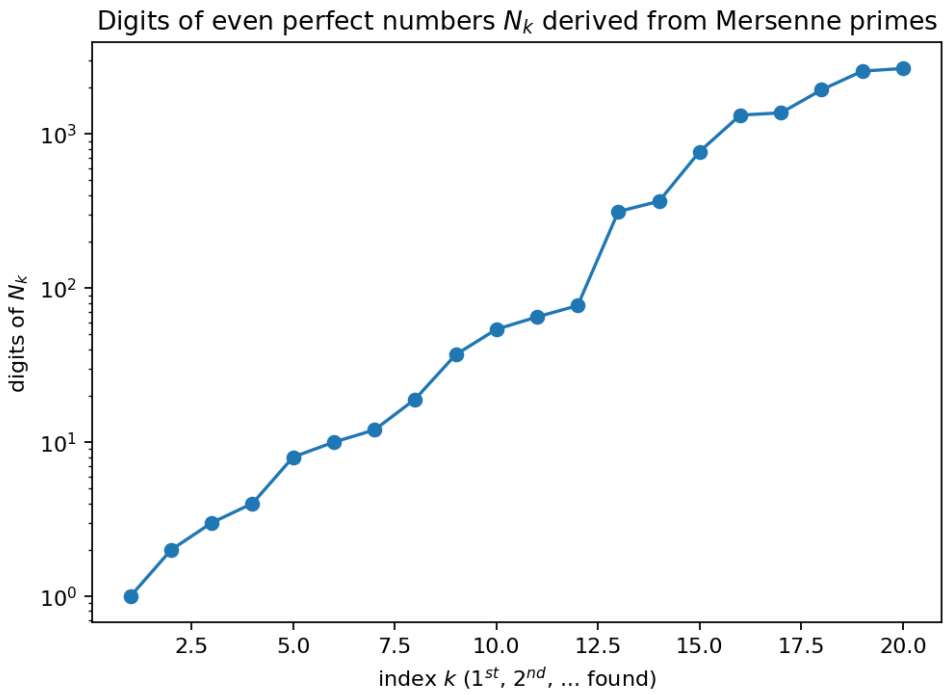
Goal

Make the Mersenne $\leftrightarrow$ perfect-number connection computationally explicit:

If $M_p = 2^p - 1$ is prime, then:

$$N_p = 2^{p-1}(2^p - 1)$$

is an **even perfect number**.



Background (quick refresher)

- *Mersenne numbers and primes refresher*
- *Perfect numbers refresher*

Research question

Across the Mersenne prime exponents discovered within your scan bounds:

- how fast do the corresponding even perfect numbers grow?
- can we verify perfectness ($\sigma(N) = 2N$) efficiently for these cases?

Why this qualifies as a mathematical experiment

- **Finite procedure:** find a set of Mersenne primes within a finite bound and generate perfect numbers.
- **Observable(s):** size metrics (digits), and validation checks of $\sigma(N) = 2N$.
- **Parameter space:** vary the exponent bound and validation depth.
- **Outcome:** concrete examples + growth plots that support intuition.
- **Reproducibility:** exponents tested and successes recorded in artifacts.

Experiment design

Computation

- Obtain a list of Mersenne prime exponents p from a scan (or a fixed list for small p).
- For each p, compute $N_p = 2^{p-1}(2^p - 1)$.
- Verify perfectness for these cases:

$$\sigma(N_p) = 2N_p.$$

For modest p, exact computation is feasible; for larger p, report size metrics and skip expensive checks.

Outputs

- table: p, M_p size, N_p size, and validation status
- plot: p vs digits of N_p
- optional: prime-factor structure display for small cases

How to run

```
make run EXP=e010
```

or:

```
uv run python -m mathxlab.experiments.e010
```

Notes / pitfalls

- Don't attempt divisor-sum sieves for huge N_p; validation must be bounded and explicit.
- Clearly separate "constructed from theorem" (conditional on M_p being prime) from "validated by computation".

Extensions

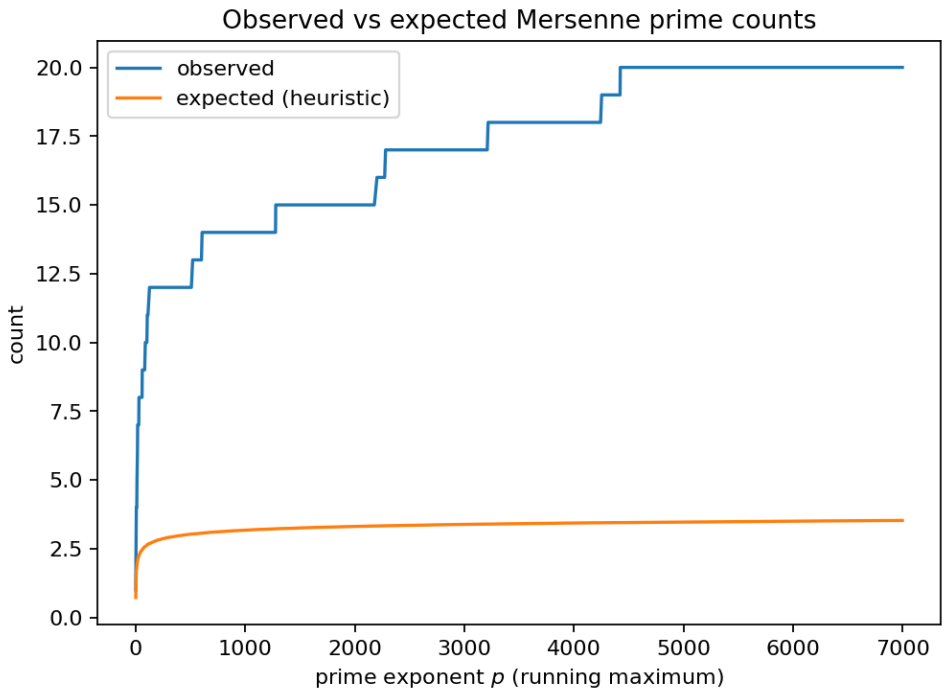
- Cross-link to E002 outputs for perfect numbers and compare growth on the same axes.
- Explore which parts of the perfectness check can be done symbolically using known factorization.

References

See *References*.

[Cald.a, con25b, Inc25a]

3.3.11 E011: Heuristic rarity of Mersenne primes



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- Compare observed Mersenne-prime counts to a simple heuristic expectation curve.
- Visualize cumulative "found vs expected" as exponent bounds increase.
- Keep the discussion explicitly empirical (evidence, not proof).

Goal

Mersenne primes are extremely rare. This experiment compares:

- the observed count of Mersenne primes among prime exponents $p \le P$
- a lightweight heuristic curve that grows slowly with P

The aim is intuition-building and trend visualization, not a rigorous model.

Background (quick refresher)

- *Mersenne numbers and primes refresher*

Research question

For increasing prime exponent bounds P:

- how does the cumulative count of "passes" from your scan grow?
- how does it compare to a simple expectation curve of the form:

$$E(P) \approx \sum_{p \le P,\ p \text{ prime}} \frac{1}{p \ln 2}$$

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a finite scan and compute a finite expected sum.
- **Observable(s):** cumulative count of passes vs. cumulative expected value.
- **Parameter space:** vary P (and optionally cap runtime per test).
- **Outcome:** plots that show agreement/divergence and suggest questions for deeper study.
- **Failure modes:** small ranges are noisy; the experiment should clearly label bounds.

Experiment design

Computation

- From E008 (or a fixed list), get the set of exponents tested and which passed.
- For each bound P, compute:
  - Found(P) = number of passing exponents $p \le P$
  - $E(P)$ = cumulative heuristic sum
- Plot both curves against P.

Outputs

- plot: P vs Found(P) and E(P)
- table: selected checkpoints (P, Found, E)

How to run

```
make run EXP=e011
```

or:

```
uv run python -m mathxlab.experiments.e011
```

Notes / pitfalls

- The heuristic curve is a **comparison tool**, not a theorem.
- Results depend heavily on which exponents were actually tested; record the tested set in the report.
- A single large Mersenne prime can dominate impressions—keep axes and annotations clear.

Extensions

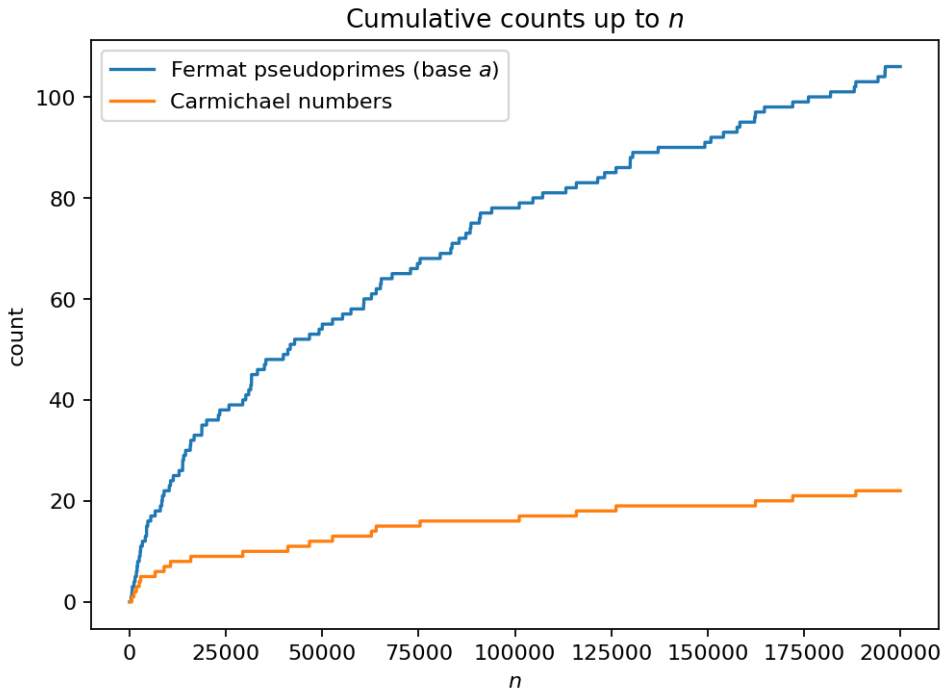
- Compare multiple heuristics (still empirical) and see which best matches the observed range.
- Extend the scan bound and see whether the deviation grows or stabilizes.

References

See *References*.

[Cal21, MR25]

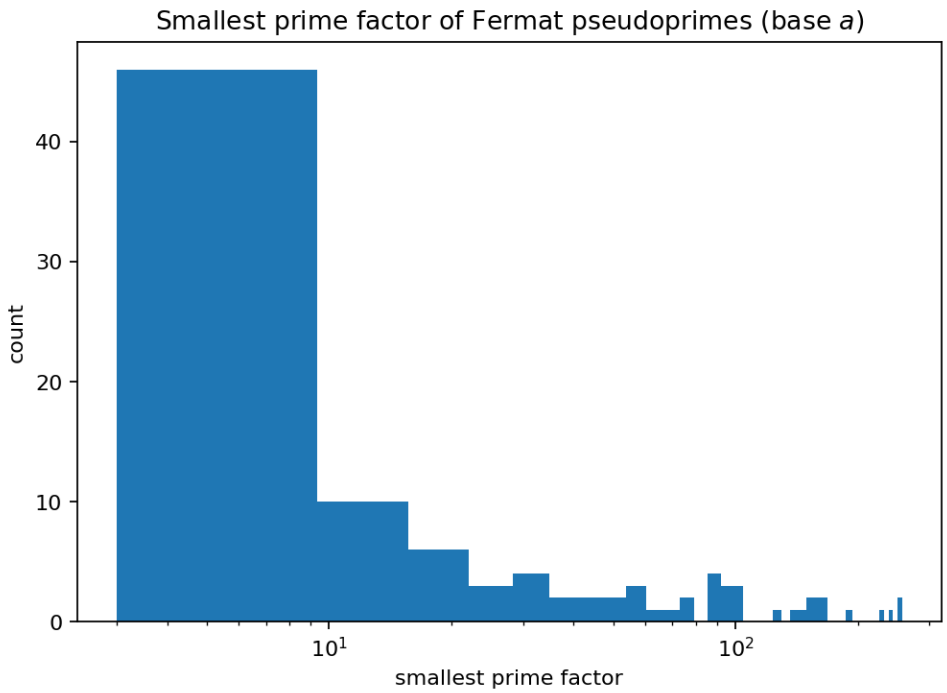
3.3.12 E012: Fermat pseudoprimes and Carmichael numbers (counterexamples)



Tags: `number-theory`, `counterexample-search`, `visualization` See: *Valid Tags*.

Highlights

- Counterexamples to naive Fermat primality testing: base-a pseudoprimes and Carmichael numbers.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.



Goal

Counterexamples to naive Fermat primality testing: base-a pseudoprimes and Carmichael numbers.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e012/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_01_cumulative_counts.png`
- `figures/fig_02_smallest_factor_hist.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e012
```

or:

```
uv run python -m mathxlab.experiments.e012
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

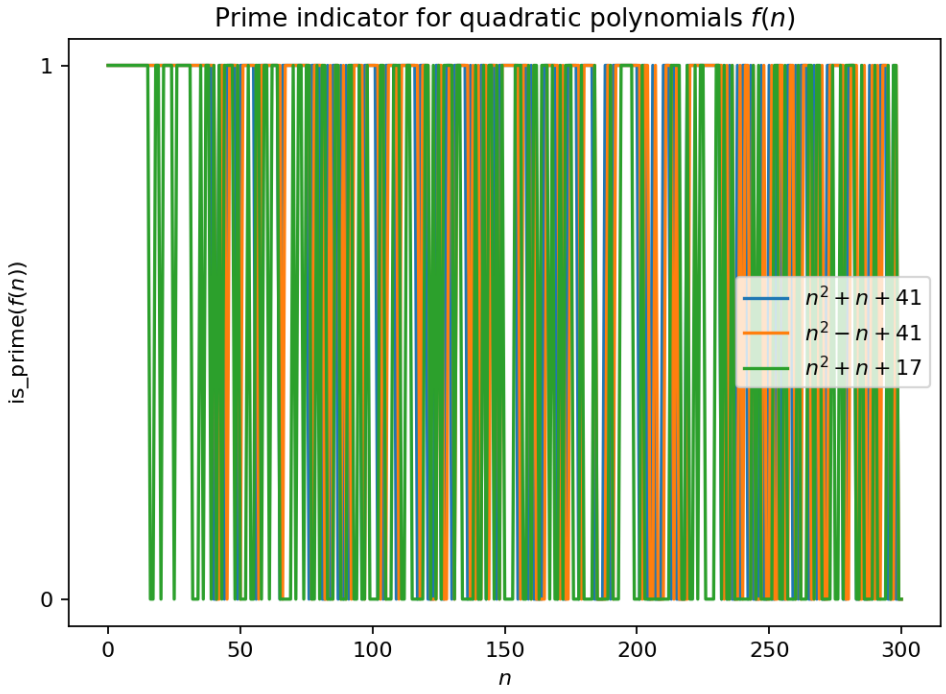
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

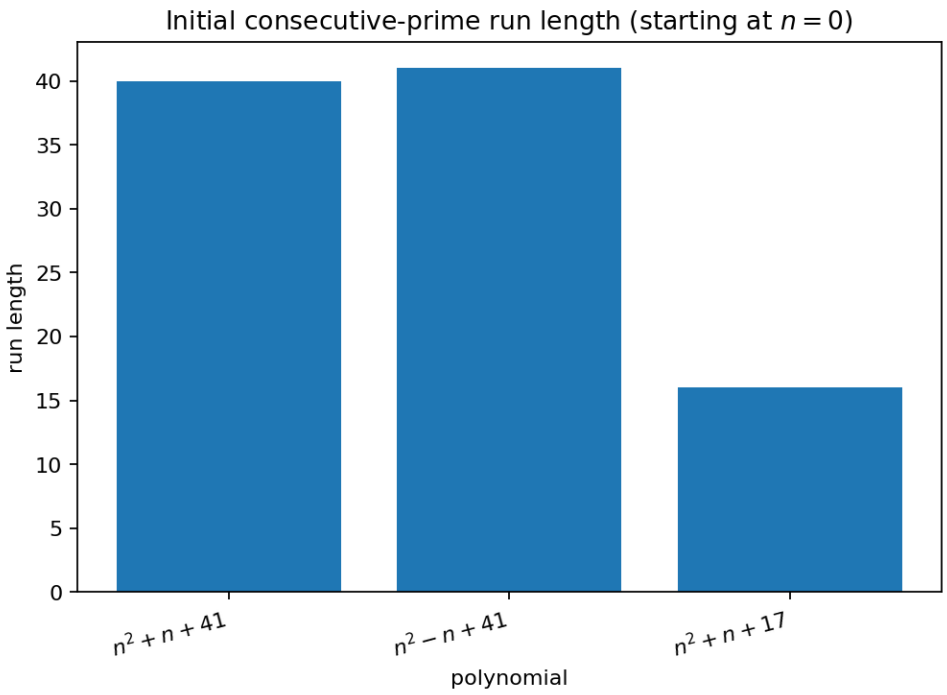
3.3.13 E013: Prime-polynomial counterexamples (Euler's n^2 + n + 41)



Tags: `number-theory`, `counterexample-search`, `visualization` See: *Valid Tags*.

Highlights

- Turn "prime-generating polynomials" folklore into a crisp counterexample (Euler's n^2+n+41).
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.



Goal

Turn "prime-generating polynomials" folklore into a crisp counterexample (Euler's n^2+n+41).

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e013/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_01_prime_indicator.png`
- `figures/fig_02_initial_prime_run_lengths.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e013
```

or:

```
uv run python -m mathxlab.experiments.e013
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

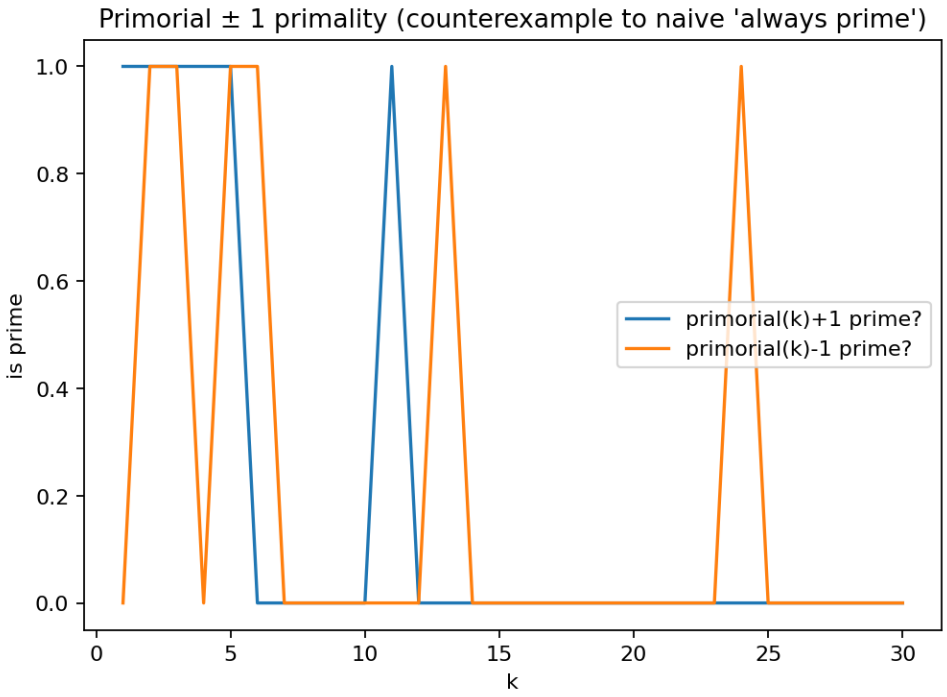
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.14 E014: Primorial ± 1 counterexamples



Tags: `number-theory`, `counterexample-search`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e014/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e014
```

or:

```
uv run python -m mathxlab.experiments.e014
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

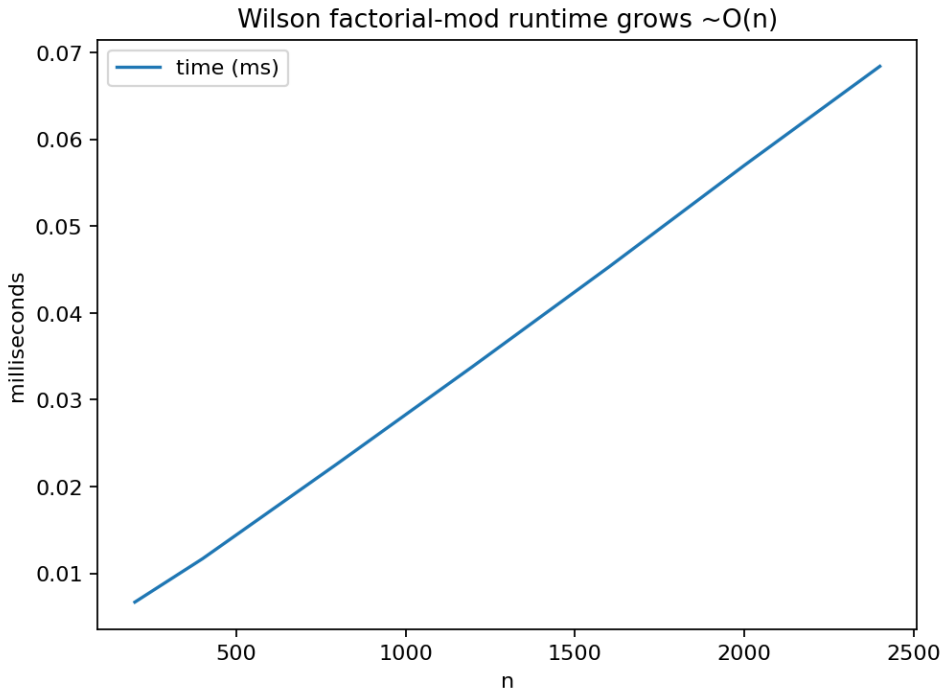
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.15 E015: Wilson test infeasibility



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e015/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e015
```

or:

```
uv run python -m mathxlab.experiments.e015
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.16 E016: Trial division vs Miller–Rabin scaling

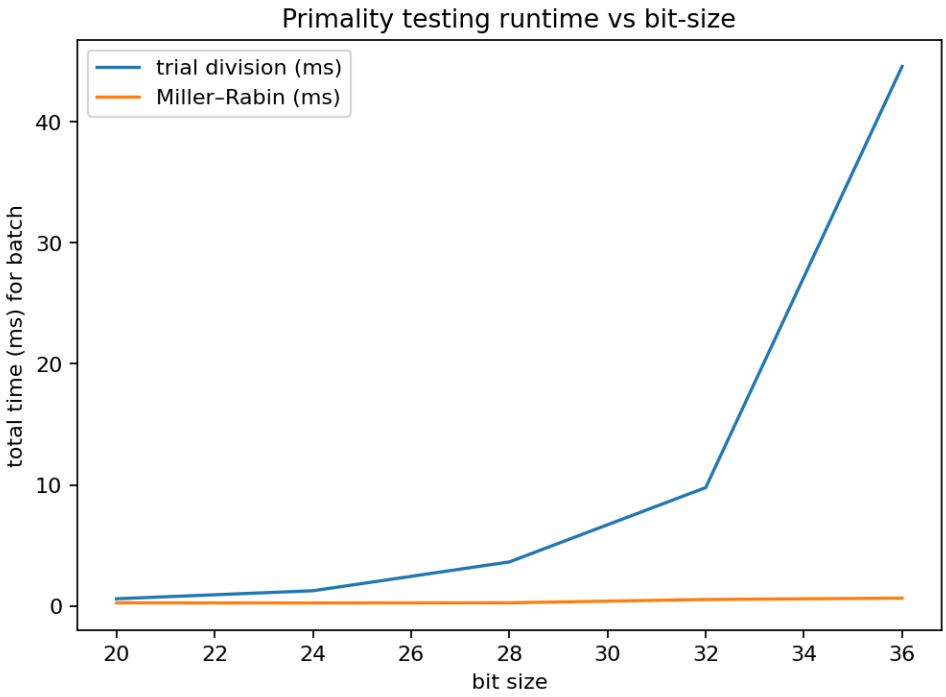
Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e016/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e016
```

or:

```
uv run python -m mathxlab.experiments.e016
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

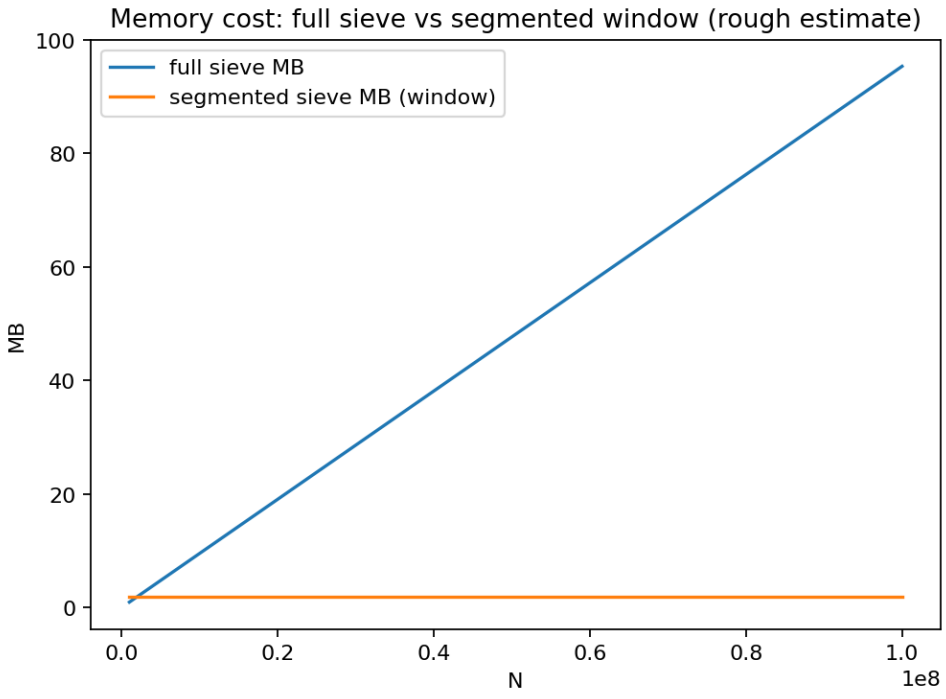
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.17 E017: Sieve memory blow-up vs segmented sieve



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e017/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e017
```

or:

```
uv run python -m mathxlab.experiments.e017
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

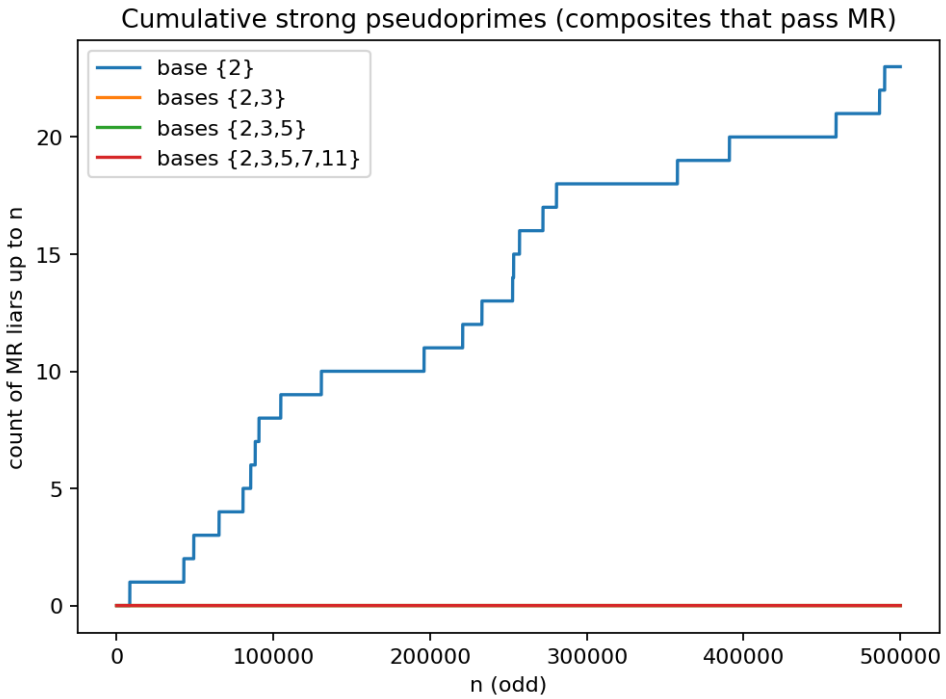
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.18 E018: Miller–Rabin base choice counterexamples



Tags: `number-theory`, `counterexample-search`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e018/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e018
```

or:

```
uv run python -m mathxlab.experiments.e018
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.19 E019: Prime density and PNT visualization

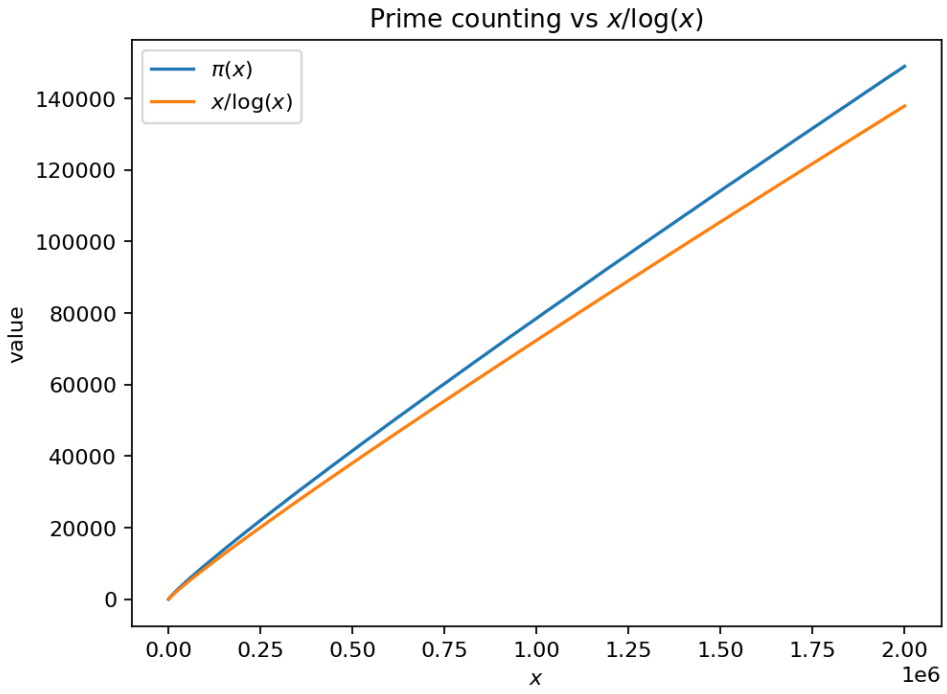
Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

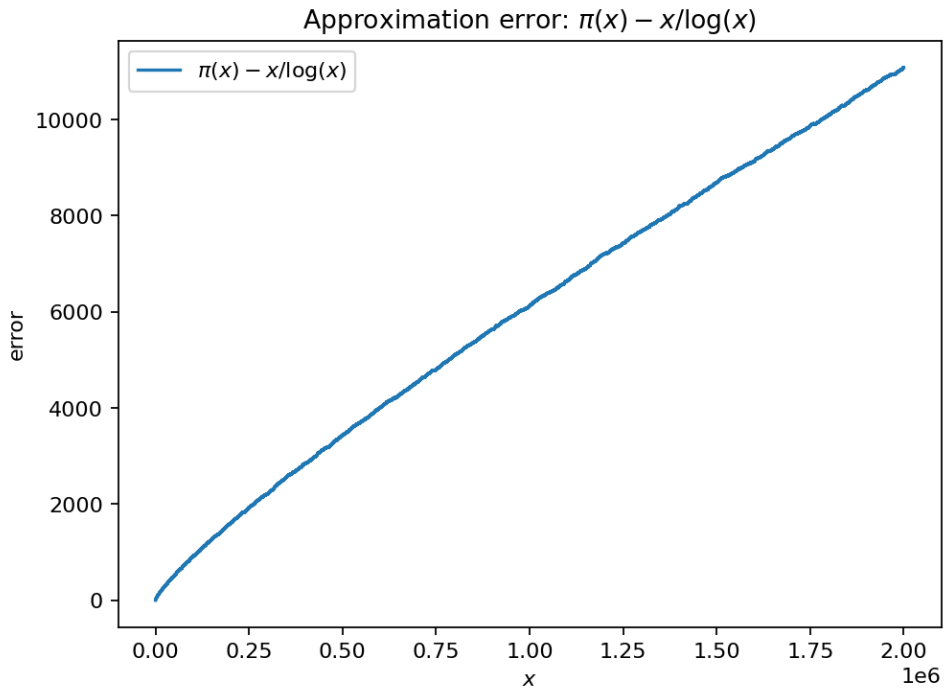
Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.


Prime counting vs x/log(x)
π(x)
x/log(x)
value
140000
120000
100000
80000
60000
40000
20000
0
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
x
1e6


Approximation error: π(x) − x/log(x)
π(x) − x/log(x)
error
10000
8000
6000
4000
2000
0
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
x
1e6

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e019/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e019
```

or:

```
uv run python -m mathxlab.experiments.e019
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

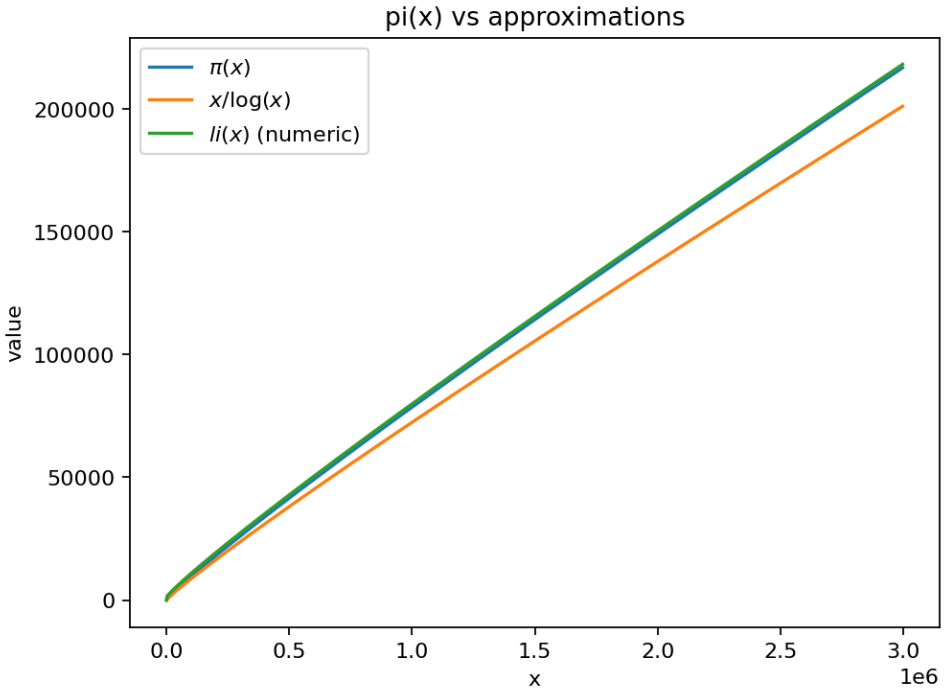
- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

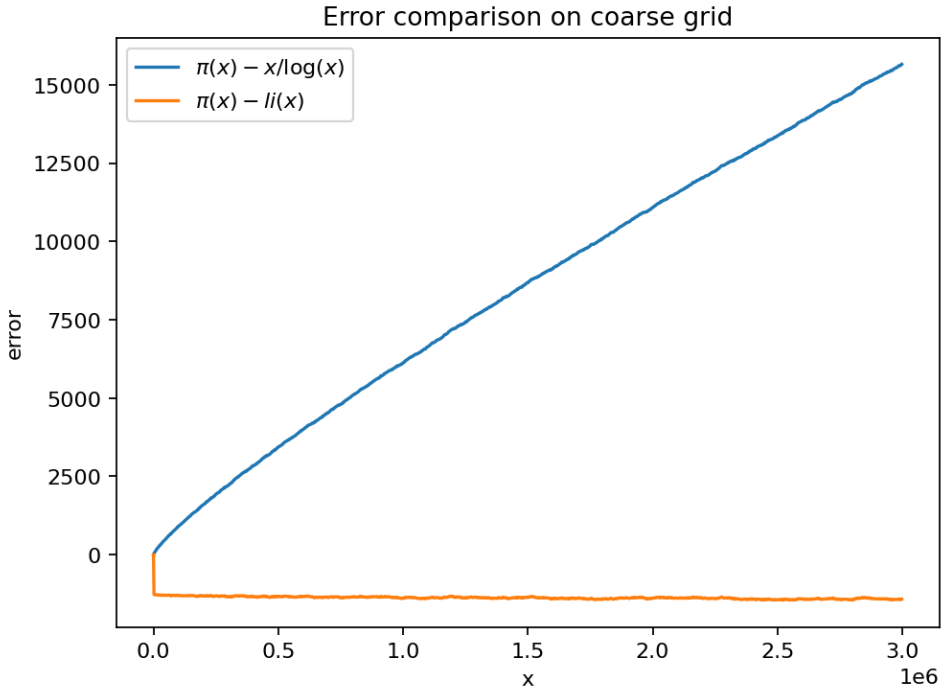
References

See *References*.

3.3.20 E020: Compare pi(x) to li(x) numerically

Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.


pi(x) vs approximations
π(x)
x/log(x)
li(x) (numeric)
value
200000
150000
100000
50000
0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
x
1e6


Error comparison on coarse grid
π(x) − x/log(x)
π(x) − li(x)
error
15000
12500
10000
7500
5000
2500
0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
x
1e6

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e020/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e020
```

or:

```
uv run python -m mathxlab.experiments.e020
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

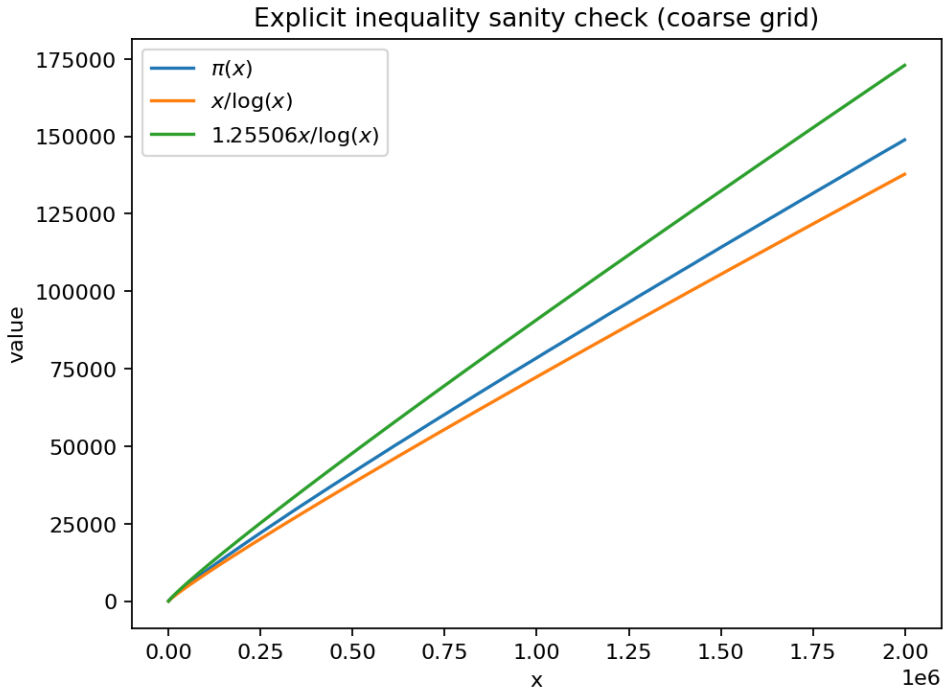
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.21 E021: Explicit bounds sanity checks



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e021/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e021
```

or:

```
uv run python -m mathxlab.experiments.e021
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

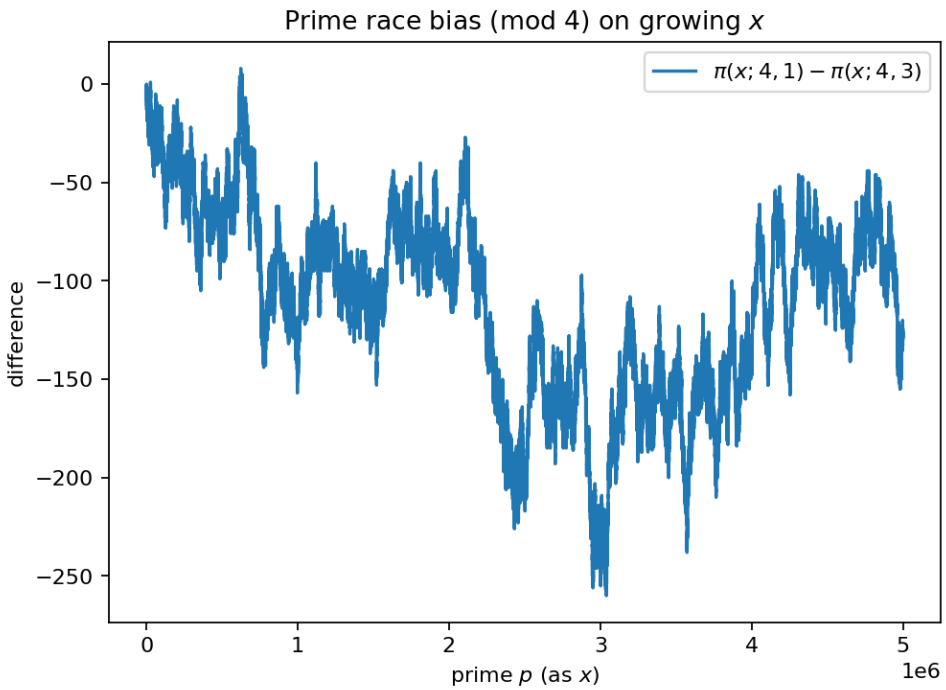
- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.22 E022: Prime race modulo 4

Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.



Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e022/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e022
```

or:

```
uv run python -m mathxlab.experiments.e022
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.23 E023: Residue class distribution mod q

Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

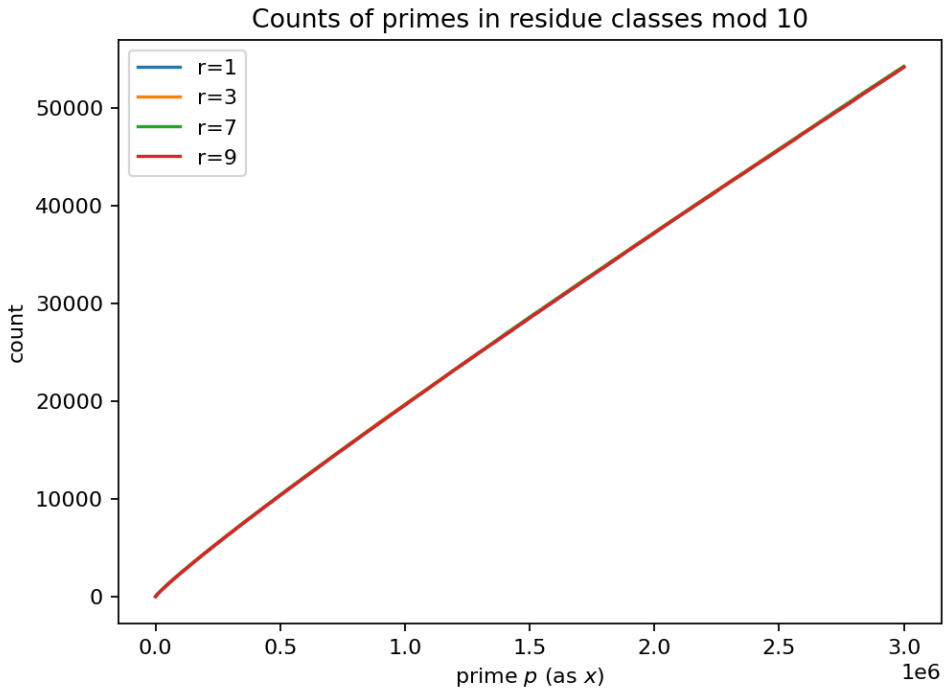
- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*



Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e023/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e023
```

or:

```
uv run python -m mathxlab.experiments.e023
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

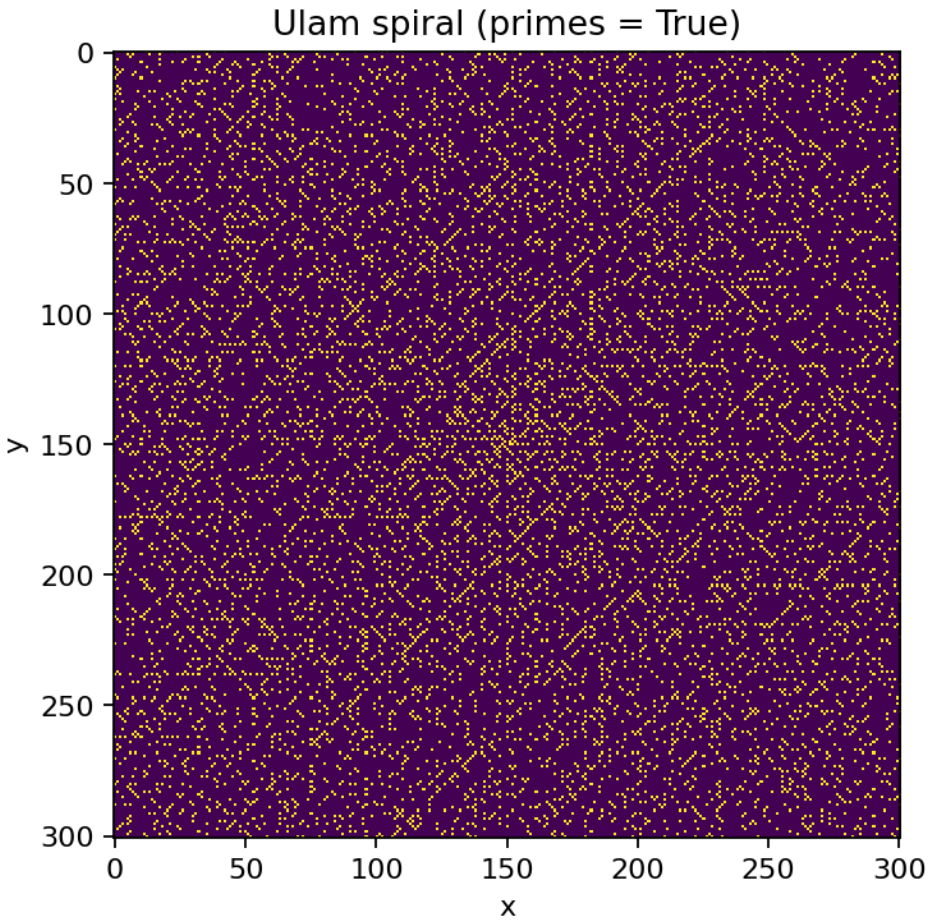
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.24 E024: Ulam spiral structure



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e024/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e024
```

or:

```
uv run python -m mathxlab.experiments.e024
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

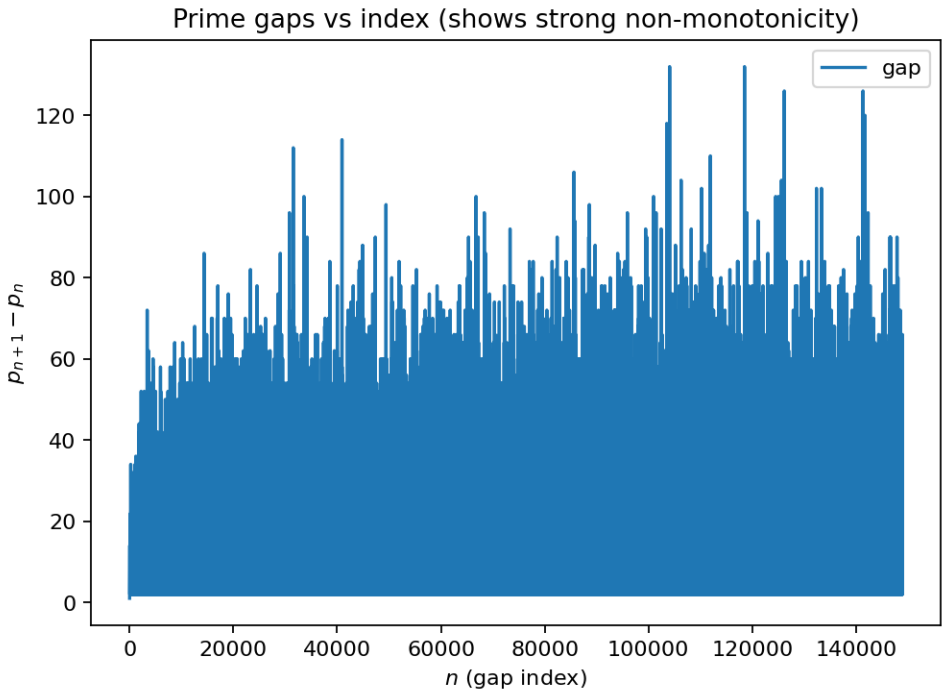
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.25 E025: Prime gaps are not monotone



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e025/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e025
```

or:

```
uv run python -m mathxlab.experiments.e025
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.26 E026: Normalized prime gaps

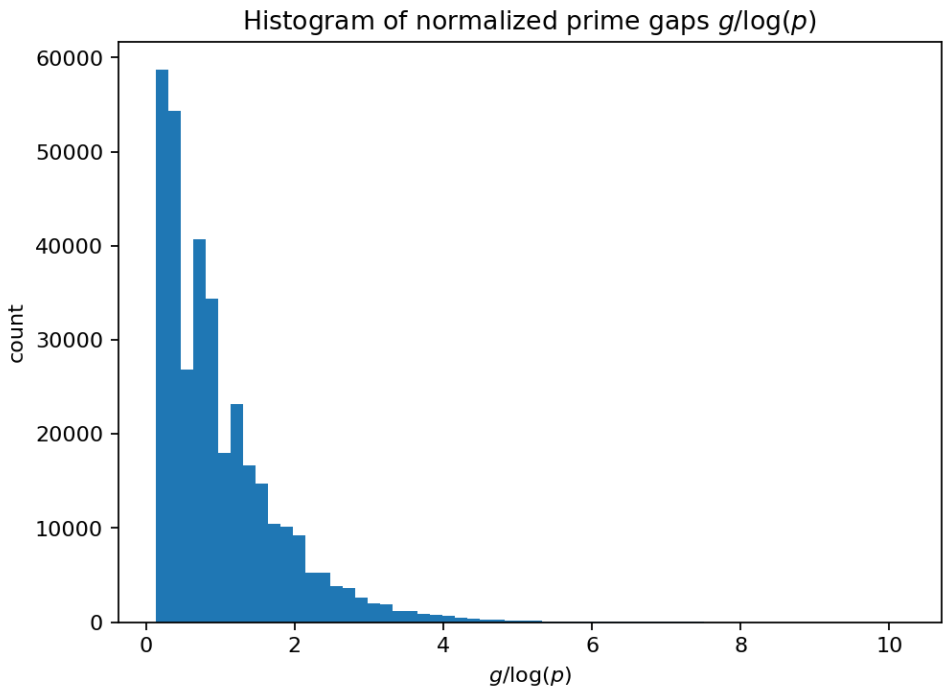
Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e026/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e026
```

or:

```
uv run python -m mathxlab.experiments.e026
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

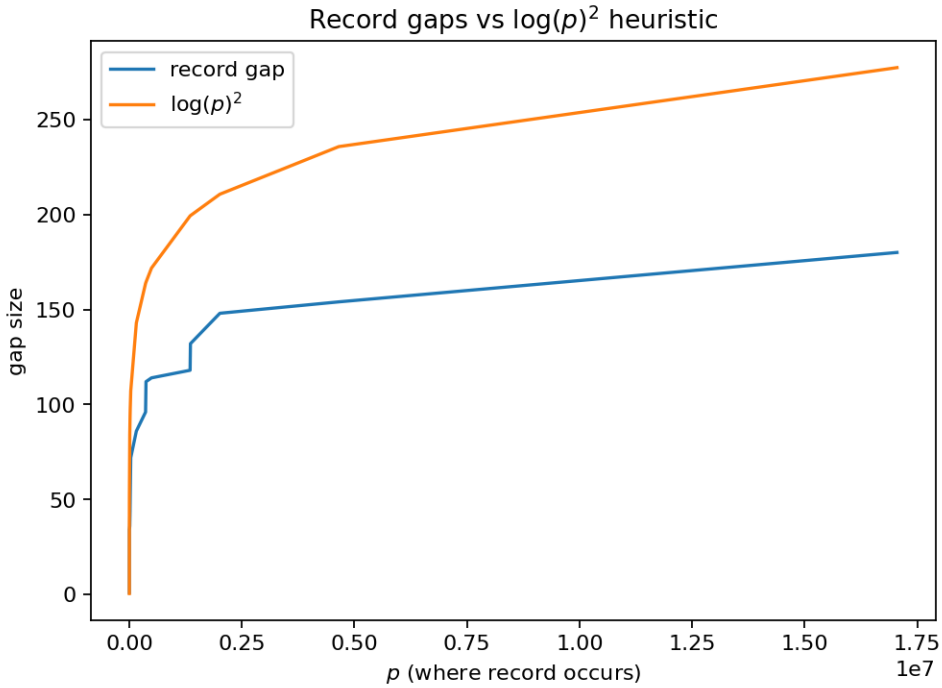
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.27 E027: Record prime gaps vs log^2 heuristic



Tags: `number-theory`, `conjecture-generation`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e027/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e027
```

or:

```
uv run python -m mathxlab.experiments.e027
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

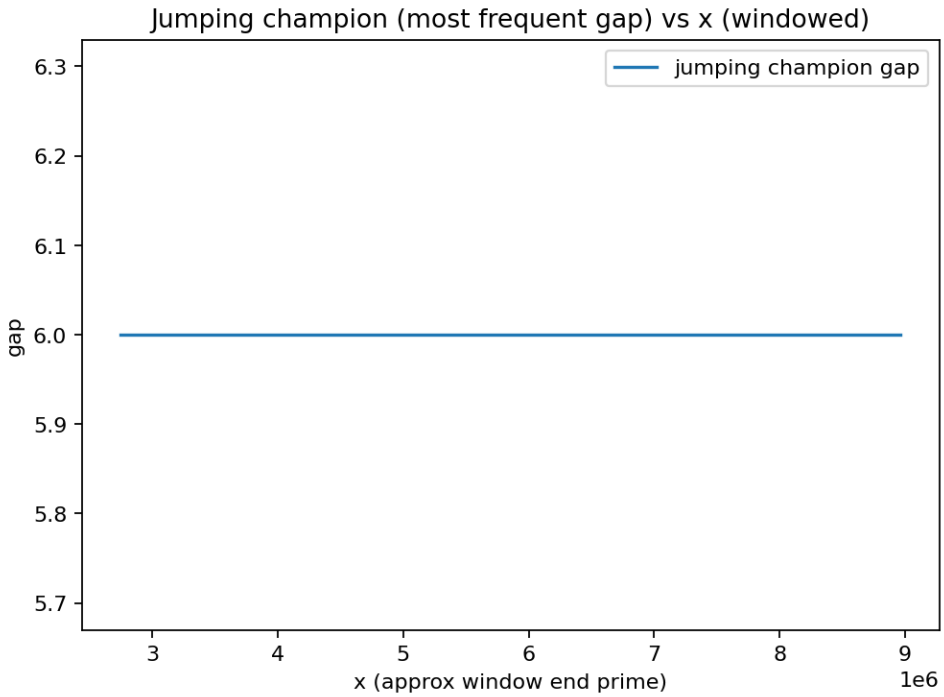
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.28 E028: Jumping champions (most frequent gaps)



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e028/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e028
```

or:

```
uv run python -m mathxlab.experiments.e028
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.29 E029: Twin primes: observed vs heuristic

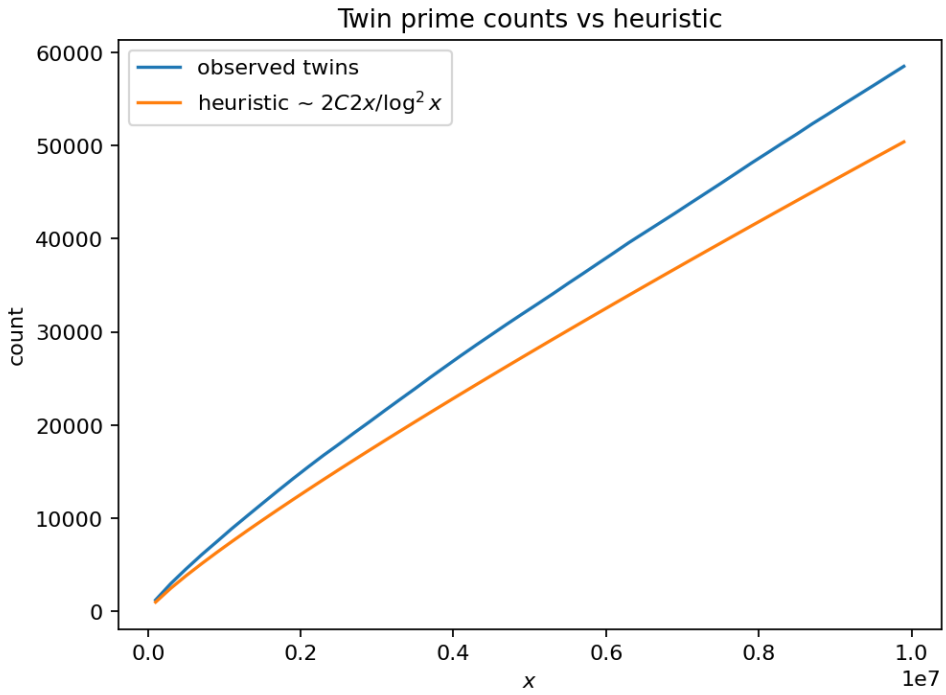
Tags: `number-theory`, `conjecture-generation`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e029/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e029
```

or:

```
uv run python -m mathxlab.experiments.e029
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

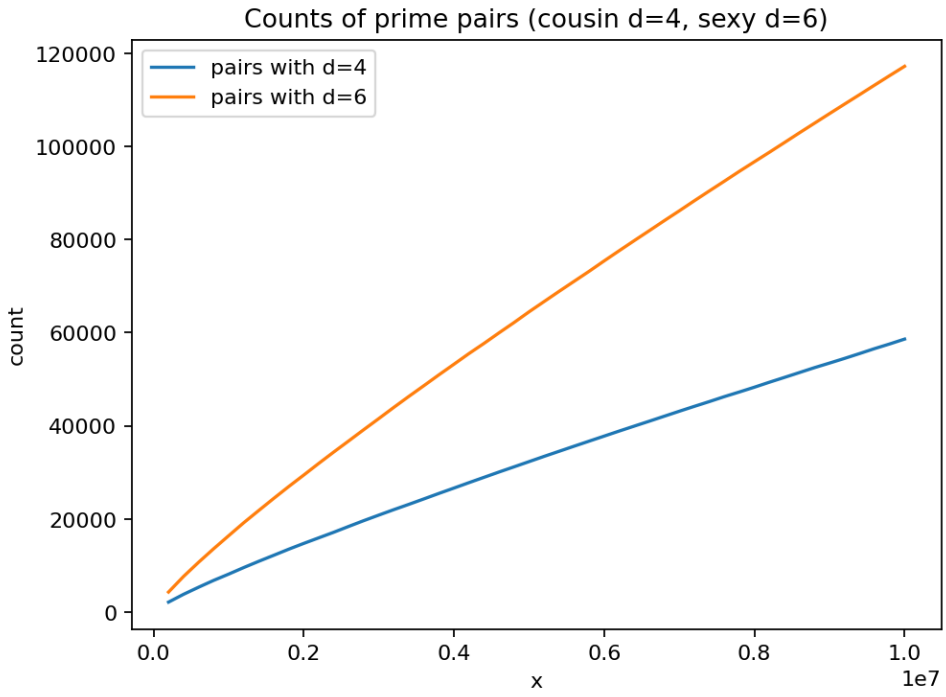
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.30 E030: Cousin and sexy prime pairs



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e030/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e030
```

or:

```
uv run python -m mathxlab.experiments.e030
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

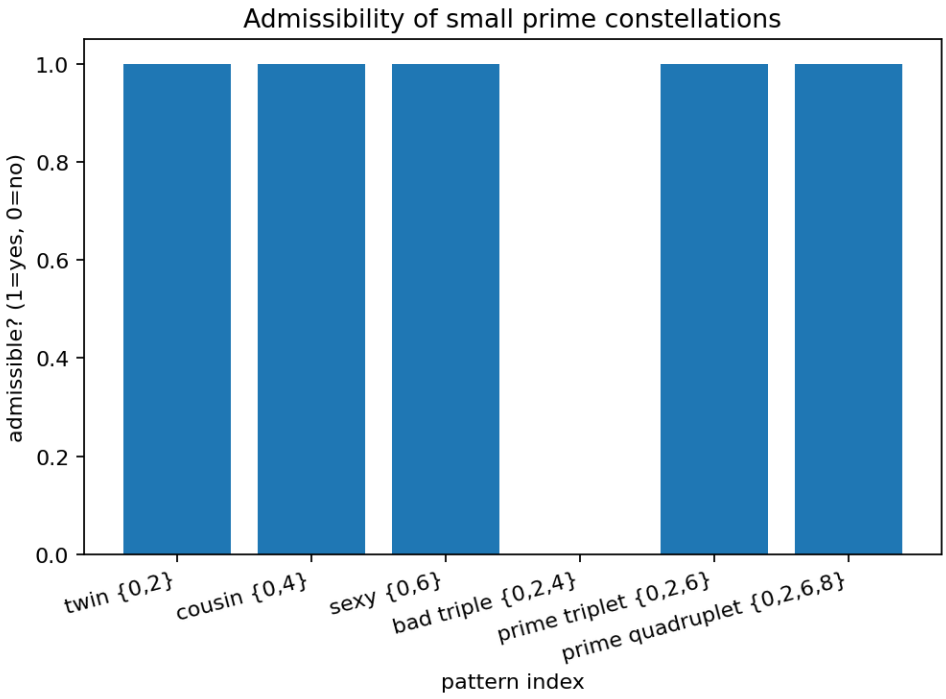
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.31 E031: Admissibility and modular obstructions



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e031/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e031
```

or:

```
uv run python -m mathxlab.experiments.e031
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.32 E032: Prime triplets and quadruplets

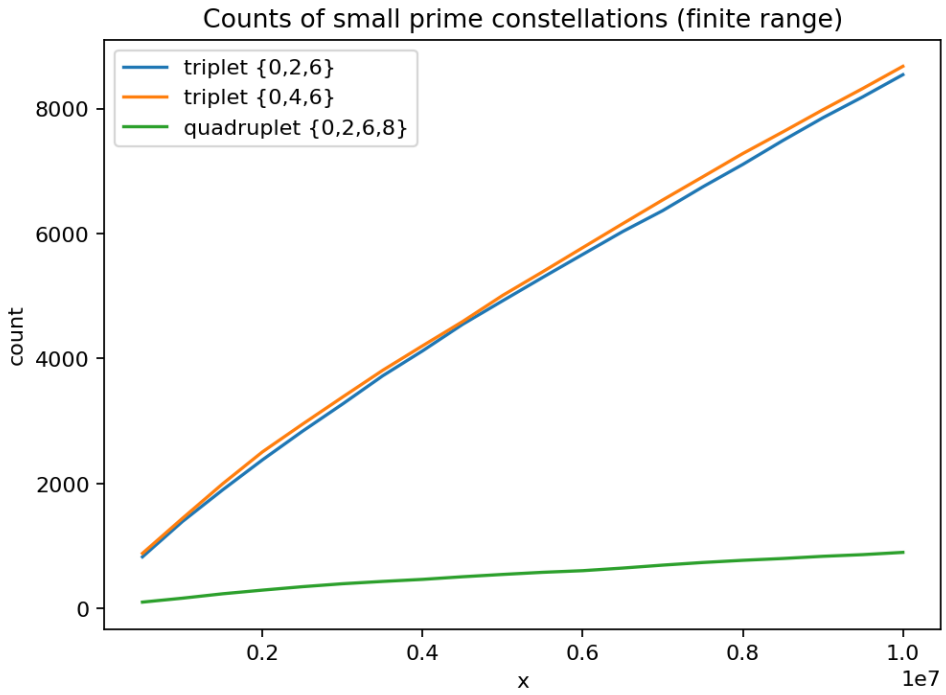
Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e032/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e032
```

or:

```
uv run python -m mathxlab.experiments.e032
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

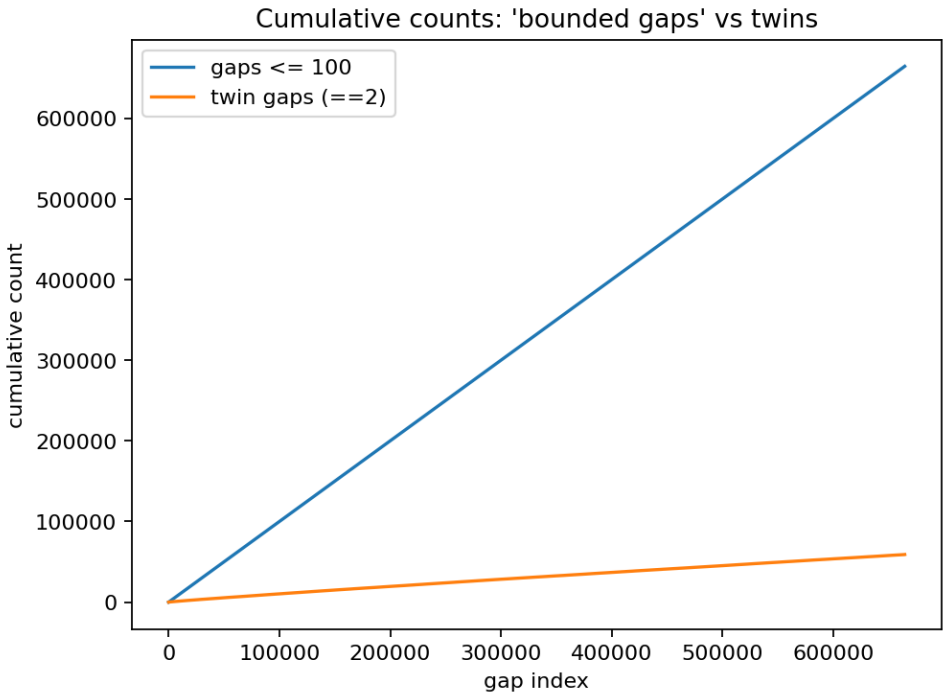
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.33 E033: Bounded gaps vs twin primes (not the same)



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e033/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e033
```

or:

```
uv run python -m mathxlab.experiments.e033
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

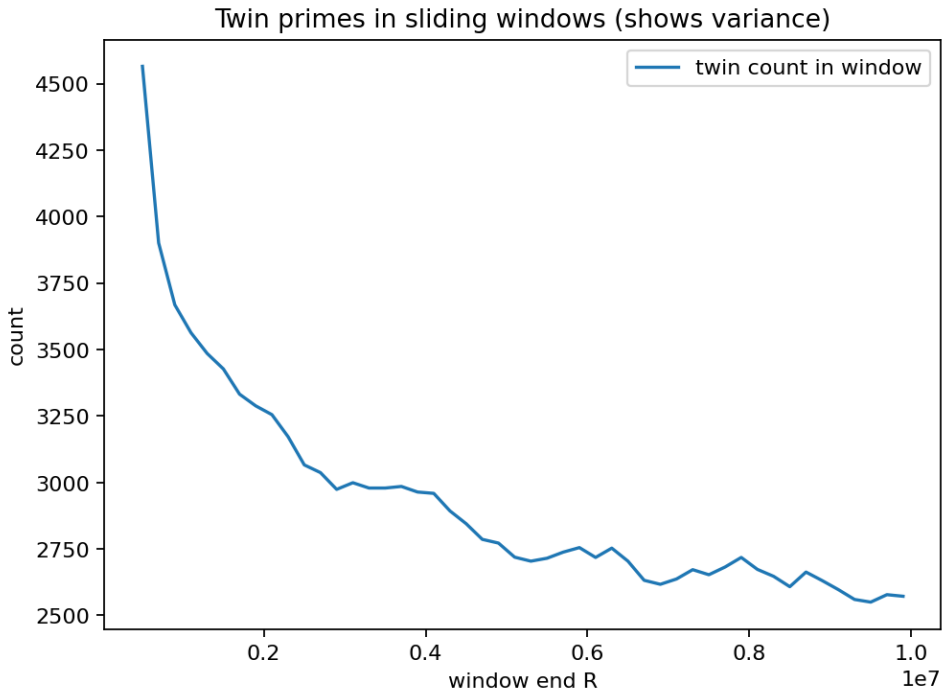
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.34 E034: Twin primes in sliding windows



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e034/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e034
```

or:

```
uv run python -m mathxlab.experiments.e034
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.35 E035: Primes in arithmetic progressions mod q

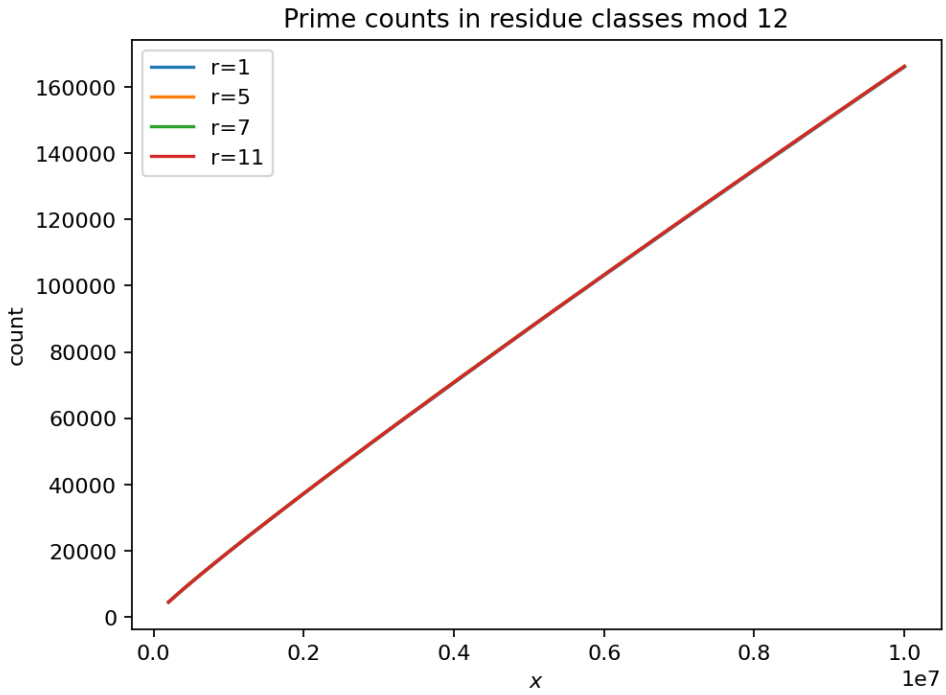
Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e035/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e035
```

or:

```
uv run python -m mathxlab.experiments.e035
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

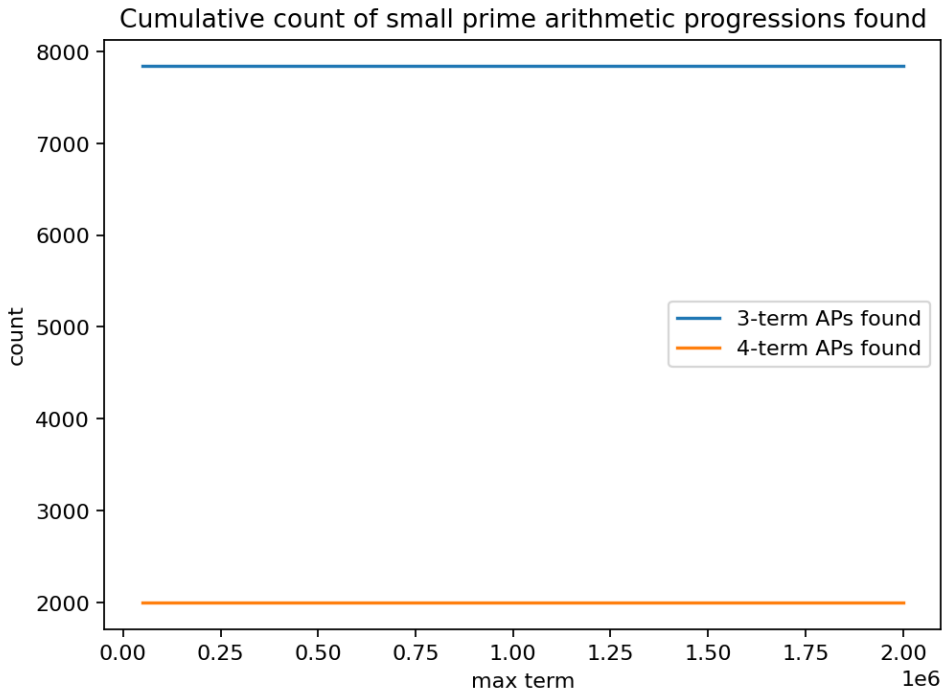
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.36 E036: Prime arithmetic progressions (small search)



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e036/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e036
```

or:

```
uv run python -m mathxlab.experiments.e036
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

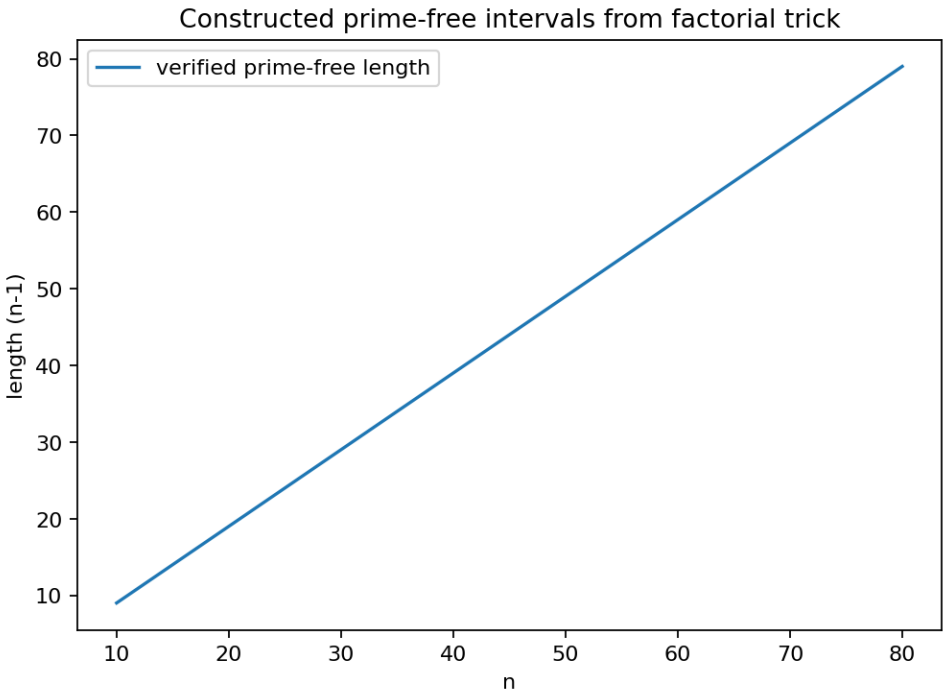
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.37 E037: Prime-free intervals via factorial construction



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e037/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e037
```

or:

```
uv run python -m mathxlab.experiments.e037
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.38 E038: Bertrand's postulate (computational verification)

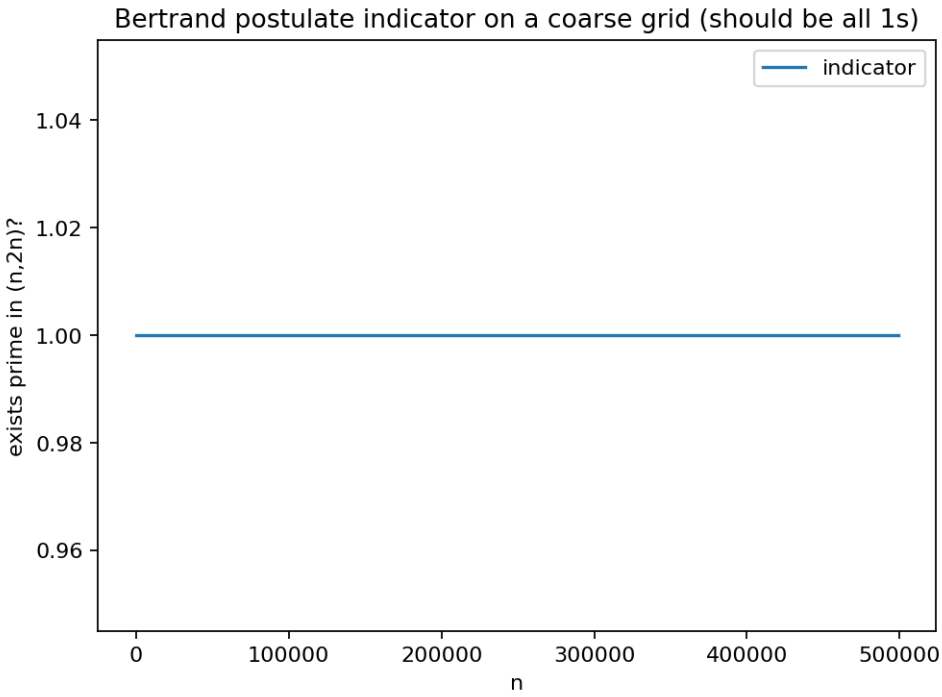
Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e038/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e038
```

or:

```
uv run python -m mathxlab.experiments.e038
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

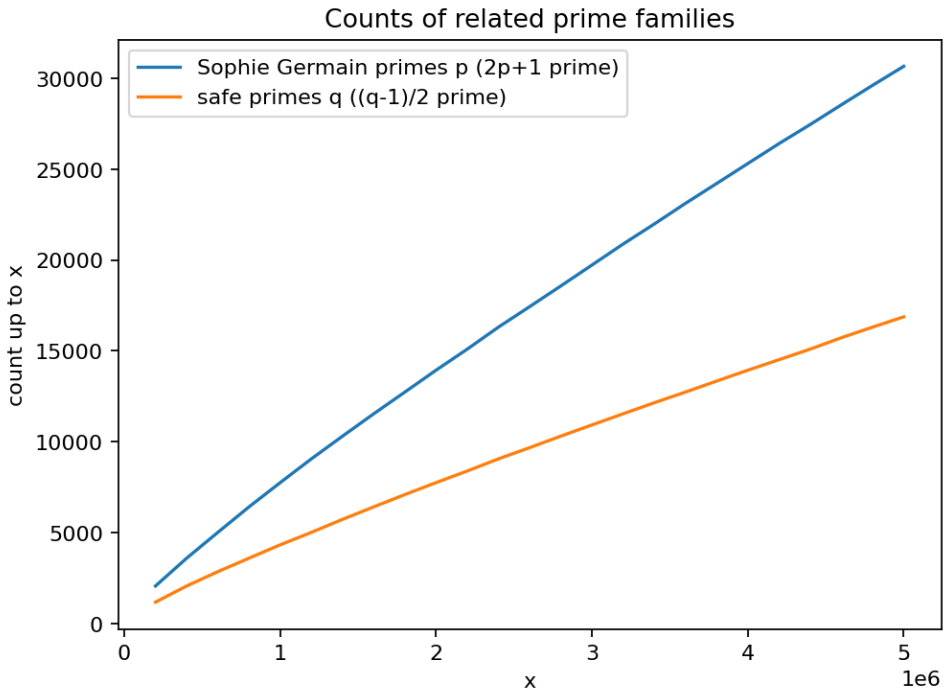
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.39 E039: Sophie Germain and safe primes



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e039/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e039
```

or:

```
uv run python -m mathxlab.experiments.e039
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

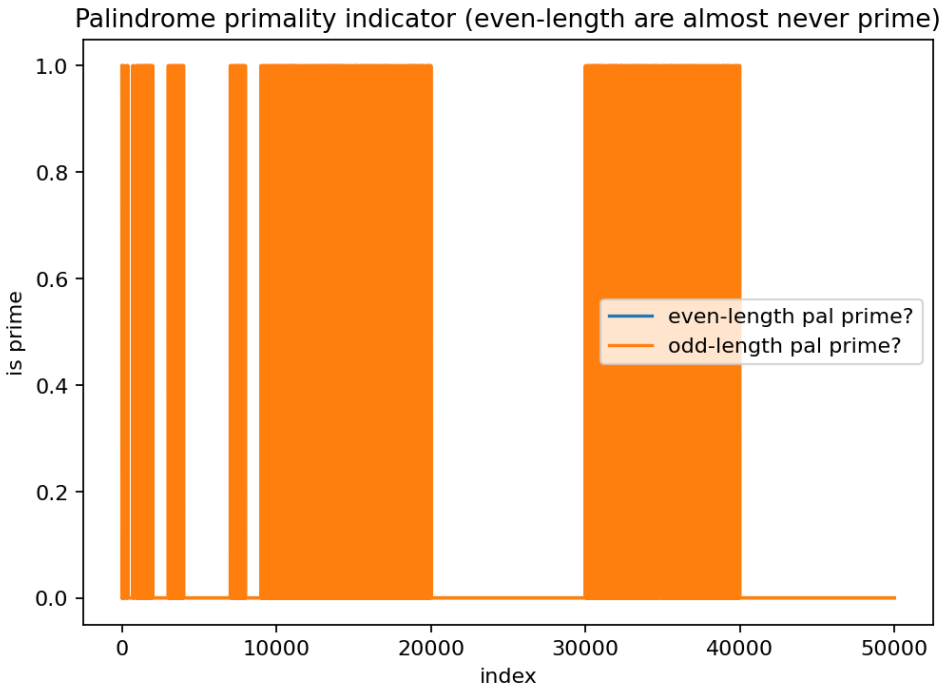
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.40 E040: Palindromic primes and the '11 trap'



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e040/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e040
```

or:

```
uv run python -m mathxlab.experiments.e040
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.41 E041: Fermat numbers: not all prime

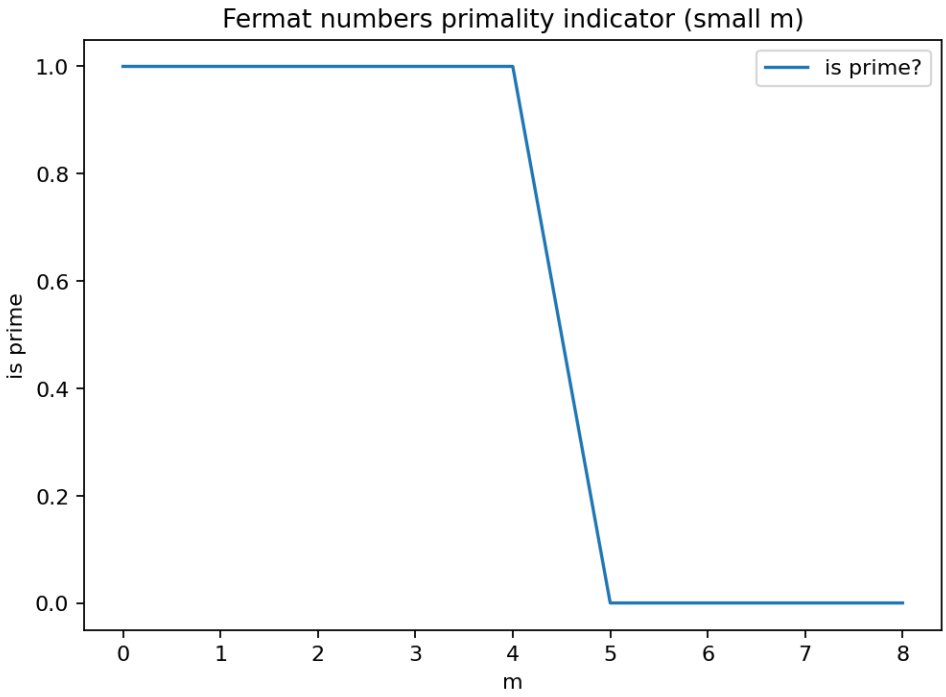
Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e041/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e041
```

or:

```
uv run python -m mathxlab.experiments.e041
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

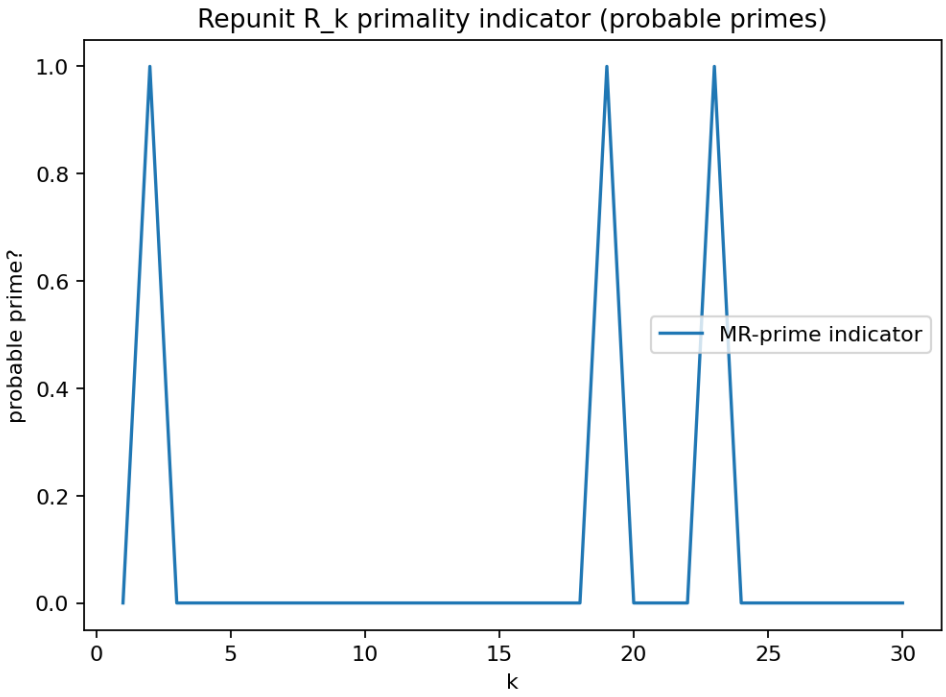
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.42 E042: Repunit primes (small k scan)



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e042/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e042
```

or:

```
uv run python -m mathxlab.experiments.e042
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

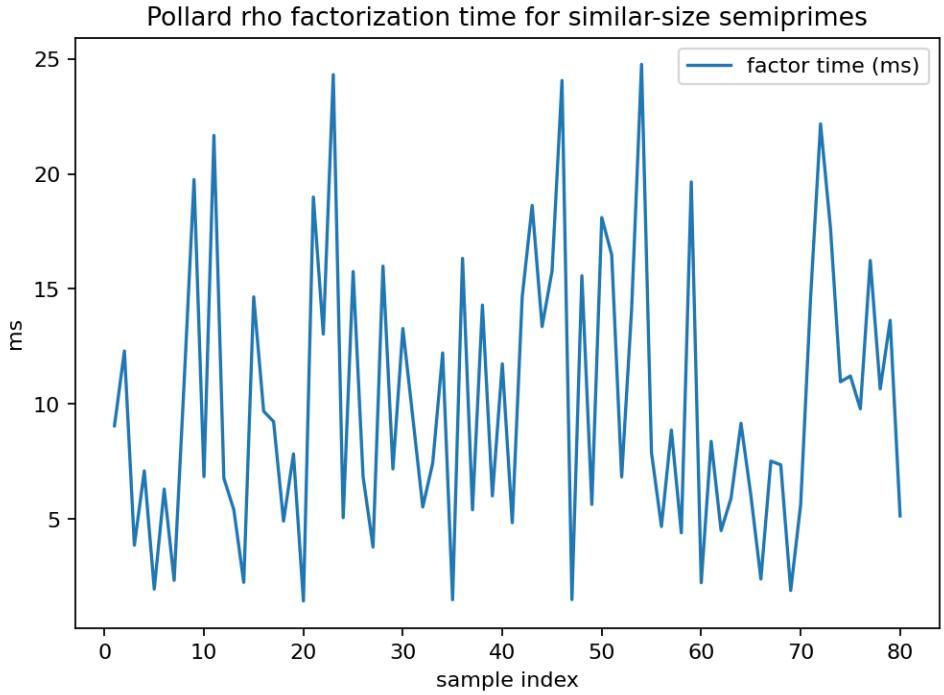
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.43 E043: Pollard rho runtime variability



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e043/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e043
```

or:

```
uv run python -m mathxlab.experiments.e043
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.44 E044: Solovay–Strassen vs Miller–Rabin (liars)

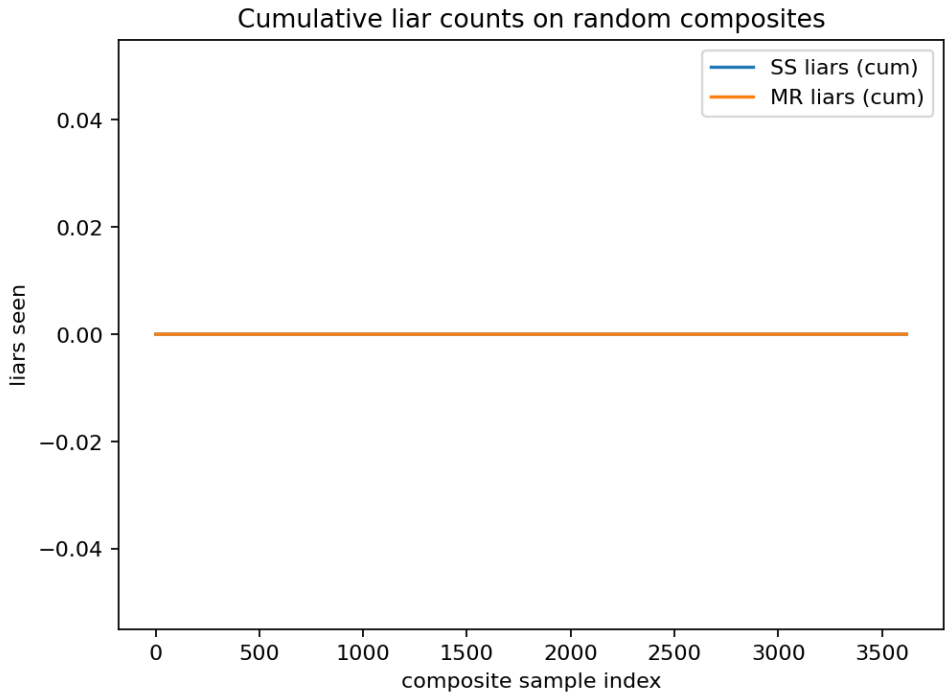
Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e044/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e044
```

or:

```
uv run python -m mathxlab.experiments.e044
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

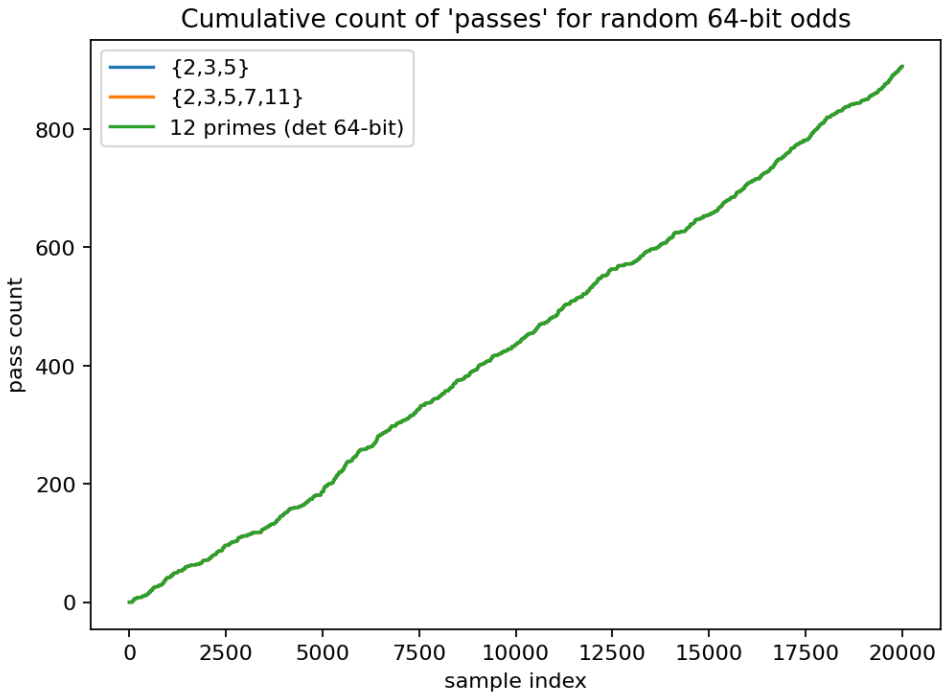
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.45 E045: Deterministic 64-bit MR base sets



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e045/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e045
```

or:

```
uv run python -m mathxlab.experiments.e045
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

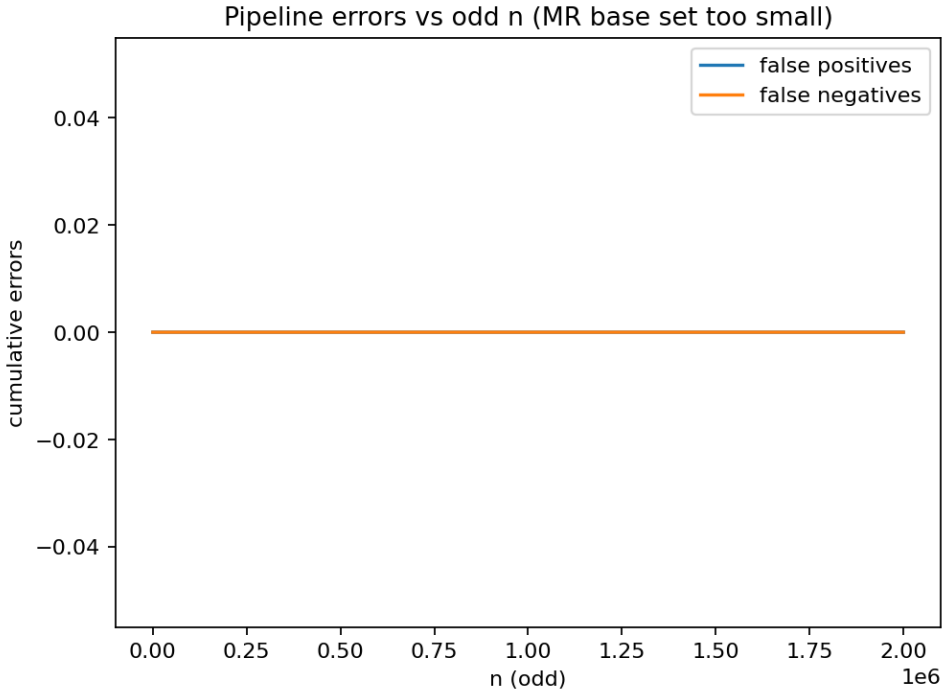
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.46 E046: Prime-testing pipeline and tuning pitfalls



Tags: `number-theory`, `quantitative-exploration`, `visualization` See: *Valid Tags*.

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* "looks-true-until-it-breaks" behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to :mod:`mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + "witness objects" for failures.
- **Reproducibility:** outputs saved to `out/e046/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
- `figures/fig_*.png`
- `params.json`
- `report.md`

How to run

```
make run EXP=e046
```

or:

```
uv run python -m mathxlab.experiments.e046
```

Notes / pitfalls

- "No counterexample found" only means "none found within the configured bounds".
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

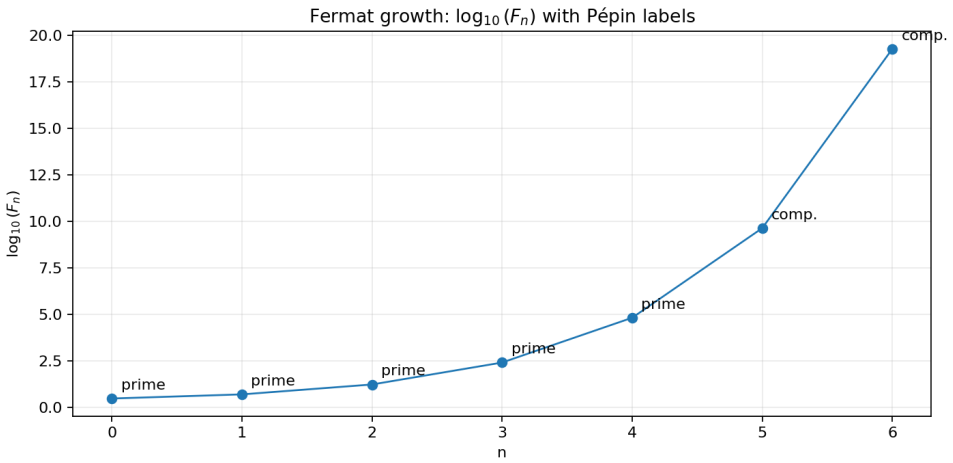
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

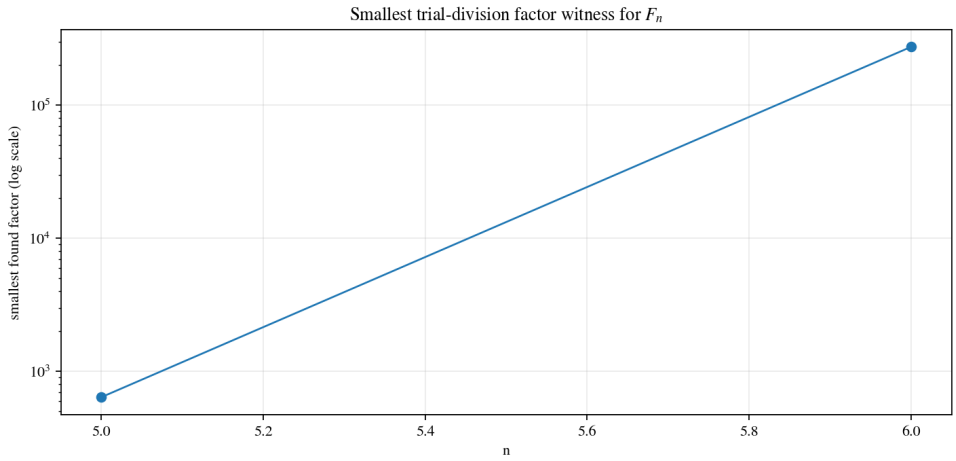
References

See *References*.

3.3.47 E047: Fermat numbers: Pépin test + factor witnesses



Tags: `number-theory`, `counterexample-search`, `visualization`, `fermat` See: *Valid Tags*.



Highlights

- Runs Pépin's test for $F_n = 2^{2^n} + 1$ (definitive for Fermat numbers).
- Finds small, explicit factor witnesses by bounded trial division.
- Visualizes how explosively F_n grows and where the first counterexample appears.

Goal

Demonstrate (computationally) why the statement "all Fermat numbers are prime" fails, and produce concrete witnesses.

Background (quick refresher)

- *Fermat numbers*

Research question

How quickly do Fermat numbers become composite in practice, and how easy is it to exhibit a *witness* factor for the first failures?

Experiment design

- Compute F_n for $n \le n_{max}$.
- Apply Pépin's test (for $n \ge 1$) to classify prime vs composite.
- If composite, attempt to find a small factor by trial division up to a configurable bound.
- Plot $\log_{10}(F_n)$ and the smallest discovered factor (if any).

Reproducibility

- `params.json` records the run settings.
- `report.md` summarizes the key findings.
- `figures/*.png` contains the plots for the run.

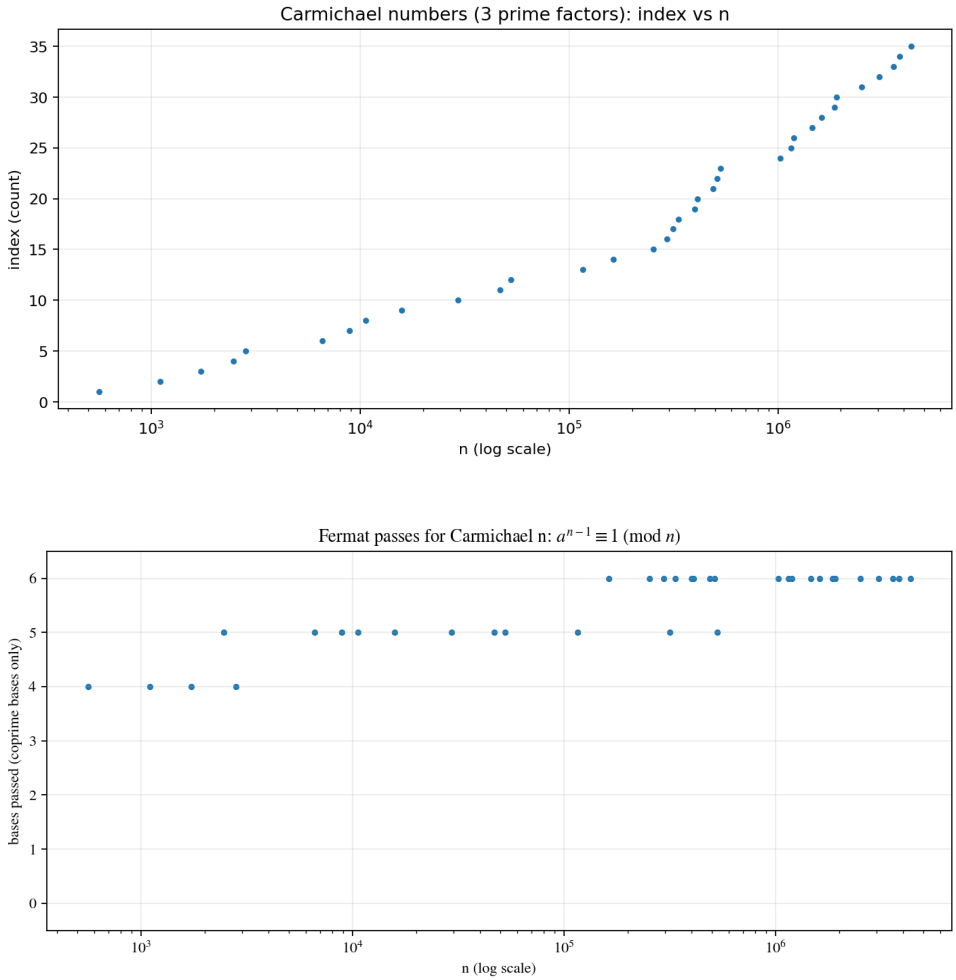
Interpreting the results

- If Pépin says "composite", the number is definitively composite (not a probabilistic claim).
- The factor search is bounded: "no factor found" does not mean "prime".
- The classic factor for F_5 (and often for F_6) appears quickly under the default bounds.

References

See Kř\'ıžek *et al.* [KvrivzekLvSomer13], Wikipedia contributors [Wikipediacontributors25b], The OEIS Foundation Inc. [TheOFInc25a].

3.3.48 E048: Carmichael numbers: Korselt scan + Fermat counterexamples



Tags: `number-theory`, `counterexample-search`, `quantitative-exploration`, `visualization`, `carmichael`, `pseudoprime` See: *Valid Tags*.

Highlights

- Enumerates many small Carmichael numbers by scanning squarefree products of three primes.
- Verifies that these composites pass Fermat's test for several common bases.
- Visualizes the distribution of discovered Carmichael numbers on a log scale.

Goal

Show that Fermat's primality test can fail spectacularly, and produce a dataset of explicit counterexamples.

Background (quick refresher)

- *Carmichael numbers*
- *Prime numbers refresher*

Research question

Within a moderate bound N, how many Carmichael numbers can we find by a simple Korselt scan, and how many Fermat bases do they fool?

Experiment design

- Enumerate candidates $n = pqr \leq N$ with distinct primes $p < q < r$.
- Apply Korselt's criterion: $(p-1) \mid (n-1)$ for each prime factor $p \mid n$.
- For each Carmichael n, test Fermat congruence for a small base set (skipping non-coprime bases).
- Plot index vs n and "bases passed" vs n.

Reproducibility

- `params.json` records the run settings.
- `report.md` summarizes the key findings.
- `figures/*.png` contains the plots for the run.

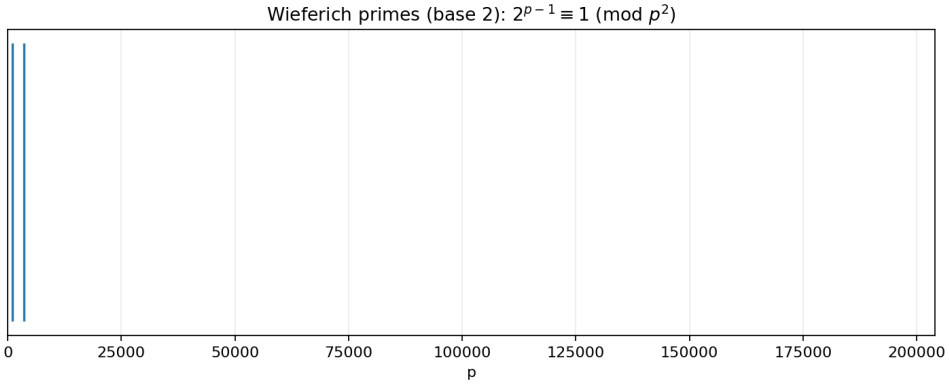
Interpreting the results

- All numbers found are composite by construction, yet they pass Fermat congruences for *all* coprime bases (in theory).
- The scan is restricted to 3-prime-factor Carmichael numbers; larger-factor Carmichaels exist too.
- Seeing "all bases passed" in the plot is a strong reminder: Fermat alone is not a reliable primality test.

References

See Alford *et al.* [AGP94], Carmichael [Car12], The OEIS Foundation Inc. [TheOFInc25e], Wikipedia contributors [Wikipediacontributors25a].

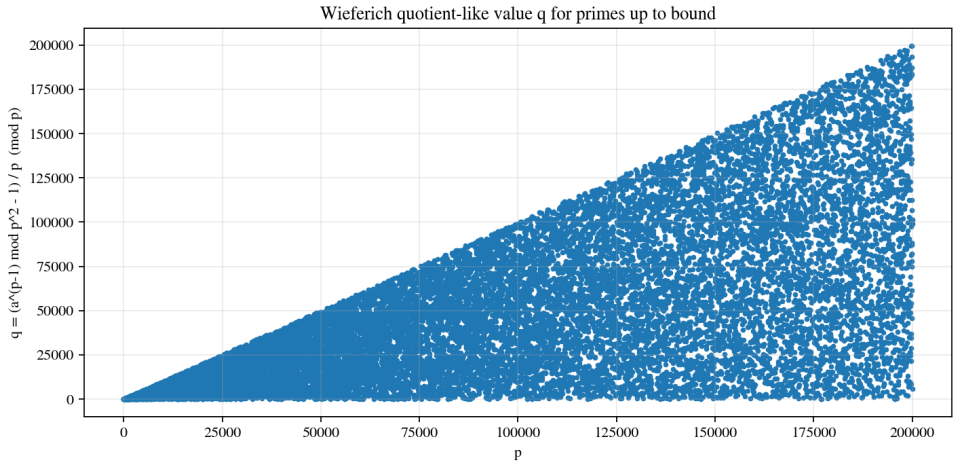
3.3.49 E049: Wieferich primes (base 2): scan and quotient visualization



Tags: `number-theory`, `counterexample-search`, `visualization`, `wieferich` See: *Valid Tags*.

Highlights

- Scans primes up to a bound for the Wieferich condition $2^{p-1} \equiv 1 \pmod{p^2}$.
- Recovers the known small hits (1093 and 3511) under default settings.
- Visualizes a quotient-like value that is exactly zero for Wieferich primes.



Goal

Explore a rare strengthening of Fermat's congruence and visualize how exceptional Wieferich primes are.

Background (quick refresher)

- *Wieferich primes*
- *Prime numbers refresher*

Research question

Up to a bound B, which primes satisfy the Wieferich condition (base 2), and how does the quotient-like statistic vary across primes?

Experiment design

- Generate all primes $p \leq B$ (excluding $p = 2$ for base 2).
- Compute $r = 2^{p-1} \bmod p^2$ and derive $q = (r - 1)/p \pmod{p}$.
- Mark primes with $q = 0$ as Wieferich hits.
- Plot hit positions and a scatter of q values.

Reproducibility

- `params.json` records the run settings.
- `report.md` summarizes the key findings.
- `figures/*.png` contains the plots for the run.

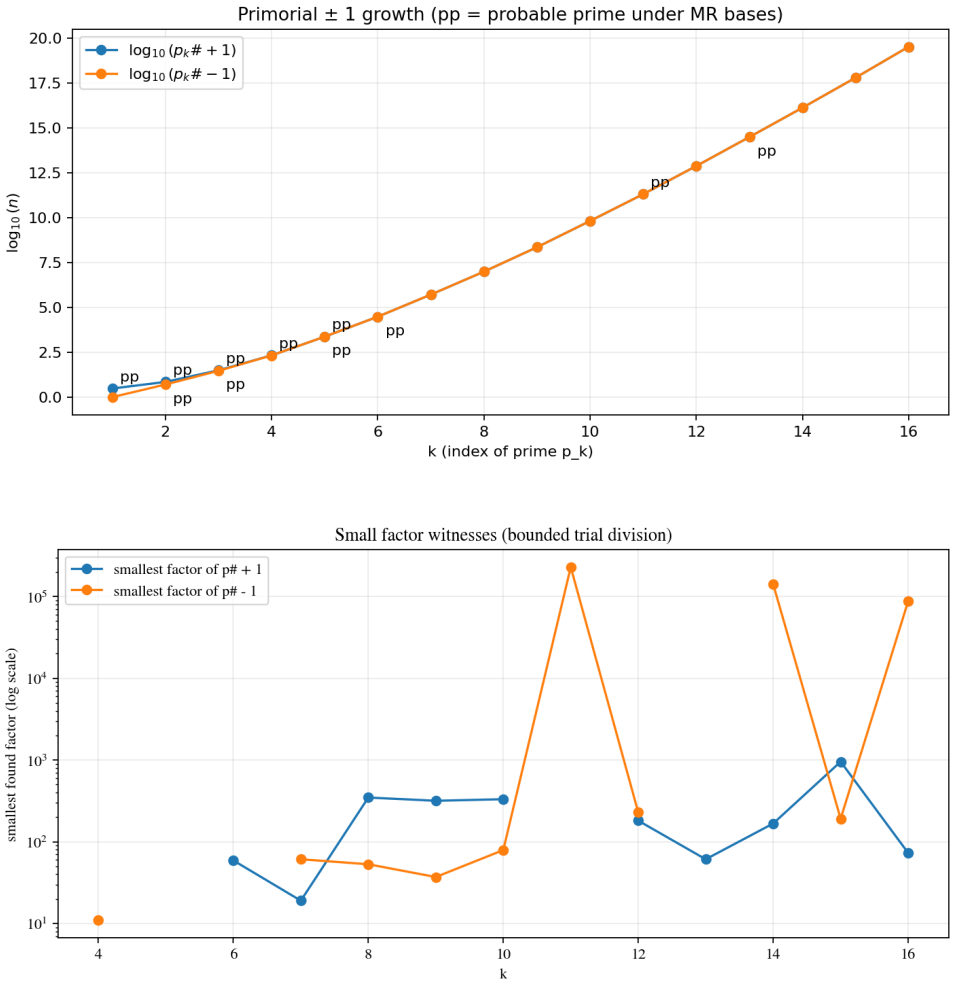
Interpreting the results

- Hits at 1093 and 3511 confirm the implementation.
- The quotient scatter is purely exploratory; it's a compact way to see "how far" a prime is from the Wieferich condition.
- Increasing the bound quickly becomes compute-heavy (but remains feasible for moderate bounds in pure Python).

References

See Wieferich [Wie09], The OEIS Foundation Inc. [TheOFInc25b], Wikipedia contributors [Wikipedia-contributors25k].

3.3.50 E050: Primorials and Euclid numbers: $p\# \pm 1$ are usually composite



Tags: `number-theory`, `counterexample-search`, `quantitative-exploration`, `visualization`, `primorial` See: *Valid Tags*.

Highlights

- Computes primorials $p_k\# = \prod_{i \le k} p_i$ and Euclid-style numbers $p_k\# \pm 1$.
- Uses a probable-prime test to show "often composite" behavior early.
- Finds small factor witnesses via bounded trial division for many early k.

Goal

Demonstrate that Euclid-style numbers are coprime to small primes but are not reliably prime, and produce explicit factor witnesses.

Background (quick refresher)

- *Primorials*
- *Prime numbers refresher*

Research question

For small k, how often are $p_k\# \pm 1$ prime (or probable prime), and how easy is it to find a small factor when they are composite?

Experiment design

- Compute primorials for $k \leq k_{max}$.
- Test $p_k\# \pm 1$ with Miller–Rabin (probable prime).
- If composite under MR, try to find a small trial-division factor as a concrete witness.
- Plot $\log_{10}(p_k\# \pm 1)$ and smallest factor witnesses (log scale).

Reproducibility

- `params.json` records the run settings.
- `report.md` summarizes the key findings.
- `figures/*.png` contains the plots for the run.

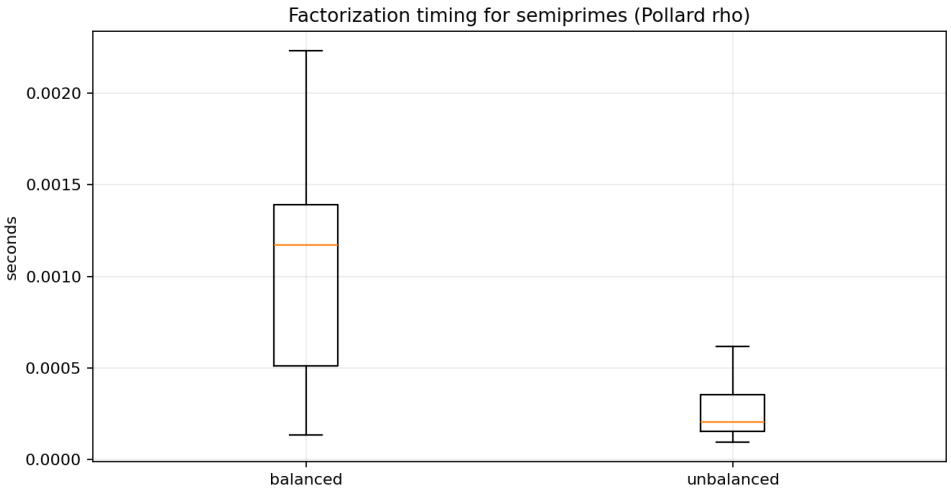
Interpreting the results

- "Probable prime" is not a proof; it's a fast filter for this exploratory experiment.
- Factor witnesses come from bounded trial division: not finding a factor is expected for some k.
- You should see early composite examples such as $p_6\# + 1 = 30031$ (with a small factor) under default bounds.

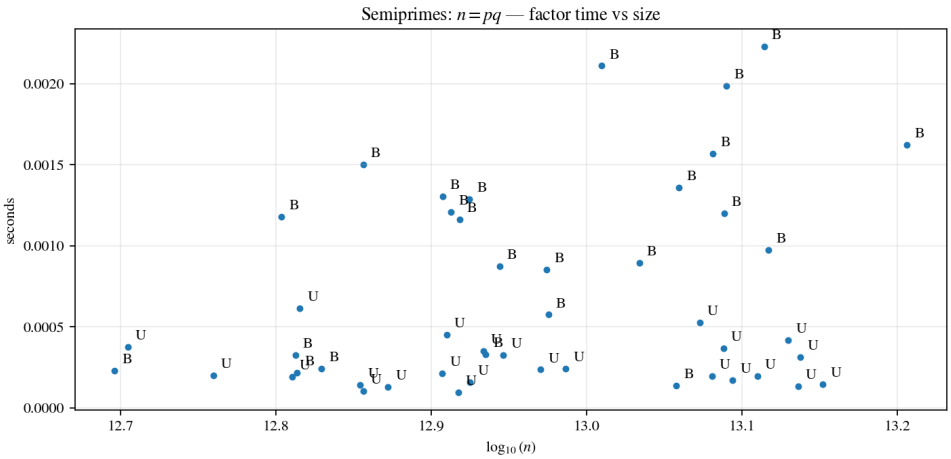
References

See Hardy and Wright [HW08a], The OEIS Foundation Inc. [TheOFInc25d], Prime Pages (UTM) [PrimePUTM25b], Wikipedia contributors [Wikipediacontributors25g].

3.3.51 E051: Semiprimes: balanced vs unbalanced factorization timing



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `factorization`, `semiprime` See: *Valid Tags*.



Highlights

- Generates small semiprimes $n = pq$ in balanced and unbalanced regimes.
- Factors them using trial division + Pollard's rho and records timings.
- Visualizes time distributions to make "balanced is harder" tangible.

Goal

Compare how factorization difficulty changes when one prime factor is much smaller than the other.

Background (quick refresher)

- *Semiprimes*
- *Prime numbers refresher*

Research question

For semiprimes of similar overall size, how does Pollard-rho timing differ between balanced ($p \approx q$) and unbalanced ($p \ll q$) cases?

Experiment design

- Generate balanced semiprimes with roughly equal factor bit lengths.
- Generate unbalanced semiprimes with a fixed small factor size.
- Factor each using trial division pre-pass + Pollard's rho, measuring wall-clock time.
- Plot boxplots and a size-vs-time scatter.

Reproducibility

- `params.json` records the run settings.
- `report.md` summarizes the key findings.
- `figures/*.png` contains the plots for the run.

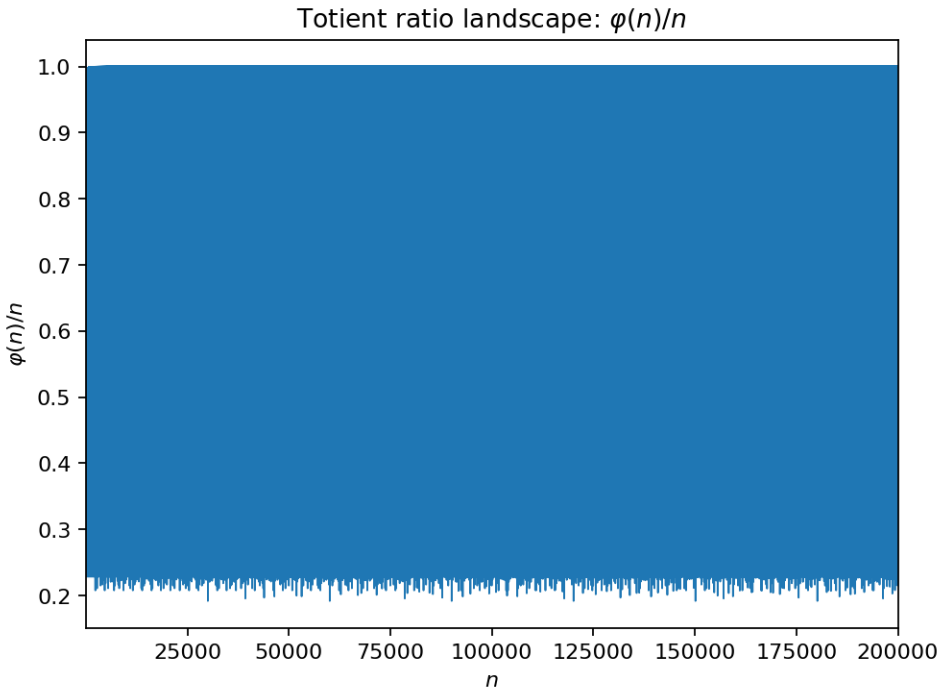
Interpreting the results

- Timings are noisy: random instance structure matters a lot at this size range.
- The trend should still show unbalanced instances often factoring faster (small factor is easier to find).
- This is an educational-scale experiment, not a cryptographic benchmark.

References

See Rivest *et al.* [RSA78], The OEIS Foundation Inc. [TheOFInc25c].

3.3.52 E052: Totient ratio landscape



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `arithmetic-functions`, `totient`, `multiplicative` See: *Valid Tags*.

Highlights

- Plot $\varphi(n)/n$ for a range of n and annotate record lows.
- Relate deep dips to integers with many small prime factors (primorial-like structure).

  #### Goal

  Make the multiplicative structure of the totient function visible, especially the role of small primes.

  #### Background (quick refresher)

  – *Arithmetic functions refresher*
- *Euler's totient function \varphi(n) refresher*
- *Primorials*

  #### Research question

  Which n minimize $\varphi(n)/n$ up to a bound, and how do they relate to primorial products?

  #### Method

  – Compute $\varphi(n)$ for $n \leq N$ (sieve / SPF-based).
- Plot the ratio $\varphi(n)/n$ and track record minima.
- Optionally compare record minima to primorials and nearby multiples.

How to run

- `make run EXP=e052`
- `uv run python -m mathxlab.experiments.e052`

Outputs

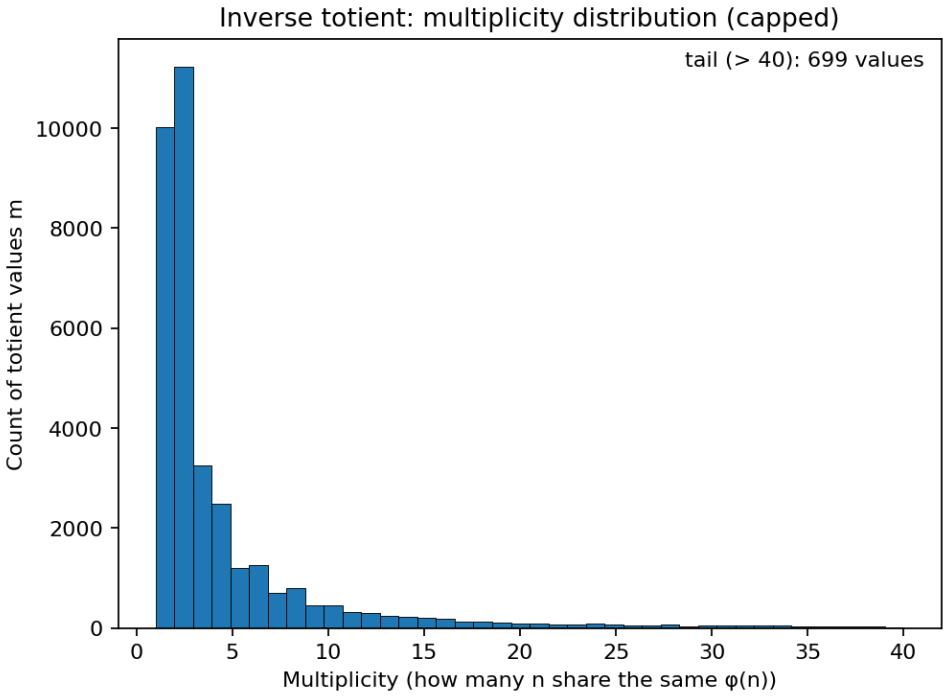
This experiment follows the standard output contract:

- `out/e052/figures/` — generated figures (PNG)
- `out/e052/report.md` — short narrative report
- `out/e052/manifest.json` — snapshot metadata for the gallery

References

See Apostol [Apo76], Tenenbaum [Ten15], Niven *et al.* [NZM91].

3.3.53 E053: Inverse totient multiplicities



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `arithmetic-functions`, `totient`, `search` See: *Valid Tags*.

Highlights

- Compute $\varphi(n)$ for $n \leq N$ and count how many preimages each value has.
- Plot the histogram of multiplicities and list the top collisions.

Goal

Empirically study the many-to-one nature of the totient function.

Background (quick refresher)

- *Euler's totient function \varphi(n) refresher*
- *Arithmetic functions refresher*

Research question

For $n \le N$, how large can the multiplicity of a totient value get, and what do the maximizers look like?

Method

- Compute $\varphi(n)$ on a range and build a frequency table for values.
- Visualize multiplicities (histogram) and inspect the most frequent values.

How to run

- `make run EXP=e053`
- `uv run python -m mathxlab.experiments.e053`

Outputs

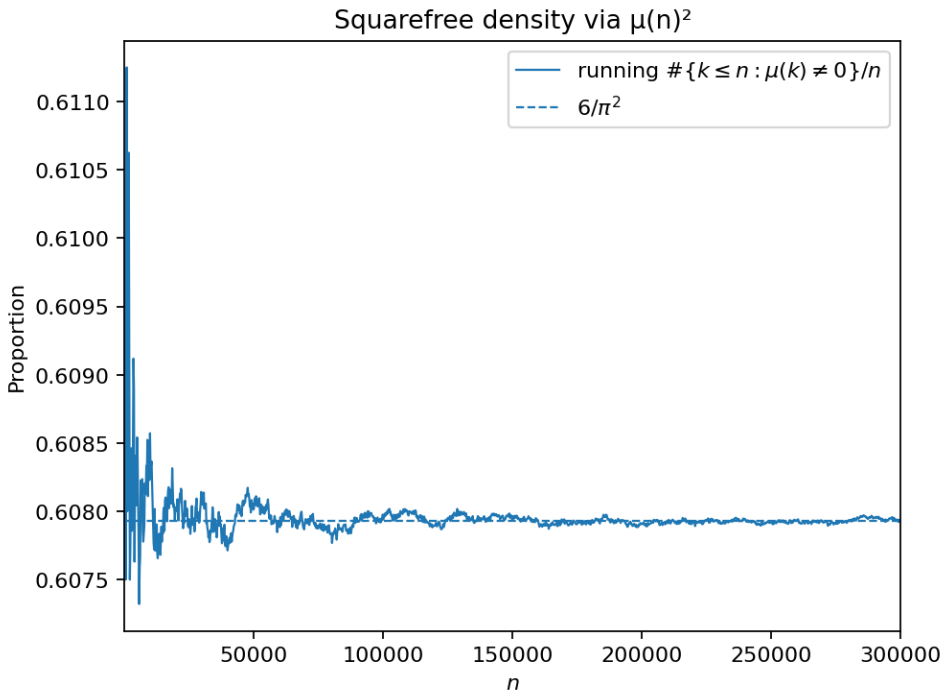
This experiment follows the standard output contract:

- `out/e053/figures/` — generated figures (PNG)
- `out/e053/report.md` — short narrative report
- `out/e053/manifest.json` — snapshot metadata for the gallery

References

See Apostol [Apo76], Niven *et al.* [NZM91].

3.3.54 E054: Squarefree density via Möbius



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `arithmetic-functions`, `mobius`, `summatory` See: *Valid Tags*.

Highlights

- Use $\mu(n)^2$ as an indicator for squarefree integers.
- Plot the running density and compare to $6/\pi^2$.

Goal

Show how an arithmetic function encodes a classic density result.

Background (quick refresher)

- *Möbius function \mu(n) and Mertens function M(x) refresher*
- *Arithmetic functions refresher*

Research question

How quickly does the empirical density of squarefree numbers approach $6/\pi^2$?

Method

- Compute $\mu(n)$ up to N and accumulate $\sum_{n \le x} \mu(n)^2$.
- Plot the ratio $\frac{1}{x} \sum_{n \le x} \mu(n)^2$ with a reference line at $6/\pi^2$.

How to run

- `make run EXP=e054`
- `uv run python -m mathxlab.experiments.e054`

Outputs

This experiment follows the standard output contract:

- `out/e054/figures/` — generated figures (PNG)
- `out/e054/report.md` — short narrative report
- `out/e054/manifest.json` — snapshot metadata for the gallery

References

See Apostol [Apo76], Tenenbaum [Ten15].

3.3.55 E055: Mertens function walk

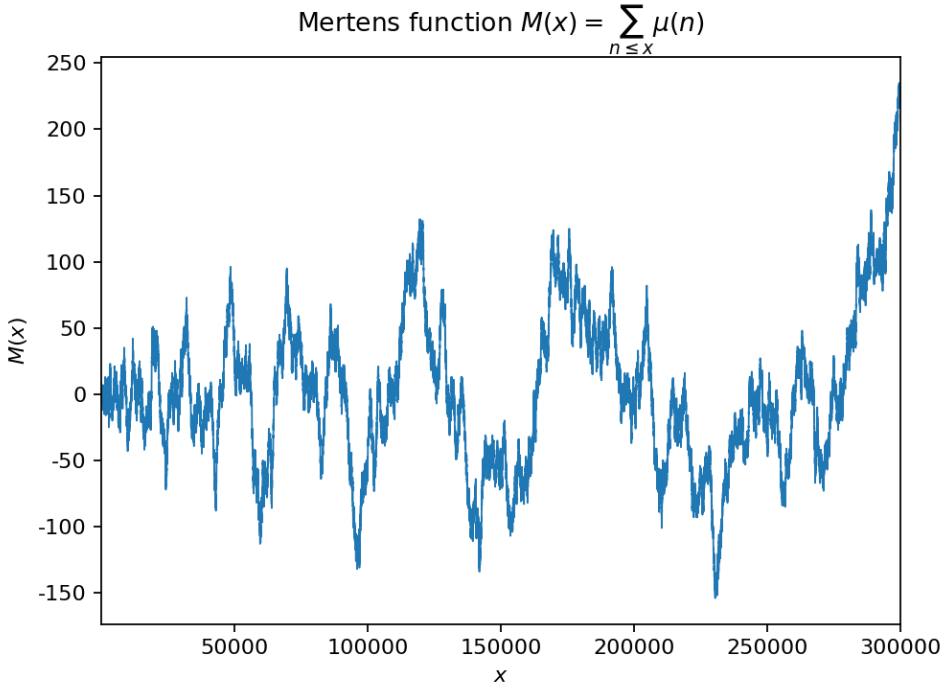
Tags: `number-theory`, `quantitative-exploration`, `visualization`, `arithmetic-functions`, `mobius`, `summatory`, `model-checking` See: *Valid Tags*.

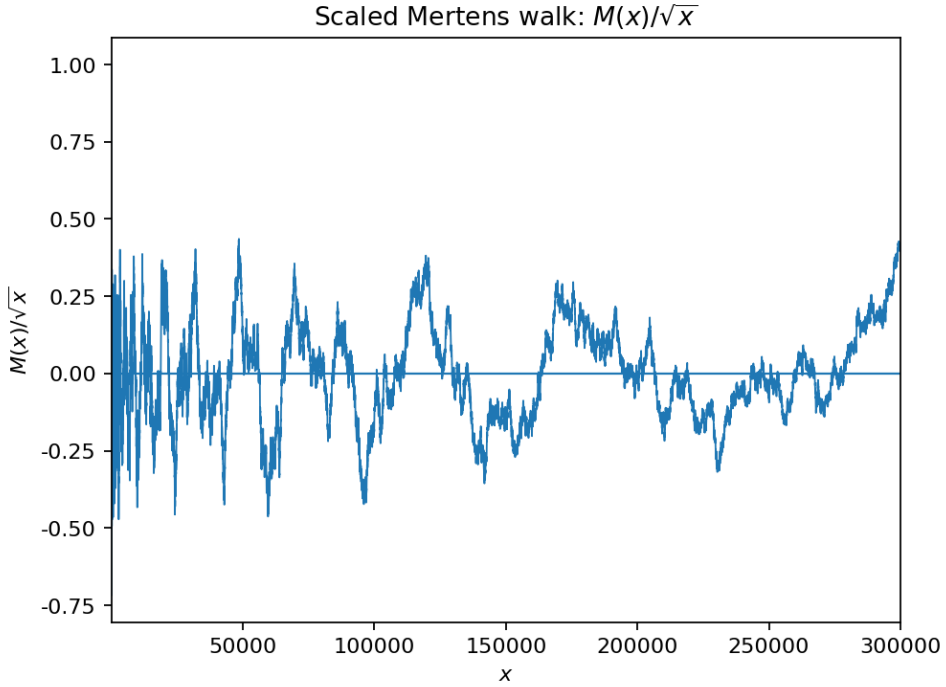
Highlights

- Plot $M(x)$ and scaled variants such as $M(x)/\sqrt{x}$.
- Compare qualitative behavior to a simple random-walk heuristic (without overclaiming).

Goal

Build intuition for summatory Möbius behavior and typical fluctuation scales.


Mertens function $M(x) = \sum_{n \le x} \mu(n)$
250
200
150
100
50
0
-50
-100
-150
$M(x)$
50000
100000
150000
200000
250000
300000
x


Scaled Mertens walk: $M(x)/\sqrt{x}$
1.00
0.75
0.50
0.25
0.00
-0.25
-0.50
-0.75
$M(x)/\sqrt{x}$
50000
100000
150000
200000
250000
300000
x

Background (quick refresher)

- *Möbius function \mu(n) and Mertens function M(x) refresher*
- *Average orders and the Erdős–Kac viewpoint*

Research question

What does $M(x)/\sqrt{x}$ look like numerically over moderate ranges?

Method

- Compute $\mu(n)$ up to N and cumulative sums $M(x)$.
- Plot $M(x)$ and $M(x)/\sqrt{x}$; optionally add moving-window statistics.

How to run

- `make run EXP=e055`
- `uv run python -m mathxlab.experiments.e055`

Outputs

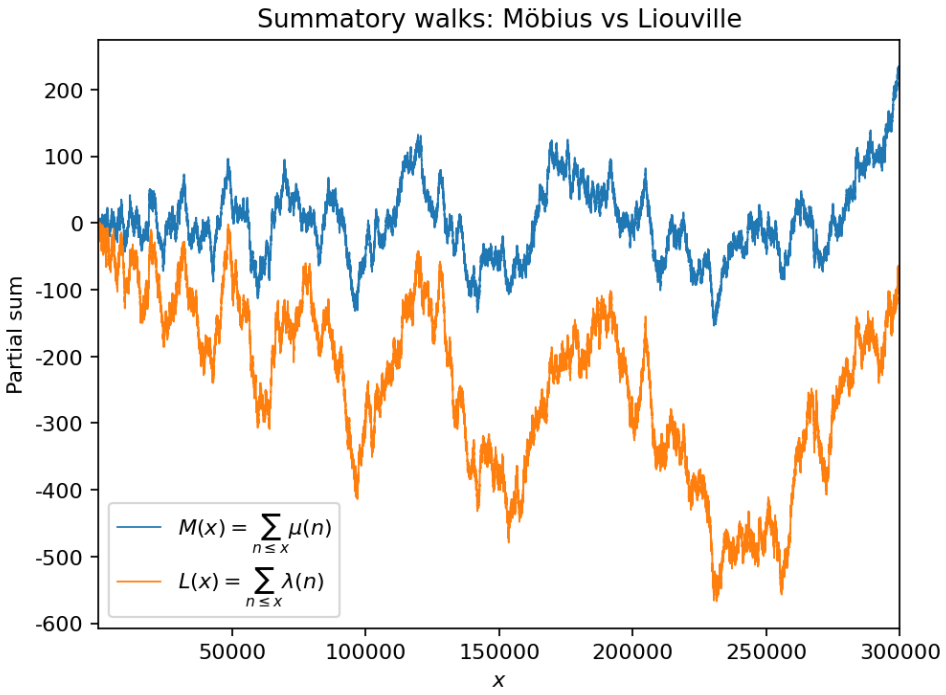
This experiment follows the standard output contract:

- `out/e055/figures/` — generated figures (PNG)
- `out/e055/report.md` — short narrative report
- `out/e055/manifest.json` — snapshot metadata for the gallery

References

See Tenenbaum [Ten15], Odlyzko and te Riele [OtR85].

3.3.56 E056: Liouville vs Möbius walks



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `arithmetic-functions`, `summatory`, `liouville`, `mobius` See: *Valid Tags*.

Highlights

- Plot partial sums of $\lambda(n)$ and $\mu(n)$ side by side.
- Compare scaled versions to see how the inclusion/exclusion of squareful terms changes the walk.

Goal

Contrast two closely related $\pm 1/0$ sequences arising from prime factorizations.

Background (quick refresher)

- *Liouville function \lambda(n) refresher*
- *Möbius function \mu(n) and Mertens function M(x) refresher*
- *Prime-factor counting: \omega(n) and \Omega(n) refresher*

Research question

How do the fluctuations of $\sum_{n \le x} \lambda(n)$ compare to $\sum_{n \le x} \mu(n)$?

Method

- Compute $\Omega(n)$, then $\lambda(n) = (-1)^{\Omega(n)}$; compute $\mu(n)$.
- Plot partial sums and scaled partial sums (e.g., divide by $\sqrt{x}$).

How to run

- `make run EXP=e056`
- `uv run python -m mathxlab.experiments.e056`

Outputs

This experiment follows the standard output contract:

- `out/e056/figures/` — generated figures (PNG)
- `out/e056/report.md` — short narrative report
- `out/e056/manifest.json` — snapshot metadata for the gallery

References

See Tenenbaum [Ten15].

3.3.57 E057: Erdős–Kac in practice

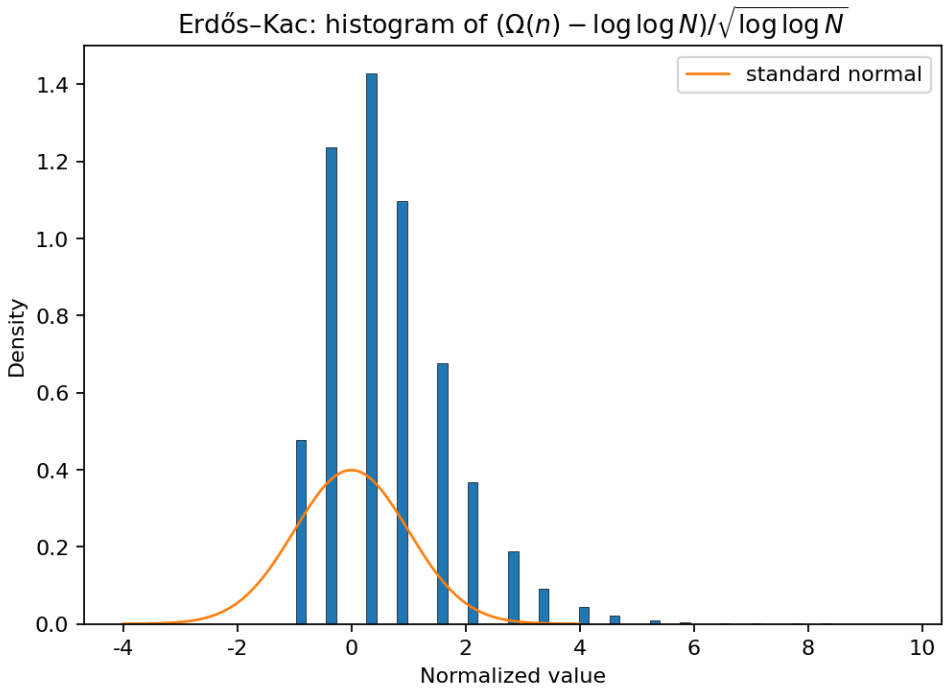
Tags: `number-theory`, `quantitative-exploration`, `visualization`, `arithmetic-functions`, `omega`, `heuristics` See: *Valid Tags*.

Highlights

- Compute $\Omega(n)$ for $n \le N$ and plot its histogram.
- Optionally normalize to compare with a Gaussian curve (Erdős–Kac heuristic).

Goal

Visualize the probabilistic number-theory flavor of prime-factor counts.



Background (quick refresher)

- *Prime-factor counting: \omega(n) and \Omega(n) refresher*
- *Average orders and the Erdős–Kac viewpoint*

Research question

How close does the distribution of $\Omega(n)$ (after normalization) look to a normal curve for moderate N?

Method

- Compute $\Omega(n)$ using an SPF sieve.
- Plot histogram; optionally overlay a normal density with matching mean/variance approximations.

How to run

- `make run EXP=e057`
- `uv run python -m mathxlab.experiments.e057`

Outputs

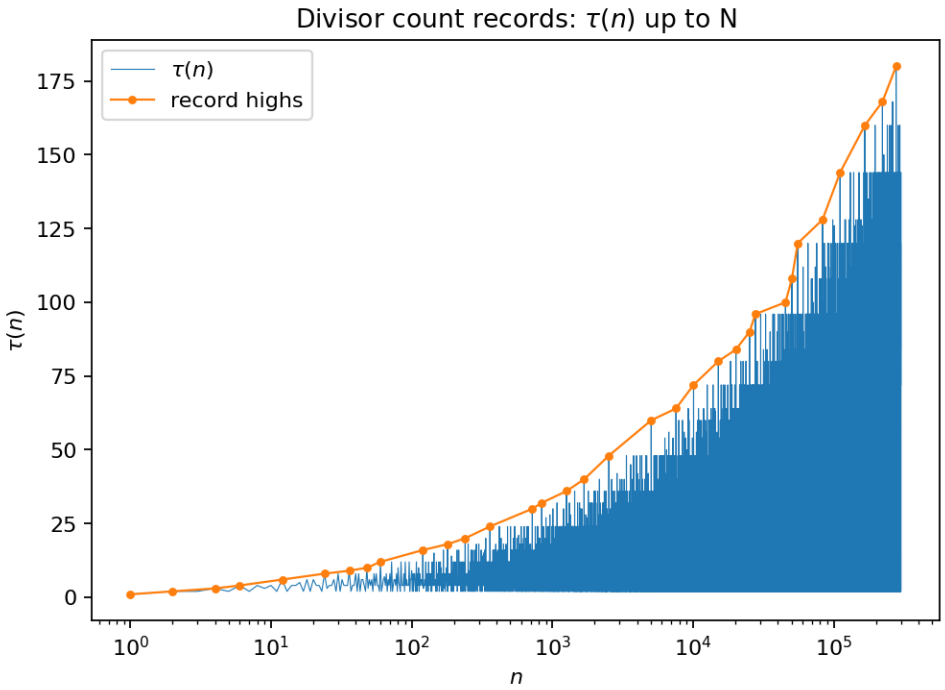
This experiment follows the standard output contract:

- `out/e057/figures/` — generated figures (PNG)
- `out/e057/report.md` — short narrative report
- `out/e057/manifest.json` — snapshot metadata for the gallery

References

See Erdős and Kac [ErdHosK40], Tenenbaum [Ten15].

3.3.58 E058: Divisor-count record highs



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `arithmetic-functions`, `divisor-function`, `bounds` See: *Valid Tags*.

Highlights

- Track record highs of $\tau(n)$ as n grows.
- Inspect factorizations of record-holders (many small primes with decreasing exponents).

Goal

Make the record phenomenon behind highly composite numbers tangible.

Background (quick refresher)

- *Divisor functions d(n) and \sigma_k(n) refresher*
- *Arithmetic functions refresher*

Research question

How do record values of $\tau(n)$ grow, and what structure do record-holders share?

Method

- Compute $\tau(n)$ up to N and track record positions.
- Plot record curve; list factorizations of top records.

How to run

- `make run EXP=e058`
- `uv run python -m mathxlab.experiments.e058`

Outputs

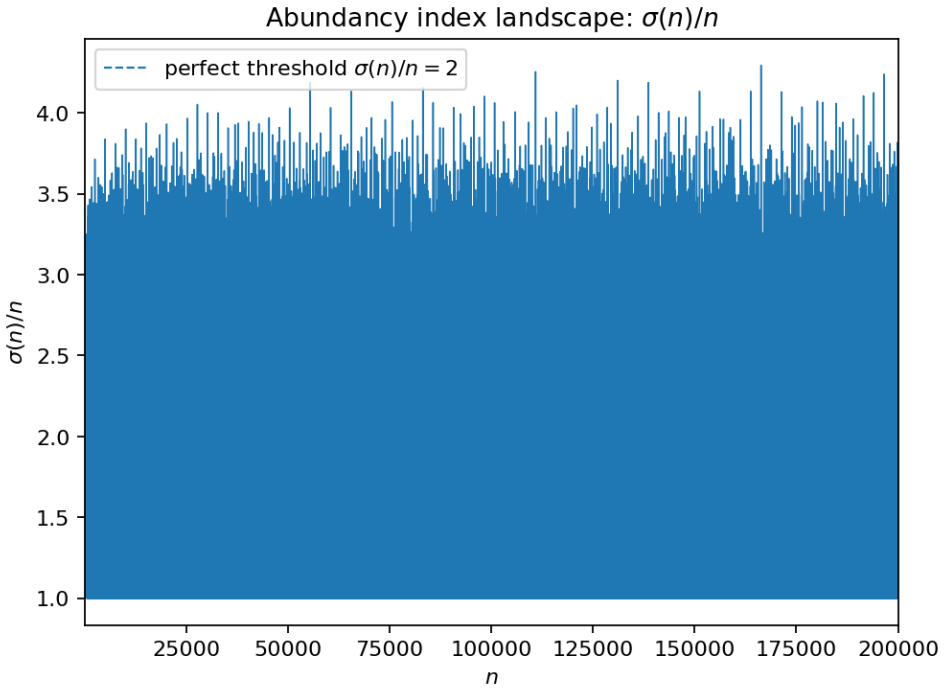
This experiment follows the standard output contract:

- `out/e058/figures/` — generated figures (PNG)
- `out/e058/report.md` — short narrative report
- `out/e058/manifest.json` — snapshot metadata for the gallery

References

See Ramanujan [Ram15], Tenenbaum [Ten15].

3.3.59 E059: Abundancy index landscape



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `arithmetic-functions`, `sigma`, `divisor-function`, `perfect` See: *Valid Tags*.

Highlights

- Plot $\sigma(n)/n$ and mark the threshold 2 (perfect numbers).
- Highlight spikes caused by many small prime factors.

Goal

Connect a divisor-sum function to the perfect/abundant classification visually.

Background (quick refresher)

- *Divisor functions d(n) and \sigma_k(n) refresher*
- *Perfect numbers refresher*

Research question

What shapes and spikes appear in $\sigma(n)/n$ over moderate ranges, and where do perfect numbers sit?

Method

- Compute $\sigma(n)$ up to N using factorization via SPF.
- Plot $\sigma(n)/n$ and annotate known perfect numbers in range.

How to run

- `make run EXP=e059`
- `uv run python -m mathxlab.experiments.e059`

Outputs

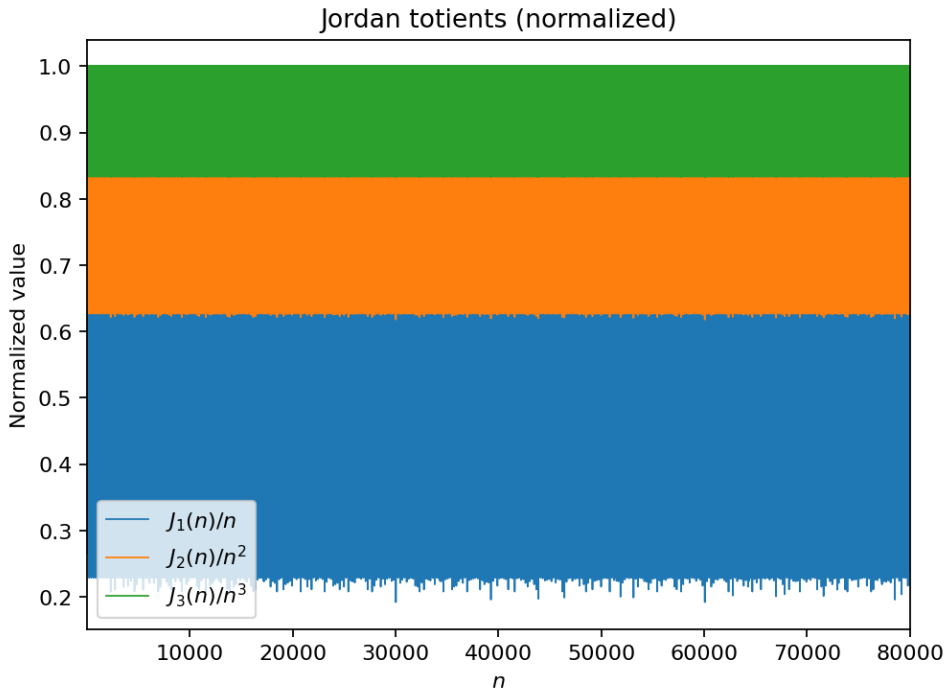
This experiment follows the standard output contract:

- `out/e059/figures/` — generated figures (PNG)
- `out/e059/report.md` — short narrative report
- `out/e059/manifest.json` — snapshot metadata for the gallery

References

See Apostol [Apo76], Niven *et al.* [NZM91].

3.3.60 E060: Jordan totients



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `arithmetic-functions`, `totient`, `multiplicative` See: *Valid Tags*.

Highlights

- Compute $J_k(n)$ for k=1,2,3 and plot $J_k(n)/n^k$.
- Compare how the small-prime structure changes as k increases.

Goal

Show a clean generalization of (n) and how normalization reveals multiplicative structure.

Background (quick refresher)

- *Jordan totient J_k(n) refresher*
- *Euler's totient function \varphi(n) refresher*

Research question

How do the normalized Jordan totients $J_k(n)/n^k$ behave for k=1..3?

Method

- Compute $J_k(n) = n^k \prod_{p|n}(1 - p^{-k})$ via prime factors.
- Plot normalized values for k=1..3.

How to run

- `make run EXP=e060`
- `uv run python -m mathxlab.experiments.e060`

Outputs

This experiment follows the standard output contract:

- `out/e060/figures/` — generated figures (PNG)
- `out/e060/report.md` — short narrative report
- `out/e060/manifest.json` — snapshot metadata for the gallery

References

See Apostol [Apo76].

3.3.61 E061: Chebyshev (x) and prime powers

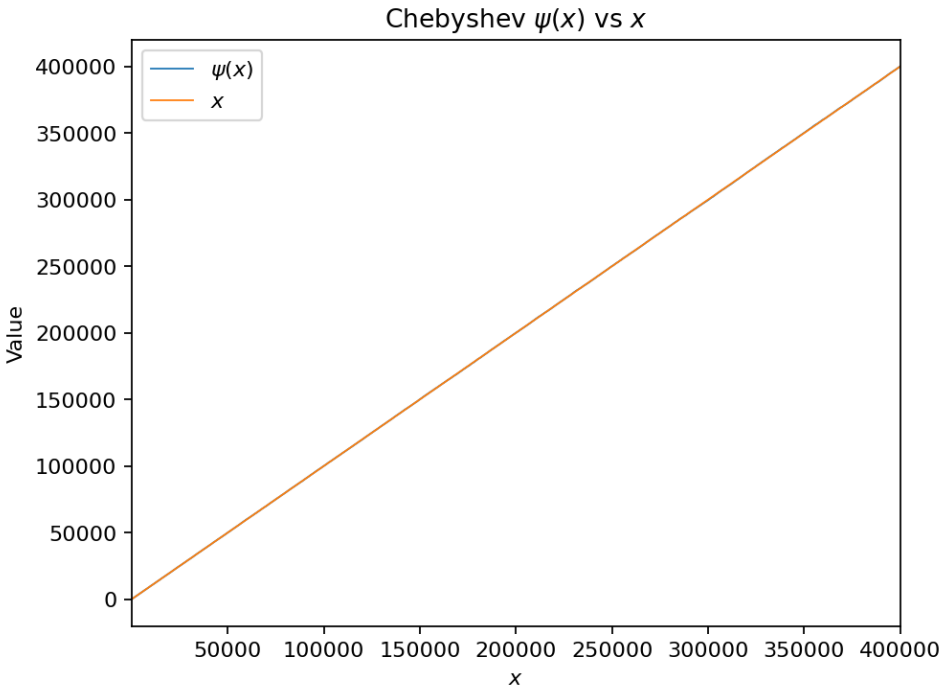
Tags: `number-theory`, `quantitative-exploration`, `visualization`, `arithmetic-functions`, `mangoldt`, `pnt`, `summatory` See: *Valid Tags*.

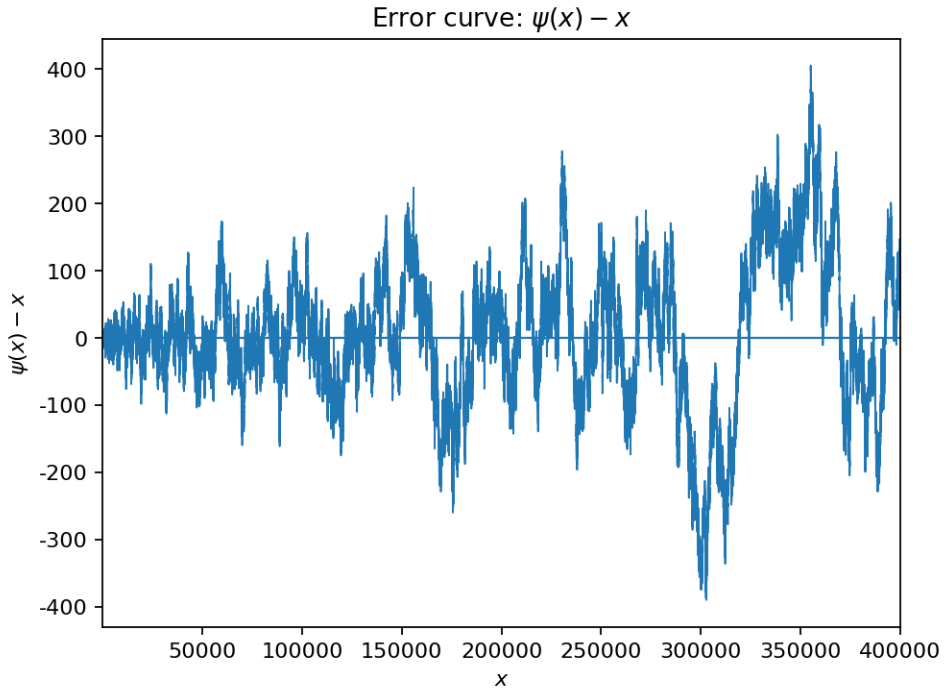
Highlights

- Plot $\psi(x)$ and the error $\psi(x) - x$.
- Make the role of prime powers visible (jumps at p^k).

Goal

Build intuition for Chebyshev functions as "smoothed" prime counters.


Chebyshev $\psi(x)$ vs x
$\psi(x)$
x
Value
400000
350000
300000
250000
200000
150000
100000
50000
0
50000
100000
150000
200000
250000
300000
350000
400000
x


Error curve: $\psi(x) - x$
$\psi(x) - x$
400
300
200
100
0
-100
-200
-300
-400
50000
100000
150000
200000
250000
300000
350000
400000
x

Background (quick refresher)

- *von Mangoldt \Lambda(n) and Chebyshev functions refresher*
- *Prime numbers refresher*

Research question

Over a moderate range, what does $\psi(x) - x$ look like and where do the main jumps occur?

Method

- Compute Λ(n) (log p on prime powers) and cumulative (x).
- Plot (x) and (x)-x; optionally annotate largest jumps.

How to run

- `make run EXP=e061`
- `uv run python -m mathxlab.experiments.e061`

Outputs

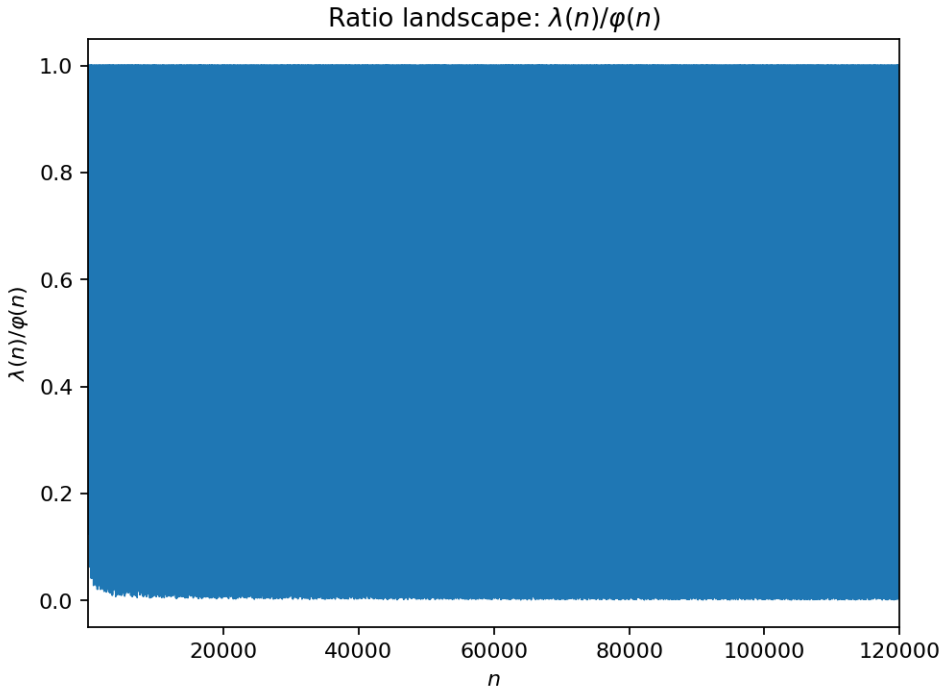
This experiment follows the standard output contract:

- `out/e061/figures/` — generated figures (PNG)
- `out/e061/report.md` — short narrative report
- `out/e061/manifest.json` — snapshot metadata for the gallery

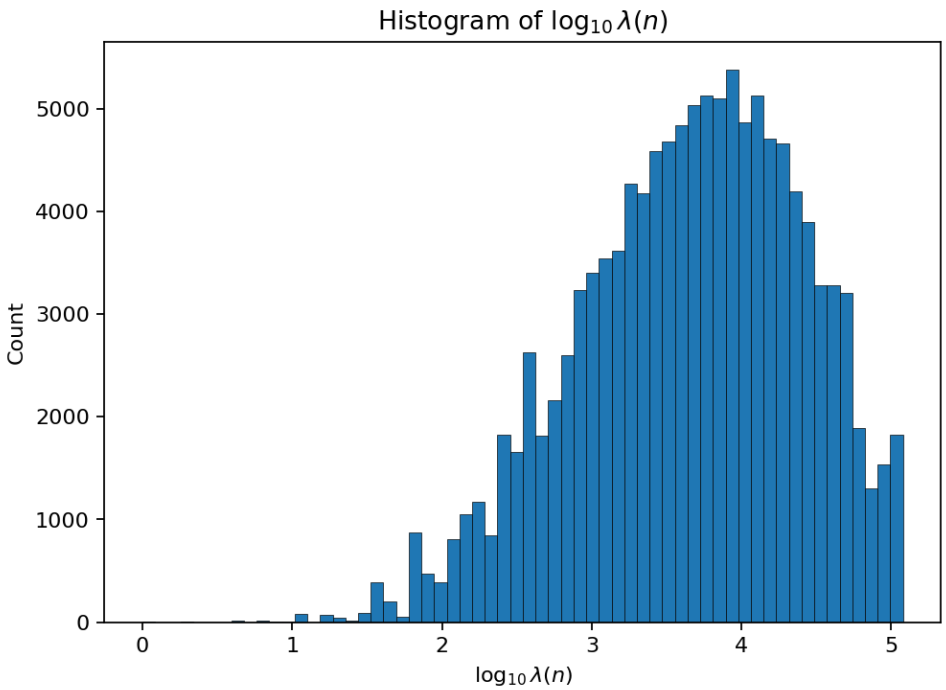
References

See Montgomery and Vaughan [MV06], Apostol [Apo76].

3.3.62 E062: Carmichael (n) vs (n)



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `arithmetic-functions`, `carmichael-lambda`, `totient` See: *Valid Tags*.



Highlights

- Plot or histogram the ratio $\varphi(n)/\lambda(n)$.
- Show how 2-adic structure affects (n) (notably for powers of 2).

Goal

See how the exponent of the multiplicative group mod n differs from its size.

Background (quick refresher)

- *Carmichael's \lambda(n) function refresher*
- *Euler's totient function \varphi(n) refresher*

Research question

How large can $\varphi(n)/\lambda(n)$ get for $n \leq N$, and what kinds of n cause large gaps?

Method

- Compute (n) and (n) from prime-power formulas and lcm composition.
- Visualize ratios (histograms, quantiles) and list top outliers.

How to run

- `make run EXP=e062`
- `uv run python -m mathxlab.experiments.e062`

Outputs

This experiment follows the standard output contract:

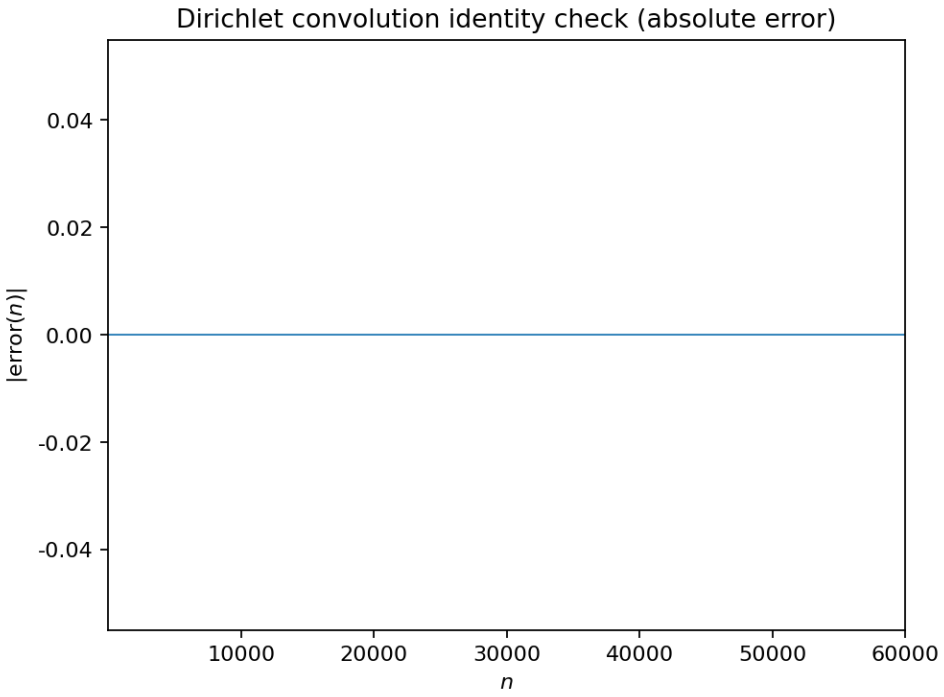
- `out/e062/figures/` — generated figures (PNG)
- `out/e062/report.md` — short narrative report

- `out/e062/manifest.json` — snapshot metadata for the gallery

References

See Erdős *et al.* [ErdHosPS91], Montgomery and Vaughan [MV06].

3.3.63 E063: Dirichlet convolution playground



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `arithmetic-functions`, `dirichlet-convolution`, `model-checking` See: *Valid Tags*.

Highlights

- Compute Dirichlet convolutions numerically on a finite range.
- Check exact identities and report any mismatches (should be zero).

Goal

Turn algebraic identities of arithmetic functions into concrete computed checks.

Background (quick refresher)

- *Dirichlet convolution refresher*
- *Arithmetic functions refresher*
- *Euler's totient function \varphi(n) refresher*

Research question

Can we numerically verify standard Dirichlet-convolution identities up to a bound without mistakes?

Method

- Implement divisor-sum convolution on [1..N].
- Compute *1 and* id and compare to and respectively.

How to run

- `make run EXP=e063`
- `uv run python -m mathxlab.experiments.e063`

Outputs

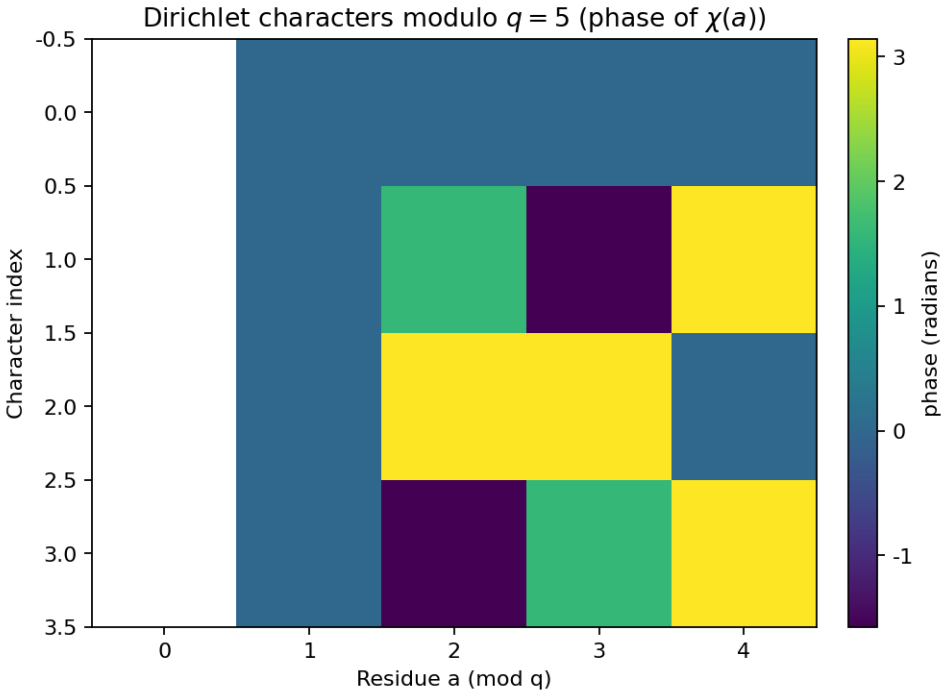
This experiment follows the standard output contract:

- `out/e063/figures/` — generated figures (PNG)
- `out/e063/report.md` — short narrative report
- `out/e063/manifest.json` — snapshot metadata for the gallery

References

See Apostol [Apo76].

3.3.64 E064: Dirichlet character tables (phase view).



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `dirichlet-characters`

Highlights

- Constructs full character tables for a small modulus q.
- Visualizes values as phases / real parts to spot structure and symmetries.
- Checks orthogonality and basic sanity constraints numerically.

What this experiment does

This experiment enumerates all Dirichlet characters modulo a small modulus q and visualizes the values (a) for residues a=0..q-1.

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e064/`:

- `figures/fig_01_character_phases.png`

How to run

```
make run EXP=e064
```

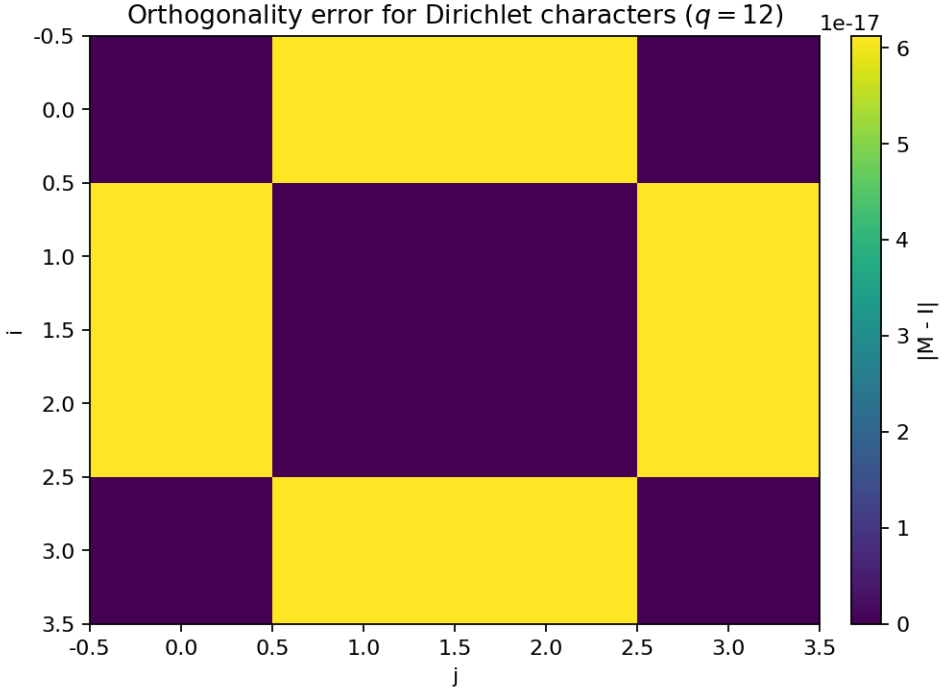
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Davenport [Dav00], Niven *et al.* [NZM91]

3.3.65 E065: Orthogonality matrix for Dirichlet characters.



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `dirichlet-characters`

Highlights

- Constructs full character tables for a small modulus q.
- Visualizes values as phases / real parts to spot structure and symmetries.
- Checks orthogonality and basic sanity constraints numerically.

What this experiment does

For characters _i modulo q, orthogonality says:

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e065/`:

- `figures/fig_01_orthogonality_error.png`

How to run

```
make run EXP=e065
```

Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Davenport [Dav00], Niven *et al.* [NZM91]

3.3.66 E066: Character partial sums: cancellation profiles.

Tags: `number-theory`, `quantitative-exploration`, `visualization`, `dirichlet-characters`

Highlights

- Constructs full character tables for a small modulus q.
- Visualizes values as phases / real parts to spot structure and symmetries.
- Checks orthogonality and basic sanity constraints numerically.

What this experiment does

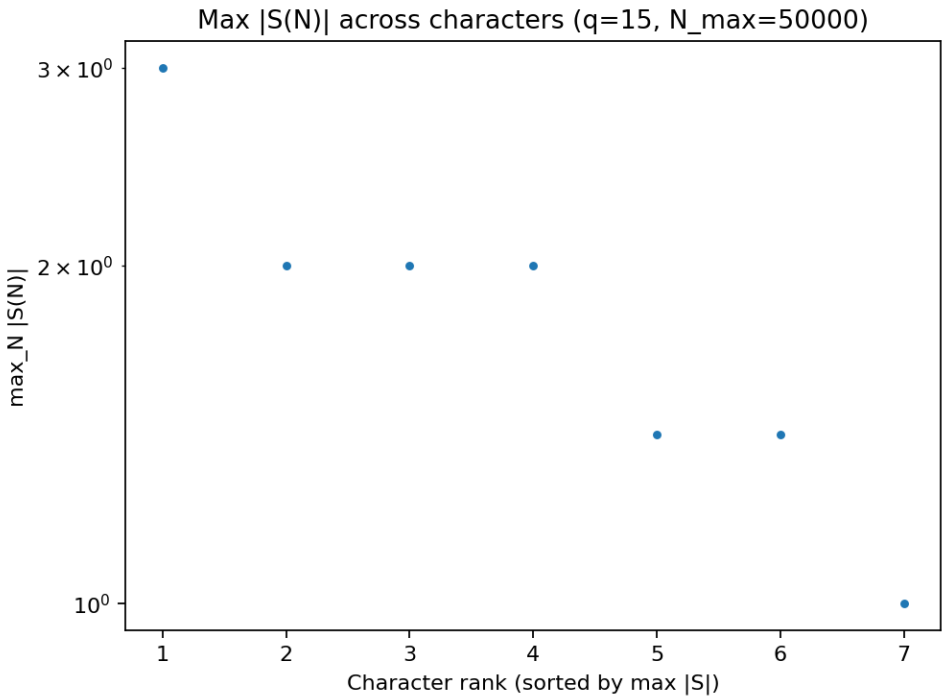
For a Dirichlet character modulo q, consider partial sums:

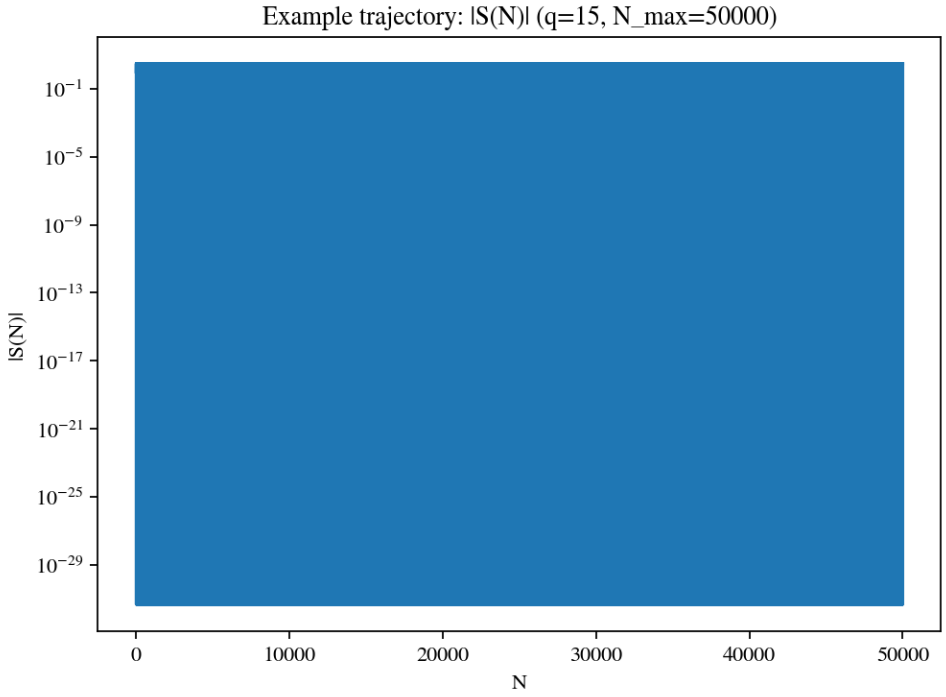
The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e066/`:

- `figures/fig_01_max_partial_sums.png`
- `figures/fig_02_example_partial_sum.png`


Max |S(N)| across characters (q=15, N_max=50000)
3 × 10^0
2 × 10^0
10^0
max_N |S(N)|
1
2
3
4
5
6
7
Character rank (sorted by max |S|)


Example trajectory: |S(N)| (q=15, N_max=50000)
10^-1
10^-5
10^-9
10^-13
10^-17
10^-21
10^-25
10^-29
|S(N)|
0
10000
20000
30000
40000
50000
N

How to run

```
make run EXP=e066
```

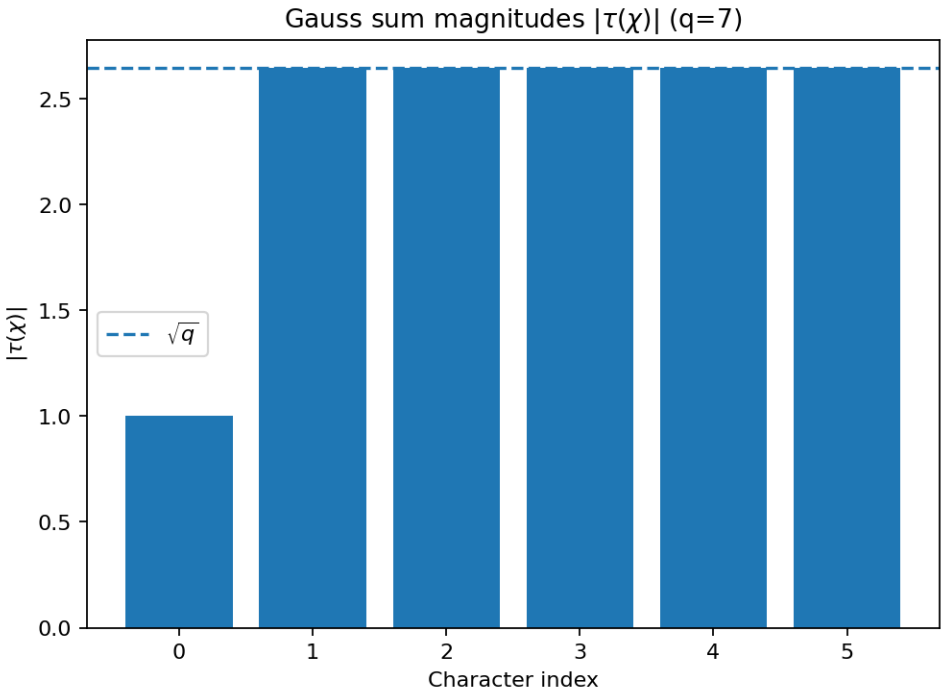
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Davenport [Dav00], Niven *et al.* [NZM91]

3.3.67 E067: Gauss sums: magnitude vs sqrt(q).



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `dirichlet-characters`

Highlights

- Constructs full character tables for a small modulus q.
- Visualizes values as phases / real parts to spot structure and symmetries.
- Checks orthogonality and basic sanity constraints numerically.

What this experiment does

For a Dirichlet character modulo q, define the Gauss sum

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e067/`:

- `figures/fig_01_gauss_sum_magnitudes.png`

How to run

```
make run EXP=e067
```

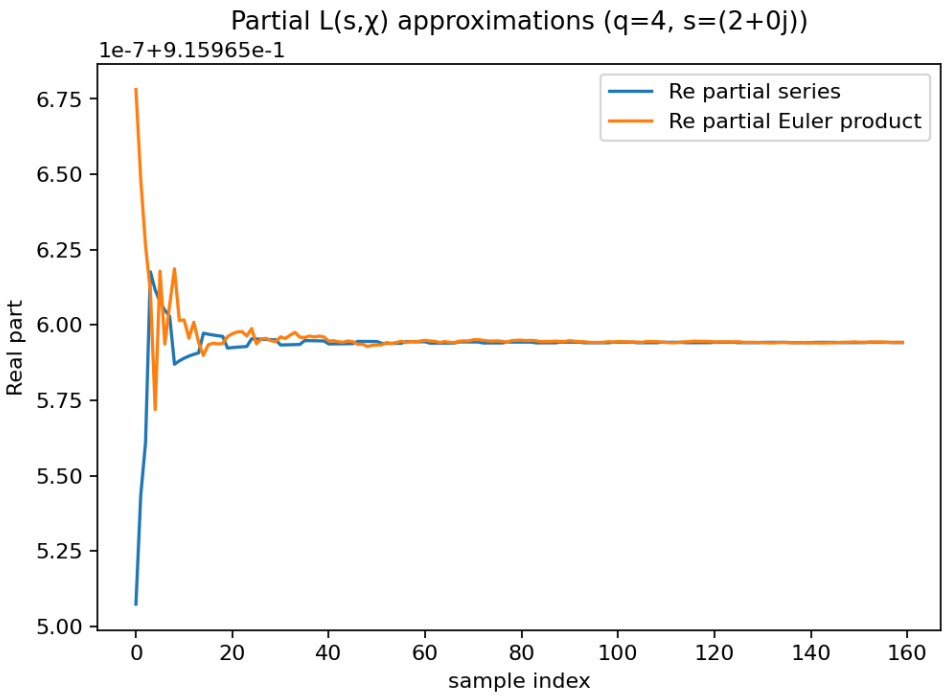
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Davenport [Dav00], Niven *et al.* [NZM91]

3.3.68 E068: Dirichlet L(s,): series vs Euler product (partial approximations).



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `dirichlet-characters`, `l-functions`

Highlights

- Computes truncated Dirichlet L-series for a nontrivial character.
- Compares series truncation against Euler-product truncation (where meaningful).
- Shows convergence behavior (fast for s>1, slow at s=1).

What this experiment does

For Re(s) > 1, Dirichlet L-functions admit both:

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e068/`:

- `figures/fig_01_series_vs_euler.png`

How to run

```
make run EXP=e068
```

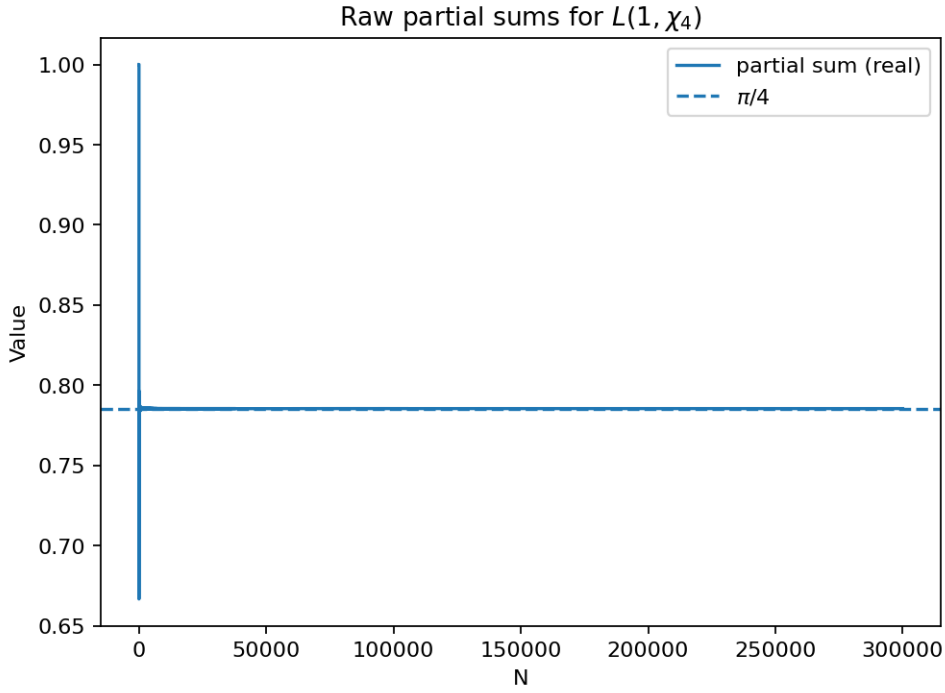
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

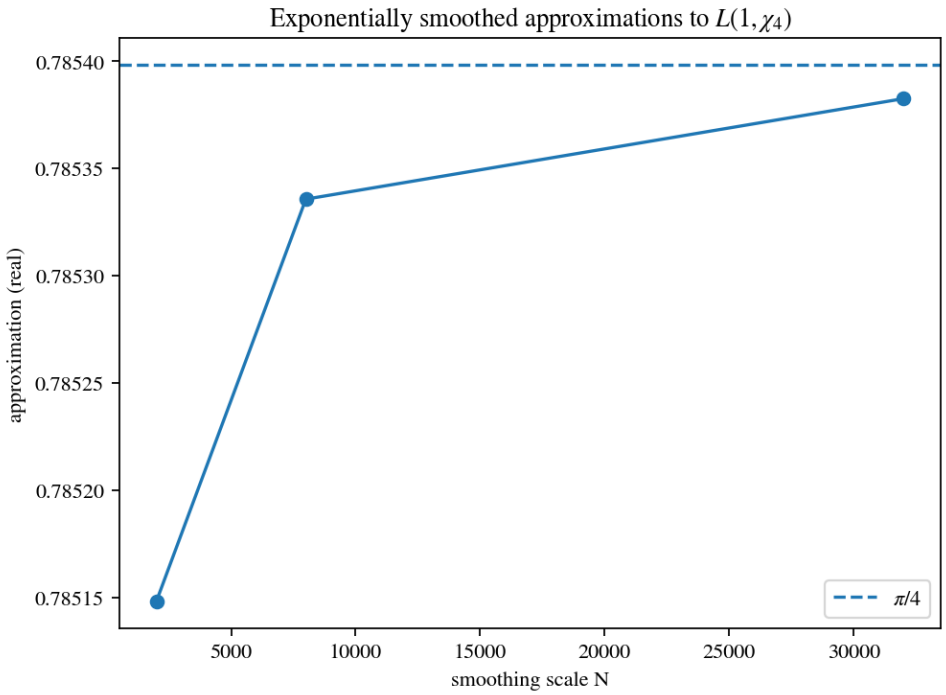
References

Apostol [Apo76], Montgomery and Vaughan [MV06]

3.3.69 E069: L(1,): slow convergence and smoothing.



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `dirichlet-characters`, `l-functions`



Highlights

- Computes truncated Dirichlet L-series for a nontrivial character.
- Compares series truncation against Euler-product truncation (where meaningful).
- Shows convergence behavior (fast for s>1, slow at s=1).

What this experiment does

At s=1, the Dirichlet series for L(1,) converges very slowly. For the nontrivial character modulo 4:

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e069/`:

- `figures/fig_01_l1_partial_sums.png`
- `figures/fig_02_l1_smoothed.png`

How to run

```
make run EXP=e069
```

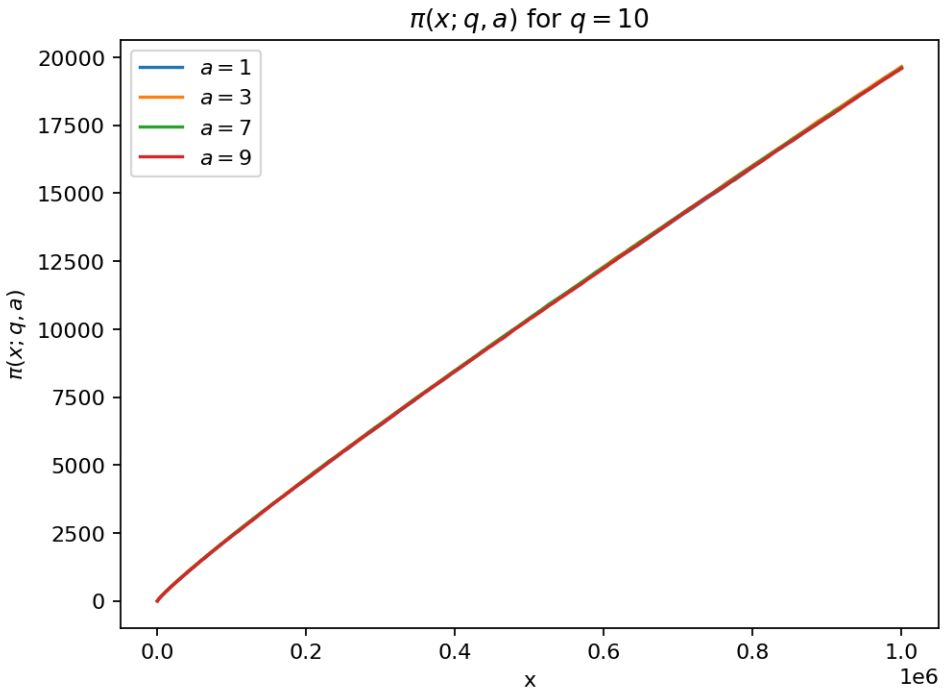
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Apostol [Apo76], Montgomery and Vaughan [MV06]

3.3.70 E070: Primes in residue classes: pi(x; q, a).



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `aps`

Highlights

- Counts primes in residue classes a mod q for several a.
- Compares empirical counts to the first-order prediction li(x)/ (q).
- Tracks a simple error proxy across x to visualize deviation patterns.

What this experiment does

This experiment counts primes in selected reduced residue classes modulo q:

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e070/`:

- `figures/fig_01_pi_x_q_a.png`

How to run

```
make run EXP=e070
```

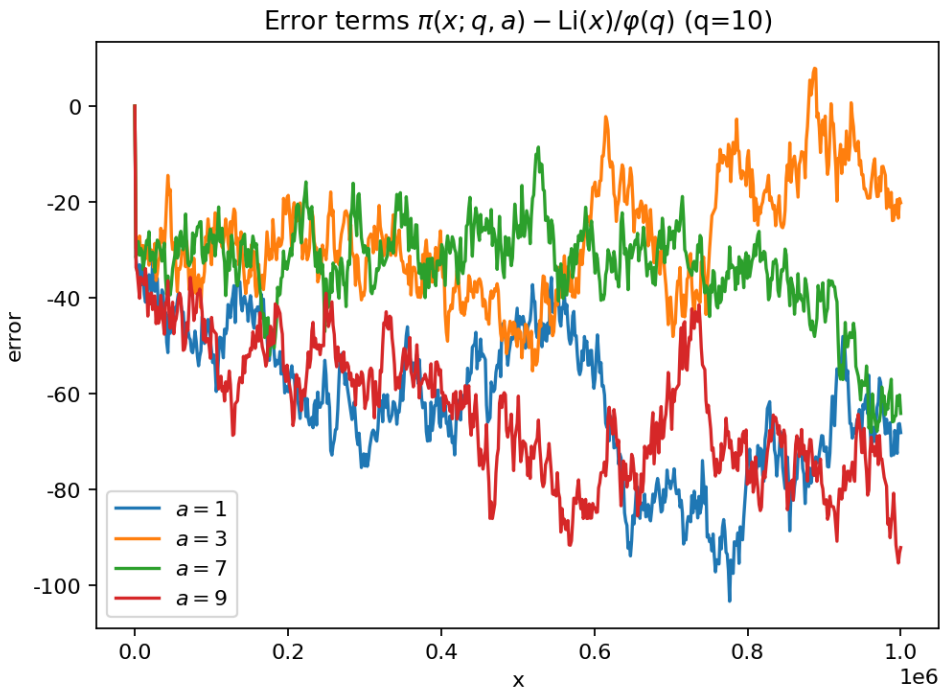
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Apostol [Apo76], Davenport [Dav00]

3.3.71 E071: PNT(AP) numerics: pi(x;q,a) - Li(x)/phi(q).



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `aps`

Highlights

- Counts primes in residue classes a mod q for several a.
- Compares empirical counts to the first-order prediction li(x)/ (q).
- Tracks a simple error proxy across x to visualize deviation patterns.

What this experiment does

The prime number theorem in arithmetic progressions suggests:

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e071/`:

- `figures/fig_01_error_terms.png`

How to run

```
make run EXP=e071
```

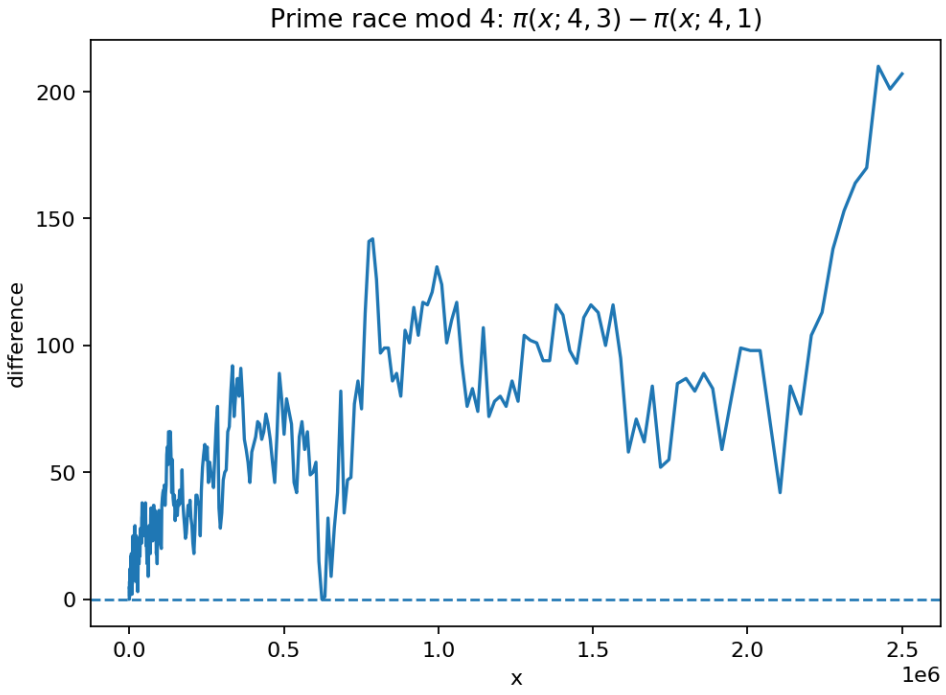
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Apostol [Apo76], Davenport [Dav00]

3.3.72 E072: Prime race mod 4: pi(x;4,3) vs pi(x;4,1).



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `prime-races`, `aps`

Highlights

- Computes prime-race differences (x;q,a) − (x;q,b) on a grid of x values.
- Visualizes lead changes and the size of fluctuations as x grows.
- Adds a derived statistic (normalization / -variant) to compare behaviors.

What this experiment does

A classical "prime race" compares how often one residue class leads another in prime counts. The most famous is mod 4:

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e072/`:

- `figures/fig_01_race_mod4_diff.png`

How to run

```
make run EXP=e072
```

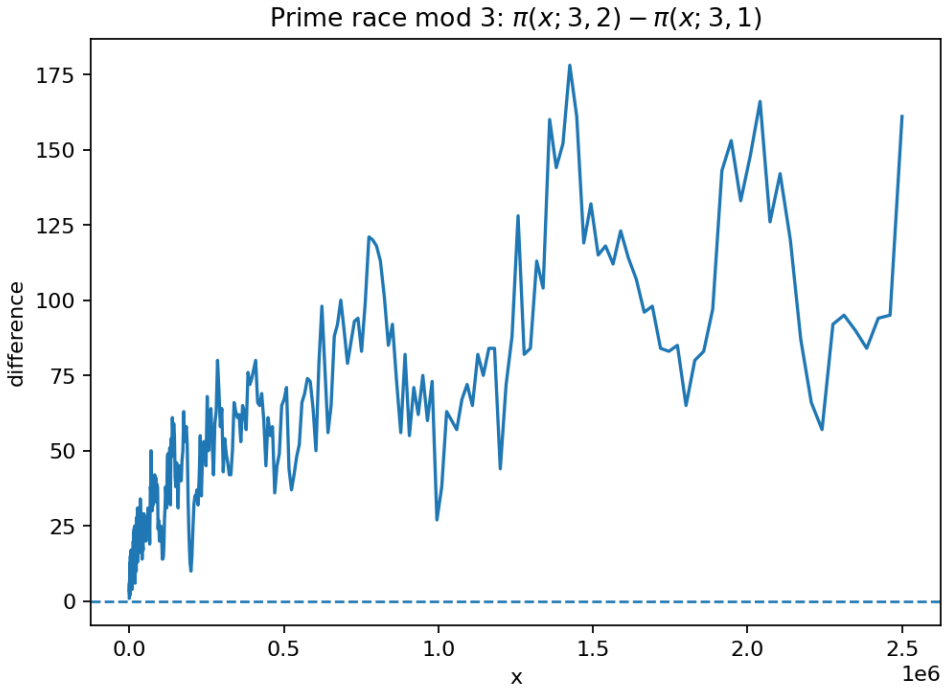
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Granville and Martin [GM06], Rubinstein and Sarnak [RS94]

3.3.73 E073: Prime race mod 3: pi(x;3,2) vs pi(x;3,1).



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `prime-races`, `aps`

Highlights

- Computes prime-race differences (x;q,a) − (x;q,b) on a grid of x values.
- Visualizes lead changes and the size of fluctuations as x grows.
- Adds a derived statistic (normalization / -variant) to compare behaviors.

What this experiment does

Another small prime race compares the two reduced residue classes modulo 3:

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e073/`:

- `figures/fig_01_race_mod3_diff.png`

How to run

```
make run EXP=e073
```

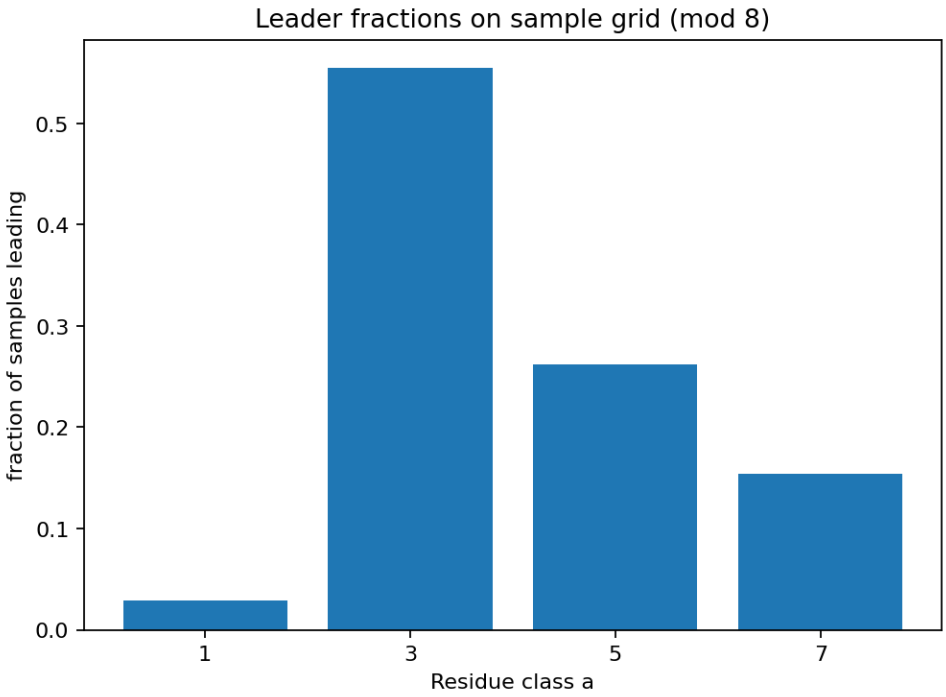
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

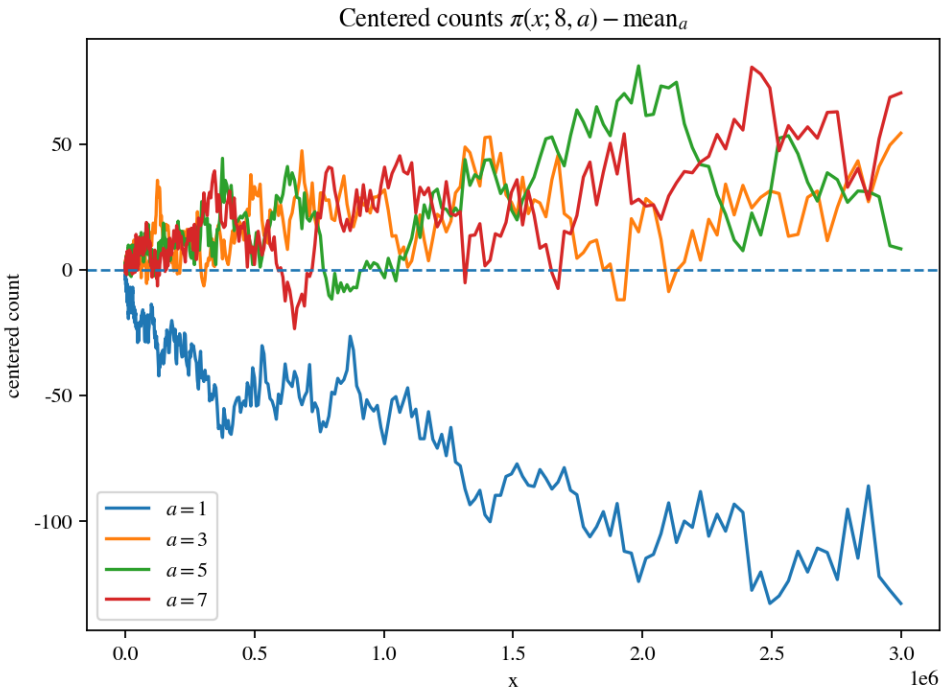
References

Granville and Martin [GM06], Rubinstein and Sarnak [RS94]

3.3.74 E074: Prime race mod 8: leaderboard among 1,3,5,7.



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `prime-races`, `aps`



Highlights

- Computes prime-race differences (x;q,a) − (x;q,b) on a grid of x values.
- Visualizes lead changes and the size of fluctuations as x grows.
- Adds a derived statistic (normalization / -variant) to compare behaviors.

What this experiment does

This experiment tracks four residue classes modulo 8:

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e074/`:

- `figures/fig_01_leader_fractions.png`
- `figures/fig_02_counts_minus_mean.png`

How to run

```
make run EXP=e074
```

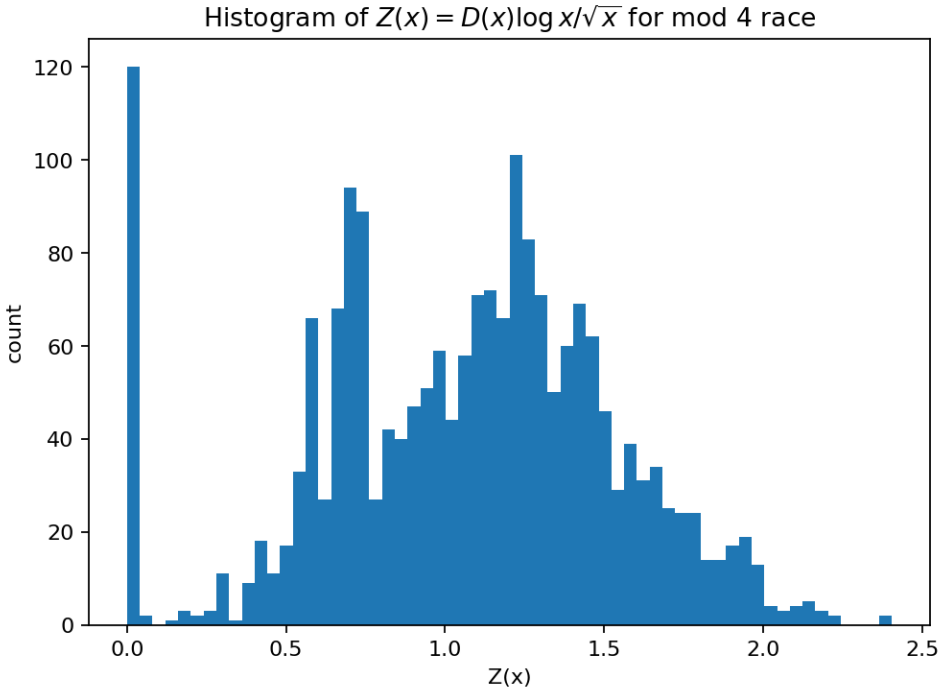
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Granville and Martin [GM06], Rubinstein and Sarnak [RS94]

3.3.75 E075: Prime race statistic: distribution on a log-grid.



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `prime-races`, `aps`

Highlights

- Computes prime-race differences (x;q,a) − (x;q,b) on a grid of x values.
- Visualizes lead changes and the size of fluctuations as x grows.
- Adds a derived statistic (normalization / -variant) to compare behaviors.

What this experiment does

For a prime race difference D(x), a common heuristic normalization is:

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e075/`:

- `figures/fig_01_statistic_hist.png`

How to run

```
make run EXP=e075
```

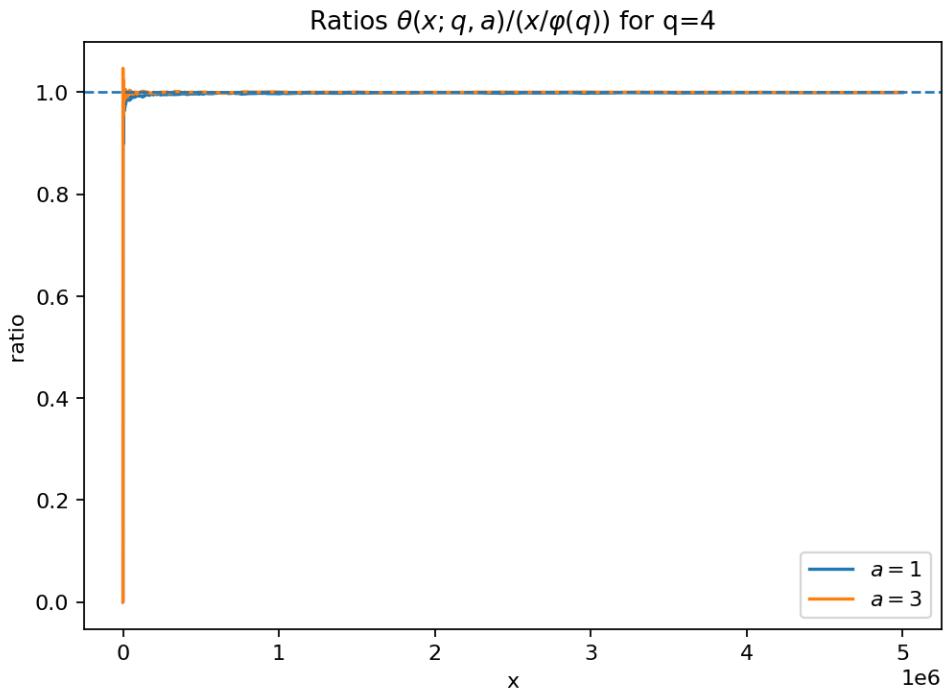
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Granville and Martin [GM06], Rubinstein and Sarnak [RS94]

3.3.76 E076: Chebyshev (x;q,a): weighted prime counts in progressions.



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `prime-races`, `aps`

Highlights

- Computes prime-race differences (x;q,a) − (x;q,b) on a grid of x values.
- Visualizes lead changes and the size of fluctuations as x grows.
- Adds a derived statistic (normalization / -variant) to compare behaviors.

What this experiment does

Define the Chebyshev theta function in a residue class:

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e076/`:

- `figures/fig_01_theta_ratios.png`

How to run

```
make run EXP=e076
```

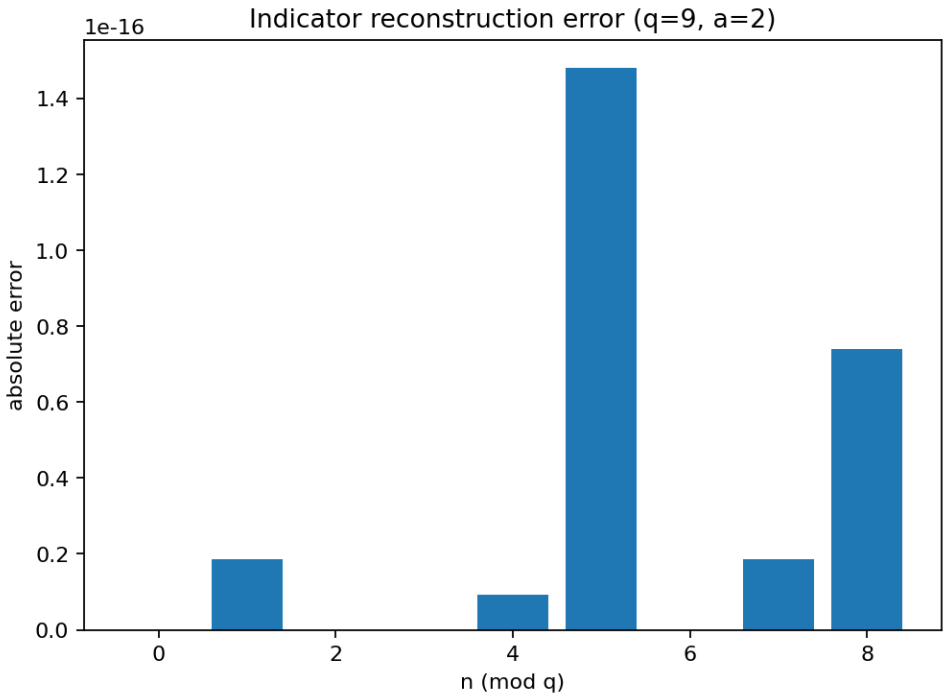
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Granville and Martin [GM06], Rubinstein and Sarnak [RS94]

3.3.77 E077: Indicator via character orthogonality (sanity check).



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `dirichlet-characters`

Highlights

- Uses character orthogonality to reconstruct residue indicators.
- Measures reconstruction error / maximal partial sums across characters.
- Explores how primitive characters vary with the modulus.

What this experiment does

A standard identity expresses an indicator of a residue class using Dirichlet characters. For gcd(a,q)=1 and gcd(n,q)=1:

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e077/`:

- `figures/fig_01_indicator_error.png`

How to run

```
make run EXP=e077
```

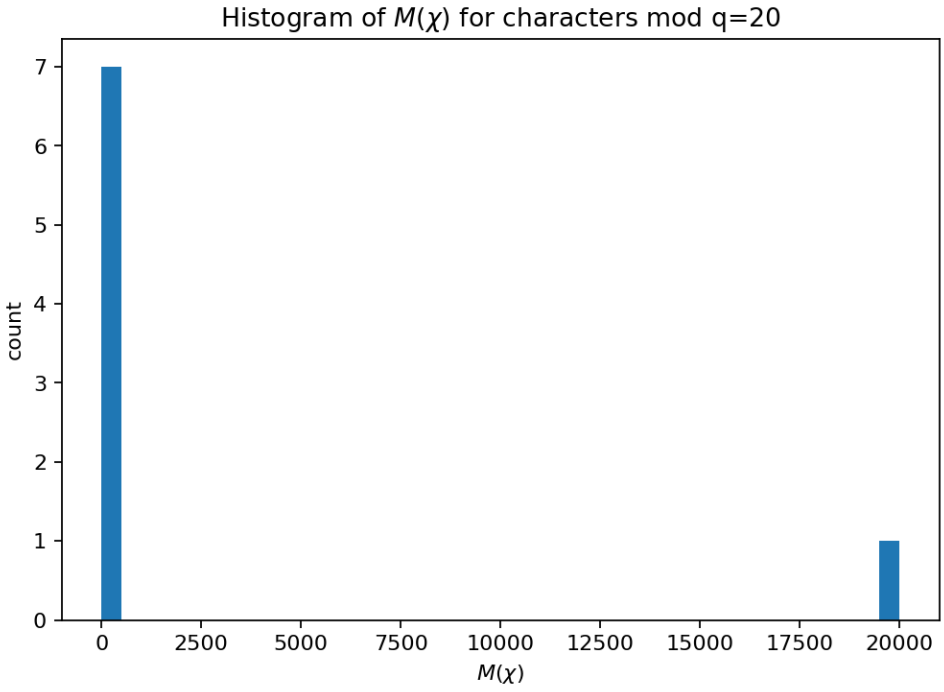
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Davenport [Dav00], Niven *et al.* [NZM91]

3.3.78 E078: Max partial sums across characters.



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `dirichlet-characters`

Highlights

- Uses character orthogonality to reconstruct residue indicators.
- Measures reconstruction error / maximal partial sums across characters.
- Explores how primitive characters vary with the modulus.

What this experiment does

For each Dirichlet character modulo q, define

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e078/`:

- `figures/fig_01_max_sums_hist.png`

How to run

```
make run EXP=e078
```

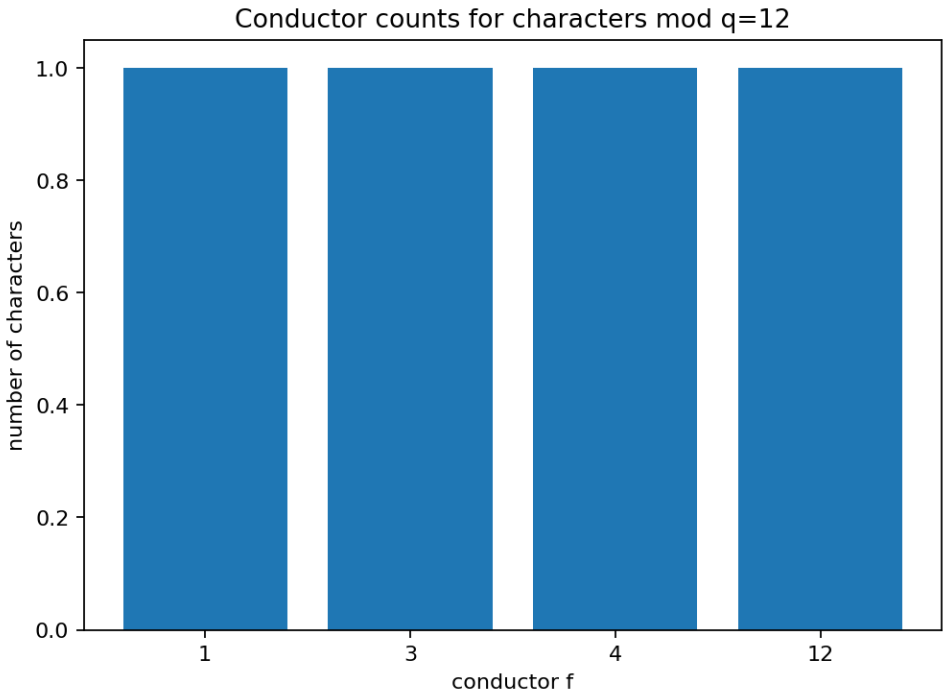
Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Davenport [Dav00], Niven *et al.* [NZM91]

3.3.79 E079: Primitive vs imprimitive characters: conductors.



Tags: `number-theory`, `quantitative-exploration`, `visualization`, `dirichlet-characters`

Highlights

- Uses character orthogonality to reconstruct residue indicators.
- Measures reconstruction error / maximal partial sums across characters.
- Explores how primitive characters vary with the modulus.

What this experiment does

A character modulo q may factor through a smaller modulus f | q. The smallest such modulus is the **conductor** of the character.

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e079/`:

- `figures/fig_01_conductor_counts.png`

How to run

```
make run EXP=e079
```

Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Davenport [Dav00], Niven *et al.* [NZM91]

3.3.80 E080: Chebyshev bias: leader fraction vs x.

Tags: `number-theory`, `quantitative-exploration`, `visualization`, `prime-races`

Highlights

- Estimates the fraction of x where one residue class leads another.
- Plots a running bias estimate over log-sampled x values.
- Detects coarse sign-change neighborhoods for the race difference.

What this experiment does

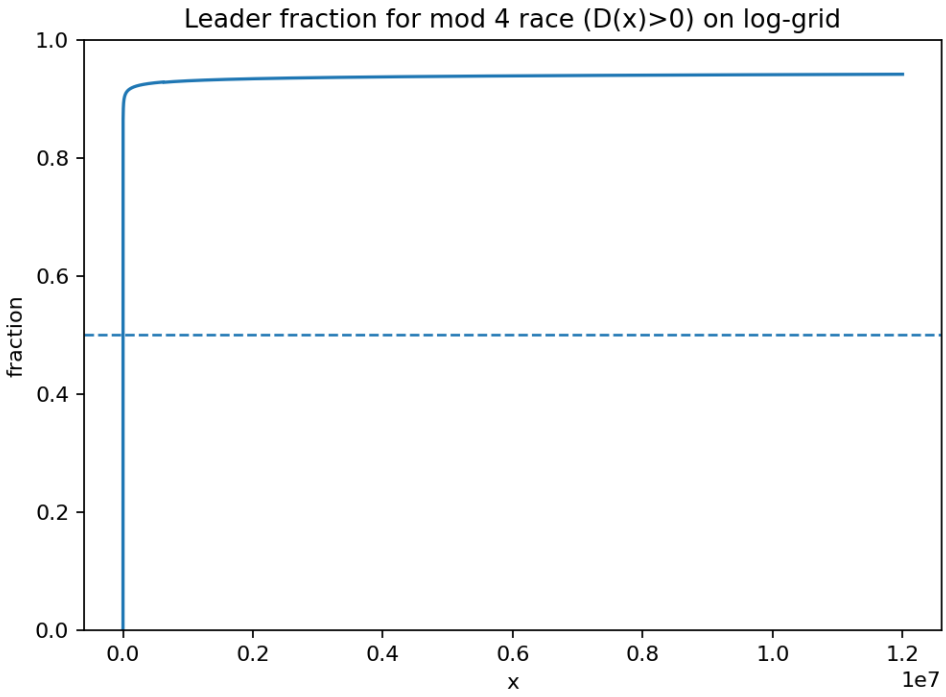
For the mod 4 race D(x)= (x;4,3)- (x;4,1), define the empirical leader fraction:

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.

Outputs

This experiment writes into `out/e080/`:

- `figures/fig_01_bias_fraction.png`



How to run

```
make run EXP=e080
```

Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Granville and Martin [GM06], Rubinstein and Sarnak [RS94]

3.3.81 E081: Prime race sign changes: first crossings table.

Tags: `number-theory`, `quantitative-exploration`, `visualization`, `prime-races`

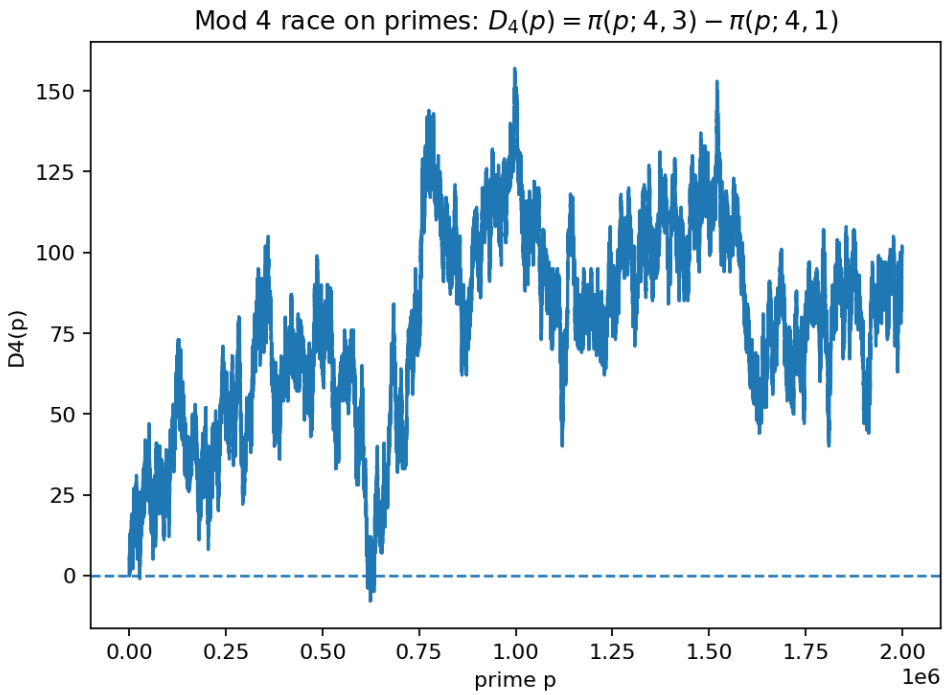
Highlights

- Estimates the fraction of x where one residue class leads another.
- Plots a running bias estimate over log-sampled x values.
- Detects coarse sign-change neighborhoods for the race difference.

What this experiment does

A prime race difference D(x) only changes when x passes a prime. We can track sign changes by iterating primes in order.

The implementation focuses on a compact, reproducible numerical workflow: deterministic parameter defaults, structured output folders, and one or more figures saved for the gallery.



Outputs

This experiment writes into `out/e081/`:

- `figures/fig_01_mod4_diff_on_primes.png`

How to run

```
make run EXP=e081
```

Notes

- The gallery preview figure shipped with the documentation uses conservative cutoffs so builds stay fast. If you run the experiment locally, increase the cutoffs to see the asymptotic regime more clearly.
- Prime-race plots depend on the chosen sampling of `x` (linear vs log grid). The qualitative "who leads" picture can change when you zoom in.

References

Granville and Martin [GM06], Rubinstein and Sarnak [RS94]

3.3.82 E082 — Zeta(s) series convergence

Tags: `analysis`, `quantitative-exploration`, `visualization`, `riemann-zeta`, `numerics`

Highlights

- Focused numeric experiment with a single main figure.
- Parameters saved to `params.json` for reproducibility.
- Lightweight computation suitable for CI "slow" suite (small defaults).

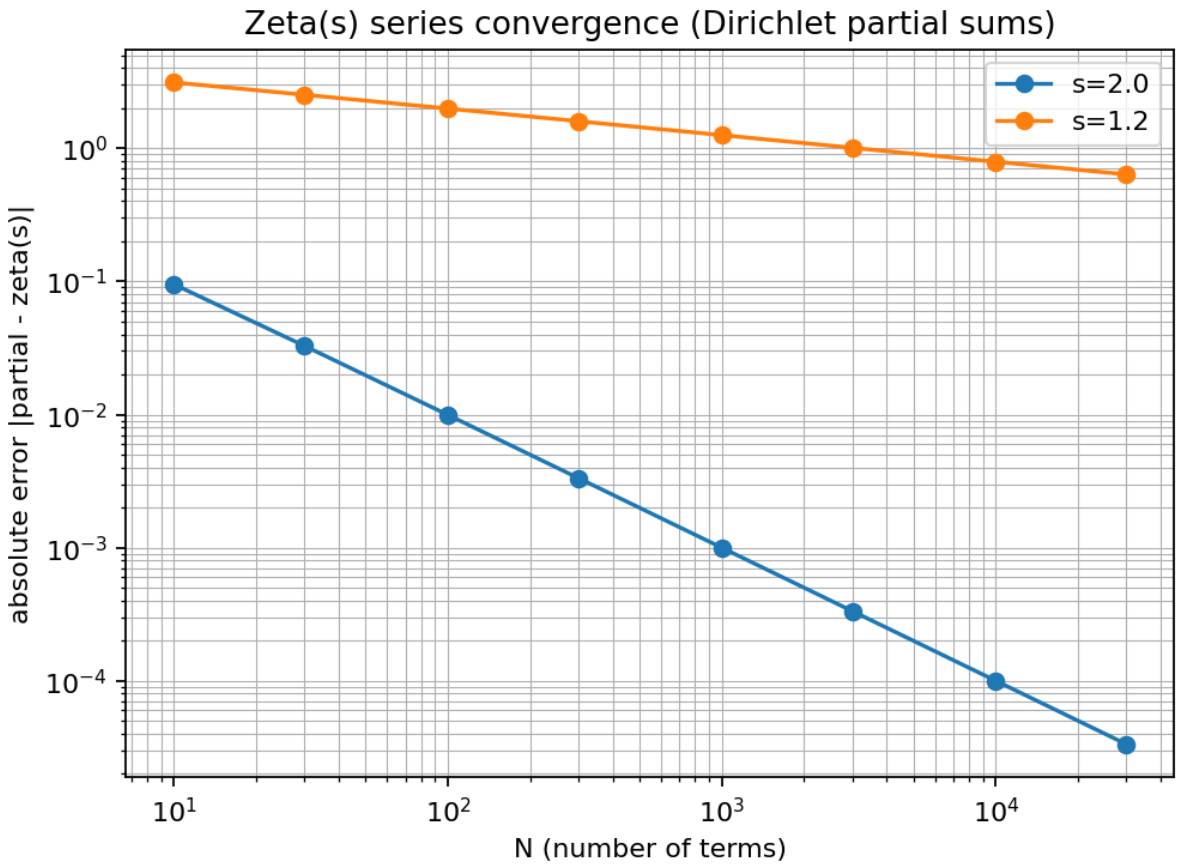


Fig. 1: E082 — Zeta(s) series convergence

What is computed

- A parameterized numeric evaluation related to the Riemann zeta function.
- A visualization summarizing the main phenomenon for the chosen parameter range.

Algorithm sketch

1. Build the numeric grid / sampling points.
2. Evaluate the target quantity with controlled truncation.
3. Render the figure and write a short report.

Outputs

- `report.md` — short narrative summary
- `params.json` — experiment parameters snapshot
- `figures/fig_01_series_convergence.png` — main figure

References

Edwards [Edw74], Titchmarsh [Tit86]

3.3.83 E083 — Series vs Euler product ( )

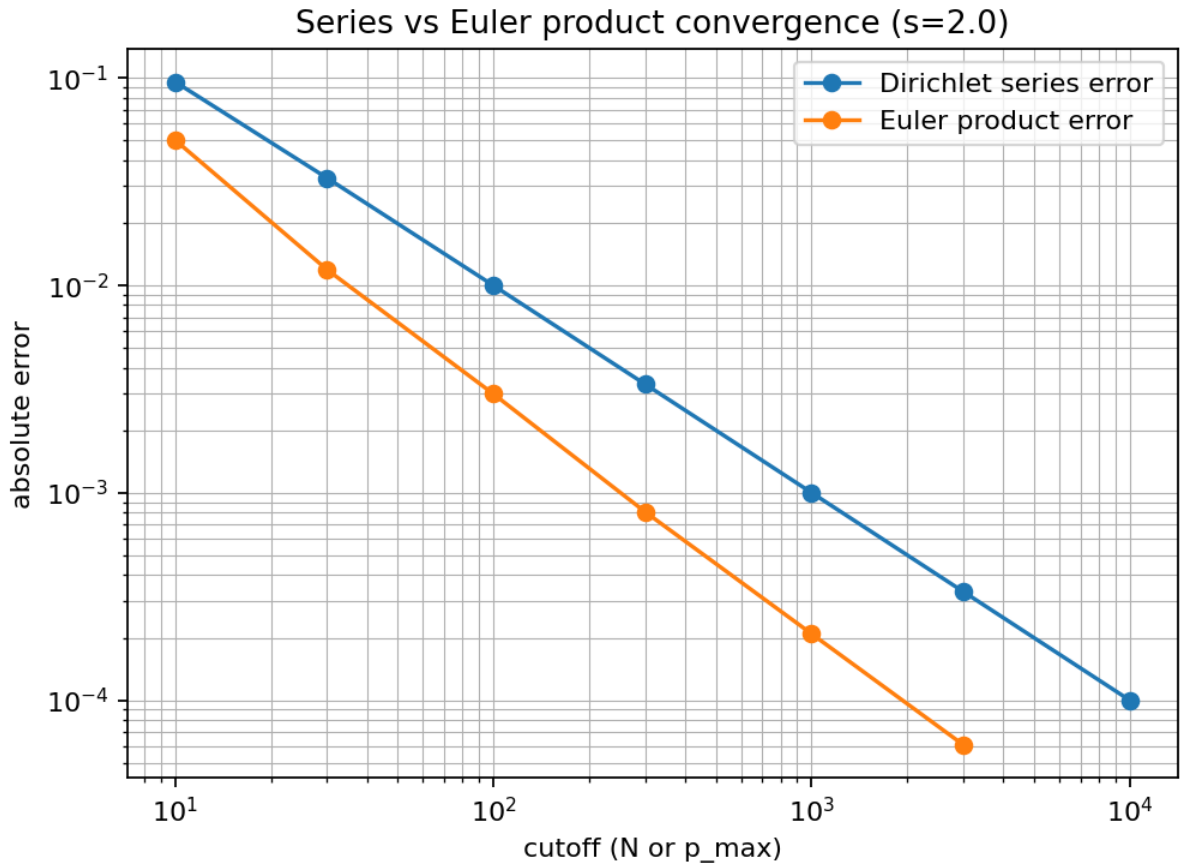


Fig. 2: E083 — Series vs Euler product ( )

Tags: `analysis`, `quantitative-exploration`, `visualization`, `riemann-zeta`, `numerics`

Highlights

- Focused numeric experiment with a single main figure.
- Parameters saved to `params.json` for reproducibility.
- Lightweight computation suitable for CI "slow" suite (small defaults).

What is computed

- A parameterized numeric evaluation related to the Riemann zeta function.
- A visualization summarizing the main phenomenon for the chosen parameter range.

Algorithm sketch

1. Build the numeric grid / sampling points.
2. Evaluate the target quantity with controlled truncation.
3. Render the figure and write a short report.

Outputs

- `report.md` — short narrative summary
- `params.json` — experiment parameters snapshot
- `figures/fig_01_series_vs_euler_product.png` — main figure

References

Edwards [Edw74], Titchmarsh [Tit86]

3.3.84 E084 — | (1/2+it)| growth snapshots

Tags: `analysis`, `quantitative-exploration`, `visualization`, `riemann-zeta`, `critical-line`, `numerics`

Highlights

- Focused numeric experiment with a single main figure.
- Parameters saved to `params.json` for reproducibility.
- Lightweight computation suitable for CI "slow" suite (small defaults).

What is computed

- A parameterized numeric evaluation related to the Riemann zeta function.
- A visualization summarizing the main phenomenon for the chosen parameter range.

Algorithm sketch

1. Build the numeric grid / sampling points.
2. Evaluate the target quantity with controlled truncation.
3. Render the figure and write a short report.

Outputs

- `report.md` — short narrative summary
- `params.json` — experiment parameters snapshot
- `figures/fig_01_critical_line_logabs.png` — main figure

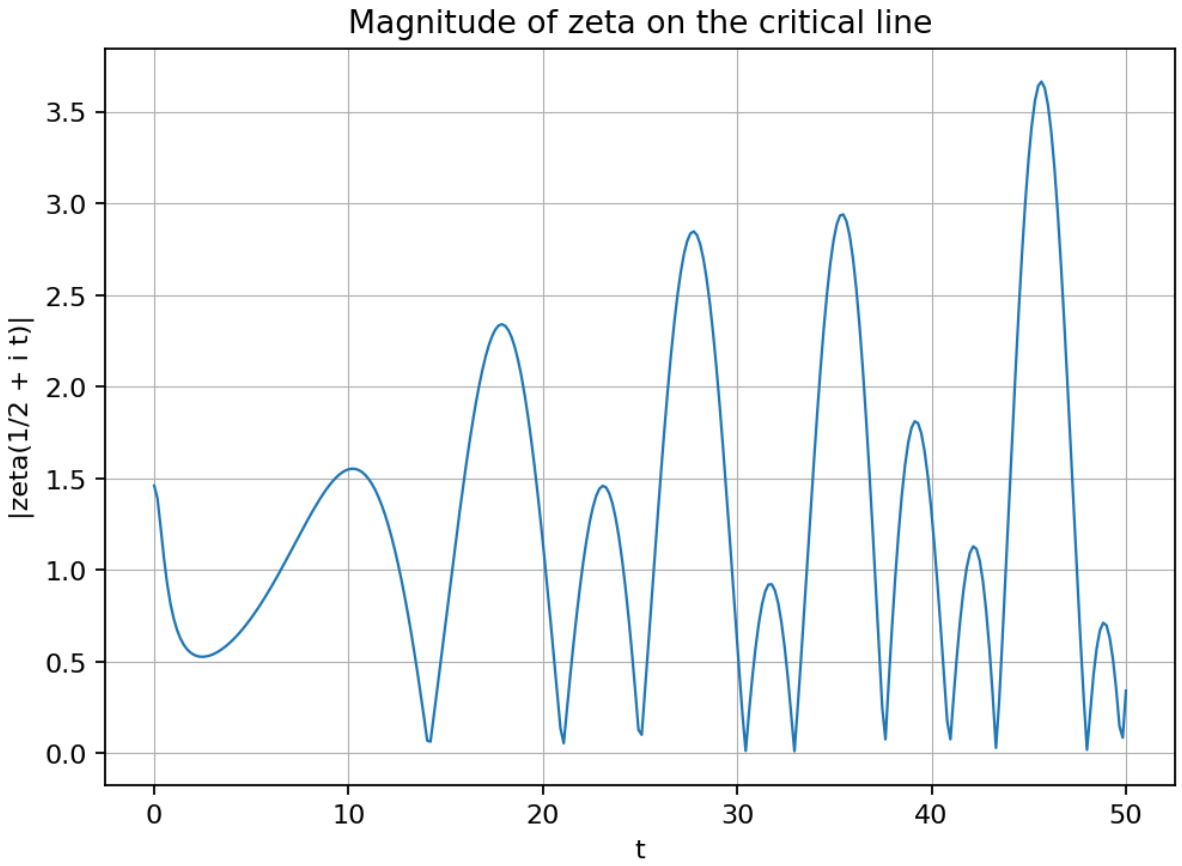


Fig. 3: E084 — | (1/2+it)| growth snapshots

References

Edwards [Edw74], Titchmarsh [Tit86]

3.3.85 E085 — Dirichlet eta acceleration for (s)

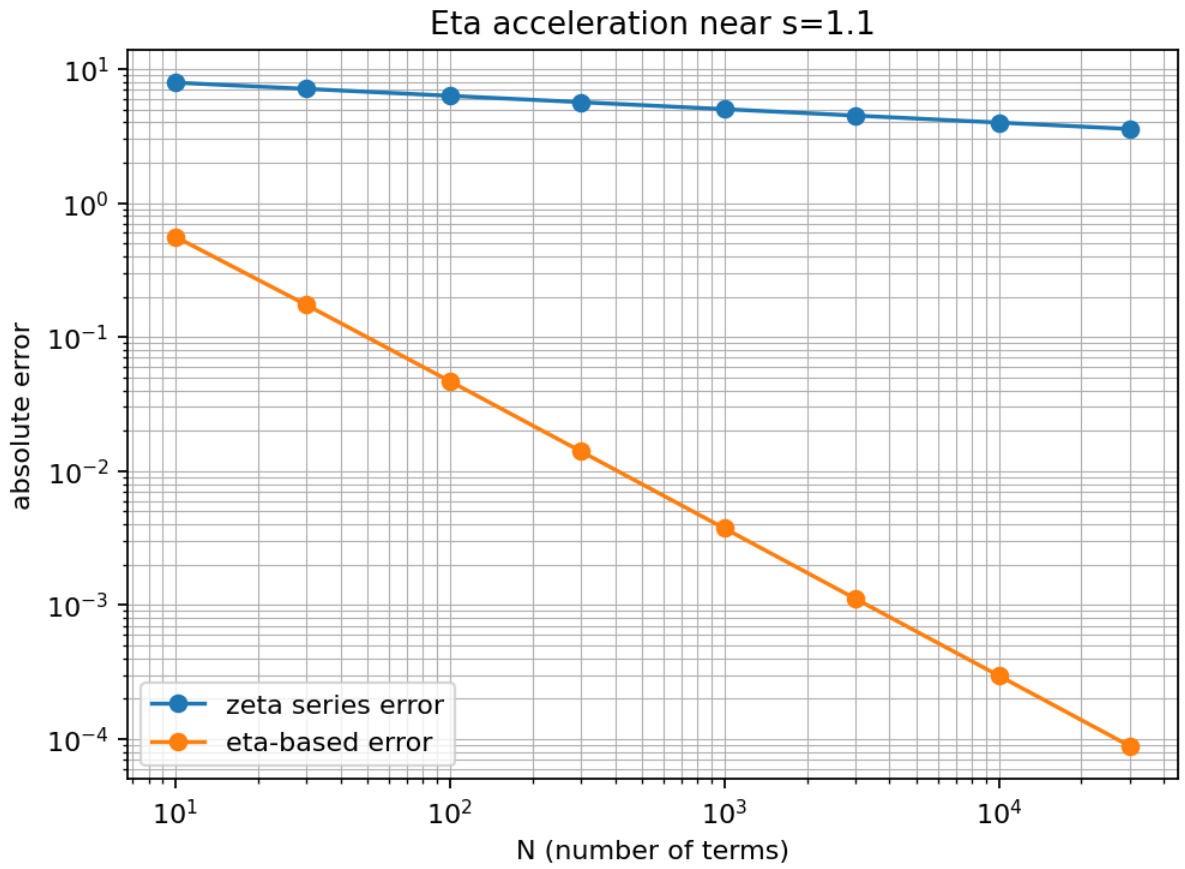


Fig. 4: E085 — Dirichlet eta acceleration for (s)

Tags: `analysis`, `quantitative-exploration`, `visualization`, `riemann-zeta`, `numerics`

Highlights

- Focused numeric experiment with a single main figure.
- Parameters saved to `params.json` for reproducibility.
- Lightweight computation suitable for CI “slow” suite (small defaults).

What is computed

- A parameterized numeric evaluation related to the Riemann zeta function.
- A visualization summarizing the main phenomenon for the chosen parameter range.

Algorithm sketch

1. Build the numeric grid / sampling points.
2. Evaluate the target quantity with controlled truncation.
3. Render the figure and write a short report.

Outputs

- `report.md` — short narrative summary
- `params.json` — experiment parameters snapshot
- `figures/fig_01_eta_acceleration.png` — main figure

References

Edwards [Edw74], Titchmarsh [Tit86]

3.3.86 E086 — Hardy Z(t) near zeros

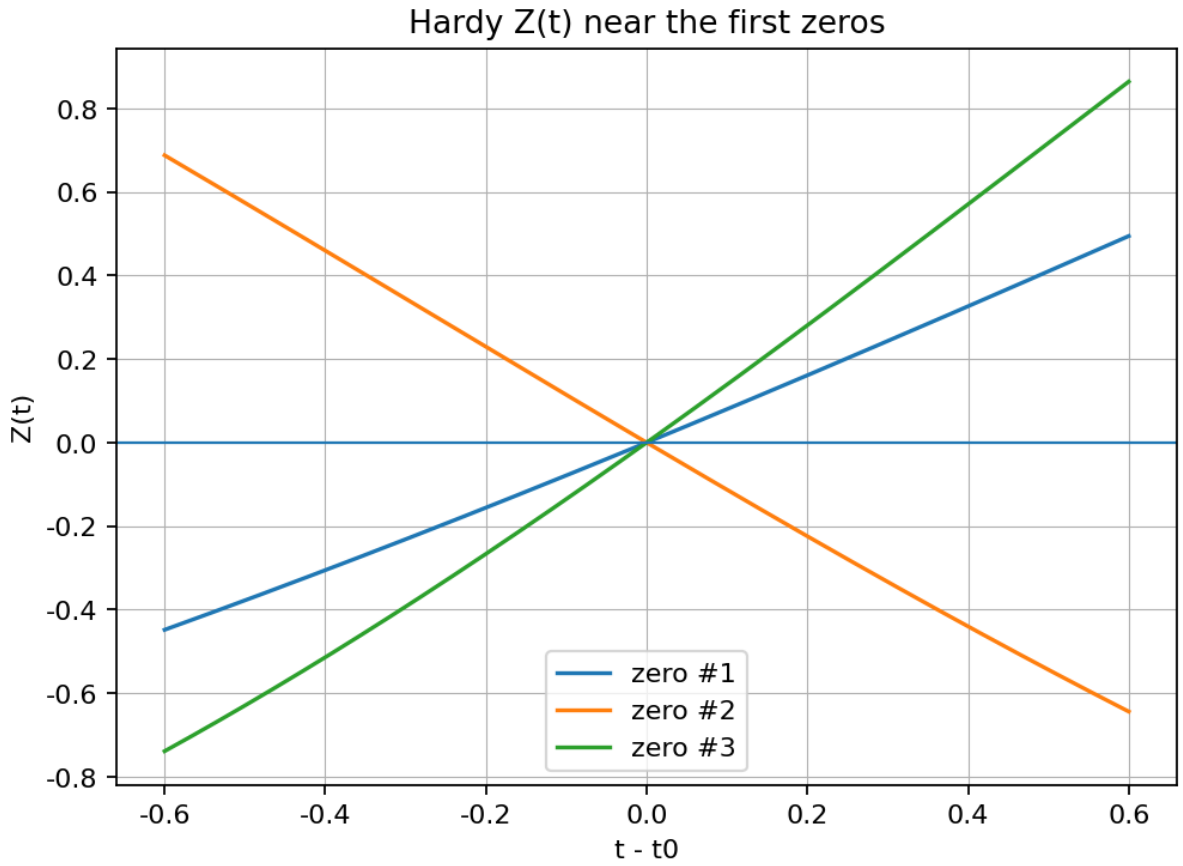


Fig. 5: E086 — Hardy Z(t) near zeros

Tags: `analysis`, `quantitative-exploration`, `visualization`, `riemann-zeta`, `zeta-zeros`, `critical-line`, `numerics`

Highlights

- Focused numeric experiment with a single main figure.
- Parameters saved to `params.json` for reproducibility.
- Lightweight computation suitable for CI “slow” suite (small defaults).

What is computed

- A parameterized numeric evaluation related to the Riemann zeta function.
- A visualization summarizing the main phenomenon for the chosen parameter range.

Algorithm sketch

1. Build the numeric grid / sampling points.
2. Evaluate the target quantity with controlled truncation.
3. Render the figure and write a short report.

Outputs

- `report.md` — short narrative summary
- `params.json` — experiment parameters snapshot
- `figures/fig_01_hardy_Z_near_zeros.png` — main figure

References

Edwards [Edw74], Titchmarsh [Tit86]

3.3.87 E087 — Gram points and spacing

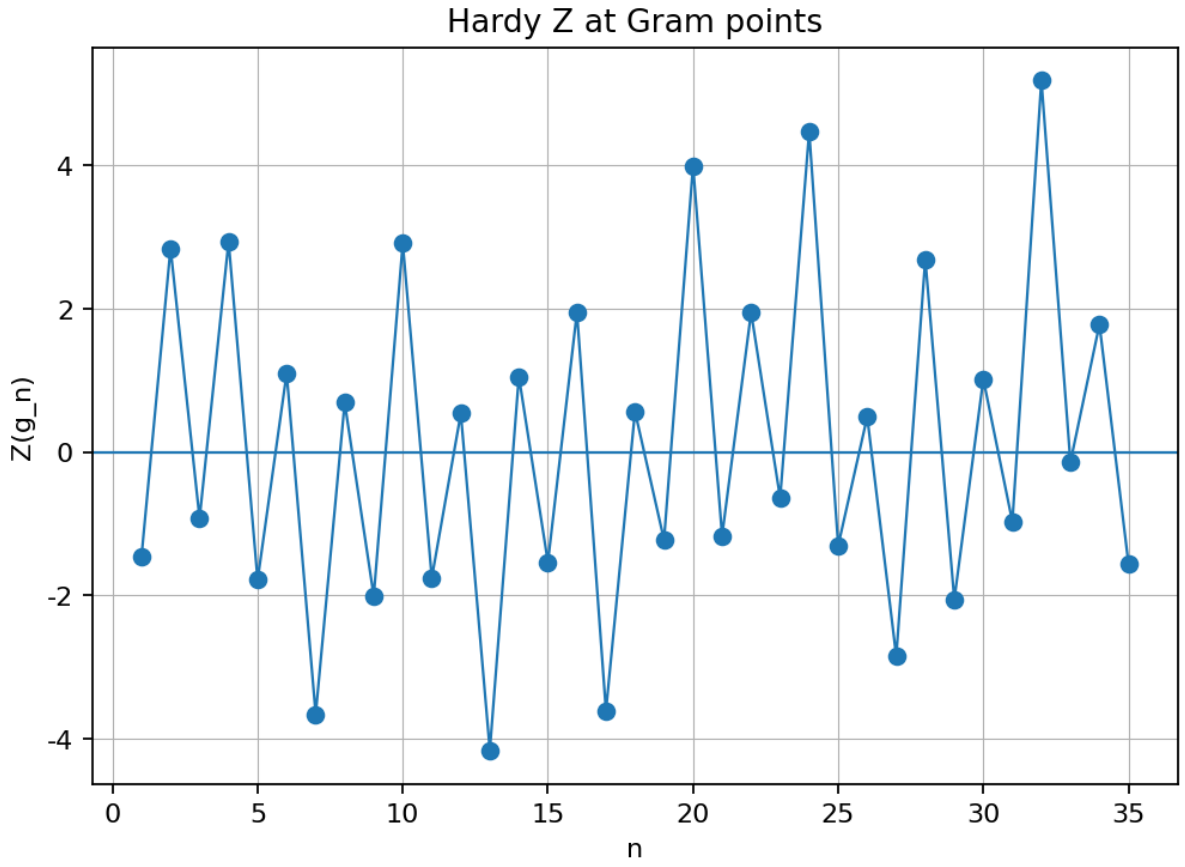


Fig. 6: E087 — Gram points and spacing

Tags: `analysis`, `quantitative-exploration`, `visualization`, `riemann-zeta`, `gram-points`, `zeta-zeros`, `numerics`

Highlights

- Focused numeric experiment with a single main figure.
- Parameters saved to `params.json` for reproducibility.
- Lightweight computation suitable for CI “slow” suite (small defaults).

What is computed

- A parameterized numeric evaluation related to the Riemann zeta function.
- A visualization summarizing the main phenomenon for the chosen parameter range.

Algorithm sketch

1. Build the numeric grid / sampling points.
2. Evaluate the target quantity with controlled truncation.
3. Render the figure and write a short report.

Outputs

- `report.md` — short narrative summary
- `params.json` — experiment parameters snapshot
- `figures/fig_01_gram_points_spacing.png` — main figure

References

Edwards [Edw74], Montgomery [Mon73], Titchmarsh [Tit86]

3.3.88 E088 — Zero counting via Riemann–von Mangoldt

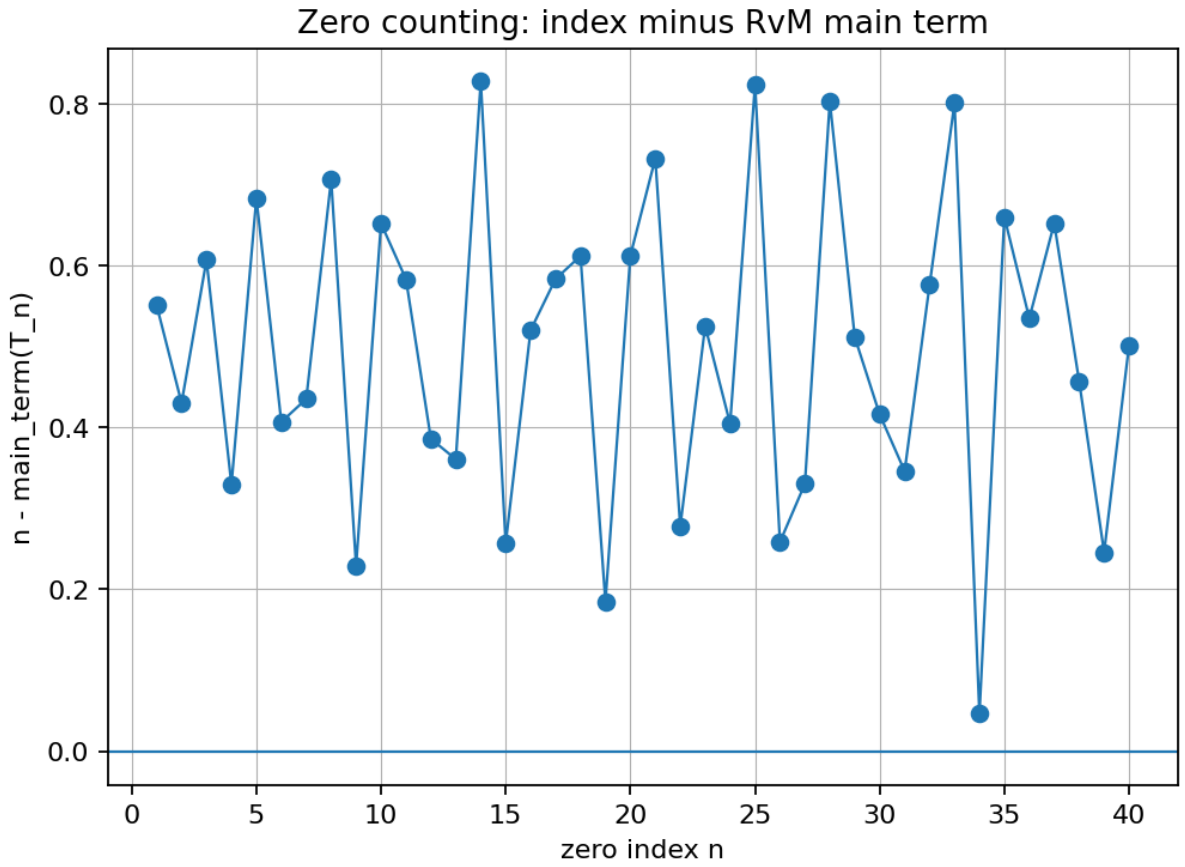


Fig. 7: E088 — Zero counting via Riemann–von Mangoldt

Tags: `analysis`, `quantitative-exploration`, `visualization`, `riemann-zeta`, `zeta-zeros`, `numerics`

Highlights

- Focused numeric experiment with a single main figure.
- Parameters saved to `params.json` for reproducibility.
- Lightweight computation suitable for CI "slow" suite (small defaults).

What is computed

- A parameterized numeric evaluation related to the Riemann zeta function.
- A visualization summarizing the main phenomenon for the chosen parameter range.

Algorithm sketch

1. Build the numeric grid / sampling points.
2. Evaluate the target quantity with controlled truncation.
3. Render the figure and write a short report.

Outputs

- `report.md` — short narrative summary
- `params.json` — experiment parameters snapshot
- `figures/fig_01_rvm_zero_counting.png` — main figure

References

Edwards [Edw74], Montgomery [Mon73], Titchmarsh [Tit86]

3.3.89 E089 — log| (s)| heatmap

Tags: `analysis`, `quantitative-exploration`, `visualization`, `riemann-zeta`, `numerics`

Highlights

- Focused numeric experiment with a single main figure.
- Parameters saved to `params.json` for reproducibility.
- Lightweight computation suitable for CI "slow" suite (small defaults).

What is computed

- A parameterized numeric evaluation related to the Riemann zeta function.
- A visualization summarizing the main phenomenon for the chosen parameter range.

Algorithm sketch

1. Build the numeric grid / sampling points.
2. Evaluate the target quantity with controlled truncation.
3. Render the figure and write a short report.

Outputs

- `report.md` — short narrative summary
- `params.json` — experiment parameters snapshot
- `figures/fig_01_zeta_heatmap_logabs.png` — main figure

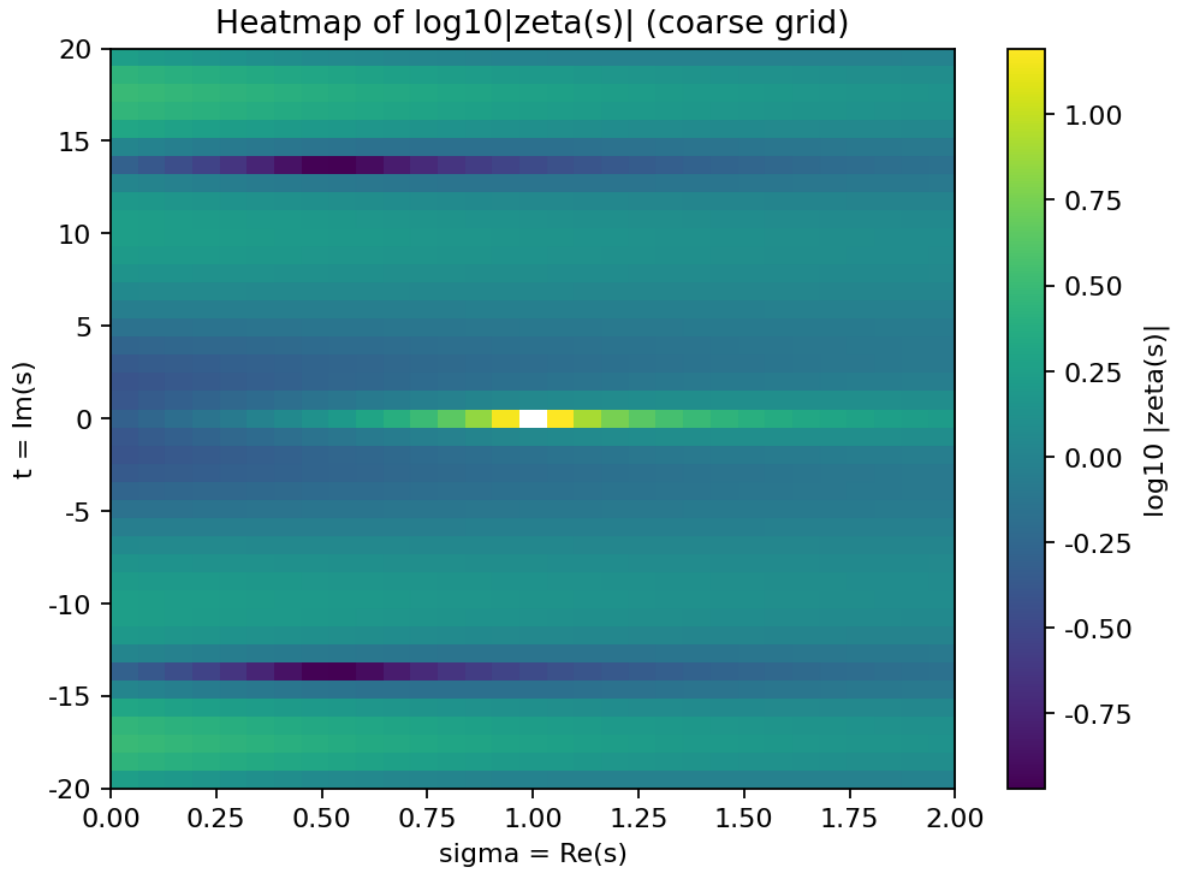


Fig. 8: E089 — log| (s)| heatmap

References

Edwards [Edw74], Titchmarsh [Tit86]

3.3.90 E090 — Functional equation residual heatmap

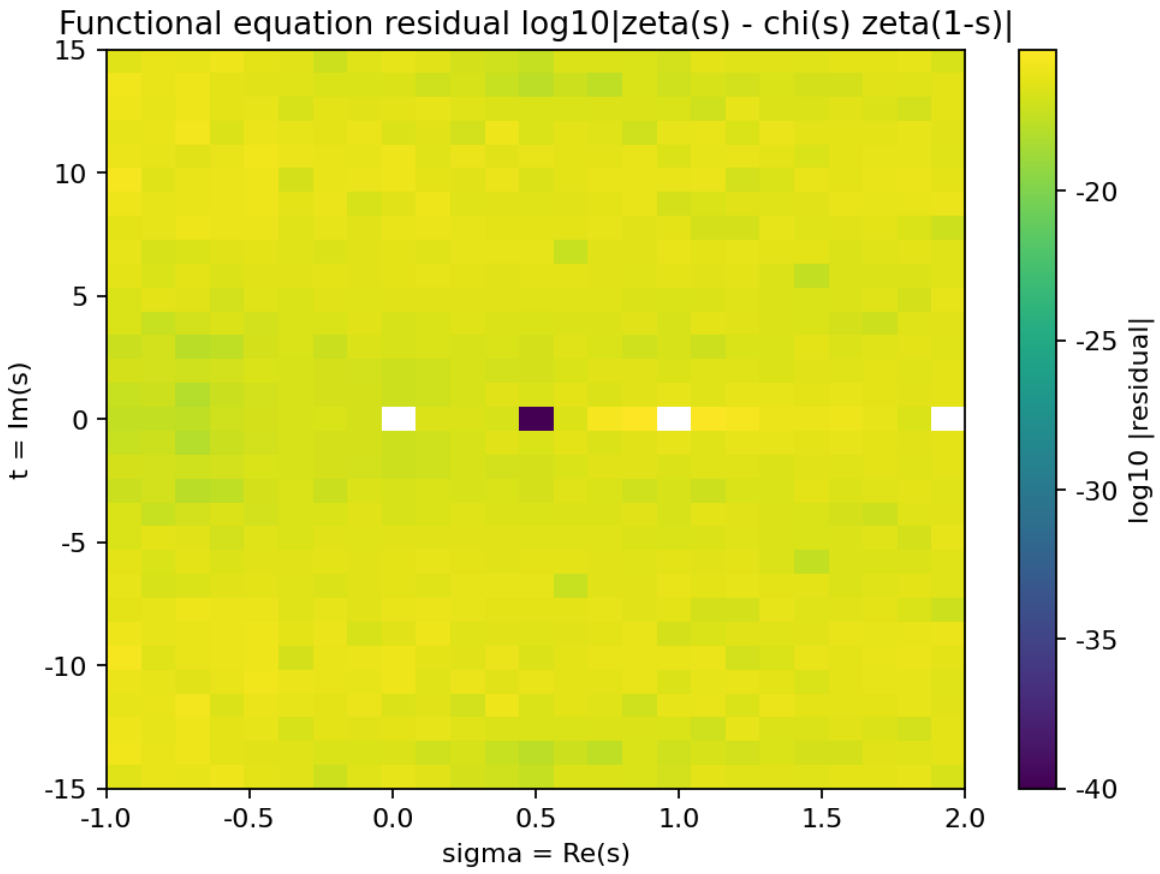


Fig. 9: E090 — Functional equation residual heatmap

Tags: `analysis`, `quantitative-exploration`, `visualization`, `riemann-zeta`, `numerics`

Highlights

- Focused numeric experiment with a single main figure.
- Parameters saved to `params.json` for reproducibility.
- Defaults chosen so the experiment remains feasible for the CI "slow" suite.

What is computed

- Evaluate the functional-equation residual (R(s)=\zeta(s)-\chi(s),\zeta(1-s)) on a small rectangle in the complex plane.
- Visualize `log10(|R(s)| + eps)` as a heatmap.

Notes

- This is a sanity-check style plot: away from poles/zeros and with sufficient precision, the residual should be small.
- Precision and grid size are controlled by parameters.

References

- See the zeta / Dirichlet-series references in `refs.bib`.

3.3.91 E091 — Partial Euler products on the critical line

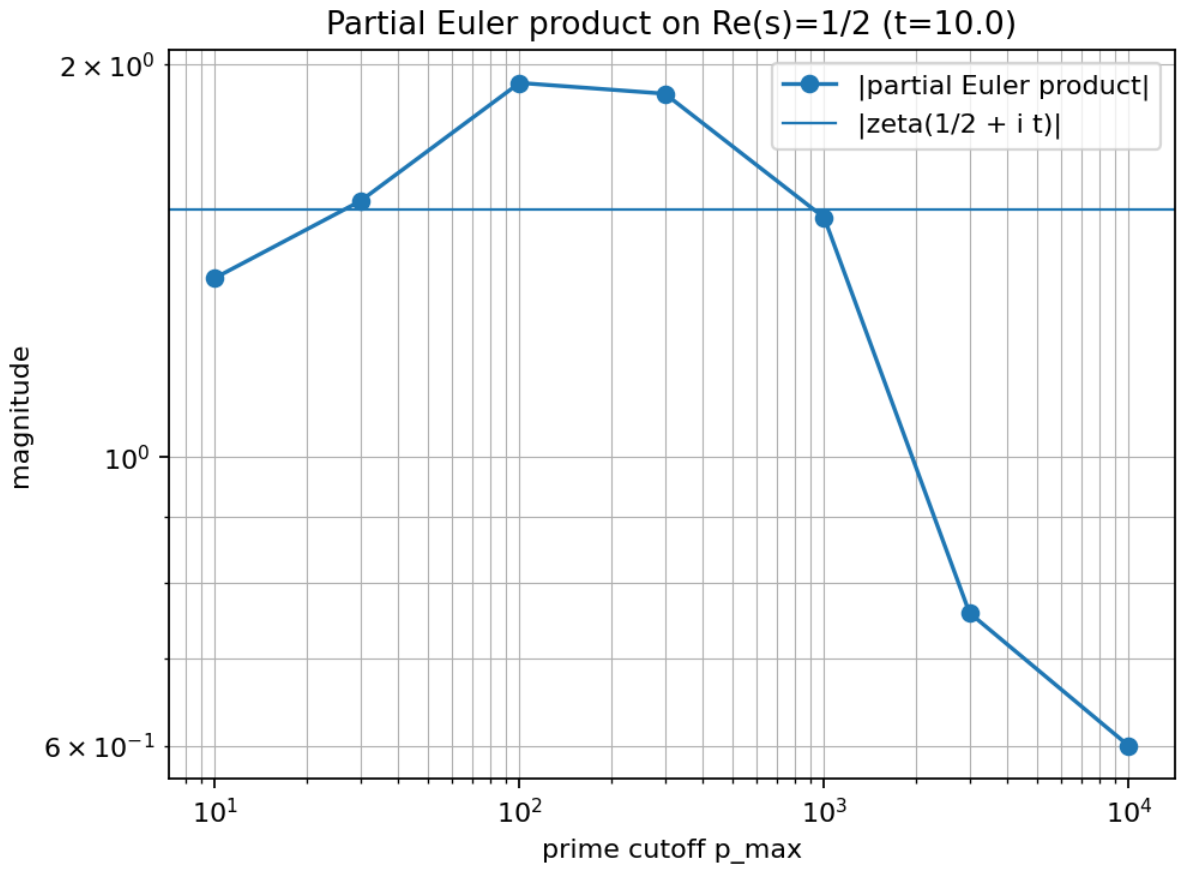


Fig. 10: E091 — Partial Euler products on the critical line

Tags: `analysis`, `quantitative-exploration`, `visualization`, `riemann-zeta`, `dirichlet-series`, `numerics`

Highlights

- Focused numeric experiment with a single main figure.
- Parameters saved to `params.json` for reproducibility.
- Defaults chosen so the experiment remains feasible for the CI "slow" suite.

What is computed

- Compare (\zeta(1/2+it)) to partial Euler products (\prod_{p\le P} (1-p^{-s})^{-1}) at a fixed `t` while increasing the prime cutoff `P`.
- Plot the mismatch to illustrate non-convergence on the critical line.

Notes

- The Euler product is only convergent for (\Re(s)>1); this experiment visualizes the failure mode at (\Re(s)=1/2).

References

- See the zeta / Dirichlet-series references in `refs.bib`.

3.3.92 E092 — 1/ (s) via the Möbius Dirichlet series

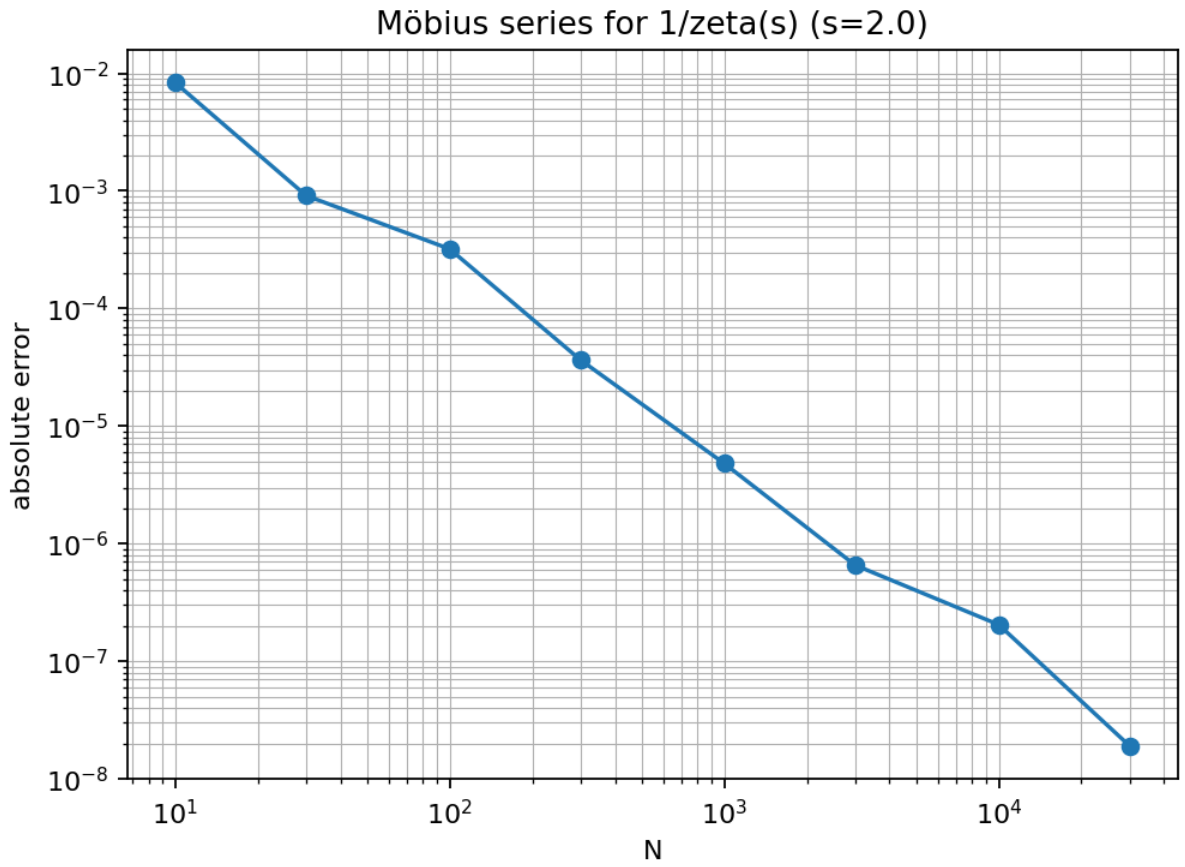


Fig. 11: E092 — 1/ (s) via the Möbius Dirichlet series

Tags: `analysis`, `quantitative-exploration`, `visualization`, `riemann-zeta`, `mobius`, `dirichlet-series`, `numerics`

Highlights

- Focused numeric experiment with a single main figure.
- Parameters saved to `params.json` for reproducibility.
- Defaults chosen so the experiment remains feasible for the CI "slow" suite.

What is computed

- For (\Re(s)>1), compare (1/\zeta(s)) to the partial sums (\sum_{n\le N} \mu(n)/n^s).
- Plot convergence as `N` increases, and highlight dependence on `Re(s)`.

Notes

- This is the classic identity (\sum_{n\ge1} \mu(n)/n^s = 1/\zeta(s)) (absolute convergence for (\Re(s)>1)).

References

- See the zeta / Dirichlet-series references in `refs.bib`.

3.3.93 E093 — — (s)/ (s) via the von Mangoldt series

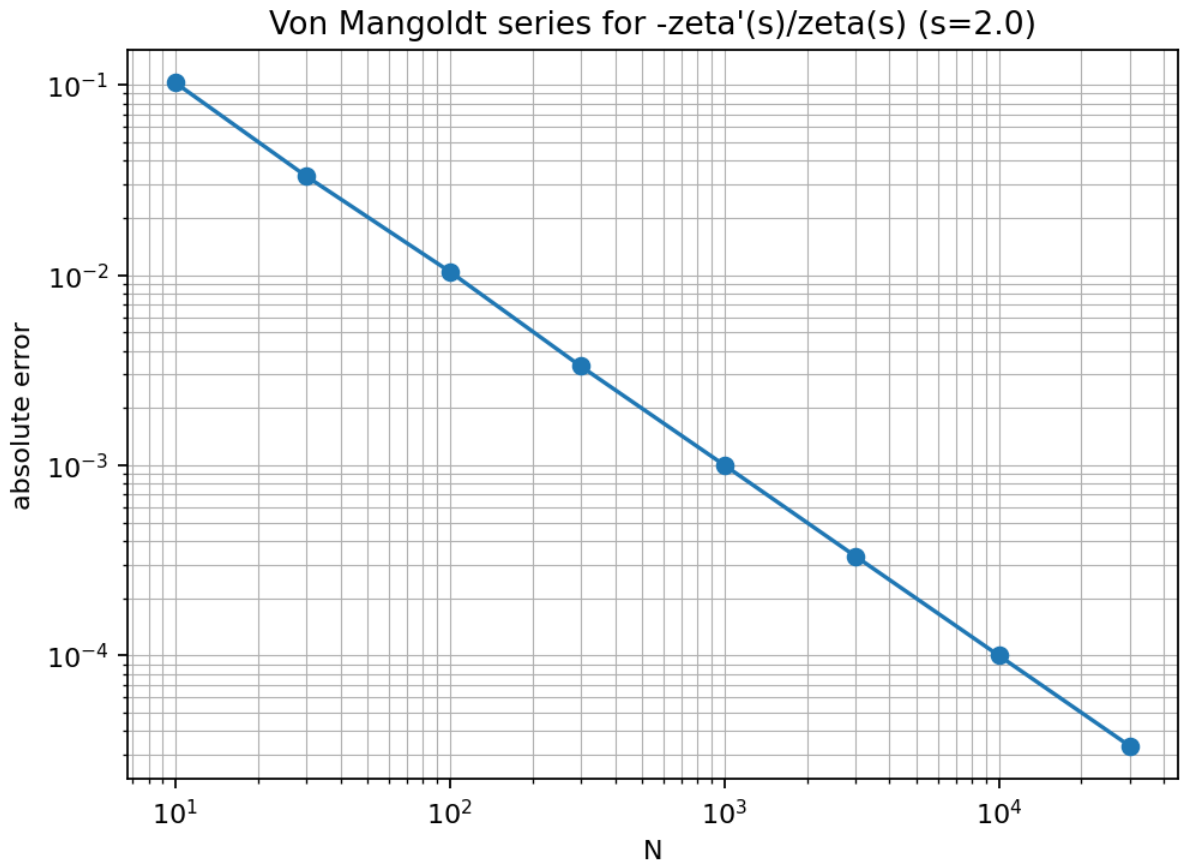


Fig. 12: E093 — — (s)/ (s) via the von Mangoldt series

Tags: `analysis`, `quantitative-exploration`, `visualization`, `riemann-zeta`, `dirichlet-series`, `numerics`

Highlights

- Focused numeric experiment with a single main figure.
- Parameters saved to `params.json` for reproducibility.
- Defaults chosen so the experiment remains feasible for the CI “slow” suite.

What is computed

- For (\Re(s)>1), compare (-\zeta’(s)/\zeta(s)) to the partial sums (\sum_{n\le N} \Lambda(n)/n^s).
- Plot the truncation error as a function of `N`.

Notes

- Uses the von Mangoldt function (\Lambda(n)); the series converges absolutely for (\Re(s)>1).

References

- See the zeta / Dirichlet-series references in `refs.bib`.

3.4 Background

This section provides mathematical foundations for the experiments.

3.4.1 Arithmetic functions refresher

This page is a *beginner-friendly* refresher for experiments about **arithmetic functions** (i.e. functions $f : \mathbb{N} \to \mathbb{C}$ that encode number-theoretic structure).

For deeper treatments, see [Apo76], [NZM91], and [Ten15].

What "arithmetic function" usually means

An **arithmetic function** is any function of a positive integer n. Common themes:

- prime factorization drives the behavior of many functions
- *multiplicativity* lets you reduce to prime powers
- *averages* and *summatory functions* reveal global structure

Examples you will meet often:

- Euler's totient $\varphi(n)$ (coprime residues)
- Möbius $\mu(n)$ and Mertens $M(x) = \sum_{n \le x} \mu(n)$
- divisor counts $d(n)$ and sums $\sigma_k(n)$
- prime-factor counting $\omega(n), \Omega(n)$
- von Mangoldt $\Lambda(n)$ (prime powers)
- Carmichael's $\lambda(n)$ (group exponent mod n)

Multiplicative vs. additive

Multiplicative

An arithmetic function f is **multiplicative** if

$$f(1) = 1, \qquad f(ab) = f(a)f(b) \ \text{ whenever } \ \gcd(a, b) = 1.$$

If $n = \prod p_i^{\alpha_i}$ then multiplicativity gives

$$f(n) = \prod f(p_i^{\alpha_i}).$$

Typical: $\varphi, \mu, \sigma_k, d, J_k, \lambda$.

Additive

A function g is **additive** if

$$g(ab) = g(a) + g(b) \ \text{ whenever } \ \gcd(a, b) = 1,$$

and **completely additive** if the relation holds for all a, b (no gcd condition). Typical: $\Omega(n)$ is completely additive; $\omega(n)$ is additive.

Why averages matter

Many experiments compare:

- pointwise behavior of $f(n)$
- cumulative behavior $\sum_{k \le n} f(k)$
- distribution of $f(n)$ over ranges

Analytic/probabilistic number theory focuses on these averages; see [MV06] and [Ten15].

Common experiment patterns

- **Value distribution:** histograms of $f(n)$ for $n \le N$
- **Scatter vs. factorization features:** compare $f(n)$ with $\log n$, $\omega(n)$, etc.
- **Normal order:** show "typical size" vs. rare extremes (e.g. Erdős–Kac behavior)
- **Summatory oscillations:** $M(x)$, $\sum_{n \le x} \lambda(n)$, etc.
- **Extremal orders:** highly composite numbers maximize $d(n)$ (see [Ram15])

Experiments in this repository

- **E121** — Multiplicativity stress tests across core arithmetic functions.
- **E123** — Correlation matrix of arithmetic functions on 1..N (empirical relationships).

3.4.2 Average orders and the Erdős–Kac viewpoint

Many arithmetic functions are "wild" pointwise but have predictable average behavior. This page gives a short refresher on *average order*, *normal order*, and the Erdős–Kac phenomenon. See [Ten15] and [ErdHosK40].

Average order vs. normal order

- **Average order:** a function $A(x)$ such that

$$\sum_{n \le x} f(n) \approx \sum_{n \le x} A(n)$$

  or f has mean value described by A.
- **Normal order:** a function $N(n)$ such that "most" integers satisfy

$$f(n) \approx N(n).$$

A classic example: for most n, $\omega(n)$ is close to $\log \log n$.

Erdős–Kac theorem (informal statement)

Let $\omega(n)$ be the number of distinct prime factors. Erdős and Kac proved that the normalized variable

$$\frac{\omega(n) - \log \log n}{\sqrt{\log \log n}}$$

has an approximately standard normal distribution over $n \le x$ as $x \to \infty$. [ErdHosK40]

Experiment ideas

- for a chosen N, compute $\omega(n)$ for $n \le N$ and histogram the normalized values
- compare the empirical mean/variance with $\log \log N$
- explore how the fit improves as N grows

Experiments in this repository

- **E094** — Erdős–Kac side-by-side for and Ω (normal order / scaling).

3.4.3 Carmichael's $\lambda(n)$ function refresher

Carmichael's function $\lambda(n)$ is the exponent of the multiplicative group modulo n. It refines Euler's theorem by giving the *smallest* universal exponent. See [ErdHosPS91] and [NZM91].

Definition

Let $(\mathbb{Z}/n\mathbb{Z})^\times$ be the multiplicative group of units modulo n. The **exponent** of a finite group G is lcm of the orders of all elements. Carmichael's function $\lambda(n)$ is the exponent of $(\mathbb{Z}/n\mathbb{Z})^\times$.

Equivalently, $\lambda(n)$ is the smallest positive integer such that for all integers a with $\gcd(a, n) = 1$:

$$a^{\lambda(n)} \equiv 1 \pmod{n}.$$

Always $\lambda(n) \mid \varphi(n)$.

Prime power formula (useful in code)

For odd primes p and $k \geq 1$:

$$\lambda(p^k) = \varphi(p^k) = p^{k-1}(p-1).$$

For 2^k:

$$\lambda(2) = 1, \quad \lambda(4) = 2, \quad \lambda(2^k) = 2^{k-2} \text{ for } k \geq 3.$$

For general $n = \prod p_i^{\alpha_i}$:

$$\lambda(n) = \operatorname{lcm}\big(\lambda(p_i^{\alpha_i})\big).$$

Why it is interesting experimentally

- highlights differences between "group size" (φ) and "group exponent" (λ)
- connects to orders modulo n and to cryptographic-style modular arithmetic
- has rich average-order results and rare extreme behavior (see [ErdHosPS91])

Experiment ideas

- compare $\lambda(n)$ vs. $\varphi(n)$ on ranges
- visualize distribution of $\varphi(n)/\lambda(n)$
- record-breakers for large $\varphi(n)/\lambda(n)$

Experiments in this repository

- **E100** — Carmichael (n) vs Euler (n): ratios, typical size, and extremes.

3.4.4 Carmichael numbers

A Carmichael number is a composite n such that

[a^{n-1} \equiv 1 \pmod{n}]

for every integer a coprime to n. They are **absolute Fermat pseudoprimes**, so they defeat the naive Fermat primality test. [PrimePUTM25a, Wikipediacontributors25a]

Key facts

- **Korselt's criterion (1899):** n is Carmichael iff (i) n is squarefree and (ii) for every prime $p \mid n$, we have $(p-1) \mid (n-1)$. [Con04, Wikipediacontributors25a]
- **At least three prime factors:** No Carmichael number is the product of only two primes. [Wikipediacontributors25a]
- **Infinitely many:** Alford–Granville–Pomerance proved there are infinitely many Carmichael numbers. [AGP94]
- **Historical origin:** Carmichael studied these numbers in the early 20th century. [Car12]

What to experiment with

- **Counterexample search:** Find Carmichael numbers by generating squarefree products and testing Korselt's criterion.
- **"Fermat test is not enough":** Show that all bases a with $\gcd(a, n) = 1$ pass Fermat for a Carmichael n, while a Miller–Rabin test quickly finds witnesses.
- **Distribution experiments:** Count Carmichael numbers up to X and compare growth to simple heuristics.
- **Construction families:** Implement Chernick-style constructions and see when factors are prime.

References

See Alford *et al.* [AGP94], Conrad [Con04], The OEIS Foundation Inc. [TheOFInc25e], Wikipedia contributors [Wikipediacontributors25a].

3.4.5 Dirichlet characters refresher

This page is a *lightweight* background for experiments about primes in residue classes, Dirichlet L-functions, and prime races.

Core definitions

Fix an integer modulus $q \geq 1$. A **Dirichlet character modulo** q is a function $\chi : \mathbb{Z} \to \mathbb{C}$ such that:

1. (Periodicity) $\chi(n+q) = \chi(n)$ for all n.
2. (Multiplicativity) $\chi(mn) = \chi(m)\chi(n)$ for all m, n.
3. (Support) $\chi(n) = 0$ if $\gcd(n, q) > 1$.
4. For $\gcd(n, q) = 1$, we have $|\chi(n)| = 1$ (so values are roots of unity).

The **principal character** χ_0 modulo q is: [\chi_0(n) = \begin{cases} 1, & \gcd(n,q)=1,\ 0, & \gcd(n,q)>1. \end{cases}]

A character is **primitive** if it does not "factor through" a smaller modulus (its conductor equals q).

Orthogonality (the workhorse identity)

Let χ, ψ be Dirichlet characters modulo q. Then (over a reduced residue system): [\sum_{\substack{a \bmod q\ \gcd(a,q)=1}} \chi(a)\overline{\psi(a)} = \begin{cases} \varphi(q), & \chi=\psi,\ 0, & \chi\ne \psi. \end{cases}]

A dual identity (summing over characters) is: [\sum_{\chi \bmod q} \chi(a)\overline{\chi(b)} = \begin{cases} \varphi(q), & a \equiv b \pmod q,\ \gcd(a,q)=1,\ 0, & \text{otherwise.} \end{cases}]

These are the algebraic "Fourier rules" that make Dirichlet's argument work.

Example: modulus $q = 4$

There are two characters modulo 4:

- χ_0 (principal).
- The nontrivial character χ_4: [\chi_4(n)= \begin{cases} 0,& 2 \mid n,\ 1,& n \equiv 1 \pmod 4,\ -1,& n \equiv 3 \pmod 4. \end{cases}]

This single character already explains the classic race between primes 1 mod 4 and 3 mod 4.

What experiments usually measure

- How $\sum_{n \le N} \chi(n)$ behaves (cancellation, max partial sums).
- How orthogonality isolates a residue class.
- How many primitive characters exist for a given modulus, and how they "look" as tables.

Practical numerical caveats

- Always define $\chi(n) = 0$ when $\gcd(n, q) > 1$ (otherwise identities break).
- Represent χ as a lookup table on residues $0, \ldots, q-1$ and reduce via `n % q`.
- Be careful with complex floats: for most experiments, values are in $\{0, \pm 1\}$ or small roots of unity, so you can use exact complex numbers.

References

See *References*.

[Dav00, IR90, LD37, Ser73]

Experiments in this repository

- **E106** — Character gallery: real vs complex characters; conjugate pairing.
- **E107** — Primitive vs imprimitive via conductor (induced characters).
- **E108** — Orthogonality relations as a heatmap; numeric sanity checks.

3.4.6 Dirichlet convolution refresher

Dirichlet convolution is a key "algebra" on arithmetic functions and shows up in many proofs and experiments. See [Apo76] and [Ten15].

Definition

For arithmetic functions $f, g : \mathbb{N} \to \mathbb{C}$, their **Dirichlet convolution** is

$$(f * g)(n) = \sum_{d \mid n} f(d)\, g\left(\frac{n}{d}\right).$$

The function $1(n) \equiv 1$ acts like an identity in many formulas, and the **delta function** $\varepsilon(n)$ (with $\varepsilon(1) = 1$ and $\varepsilon(n) = 0$ for $n > 1$) is the convolution identity:

$$f * \varepsilon = f.$$

Möbius inversion

If

$$F(n) = \sum_{d \mid n} f(d) \quad \Longleftrightarrow \quad F = f * 1,$$

then **Möbius inversion** says

$$f = F * \mu.$$

This is one of the main reasons $\mu(n)$ appears everywhere.

Classic identities

- Sum of divisors:

$$\sigma(n) = (1 * \mathrm{id})(n),$$

  where $\mathrm{id}(n) = n$.

- Totient:

$$\varphi = \mu * \mathrm{id}.$$

- Jordan totient:

$$J_k = \mu * \mathrm{id}^k.$$

- von Mangoldt:

$$\Lambda = \mu * \log,$$

  in the sense of Dirichlet series / logarithmic derivatives of $\zeta(s)$.

Dirichlet series viewpoint (why it's computationally useful)

If $F(s) = \sum_{n\ge1} f(n)n^{-s}$ and $G(s) = \sum_{n\ge1} g(n)n^{-s}$ converge absolutely, then

$$\sum_{n\ge1} \frac{(f*g)(n)}{n^s} = F(s)\,G(s).$$

This "turns convolution into multiplication" and underlies many analytic estimates; see [MV06].

Experiments in this repository

- **E102** — Dirichlet convolution identity zoo (*1=* , 1=id, 1*1=* , *id*1= , …).
- **E121** — Multiplicativity stress tests and convolution sanity checks (random coprime tests).

3.4.7 Dirichlet eta function (s)

The **Dirichlet eta function** is the alternating Dirichlet series [\eta(s)=\sum_{n\ge 1} \frac{(-1)^{n-1}}{n^s},] which converges for (\Re(s)>0) (by alternating-series arguments). It is related to the Riemann zeta function by [\eta(s) = \left(1-2^{1-s}\right)\zeta(s).]

Key ideas

- **Faster convergence:** for real (s>1), the alternating series often converges more rapidly than the plain -series.
- **Analytic continuation:** the identity above provides an analytic continuation of (s) away from the factor (1-2^{1-s}).
- **Numerical gateway:** (s) is a convenient "entry point" for simple (s) numerics without complex contour methods.

Why it matters in this project

When we want quick, small-scale experiments (e.g. comparing partial sums, smoothing, or convergence acceleration), (s) provides stable numerics with minimal infrastructure.

Experiments in this repository

- **E114** — (1/2+it) via (s): truncation/precision stability map.

References

See *References*.

[Edw74, Tit86, Wikipediacontributors25h]

3.4.8 Dirichlet L-functions refresher

Dirichlet characters package congruence information; Dirichlet L-functions turn that into analytic objects whose zeros control prime distribution in residue classes.

Core definitions

Let χ be a Dirichlet character modulo q. The **Dirichlet L-function** is [L(s,\chi) = \sum_{n=1}^\infty \frac{\chi(n)}{n^s}, \qquad \Re(s) > 1.]

For $\Re(s) > 1$, it has an **Euler product**: [L(s,\chi) = \prod_{p \nmid q}\left(1-\frac{\chi(p)}{p^s}\right)^{-1}.]

For the principal character: [L(s,\chi_0) = \zeta(s)\prod_{p\mid q}\left(1-p^{-s}\right).]

The decisive analytic fact (used in Dirichlet's theorem) is: [L(1,\chi) \ne 0 \quad \text{for every nonprincipal character } \chi.]

What experiments usually visualize or measure

- Convergence of partial sums $\sum_{n\le N}\chi(n)n^{-s}$ for various s.
- Convergence of partial Euler products over primes.
- Sensitivity near $s = 1$ (slow convergence) and how to stabilize it.

Practical numerical caveats

- Near $s = 1$, both series and Euler products converge painfully slowly.
- For Euler products, compute via logs: [\log L(s,\chi) \approx -\sum_{p\le P}\log\left(1-\chi(p)p^{-s}\right)] to reduce catastrophic multiplication error.
- If you compare characters, always keep the same prime cutoff P to make plots comparable.

References

See *References*.

[Dav00, LD37, Ser73, Was97]

Experiments in this repository

- **E110** — Dirichlet L-series partial sums at s=1 and s=1/2 (principal vs nonprincipal).
- **E111** — Euler product vs Dirichlet series truncations for L(s,): error vs cutoff.

3.4.9 Divisor functions $d(n)$ and $\sigma_k(n)$ refresher

Divisor functions measure how many divisors a number has, and how large they are. They are classical examples of multiplicative functions. See [Apo76].

Counting divisors: $d(n)$

Let $d(n)$ (also written $\tau(n)$) be the number of positive divisors of n.

If $n = \prod p_i^{\alpha_i}$, then

$$d(n) = \prod(\alpha_i + 1).$$

Sum of divisors: $\sigma_k(n)$

For $k \geq 0$,

$$\sigma_k(n) = \sum_{d|n} d^k.$$

The case $k = 1$ is the usual sum-of-divisors function $\sigma(n)$.

For a prime power,

$$\sigma_k(p^\alpha) = 1 + p^k + p^{2k} + \cdots + p^{\alpha k} = \frac{p^{(\alpha+1)k} - 1}{p^k - 1}.$$

Again, multiplicativity gives $\sigma_k(n)$ from prime powers.

Highly composite numbers (extremal behavior)

Numbers that maximize $d(n)$ up to a range are called **highly composite numbers**. Ramanujan's classic paper studies their structure. [Ram15]

Experiment ideas

- visualize $d(n)$ up to N and highlight record-breakers
- compare $\sigma(n)$ to n (abundant / perfect / deficient classification)
- log-scale plots of $d(n)$ to make extremes visible

Experiments in this repository

- **E096** — Record-holders for (n) up to N (highly composite flavor).
- **E097** — (n)/n landscape: deficient / perfect / abundant classification.
- **E098** — Extremals of (n)/n^ across (phase changes / superabundant intuition).

References

See *References*.

[AErdHos44, Apo76, Lag02, Ram15, Rob84]

3.4.10 Euler's totient function $\varphi(n)$ refresher

Euler's totient function $\varphi(n)$ counts reduced residues modulo n. Standard references: [Apo76], [NZM91].

Definition

For $n \geq 1$,

$$\varphi(n) = \#\{\, 1 \leq k \leq n : \gcd(k, n) = 1 \,\}.$$

Equivalently: $\varphi(n)$ is the size of the multiplicative group $(\mathbb{Z}/n\mathbb{Z})^\times$.

Prime power formula

If p is prime and $k \geq 1$,

$$\varphi(p^k) = p^k - p^{k-1} = p^k \left(1 - \frac{1}{p}\right).$$

Since φ is multiplicative, for $n = \prod p_i^{\alpha_i}$:

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right).$$

Important properties

- **Euler's theorem:** if $\gcd(a, n) = 1$ then $a^{\varphi(n)} \equiv 1 \pmod{n}$.
- **Average order:** roughly $\sum_{n \leq x} \varphi(n) \sim \frac{3}{\pi^2} x^2$ (analytic methods).
- **Extremes:** $\varphi(n)$ is small when n has many small prime factors.

Computational notes

Given the prime factorization of n, $\varphi(n)$ is cheap to compute via the product formula. For experiments, you can often compute it for all $n \leq N$ efficiently using a sieve-style method.

Experiments in this repository

- **E101** — Reduced residues (/q)× as an explicit set; verify size (q).

3.4.11 Explicit formulas: primes zeros

"Explicit formulas" are identities that connect prime counting (typically via Chebyshev functions like (\psi(x))) to sums over zeros of (s) (or more general L-functions). Conceptually, they are a precise form of the slogan:

> **Primes are controlled by zeros.**

A prototypical form expresses (\psi(x)) as a main term (x) plus oscillatory corrections coming from nontrivial zeros.

Key ideas

- **From Euler product to primes:** differentiating (\log\zeta(s)) connects (s) to the von Mangoldt function (\Lambda(n)).
- **Zeros drive oscillations:** sums over (\rho) (zeros) produce fluctuating terms that explain biases and sign changes seen in finite ranges.
- **Numerical truncation:** in experiments, one typically truncates zero sums and studies stability vs cutoff parameters.

Why it matters in this project

This is the bridge between the "Dirichlet character / L-function" block and the "prime race / bias" phenomena: you can literally watch zero contributions modulate prime-counting curves.

Experiments in this repository

- **E119** — (x) − x oscillations and "explicit-formula intuition" (visual).

References

See *References*.

[Ivic85, IK04, Sch76, Tit86, Wikipediacontributors25h]

3.4.12 Exploratory visualizations for arithmetic functions

Several experiments in this project are intentionally "EDA-like": they produce **heatmaps**, **atlases**, and **correlation matrices** for many functions at once. This page collects best practices so those figures remain interpretable and comparable.

Core definitions

Given a function table $F(n)$ for $n = 1, \ldots, N$ and a list of arithmetic functions $f_1, \ldots, f_m$, two common derived views are:

- **Heatmap atlas:** visualize values $f_j(n)$ as an image (rows = functions, columns = n or blocks of n).
- **Correlation matrix:** compute an empirical correlation between columns $f_i(n)$ and $f_j(n)$ (Pearson or rank-based).

What experiments usually visualize or measure

- "Texture": squarefree bands, prime powers, smooth numbers, record-holders, etc.
- Clustering: which functions behave similarly on $1..N$ (after normalization).
- Stability: how patterns change when N grows or when you sample on a log grid.

Practical numerical caveats

- **Normalize first.** Raw scales differ wildly (e.g. $\sigma(n)$ vs $\mu(n)$). Typical choices: z-score, log1p, or scaling by n^{α}.
- **Beware heavy tails.** Extremal values can dominate Pearson correlation; consider Spearman correlation or winsorization for robustness.
- **Sampling matters.** Linear n emphasizes small structure; log-spaced n emphasizes asymptotic behavior. Record the sampling rule in the figure caption.

References

See *References*.

[Arn15, BB08]

Experiments in this repository

- **E122** — Heatmap atlas of , , Ω and related "squarefree texture" features.
- **E123** — Correlation matrix of arithmetic functions (with normalization variants).

3.4.13 Fermat numbers

Fermat numbers are the integers

[F_n = 2^{2^n} + 1 \quad (n \ge 0).]

They grow extremely quickly and are central to a classic "counterexample" story: Fermat conjectured that all F_n are prime, but F_5 is composite (Euler). [KvrivzekLvSomer13, Wikipediacontributors25b]

Key facts

- **Coprime property:** $\gcd(F_m, F_n) = 1$ for $m \neq n$. In fact, $F_0 F_1 \cdots F_{n-1} = F_n - 2$. [KvrivzekLvSomer13]
- **Fermat primes:** If F_n is prime, it is called a Fermat prime (known: $n = 0..4$). [TheOFInc25a]
- **Pépin test (primality):** For $n \geq 1$, F_n is prime iff $3^{(F_n-1)/2} \equiv -1 \pmod{F_n}$. [KvrivzekLvSomer13]

What to experiment with

- **Factor hunting:** Search for small prime factors of F_n (using modular arithmetic and wheel/primorial filters).
- **Pépin vs. trial division:** Compare performance and failure modes as n increases.
- **Structure of known factors:** Many known prime factors have the form $k \cdot 2^{n+2} + 1$; explore how often such candidates appear.
- **Euclid-style numbers:** Explore $E_n = 1 + \prod_{i=0}^{n} p_i$ and compare "Euclid primes" vs. Fermat primes (they behave very differently).

References

See Kř\'ıžek *et al.* [KvrivzekLvSomer13], The OEIS Foundation Inc. [TheOFInc25a], Wikipedia contributors [Wikipediacontributors25b].

3.4.14 Gauss sums and the discrete Fourier viewpoint

Gauss sums are "finite Fourier transforms" of Dirichlet characters. They are a compact way to see how characters interact with additive phases, and they show up naturally in functional equations and explicit evaluations of certain L-values.

Core definitions

Let χ be a Dirichlet character modulo q and write [e_q(a)=\exp!\left(\frac{2\pi i a}{q}\right).] The (normalized) **Gauss sum** is [\tau(\chi)=\sum_{a=1}^{q} \chi(a),e_q(a).]

For a **primitive** character χ modulo q, one has the fundamental magnitude law [|\tau(\chi)|=\sqrt{q}.] (For imprimitive characters, the behavior is more subtle and often smaller.)

A key identity connecting Gauss sums to character orthogonality is: [\sum_{a \bmod q} \chi(a),e_q(an)=\overline{\chi}(n),\tau(\chi) \quad (\gcd(n,q)=1),] and it vanishes when $\gcd(n, q) > 1$.

What experiments usually visualize or measure

- The cloud of complex values $\tau(\chi)$ for all characters modulo a fixed q.
- The "circle of radius $\sqrt{q}$" phenomenon for primitive characters.
- Quadratic characters as a special case (values close to $\pm\sqrt{q}$ or $\pm i\sqrt{q}$ depending on q).

Practical numerical caveats

- Always be explicit about the representative set for residues (e.g. $a = 1, \ldots, q$).
- For imprimitive characters, confirm whether the implementation returns the induced character or the primitive lift; this changes $\tau(\chi)$ drastically.
- Use complex128 (or higher) if you push q large; cancellation is extreme in the raw sum definition.

References

See *References*.

[BEW98, Dav00]

Experiments in this repository

- **E109** — Gauss sums $\tau(\chi)$: magnitude law and geometry for characters modulo q.

3.4.15 Gram points and zero counting

A **Gram point** is (informally) a value (t) where the Riemann–Siegel theta function (\theta(t)) hits an integer multiple of (\pi). These points are useful landmarks when visualizing (Z(t)) and tracking sign changes.

The **Riemann–von Mangoldt formula** gives the asymptotic count of nontrivial zeros up to height (T): [N(T)=\frac{T}{2\pi}\log!\left(\frac{T}{2\pi}\right)-\frac{T}{2\pi}+O(\log T)\quad (T\to\infty),] with refinements that include the argument of on the critical line.

Key ideas

- **Bookkeeping:** zero counting provides a "sanity check" for numerical root-finding: we can compare a computed zero list against (N(T)).
- **Gram blocks / Gram's law:** empirical rules about how zeros sit between consecutive Gram points (useful, but not always true).
- **Practical numerics:** many experiments become clearer when plotted against (\theta(t)) or indexed by Gram points.

Experiments in this repository

- **E116** — Observed zero counts (via sign changes) vs Riemann–von Mangoldt estimate.

References

See *References*.

[Odl92, Tit86, Wikipediacontributors25i, Wikipediacontributors25l]

3.4.16 Hardy's Z-function and the critical line

For real (t), the **Hardy Z-function** is defined (up to standard conventions) by [Z(t)=e^{i\theta(t)},\zeta!\left(\tfrac12+it\right),] where (\theta(t)) is the Riemann–Siegel theta function. The key feature is that **Z(t) is real-valued for real t**, so zeros of (Z(t)) correspond to zeros of (s) on the critical line.

Key ideas

- **Real signal:** studying sign changes of (Z(t)) is a practical way to bracket zeros on (\Re(s)=\tfrac12).
- **Theta function:** (\theta(t)) captures the "oscillatory phase" of on the critical line and is closely tied to Gram points.
- **Numerical workflows:** many computational approaches to zeta zeros are formulated in terms of (Z(t)), (\theta(t)), and their approximations.

Why it matters in this project

It gives a clean, visualization-friendly path from "complex -values" to "real curves" whose roots can be bracketed and counted.

Experiments in this repository

- **E115** — Hardy Z sign-change scan and zero bracketing (bisection refinement).

References

See *References*.

[Odl92, Tit86, Wikipediacontributors25i, Wikipediacontributors25l]

3.4.17 Jordan totient $J_k(n)$ refresher

Jordan's totient function generalizes Euler's totient. It is multiplicative and appears naturally in group-counting problems. See [Apo76] and [Ten15].

Definition

For a fixed integer $k \geq 1$, the **Jordan totient** function $J_k(n)$ can be defined by

$$J_k(n) = n^k \prod_{p|n} \left(1 - \frac{1}{p^k}\right).$$

For $k = 1$, this is Euler's totient: $J_1(n) = \varphi(n)$.

Divisor-sum identity

A useful identity is

$$\sum_{d|n} J_k(d) = n^k.$$

In Dirichlet convolution language:

$$J_k * 1 = \mathrm{id}^k,$$

hence $J_k = \mu * \mathrm{id}^k$.

Experiment ideas

- compare $J_k(n)$ as k varies
- visualize $J_2(n)$ and $J_3(n)$ over $n \leq N$
- compare $J_k(n)/n^k$ as a "prime factor penalty"

Experiments in this repository

- **E099** — Jordan totients J_k: atlas, identities (J_1=), and scaling J_k(n)/n^k.

3.4.18 Liouville function $\lambda(n)$ refresher

The Liouville function is a simple, oscillatory completely multiplicative function built from $\Omega(n)$ (the total number of prime factors with multiplicity). See [Ten15].

Definition

Define

$$\lambda(n) = (-1)^{\Omega(n)}.$$

So $\lambda(n) = 1$ if n has an even total number of prime factors (with multiplicity), and $\lambda(n) = -1$ otherwise.

Examples:

- $\lambda(1) = 1$
- $\lambda(2) = -1$
- $\lambda(4) = \lambda(2^2) = 1$
- $\lambda(12) = \lambda(2^2 \cdot 3) = -1$

Key properties

- **Completely multiplicative:** $\lambda(ab) = \lambda(a)\lambda(b)$ for all a, b.
- Connected to the Möbius function, but without the "squarefree = 0" behavior.

Summatory function experiments

A standard experiment is the summatory function

$$L(x) = \sum_{n \le x} \lambda(n),$$

which oscillates and has deep connections to primes and zeta-function methods.

Experiment ideas

- compare $L(x)$ with a random walk
- compare $\lambda(n)$ and $\mu(n)$ on squarefree numbers
- visualize $\lambda(n)$ as a ± 1 texture over the integer grid

3.4.19 Mersenne numbers and primes refresher

This page is a *beginner-friendly* refresher for experiments about **Mersenne numbers** and **Mersenne primes**.

Core definitions

The **Mersenne numbers** are the integers

$$M_n = 2^n - 1 \quad (n \in \mathbb{N}).$$

A **Mersenne prime** is a Mersenne number that is prime:

$$M_p \text{ is a Mersenne prime} \iff 2^p - 1 \text{ is prime.}$$

A key (easy) fact is that if $2^n - 1$ is prime, then n must be prime.
So, in experiments, you typically only test M_p for *prime* exponents p. [con25b]

Key theorem / test: Lucas–Lehmer (why Mersenne primes are "computable")

For an odd prime exponent p, define $M_p = 2^p - 1$ and a sequence $\{s_k\}$ by

$$s_0 = 4, \qquad s_{k+1} = s_k^2 - 2.$$

Compute this sequence *modulo* M_p at each step, and then:

$$M_p \text{ is prime} \iff s_{p-2} \equiv 0 \pmod{M_p}.$$

This is the **Lucas–Lehmer test**. It's the workhorse behind most practical Mersenne-prime checks (and historically central to GIMPS). [con25a, MR24]

What experiments typically visualize

- **Growth with the exponent:** number of bits / digits of M_p as p grows.
- **Prime vs. composite behavior:** the Lucas–Lehmer residue s_{p-2} mod M_p across many prime exponents.
- **Factor patterns for composites:** quickly finding small factors of M_p (to avoid running LLT when a trivial factor exists).
- **Connections to perfect numbers:** if M_p is prime, then $2^{p-1}(2^p - 1)$ is an even perfect number (Euclid–Euler). [Cald.a]

For curated sequences (lists of exponents / known values), OEIS is a convenient "ground truth" reference. [Inc25a, Inc25b]

Practical numerical caveats

- **Always reduce modulo M_p in Lucas–Lehmer.**
  If you don't, the intermediate values explode in size (each squaring roughly doubles the bit-length).
- **Test the exponent first.**
  If p is composite, M_p is automatically composite, so there's no point running LLT.
- **Huge integers are fine, but you must be intentional.**
  Python's big integers won't overflow, but performance depends on bit-length and on the efficiency of modular squaring.
- **Separate "demo scale" from "real scale."**
  For educational experiments, keep p modest (e.g., $p \le 10^5$ is already huge for pure Python LLT). For large exponents, you'd rely on specialized implementations and careful FFT-based multiplication.

References

See *References*.

[Cal21, con25a, con25b, Inc25a, Inc25b]

3.4.20 Möbius function $\mu(n)$ and Mertens function $M(x)$ refresher

The Möbius function $\mu(n)$ is the main tool for inversion over divisors. The Mertens function $M(x) = \sum_{n \le x} \mu(n)$ is a classic "oscillatory" summatory function. See [Apo76], [Ten15].

Möbius function

Define $\mu(1) = 1$. For $n > 1$:

- $\mu(n) = 0$ if n is divisible by the square of a prime (not squarefree)
- otherwise $\mu(n) = (-1)^k$ where k is the number of distinct prime factors of n

So for squarefree $n = p_1 p_2 \cdots p_k$,

$$\mu(n) = (-1)^k.$$

Möbius inversion (most important identity)

If

$$F(n) = \sum_{d|n} f(d),$$

then

$$f(n) = \sum_{d|n} \mu(d)\, F\left(\frac{n}{d}\right).$$

Mertens function and a famous counterexample

The **Mertens function** is

$$M(x) = \sum_{n \le x} \mu(n).$$

A historic conjecture was $|M(x)| < \sqrt{x}$ for all $x \ge 1$ (the **Mertens conjecture**). It was disproved by Odlyzko and te Riele. [OtR85]

Why $M(x)$ is experiment-friendly

- $M(x)$ exhibits sign changes and irregular growth ("random walk-like" behavior).
- It is deeply connected to the Riemann zeta function and the Riemann hypothesis.

Typical experiment: plot $M(x)$ for growing x and compare to envelopes like $\pm\sqrt{x}$ or $\pm x^{1/2}\log x$.

Experiments in this repository

- **E105** — Mertens function M(x) at multiple scales (including rescaling views).

3.4.21 Prime-factor counting: $\omega(n)$ and $\Omega(n)$ refresher

Prime-factor counts are central examples of additive arithmetic functions. They are key in probabilistic number theory and in "typical behavior" experiments. See [Ten15].

Definitions

Let $n = \prod p_i^{\alpha_i}$.

- $\omega(n)$: number of **distinct** prime factors

$$\omega(n) = \#\{p : p \mid n\}.$$

- $\Omega(n)$: number of prime factors **with multiplicity**

$$\Omega(n) = \sum_i \alpha_i.$$

Examples:

- $12 = 2^2 \cdot 3$ has $\omega(12) = 2$ and $\Omega(12) = 3$.

Additivity

If $\gcd(a, b) = 1$ then

$$\omega(ab) = \omega(a) + \omega(b),$$

and Ω is even completely additive:

$$\Omega(ab) = \Omega(a) + \Omega(b) \quad \text{for all } a, b.$$

Typical size: Erdős–Kac phenomenon

A famous theorem of Erdős and Kac says (informally) that $\omega(n)$ behaves like a normal random variable with mean and variance $\log\log n$ after normalization. [ErdHosK40]

This is why histograms of $\omega(n)$ over ranges often look Gaussian.

Experiment ideas

- histogram of $\omega(n)$ for $n \le N$ and compare to a normal curve
- compare $\omega(n)$ vs. $\log\log n$ ("normal order")
- scatter $\Omega(n)$ vs. $\omega(n)$ to see multiplicities

Experiments in this repository

- **E094** — (n) vs Ω(n): Erdős–Kac side-by-side (distribution + scaling).
- **E095** — Squarefree conditioning ((n) 0): forces (n)=Ω(n).
- **E122** — Heatmap atlas of , , Ω (visual textures / patterns).

3.4.22 Partition function $p(n)$ refresher

The partition function $p(n)$ counts the number of ways to write n as a sum of positive integers, ignoring order (e.g. $4 = 4 = 3+1 = 2+2 = 2+1+1 = 1+1+1+1$ gives $p(4) = 5$). See [And84].

Definition

A **partition** of n is a multiset of positive integers with sum n. The partition function $p(n)$ is the number of such partitions.

Conventions:

- $p(0) = 1$ (empty partition)
- $p(n) = 0$ for $n < 0$

Generating function

The ordinary generating function is

$$\sum_{n\ge 0} p(n) q^n = \prod_{m\ge 1} \frac{1}{1-q^m}.$$

This identity is the starting point for many algorithms and proofs.

Growth

$p(n)$ grows very rapidly (roughly like $\exp(C\sqrt{n})$), so experiments often use log-scales or focus on modular patterns.

Experiment ideas

- compute $p(n)$ for $n \le N$ via dynamic programming and plot $\log p(n)$
- explore congruences (e.g. Ramanujan-type congruences)
- compare exact values to asymptotic approximations (later experiment)

3.4.23 Perfect numbers refresher

This page is a *beginner-friendly* refresher for experiments about **perfect numbers**. You only need basic number theory facts (divisors, primes) to follow it.

Core definitions

For a positive integer n, let

- $d \mid n$ mean "d divides n",
- $\sigma(n) = \sum_{d \mid n} d$ be the **sum-of-divisors function**,
- $s(n) = \sum_{d \mid n,\ d<n} d = \sigma(n) - n$ be the sum of **proper** divisors.

A number n is **perfect** iff its proper divisors sum to itself:

$$n \text{ is perfect} \iff s(n) = n \iff \sigma(n) = 2n.$$

Examples:

- 6 is perfect because $1 + 2 + 3 = 6$.
- 28 is perfect because $1 + 2 + 4 + 7 + 14 = 28$.

Key theorem: all even perfect numbers (Euclid–Euler)

A classic result completely characterizes **even** perfect numbers:

Euclid–Euler theorem.
An integer n is an even perfect number **iff**

$$n = 2^{p-1}(2^p - 1)$$

where $2^p - 1$ is prime (a **Mersenne prime**).

So every known perfect number is generated from a Mersenne prime exponent p. This is the main "generator" you'll use in experiments. [Cald.a, Voi98]

Why σ matters (multiplicativity)

If n factors as

$$n = \prod_{i=1}^{k} p_i^{a_i},$$

then

$$\sigma(n) = \prod_{i=1}^{k} \sigma\left(p_i^{a_i}\right) = \prod_{i=1}^{k} \frac{p_i^{a_i+1} - 1}{p_i - 1}.$$

This formula turns "sum all divisors" into a fast computation once you know the prime factorization.

The big open question: odd perfect numbers

It is **unknown** whether any **odd** perfect numbers exist. A large literature proves *constraints* (congruences, size bounds, number of prime factors, etc.). Experiments can explore these constraints (and why they make brute-force search unrealistic). [Guy04, OR14, Sto24]

What experiments typically visualize

Typical "lab" questions you can turn into plots and tables:

- **Verification by computation:** compute $\sigma(n)$ (via factorization) and check $\sigma(n) = 2n$ for candidates.
- **Generator experiment:** produce even perfect numbers from known Mersenne exponents p and confirm perfection.
- **Growth:** number of digits / bit length of $2^{p-1}(2^p - 1)$ as a function of p.

- **Divisor-function behavior:** compare $\sigma(n)/n$ (abundancy index) for random n vs. perfect numbers.
- **Odd constraints (toy models):** test necessary conditions on odd n and see how restrictive they are.

For data (lists of perfect numbers and exponents), OEIS is a convenient reference. [OEISFInc25a]

Practical numerical caveats

Even with correct math, computation has a few traps:

- **Factorization dominates.** Computing $\sigma(n)$ via divisors is slow unless you factor n. For large n, factorization becomes infeasible; prefer the Euclid–Euler generator for even perfect numbers.
- **Big integers are fine but expensive.** Python integers won't overflow, but operations on huge numbers scale with the number of bits (so be mindful in loops and plotting).
- **Prime testing vs. proof.** Testing that $2^p - 1$ is prime is nontrivial for large p. For experiments, use a curated list of known Mersenne prime exponents (e.g., from OEIS / GIMPS) rather than trying to discover new ones from scratch. [MersenneResearchIncGIMPS24, MersenneResearchIncGIMPS25]
- **Be explicit about definitions.** Some sources define "perfect" via $\sigma(n) = 2n$; others via proper divisors. Use one convention consistently in code and docs.

References

See *References*.

[Cald.a, Sto24, Voi98, OEISFInc25a]

3.4.24 Pretentious number theory refresher

"Pretentious" number theory replaces some zero-driven arguments by measuring how closely a multiplicative function *pretends* to be a simple model (typically n^{it} or a Dirichlet character). It is especially effective for **comparative** experiments: *which model explains the data best?*

Core definitions

Let f, g be multiplicative functions bounded by 1 in magnitude on primes. The **pretentious distance** up to x is [\mathbb{D}(f,g;x)^2 =\sum_{p\le x}\frac{1-\Re\big(f(p)\overline{g(p)}\big)}{p}.]

Interpretation:

- $\mathbb{D}(f, g; x)$ small means $f(p)$ and $g(p)$ "agree" on most primes (in a $1/p$-weighted sense).
- $\mathbb{D}(f, g; x)$ large means f cannot be explained well by g on primes.

A common model family is $g(n) = \chi(n)n^{it}$ with a Dirichlet character χ and real t.

What experiments usually visualize or measure

- Fit t to minimize $\mathbb{D}(f, n^{it}; x)$ and plot the best-fit $t(x)$.
- Compare distances $\mathbb{D}(f, \chi; x)$ across characters χ to see which congruence structure matches f.
- Use the distance to explain why partial sums of $f(n)$ may be large or small.

Practical numerical caveats

- Always define the prime cutoff (use the same x for fair comparisons).
- Distance is dominated by small primes; if you want to study "large-prime behavior," consider plotting contributions by prime ranges.

- When fitting t, the objective can have shallow minima; report robustness (e.g., multiple initial guesses).

References

See *References*.

[Gra09, GS07]

Experiments in this repository

- **E120** — Pretentious distance atlas for core multiplicative functions (, , /n, …).

3.4.25 Prime counting approximations: (x), Li(x), and R(x)

The prime-counting function (\pi(x)) counts primes up to (x). Two standard smooth approximations are:

- the **logarithmic integral** (\mathrm{Li}(x)), and
- **Riemann's R function** (R(x)) (a Möbius-weighted combination of (\mathrm{Li}(x^{1/n}))).

These approximations are central for numerical experiments around the Prime Number Theorem, error terms, and the visual "shape" of prime races.

Key ideas

- **PNT baseline:** (\pi(x)\sim \frac{x}{\log x}), and (\mathrm{Li}(x)) is often an excellent smooth baseline.
- **R(x) as a refinement:** R(x) folds in prime-power information and is closely connected to explicit-formula viewpoints.
- **Error terms:** plotting (\pi(x)-\mathrm{Li}(x)) or (\psi(x)-x) reveals oscillations and sign changes.

References

See *References*.

[Tit86, Wikipediacontributors25f]

3.4.26 Prime number races refresher

A **prime number race** compares prime counts in different residue classes, e.g. [\pi(x;4,3)\ \text{vs.}\ \pi(x;4,1).]

Although $\pi(x;q,a) \sim \mathrm{Li}(x)/\varphi(q)$ suggests "no long-term winner," finite ranges often show persistent preferences (biases).

The classic example: Chebyshev's bias mod 4

Empirically, for many ranges of x one observes [\pi(x;4,3) > \pi(x;4,1),] even though both are asymptotically equal.

Rubinstein–Sarnak analyze this phenomenon via the distribution of zeros of relevant L-functions (under standard hypotheses).

What experiments usually visualize or measure

- The difference curve $D(x) = \pi(x;q,a) - \pi(x;q,b)$ and its sign changes.
- The "race leaderboard" among several classes (e.g., mod 8 or mod 12).
- Histogram / empirical distribution of a normalized statistic sampled on a log-grid of x (choose a normalization and stick to it).

Practical numerical caveats

- Bias claims depend on *how you sample* x (linear vs log-grid); be explicit.
- With small cutoffs (say $x \le 10^7$), you'll see "apparent stability" that may later flip; that's expected.
- Counting primes is the bottleneck; keep an eye on runtime and memory, and record parameters in your manifest.

References

See *References*.

[GM06, RS94]

Experiments in this repository

- **E112** — Prime race curves (x;q,a) − (x;q,b): sign changes and time-in-lead.

3.4.27 Prime numbers refresher

This page is a *beginner-friendly* refresher for experiments about **prime numbers**. It is intentionally broad: it covers the prime-related ideas most commonly explored with computation (distribution, sieves, primality testing, gaps, and prime patterns).

Core definitions

Primes and composites

A positive integer $p \ge 2$ is **prime** if its only positive divisors are 1 and p. If $n \ge 2$ is not prime, it is **composite**.

Equivalently,

$$p \text{ is prime} \iff \forall\, a, b \in \mathbb{N} :\ p = ab \Rightarrow (a = 1 \text{ or } b = 1).$$

Greatest common divisor and coprimality

The **greatest common divisor** of integers a, b, written $\gcd(a, b)$, is the largest positive integer dividing both. We say a and b are **coprime** if $\gcd(a, b) = 1$.

Coprimality appears constantly in modular arithmetic and in many prime-related proofs.

Congruences (modular arithmetic)

For integers a, b and $m \ge 2$,

$$a \equiv b \pmod{m}$$

means m divides $a - b$. Many prime experiments (residue classes, sieves, primality tests) are naturally expressed modulo m.

Prime factorization (the "atomic" viewpoint)

The **Fundamental Theorem of Arithmetic** says every integer $n \ge 2$ factors uniquely (up to ordering) as

$$n = \prod_{i=1}^{k} p_i^{\alpha_i},$$

where the p_i are distinct primes and the α_i are positive integers. This makes primes the "building blocks" of the integers. [HW08b]

Key quantitative language

Infinitely many primes

Euclid's classic argument shows there are infinitely many primes. This matters experimentally because it justifies questions like "how often do primes occur as numbers grow?" [HW08b]

The prime counting function and the Prime Number Theorem

Let $\pi(x)$ be the number of primes $\leq x$. The **Prime Number Theorem** (PNT) states

$$\pi(x) \sim \frac{x}{\log x} \quad (x \to \infty).$$

Informally: primes thin out, and the "typical" gap size near x is about $\log x$. [Apo76]

For experiments, two very common comparisons are:

- $\pi(x)$ vs. $x/\log x$,
- $\pi(x)$ vs. $\mathrm{Li}(x)$ (the logarithmic integral), which is often a better approximation in practice. [Apo76, RS62]

Explicit bounds (useful sanity checks)

Many sources provide explicit inequalities for $\pi(x)$, the nth prime p_n, and related functions. These bounds are useful to validate computations and pick experiment ranges safely. [Axl19, Dus18, RS62]

Computational building blocks

Generating primes: sieves

If you need *all* primes up to a bound N, a sieve is usually best.

Sieve of Eratosthenes (idea). Start with a boolean array "possibly prime," then repeatedly cross out multiples of each found prime. Time is roughly $O(N \log \log N)$, memory is $O(N)$ booleans.

Segmented sieve. For large N, store only a window $[L, R]$ at a time; this keeps memory bounded while preserving speed. Segmented sieves are often the right choice once N grows beyond typical RAM-friendly sizes. [CP05]

Testing primality (single numbers)

If you need to test the primality of individual (possibly large) integers, common strategies are:

- **Trial division** up to $\sqrt{n}$ (great for small n, too slow for large),
- **Probabilistic tests** such as **Miller–Rabin**, fast and extremely reliable with good parameters, [Rab80]
- **Deterministic polynomial-time** primality testing (AKS), important theoretically but rarely used for practical large-number work. [AKS04]

In an experiment repo, it is common to use Miller–Rabin for speed and then either: (1) accept "probable prime" status (with a stated error bound) or (2) confirm with stronger checks for the sizes you care about. [CP05]

Factorization (often the bottleneck)

Many arithmetic functions become easy once a number is factored, but factoring large integers is hard. Even "toy" factorization methods (e.g., Pollard's rho) make excellent experiments because performance varies wildly with input structure. [CP05]

Prime gaps and prime patterns

Prime gaps

Let p_n be the nth prime. The **prime gap** sequence is

$$g_n = p_{n+1} - p_n.$$

Heuristically, "typical" gaps near p are size $\log p$, but gaps vary a lot. Two natural experimental viewpoints are:

- **local statistics**: histogram of g_n in a range,
- **record gaps**: the largest gap seen up to x (maximal gaps / first occurrences). [Cald.b, Cramer36, Nic99]

A common normalization is $g_n / \log p_n$, which helps compare different scales.

Twin, cousin, and sexy primes (prime pairs)

A **prime pair** is a pair of primes $(p, p + d)$ with fixed even difference d:

- **twin primes**: $d = 2$,
- **cousin primes**: $d = 4$,
- **sexy primes**: $d = 6$.

These are all instances of "prime constellations."

Prime k-tuples (constellations)

A **prime k-tuple** is a set of offsets $\{h_1, \ldots, h_k\}$ such that $p + h_i$ are simultaneously prime. Hardy–Littlewood heuristics predict asymptotic counts for many such patterns (including twins). [HL23]

Modern sieve breakthroughs show **bounded prime gaps** exist (there are infinitely many g_n below some constant), but we still do not know whether the twin prime conjecture is true. [May15, Zha14, DHJPolymath14]

Primes in structure (residue classes and progressions)

Residue classes

Primes (except 2) are odd, so they lie in residue classes modulo 2. More generally, you can study how primes distribute among residue classes modulo q. Experiments here include "prime races" and small biases in counts.

Arithmetic progressions of primes

A classical result (Dirichlet's theorem) says that if $\gcd(a, q) = 1$, then the arithmetic progression

$$a,\ a + q,\ a + 2q, \ldots$$

contains infinitely many primes. A landmark modern result (Green–Tao) shows primes contain arbitrarily long arithmetic progressions. [GT08]

For experiments, you can stay at an elementary level (searching for 3-term or 4-term prime APs in ranges) while still getting meaningful patterns.

Other prime families you may include (optional)

These are often nice "side quests" that still fit under "Prime Numbers":

- **Sophie Germain primes**: p prime and $2p + 1$ prime,
- **safe primes**: q prime with $(q - 1)/2$ prime,

- **Mersenne primes**: $2^p - 1$ prime (you already have a dedicated page), [con25b]
- **palindromic / repunit primes**: "pattern-based" primes useful for search experiments.

What experiments typically visualize

Typical "prime numbers lab" plots and tables include:

- **Prime density:** $\pi(x)$ vs. $x/\log x$, and error curves $\pi(x) - x/\log x$. [Apo76]
- **Bounds and approximations:** compare $\pi(x)$ with explicit inequalities (sanity checks). [Dus18, RS62]
- **Sieve scaling:** runtime/memory vs. N for classic vs. segmented sieves. [CP05]
- **Gaps:** histograms of g_n, normalized gaps $g_n/\log p$, record gaps vs. $\log^2 x$. [Cramer36, Nic99]
- **Prime pairs:** counts of twin/cousin/sexy primes in $[1, x]$, compared to Hardy–Littlewood style heuristics. [HL23]
- **Bounded gaps story:** "what bound was known when?" (Zhang → Maynard → Polymath) plus empirical gap data. [May15, Zha14, DHJPolymath14]
- **Primality tests:** speed vs. integer size (bits), and false-positive rates for weak tests vs. Miller–Rabin. [AKS04, Rab80]
- **Progressions:** frequency of 3-term prime APs in ranges; maximal AP length in a window. [GT08]

For "ground truth" sequences and curated lists, OEIS and PrimePages are practical reference points. [Cald.b, OEISFInc25b, OEISFInc25c]

Practical numerical caveats

- **Pick the right primitive:** if you need *many* primes up-to N, use a sieve; if you need primality of a few large numbers, use primality testing.
- **Memory matters for sieves:** a naive boolean list of length N can be huge; segmented sieves avoid this.
- **State what "prime" means in code:** if you use Miller–Rabin, your results are "probable primes" unless you add a deterministic guarantee for your size range. [Rab80]
- **Avoid floating-point traps:** use natural logs, guard against log(0), and remember that $\log x$ changes slowly.
- **Be precise with pair-counting:** decide whether you count pairs by their smaller prime p (common), and avoid double-counting.
- **Separate discovery from verification:** for pattern-hunting (e.g., long APs), do fast screening first, then verify candidates carefully.

References

See *References*.

[AKS04, Apo76, Cramer36, CP05, Dus18, GT08, HL23, HW08b, May15, Rab80, RS62, Zha14, DHJPolymath14]

3.4.28 Primes in arithmetic progressions refresher

For integers $q \geq 1$ and a, define [\pi(x;q,a) := #{p \le x : p \text{ prime and } p \equiv a \pmod q}.]

Core statements

Dirichlet's theorem (existence)

If $\gcd(a, q) = 1$, then there are infinitely many primes $p \equiv a \pmod q$.

Prime number theorem in arithmetic progressions (equidistribution)

For fixed q and $\gcd(a, q) = 1$, [\pi(x;q,a) \sim \frac{\mathrm{Li}(x)}{\varphi(q)} \quad (x\to\infty).]

So, asymptotically, primes split evenly among the reduced residue classes.

What experiments usually visualize or measure

- Raw counts $\pi(x; q, a)$ for small moduli q (2,3,4,5,8,10,12,...).
- Differences (races): $\pi(x; q, a) - \pi(x; q, b)$.
- "Leader changes": values of x where the sign of the difference flips.

Practical numerical caveats

- Use a segmented sieve for $\pi(x; q, a)$ if x gets large.
- For small x, visual phenomena are dominated by finite-size effects; that's fine, but label plots honestly (e.g., "up to 10^7").
- If you use $\mathrm{Li}(x)$ for normalization, document the approximation you use.

References

See *References*.

[Dav00, LD37]

Experiments in this repository

- **E113** — First prime in each residue class (smallest p a mod q for all a (/q)×).

3.4.29 Primorials

The primorial of a prime p_k is

[p_k# = \prod_{i=1}^{k} p_i]

(the product of the first k primes). Primorials are a natural "smoothing knob" in experimental number theory: they amplify small prime structure and appear in constructions like Euclid numbers $p_k\# \pm 1$. [PrimePUTM25b, TheOFInc25d]

Key facts

- **Growth:** $\log(p_k\#) = \sum_{i \le k} \log p_i$; by the prime number theorem this is asymptotic to p_k (in a coarse sense). [HW08a]
- **Euclid numbers:** $p_k\# + 1$ is coprime to all primes $\le p_k$, but is usually composite (so: "Euclid's proof does *not* generate primes"). [PrimePUTM25b]

What to experiment with

- **Euclid-number factorization:** For $E_k = p_k\# \pm 1$, try to find small factors and compare +1 vs. -1.
- **Primorial wheels:** Use primorials to build "wheel sieves" and benchmark against a simple sieve.
- **Primorial primes / plus/minus primes:** Scan for primes of the form $p_k\# \pm 1$ and visualize rarity.

References

See Prime Pages (UTM) [PrimePUTM25b], The OEIS Foundation Inc. [TheOFInc25d], Wikipedia contributors [Wikipediacontributors25g].

3.4.30 Riemann zeta function (s)

The **Riemann zeta function** (s) is the Dirichlet series [\zeta(s)=\sum_{n\ge 1} \frac{1}{n^s},] which converges for (\Re(s)>1) and extends (by analytic continuation) to a meromorphic function on (\mathbb{C}) with a single simple pole at (s=1).

Key ideas

- **Euler product (primes):** for (\Re(s)>1), [\zeta(s)=\prod_{p\ \text{prime}}\frac{1}{1-p^{-s}},] linking (s) directly to prime distribution.
- **Functional equation:** (s) satisfies a symmetry relating (s) and (1-s), which is central for studying zeros.
- **Zeros:** (s) has *trivial zeros* at negative even integers and *nontrivial zeros* in the critical strip (0<\Re(s)<1), conjecturally all on the critical line (\Re(s)=\tfrac12) (Riemann Hypothesis).

Why it matters in this project

Many "analytic prime number theory" numerics (prime counting approximations, explicit formulas, prime races, etc.) are most naturally expressed in terms of (s), its logarithmic derivative, and its zeros.

Experiments in this repository

- **E117** — -factor functional-equation consistency checks on a grid of s values.
- **E118** — Partial Euler product for (s): where it breaks as Re(s) decreases.

References

See *References*.

[Edw74, Ivic85, Odl92, Tit86, Wikipediacontributors25h]

3.4.31 Semiprimes

A semiprime is an integer that is the product of two primes (not necessarily distinct), i.e.

[n = pq \quad (p, q \text{ prime}).]

Semiprimes sit at the center of practical factorization hardness and are the basic objects behind RSA-style moduli. [RSA78, TheOFInc25c]

Key facts

- **Sequence:** The semiprimes form OEIS sequence A001358. [TheOFInc25c]
- **RSA context:** RSA uses $N = pq$ and the difficulty of factoring N (when p, q are large and random-looking). [RSA78]
- **Special cases:** $p = q$ yields squares of primes; $p \neq q$ yields "RSA-like" semiprimes.

What to experiment with

- **Distribution:** Count semiprimes up to X and compare empirical growth with simple heuristic approximations.
- **Factorization methods:** Benchmark trial division, Pollard's rho, and (for larger sizes) ECM bindings if you later allow optional deps.

- **RSA-style generation:** Generate balanced semiprimes (p, q same bitlength) and compare factorization difficulty vs unbalanced ones.
- **Smoothness:** Explore how often $p-1$ or $q-1$ is smooth (motivates $p-1$ factoring methods).

References

See Rivest *et al.* [RSA78], The OEIS Foundation Inc. [TheOFInc25c].

3.4.32 Taylor series refresher

This page is a *beginner-friendly* refresher for experiments that use Taylor polynomials. You only need basic calculus (derivatives) to follow it.

Taylor polynomial

Assume f has enough derivatives near x_0 (this is true for $\sin(x)$, $\cos(x)$, polynomials, exponentials, etc.). The Taylor polynomial of degree n around x_0 is

$$T_n(x; x_0) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k.$$

Intuition: $T_n(x; x_0)$ is the polynomial that matches $f(x_0)$ and the first n derivatives at x_0. It is usually accurate when x is close to x_0.

For $f(x) = \sin(x)$, the derivatives cycle, and around $x_0 = 0$ this becomes

$$\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots.$$

Truncation error and the remainder

The approximation error is the remainder

$$R_n(x; x_0) = f(x) - T_n(x; x_0).$$

Under standard conditions, Taylor's theorem gives a remainder representation. One common form is the (Lagrange) remainder:

$$R_n(x; x_0) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)^{n+1} \quad \text{for some } \xi \text{ between } x \text{ and } x_0.$$

This is the key qualitative message for experiments like E001:

- the factor $(x - x_0)^{n+1}$ makes the method **local** (good near x_0, potentially bad far away),
- increasing n helps most where $|x - x_0|$ is small.

What experiments typically visualize

In a numerical experiment, you often look at

- absolute error: $|R_n(x; x_0)|$
- relative error: $|R_n(x; x_0)|/|f(x)|$ (careful near zeros of f)

and plot them across a domain to see where the approximation is reliable.

Practical numerical caveats

Even when the mathematics are correct, computation can mislead:

- large $|x - x_0|$ and high n can produce huge intermediate terms,
- subtractive cancellation can reduce accuracy,
- floating-point rounding can dominate before the theoretical truncation error does.

A common "extension" experiment is to repeat the same plots using higher precision arithmetic to separate *truncation error* from *rounding error*.

Introductory reading

If you want a longer, *beginner-friendly* treatment (beyond this refresher), these are good starting points:

- A quick overview / definitions and examples: [Wikipediacontributors25j].
- A rigorous calculus textbook with a clean presentation of Taylor's theorem and remainders: [Apo91].
- A proof-oriented classic (slower, deeper): [Spi08].
- For the numerical viewpoint (truncation vs. rounding error): [BFB15].

References

See *References*.

[Apo91, BFB15, Spi08, Wikipediacontributors25j]

3.4.33 von Mangoldt $\Lambda(n)$ and Chebyshev functions refresher

The von Mangoldt function concentrates on prime powers and is central in analytic number theory. Chebyshev functions summarize prime distributions and are used in proofs of the Prime Number Theorem. See [MV06] and [Ten15].

von Mangoldt function

Define

$$\Lambda(n) = \begin{cases} \log p, & \text{if } n = p^k \text{ for a prime } p \text{ and } k \geq 1, \\ 0, & \text{otherwise.} \end{cases}$$

So $\Lambda(n)$ "picks out" primes and prime powers.

Chebyshev functions

Two common Chebyshev functions:

- $\theta(x) = \sum_{p \leq x} \log p$
- $\psi(x) = \sum_{n \leq x} \Lambda(n)$

Since ψ sums $\log p$ over prime powers, it is smoother and often easier in analysis.

Why these matter

- $\psi(x) \sim x$ is essentially equivalent to the Prime Number Theorem.
- Λ is tied to the logarithmic derivative of $\zeta(s)$ (via Dirichlet series).

Experiment ideas

- plot $\psi(x)$ and compare to x
- plot $\theta(x)$ and compare to x
- compare contributions from primes vs. higher prime powers in $\psi(x)$

Experiments in this repository

- **E103** — Chebyshev (x) jumps at prime powers (Λ support).
- **E104** — Von Mangoldt Λ statistics (support, value distribution, summatory behavior).
- **E119** — (x) − x oscillations (prime-weighted deviation view).

3.4.34 Wieferich primes

A Wieferich prime (base 2) is a prime p such that

[2^{p-1} \equiv 1 \pmod{p^2}.]

Only two are currently known: 1093 and 3511. [TheOFInc25b, Wikipediacontributors25k]

Key facts

- **Origin (Fermat's Last Theorem, first case):** Wieferich proved in 1909 that if the first case of FLT fails for an odd prime exponent p, then p must be a Wieferich prime (base 2). [Wie09]
- **Rarity:** Heuristics suggest Wieferich primes are very sparse; whether infinitely many exist is open. [Wikipediacontributors25k]
- **Generalization:** One can define Wieferich primes for any base a via $a^{p-1} \equiv 1 \pmod{p^2}$. [PrimePUTM25c]

What to experiment with

- **Fast search:** Implement modular exponentiation (pow with mod) to test primes up to a bound and recover $1093, 3511$.
- **Near-Wieferich primes:** Measure the p-adic valuation $v_p(2^{p-1} - 1)$ and look for unusually large values.
- **Connections:** Explore overlaps with other "rare prime" phenomena (e.g., Wall–Sun–Sun, Wilson) as *tags* rather than hard coupling.

References

See Wieferich [Wie09], The OEIS Foundation Inc. [TheOFInc25b], Wikipedia contributors [Wikipediacontributors25k].

3.5 Getting started

This project uses an **uv-only** workflow and a small Makefile wrapper to run everything consistently.

3.5.1 Prerequisites

You need:

- **Python 3.14**
- **uv** on your PATH
- **GNU Make**

Windows notes

- Install GNU Make (e.g., via Chocolatey).
- You can run everything from `cmd.exe`, PowerShell, or from WSL.

3.5.2 Verify tools

From the repository root:

```
make uv-check
make python-check
make python-info
```

3.5.3 First-time setup

Create a virtual environment and install dependencies:

```
make venv
make install-dev
make install-docs
```

3.5.4 Run the full development chain

```
make final
```

`make final` runs:

- formatting (Ruff)
- linting (Ruff)
- type checking (mypy)
- tests (pytest)
- documentation (sphinx)

Documentation can be built separately via:

```
make docs
```

3.5.5 Run an experiment

Example (E001):

```
make run EXP=e001 ARGS="--seed 1"
```

See the experiment overview here: *Experiments Gallery*.

3.5.6 Build documentation locally

```
make docs
```

Output:

- `docs/_build/html`

3.5.7 Troubleshooting

`error: Failed to spawn: sphinx-build`

Sphinx is not installed in the uv environment.

Fix:

```
make install-docs
make docs
```

`make python-check` **fails**

The repo enforces Python **3.14**. Install Python 3.14, then recreate the environment:

```
make clean-venv
make venv
make install-dev
make install-docs
```

3.6 Development

This page describes the development workflow and the conventions used in this repository.

3.6.1 Workflow overview

Use the Makefile targets for everything. They wrap `uv run ...` so you do not need to activate a virtual environment manually.

Typical day-to-day:

```
make final
```

Documentation-only build:

```
make docs
```

Clean caches and build artifacts:

```
make clean
```

Remove the virtual environment (full reset):

```
make clean-venv
```

3.6.2 Makefile workflow

The Makefile is the **single entry point** for development tasks (env setup, quality checks, docs builds, and running experiments).

- Prefer `make final` before pushing.
- Prefer `make docs` to validate documentation changes.
- Use `make run EXP=<id>` to execute an experiment and write artifacts to `out/<id>/`.

Dependency groups

This summary is included from the Makefile documentation:

Dependencies are organized via `pyproject.toml` extras:

- **default**: runtime dependencies needed to run the package
- **dev** (`--extra dev`): developer tooling (ruff, mypy, pytest, etc.)
- **docs** (`--extra docs`): documentation tooling (sphinx, furo, myst-parser, sphinx-design, bibtex, …)

What the Makefile does

- Most dev targets run via:

```
uv run --extra dev ...
```

- Documentation targets run via:

```
uv run --extra docs ...
```

Why this matters

CI and local development can install only what they need:

- fast "dev chain" (no docs):

```
uv sync --extra dev
```

- docs build:

```
uv sync --extra docs
```

Run logs

Experiments live under `mathxlab/experiments/` and can be run either directly with Python or through Make targets (if available in your Makefile).

Run an experiment module directly

```
uv run python -m mathxlab.experiments.e001
```

Typical output locations (convention)

Depending on the experiment runner implementation, outputs are usually placed in:

- `out/e###/` (generated artifacts, figures, manifests)
- `docs/gallery/`, `docs/reports/`, `docs/manifests/` (published snapshots)

If you add new experiments, keep the numbering stable (`e001`, `e002`, …) so the gallery and documentation can remain consistent.

Common workflows

Task	Command
Complete quality check	`make final`
Build HTML docs	`make docs-html`
Run tests	`make test`
Clean all artifacts	`make clean`
Reset environment	`make clean && make venv`

Full reference

For the complete target-by-target reference and troubleshooting, see makefile.

3.6.3 Formatting, linting, typing, tests

- Formatting: **Ruff formatter**
- Linting: **Ruff**
- Typing: **mypy**
- Tests: **pytest**

CI formatting behavior

In CI, formatting runs in check mode (`ruff format --check`). Locally it formats in place.

3.6.4 Experiment authoring guidelines

When adding a new experiment:

1. Add a new module under `mathxlab/experiments/`, e.g. `e002_...py`.
2. Prefer deterministic outputs:
   - `--seed` argument if randomness is involved
   - write results to a single `--out` directory
3. Keep the experiment runnable as a module:
   - `python -m mathxlab.experiments.e002`
4. Update the docs:
   - add a short entry to *Experiments Gallery*
   - optionally add a dedicated page under `docs/experiments/` later

3.6.5 Documentation

Docs are built with Sphinx + MyST.

Build locally:

```
make install-docs
make docs
```

Deployed website:

- GitHub Pages from the `docs` workflow

3.6.6 Contributing (high-level)

- Create a feature branch.
- Open a PR against `main`.
- CI must pass before merge.
- Keep PRs small and well-scoped.

3.7 References

3.8 PDF download

A PDF build of this documentation is generated by GitHub Actions and published alongside the HTML site.

- Download PDF

If the link is missing, it usually means you are viewing an older deployment or the PDF build step failed.

BIBLIOGRAPHY

[Expa] Experimental mathematics (project euclid archive). URL: https://projecteuclid.org/journals/experimental-mathematics (visited on 2025-12-22).

[Expb] Experimental mathematics lab (university of luxembourg). URL: https://math.uni.lu/eml/ (visited on 2025-12-22).

[Expc] Experimental mathematics website. URL: https://www.experimentalmath.info (visited on 2025-12-22).

[Exp92] Experimental mathematics (journal). 1992. URL: https://www.tandfonline.com/journals/uexm20.

[AKS04] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. PRIMES is in P. *Annals of Mathematics*, 160(2):781–793, 2004. URL: https://annals.math.princeton.edu/2004/160-2/p12 (visited on 2025-12-27), doi:10.4007/annals.2004.160.781.

[AErdHos44] L. Alaoglu and Paul Erdős. On highly composite and similar numbers. *Transactions of the American Mathematical Society*, 56(3):448–469, 1944. doi:10.1090/S0002-9947-1944-0011087-2.

[AGP94] W. R. Alford, Andrew Granville, and Carl Pomerance. There are infinitely many carmichael numbers. *Annals of Mathematics*, 139(3):703–722, 1994. URL: https://annals.math.princeton.edu/1994/139-3/p10.

[And84] George E. Andrews. *The Theory of Partitions*. Volume 2 of Encyclopedia of Mathematics and its Applications. Cambridge University Press, 1984. ISBN 978-0-521-63766-4. doi:10.1017/CBO9780511608650.

[Apo76] Tom M. Apostol. *Introduction to Analytic Number Theory*. Springer, 1976. URL: https://link.springer.com/book/10.1007/978-1-4757-5579-4 (visited on 2025-12-27), doi:10.1007/978-1-4757-5579-4.

[Apo91] Tom M. Apostol. *Calculus, Volume 1*. John Wiley & Sons, 2 edition, 1991. ISBN 9780471000051. URL: https://books.google.com/books/about/Calculus\TU\textbackslash{}_Volume\TU\textbackslash{}_1.html?id=o2D4DwAAQBAJ.

[Arn15] Vladimir I. Arnold. *Experimental Mathematics*. MSRI Mathematical Circles Library. American Mathematical Society, 2015. ISBN 9780821894163. URL: https://bookstore.ams.org/msri-13/ (visited on 2025-12-22).

[AM14] Jeremy Avigad and Rebecca Morris. The concept of “character” in dirichlet's theorem on primes in an arithmetic progression. *Archive for History of Exact Sciences*, 68(3):265–326, 2014. URL: https://arxiv.org/abs/1209.3657 (visited on 2025-12-29), doi:10.1007/s00407-013-0126-0.

[Axl19] Christian Axler. New estimates for the nth prime number. *Journal of Integer Sequences*, 22(4):Article 19.4.2, 2019. URL: https://cs.uwaterloo.ca/journals/JIS/VOL22/Axler/axler17.pdf (visited on 2025-12-27).

[Axl24] Christian Axler. Effective estimates for some functions defined over primes. *Integers*, 24:A34, 2024. URL: https://math.colgate.edu/~integers/y34/y34.pdf (visited on 2025-12-27).

[BB05] David H. Bailey and Jonathan M. Borwein. Experimental mathematics: examples, methods and implications. *Notices of the American Mathematical Society*, 52(5):502–514, 2005. URL: https://www.ams.org/notices/200505/fea-borwein.pdf.

[BBC04] David H. Bailey, Jonathan M. Borwein, and Richard E. Crandall. Ten problems in experimental mathematics. *Experimental Mathematics*, 13(2):193–207, 2004.

[BEW98] Bruce C. Berndt, Ronald J. Evans, and Kenneth S. Williams. *Gauss and Jacobi Sums*. Wiley-Interscience, 1998. ISBN 9780471128076.

[BvdPSZ14] Jonathan Borwein, Alf van der Poorten, Jeffrey Shallit, and Wadim Zudilin. *Neverending Fractions: An Introduction to Continued Fractions*. Volume 23 of Australian Mathematical Society Lecture Series. Cambridge University Press, 2014. ISBN 9780521186490. URL: https://www.cambridge.org/core/books/neverending-fractions/A3900DAB483D65CE6CB960A6B71226EE (visited on 2025-12-22).

[Bor05] Jonathan M. Borwein. The experimental mathematician: the pleasure of discovery and the role of proof. *International Journal of Computers for Mathematical Learning*, 10:75–108, 2005. doi:10.1007/s10758-005-6244-3.

[Bor09] Jonathan M. Borwein. *The Crucible: An Introduction to Experimental Mathematics*. A K Peters, Wellesley, MA, USA, 2009. ISBN 9781568813438.

[BB08] Jonathan M. Borwein and David H. Bailey. *Mathematics by Experiment: Plausible Reasoning in the 21st Century*. A K Peters, Wellesley, MA, USA, 2 edition, 2008. ISBN 9781568814421.

[BBG04] Jonathan M. Borwein, David H. Bailey, and Roland Girgensohn. *Experimentation in Mathematics: Computational Paths to Discovery*. A K Peters, Natick, MA, USA, 2004. ISBN 9781568811369. doi:10.1201/9781439864197.

[BBG+07] Jonathan M. Borwein, David H. Bailey, Roland Girgensohn, David Luke, and Victor H. Moll. *Experimental Mathematics in Action*. A K Peters, Wellesley, MA, USA, 2007. ISBN 9781568812717. URL: https://carmamaths.org/resources/jon/Preprints/Books/EMA/ema.pdf (visited on 2025-12-22).

[BB04] Jonathan M. Borwein and Richard P. Brent. *Inquiries into Experimental Mathematics*. A K Peters, Wellesley, MA, USA, 2004. ISBN 9781568812113.

[BFB15] Richard L. Burden, J. Douglas Faires, and Annette M. Burden. *Numerical Analysis*. Cengage Learning, 10 edition, 2015. ISBN 9781305253667. URL: https://www.cengage.com/c/numerical-analysis-10e-faires/9781305253667/.

[Cald.a] Chris Caldwell. Characterizing all even perfect numbers. n.d. PrimePages (The Prime Database), proof note on the Euclid–Euler characterization. URL: https://t5k.org/notes/proofs/EvenPerfect.html (visited on 2025-12-25).

[Cal21] Chris K. Caldwell. Mersenne primes: history, theorems and lists. Online, 2021. PrimePages reference page collecting history, key theorems, and curated tables related to Mersenne primes and perfect numbers. URL: https://primes.utm.edu/mersenne/ (visited on 2025-12-27).

[Cald.b] Chris K. Caldwell. Prime gaps and related records. Online, n.d. PrimePages (The Prime Database), curated notes and links on prime gaps and record computations. URL: https://t5k.org/notes/primegaps.html (visited on 2025-12-27).

[Car12] R. D. Carmichael. On composite numbers p which satisfy the fermat congruence a^P-1 \equiv 1 (mod p). *American Mathematical Monthly*, 19(2):22–27, 1912. URL: https://www.jstor.org/stable/2974142.

[Con04] Keith Conrad. Korselt's criterion for carmichael numbers. https://kconrad.math.uconn.edu/blurbs/ugradnumthy/carmichael.pdf, 2004. Online lecture note; accessed 2025-12-28.

[con25a] Wikipedia contributors. Lucas–lehmer primality test — wikipedia, the free encyclopedia. Online, 2025. Revision as of 01:16, 31 October 2025. Abstract: Description and correctness of the Lucas–Lehmer test used to prove primality of Mersenne numbers $M_p=2^p-1$. URL:

https://en.wikipedia.org/w/index.php?oldid=1319643028&title=Lucas%E2%80%93Lehmer_primality_test (visited on 2025-12-27).

[con25b] Wikipedia contributors. Mersenne prime — wikipedia, the free encyclopedia. Online, 2025. Revision as of 20:10, 11 December 2025. Abstract: Overview of Mersenne primes (numbers of the form 2^p-1), their connection to perfect numbers, and known results and records. URL: https://en.wikipedia.org/w/index.php?oldid=1326946346&title=Mersenne_prime (visited on 2025-12-27).

[Cramer36] Harald Cramér. On the order of magnitude of the difference between consecutive prime numbers. *Acta Arithmetica*, 2(1):23–46, 1936. URL: https://www.impan.pl/en/publishing-house/journals-and-series/acta-arithmetica/all/2/1/93277/on-the-order-of-magnitude-of-the-difference-between-consecutive-prime-numbers (visited on 2025-12-27), doi:10.4064/aa-2-1-23-46.

[CP05] Richard Crandall and Carl Pomerance. *Prime Numbers: A Computational Perspective*. Springer, 2005. URL: https://link.springer.com/book/10.1007/978-1-4684-9316-0 (visited on 2025-12-27), doi:10.1007/978-1-4684-9316-0.

[Dav00] Harold Davenport. *Multiplicative Number Theory*. Volume 74 of Graduate Texts in Mathematics. Springer, New York, 3 edition, 2000. ISBN 978-0-387-95097-6. URL: https://link.springer.com/book/10.1007/978-1-4757-5927-3 (visited on 2025-12-29), doi:10.1007/978-1-4757-5927-3.

[Dus18] Pierre Dusart. Explicit estimates of some functions over primes. *The Ramanujan Journal*, 45(1):227–251, 2018. URL: https://link.springer.com/article/10.1007/s11139-016-9839-4 (visited on 2025-12-27), doi:10.1007/s11139-016-9839-4.

[Edw74] Harold M. Edwards. *Riemann's Zeta Function*. Princeton University Press, 1974. ISBN 9780691113852.

[ErdHosK40] Paul Erdős and Mark Kac. The gaussian law of errors in the theory of additive number theoretic functions. *American Journal of Mathematics*, 62:738–742, 1940. doi:10.2307/2371483.

[ErdHosPS91] Paul Erdős, Carl Pomerance, and Eric Schmutz. Carmichael's lambda function. *Acta Arithmetica*, 58(4):363–385, 1991. URL: http://eudml.org/doc/206359.

[FI10] John Friedlander and Henryk Iwaniec. *Opera de Cribro*. Volume 57 of Colloquium Publications. American Mathematical Society, 2010. URL: https://bookstore.ams.org/coll-57/ (visited on 2025-12-27), doi:10.1090/coll/057.

[Gra09] Andrew Granville. Pretentiousness in analytic number theory. *Journal de Théorie des Nombres de Bordeaux*, 21(1):159–173, 2009. doi:10.5802/jtnb.664.

[GM06] Andrew Granville and Greg Martin. Prime number races. *The American Mathematical Monthly*, 113(1):1–33, 2006. URL: https://arxiv.org/abs/math/0408319 (visited on 2025-12-29), doi:10.1080/00029890.2006.11920275.

[GS07] Andrew Granville and Kannan Soundararajan. Large character sums: pretentious characters and the Pólya–Vinogradov theorem. *Journal of the American Mathematical Society*, 20(2):357–384, 2007. doi:10.1090/S0894-0347-06-00549-1.

[GT08] Ben Green and Terence Tao. The primes contain arbitrarily long arithmetic progressions. *Annals of Mathematics*, 167(2):481–547, 2008. URL: https://annals.math.princeton.edu/2008/167-2/p01 (visited on 2025-12-27), doi:10.4007/annals.2008.167.481.

[Guy04] Richard K. Guy. *Unsolved Problems in Number Theory*. Springer, 3 edition, 2004. ISBN 9780387208602. URL: https://link.springer.com/book/10.1007/978-0-387-26677-0, doi:10.1007/978-0-387-26677-0.

[HR74] Heini Halberstam and Hans-Egon Richert. *Sieve Methods*. Academic Press, 1974. URL: https://www.sciencedirect.com/book/9780123182501/sieve-methods (visited on 2025-12-27).

[HL23] G. H. Hardy and J. E. Littlewood. Some problems of `partitio numerorum'; III: on the expression of a number as a sum of primes. *Acta Mathematica*, 44:1–70, 1923. URL:

https://link.springer.com/article/10.1007/BF02403921 (visited on 2025-12-27), doi:10.1007/BF02403921.

[HW08a] G. H. Hardy and E. M. Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, 6 edition, 2008. ISBN 9780199219858.

[HW08b] G. H. Hardy and E. M. Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, 6 edition, 2008. ISBN 9780199219865. URL: https://global.oup.com/ukhe/product/an-introduction-to-the-theory-of-numbers-9780199219865 (visited on 2025-12-27).

[Inc25a] OEIS Foundation Inc. A000043 — mersenne exponents. Online, 2025. OEIS entry for primes p such that 2^p-1 is prime (exponents of Mersenne primes). URL: https://oeis.org/A000043 (visited on 2025-12-27).

[Inc25b] OEIS Foundation Inc. A001348 — mersenne numbers: 2^p-1 where p is prime. Online, 2025. OEIS entry for the Mersenne numbers sequence 2^p-1 with prime exponents p. URL: https://oeis.org/A001348 (visited on 2025-12-27).

[IR90] Kenneth Ireland and Michael Rosen. *A Classical Introduction to Modern Number Theory*. Graduate Texts in Mathematics. Springer, New York, 2 edition, 1990. ISBN 978-0-387-97329-6. URL: https://link.springer.com/book/10.1007/978-1-4757-2103-4 (visited on 2025-12-29), doi:10.1007/978-1-4757-2103-4.

[Ivic85] Aleksandar Ivić. *The Riemann Zeta-Function: Theory and Applications*. John Wiley & Sons, 1985.

[IK04] Henryk Iwaniec and Emmanuel Kowalski. *Analytic Number Theory*. Volume 53 of Colloquium Publications. American Mathematical Society, 2004. URL: https://bookstore.ams.org/coll-53/ (visited on 2025-12-27), doi:10.1090/coll/053.

[KvrivzekLvSomer13] Michal Kř\'ıžek, Florian Luca, and Ladislav Šomer. *17 Lectures on Fermat Numbers: From Number Theory to Geometry*. CMS Books in Mathematics. Springer, 2013. ISBN 9781461465250. doi:10.1007/978-1-4614-6526-7.

[Lag02] Jeffrey C. Lagarias. An elementary problem equivalent to the Riemann hypothesis. *The American Mathematical Monthly*, 109(6):534–543, 2002. doi:10.1080/00029890.2002.11919832.

[LD37] Peter Gustav Lejeune Dirichlet. Beweis des satzes, dass jede unbegrenzte arithmetische progression, deren erstes glied und differenz ganze zahlen ohne gemeinschaftlichen faktor sind, unendlich viele primzahlen enth"alt. *Abhandlungen der K"oniglichen Preussischen Akademie der Wissenschaften zu Berlin*, pages 45–81, 1837. URL: https://sites.mathdoc.fr/cgi-bin/oeitem?id=OE_DIRICHLET___1_313_0 (visited on 2025-12-29).

[LCD+03] Shangzhi Li, Falai Chen, Jiansong Deng, Yaohua Wu, and Yunhua Zhang. *Mathematics Experiments*. World Scientific, 2003. ISBN 9789812380500. doi:10.1142/5008.

[May15] James Maynard. Small gaps between primes. *Annals of Mathematics*, 181(1):383–413, 2015. URL: https://annals.math.princeton.edu/2015/181-1/p07 (visited on 2025-12-27), doi:10.4007/annals.2015.181.1.7.

[MR24] Inc. Mersenne Research. Gimps — the math — primenet. Online, 2024. Background on the mathematics and algorithms used in the GIMPS search strategy (trial factoring, P-1, PRP testing, Lucas–Lehmer, double-checking). URL: https://www.mersenne.org/various/math.php (visited on 2025-12-27).

[MR25] Inc. Mersenne Research. List of known mersenne prime numbers. Online, 2025. PrimeNet/GIMPS list of known Mersenne primes including discovery metadata and verification method. URL: https://www.mersenne.org/primes/ (visited on 2025-12-27).

[Mon73] H. L. Montgomery. The pair correlation of zeros of the zeta function. *Proceedings of Symposia in Pure Mathematics*, 24:181–193, 1973.

[MV06] Hugh L. Montgomery and Robert C. Vaughan. *Multiplicative Number Theory I: Classical Theory*. Volume 97 of Cambridge Studies in Advanced Mathematics. Cambridge University Press, 2006. ISBN 978-0-521-84903-6. doi:10.1017/CBO9780511618314.

[MM97] M. Ram Murty and V. Kumar Murty. *Non-vanishing of L-Functions and Applications*. Volume 157 of Progress in Mathematics. Birkhäuser, 1997. ISBN 9783764358013. doi:10.1007/978-3-0348-8956-8.

[Nic99] Thomas R. Nicely. New maximal prime gaps and first occurrences. *Mathematics of Computation*, 68(227):1311–1315, 1999. URL: https://www.ams.org/mcom/1999-68-227/S0025-5718-99-01065-0/ (visited on 2025-12-27), doi:10.1090/S0025-5718-99-01065-0.

[NZM91] Ivan Niven, Herbert S. Zuckerman, and Hugh L. Montgomery. *An Introduction to the Theory of Numbers*. John Wiley & Sons, 5 edition, 1991. ISBN 978-0-471-62546-0.

[OR14] Pascal Ochem and Michaël Rao. Lower bounds on odd perfect numbers. 2014. Slides (Montpellier, 2014-07-02). URL: https://www.lirmm.fr/~ochem/opn/opn\TU\textbackslash{}_slide.pdf (visited on 2025-12-25).

[Odl92] Andrew M. Odlyzko. The 10^20-th zero of the Riemann zeta function and 175 million of its neighbors. Unpublished manuscript, 1992. Revision of a 1989 manuscript; numerical computations of high zeros of $\zeta (s)$. URL: https://www-users.cse.umn.edu/~odlyzko/unpublished/zeta.10to20.1992.pdf (visited on 2025-12-30).

[OtR85] Andrew M. Odlyzko and Herman J. J. te Riele. Disproof of the mertens conjecture. *Journal für die reine und angewandte Mathematik*, 357:138–160, 1985. URL: http://eudml.org/doc/152712, doi:10.1515/crll.1985.357.138.

[PetkovvsekWZ96] Marko Petkovšek, Herbert S. Wilf, and Doron Zeilberger. *A=B*. A K Peters, Wellesley, MA, USA, 1996. ISBN 9781568810635. URL: https://sites.math.rutgers.edu/~zeilberg/expmath/ (visited on 2025-12-22).

[Polya54a] George Pólya. *Mathematics and Plausible Reasoning, Volume I: Induction and Analogy in Mathematics*. Princeton University Press, Princeton, NJ, USA, 1954.

[Polya54b] George Pólya. *Mathematics and Plausible Reasoning, Volume II: Patterns of Plausible Inference*. Princeton University Press, Princeton, NJ, USA, 1954.

[Rab80] Michael O. Rabin. Probabilistic algorithm for testing primality. *Journal of Number Theory*, 12(1):128–138, 1980. URL: https://www.sciencedirect.com/science/article/pii/0022314X80900840 (visited on 2025-12-27), doi:10.1016/0022-314X(80)90084-0.

[Ram15] Srinivasa Ramanujan. Highly composite numbers. *Proceedings of the London Mathematical Society*, s2-14(1):347–409, 1915. URL: https://ramanujan.sirinudi.org/Volumes/published/ram15.pdf, doi:10.1112/plms/s2_14.1.347.

[Rib96] Paulo Ribenboim. *The New Book of Prime Number Records*. Springer, 1996. ISBN 9780387944579. URL: https://link.springer.com/book/10.1007/978-1-4612-0759-7 (visited on 2025-12-27), doi:10.1007/978-1-4612-0759-7.

[Rie94] Hans Riesel. *Prime Numbers and Computer Methods for Factorization*. Birkhäuser, 2 edition, 1994. URL: https://link.springer.com/book/10.1007/978-0-8176-8298-9 (visited on 2025-12-27), doi:10.1007/978-0-8176-8298-9.

[RSA78] Ronald L. Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978. doi:10.1145/359340.359342.

[Rob84] Guy Robin. Grandes valeurs de la fonction somme des diviseurs et hypothèse de Riemann. *Journal de Mathématiques Pures et Appliquées*, 63:187–213, 1984.

[RS62] J. Barkley Rosser and Lowell Schoenfeld. Approximate formulas for some functions of prime numbers. *Illinois Journal of Mathematics*, 6(1):64–94, 1962. URL: https://projecteuclid.org/journals/illinois-journal-of-mathematics/volume-6/issue-1/Approximate-Formulas-for-Some-Functions-of-Prime-Numbers/ijm/1255631807.full (visited on 2025-12-27), doi:10.1215/ijm/1255631807.

[RS94] Michael Rubinstein and Peter Sarnak. Chebyshev's bias. *Experimental Mathematics*, 3(3):173–197, 1994. URL: https://projecteuclid.org/journals/experimental-mathema

tics/volume-3/issue-3/Chebyshevs-bias/em/1048515870.full (visited on 2025-12-29), doi:10.1080/10586458.1994.10504289.

[Sch76] Lowell Schoenfeld. Sharper bounds for the Chebyshev functions $\theta (x)$ and $\psi (x)$. II. *Mathematics of Computation*, 30(134):337–360, 1976. doi:10.1090/S0025-5718-1976-0457374-6.

[Ser73] Jean-Pierre Serre. *A Course in Arithmetic*. Graduate Texts in Mathematics. Springer, New York, 1973. ISBN 978-0-387-90040-7. URL: https://link.springer.com/book/10.1007/978-1-4684-9884-4 (visited on 2025-12-29), doi:10.1007/978-1-4684-9884-4.

[SS77] Robert M. Solovay and Volker Strassen. A fast Monte-Carlo test for primality. *SIAM Journal on Computing*, 6(1):84–85, 1977. URL: https://epubs.siam.org/doi/10.1137/0206006 (visited on 2025-12-27), doi:10.1137/0206006.

[Spi08] Michael Spivak. *Calculus*. Publish or Perish, Inc., 4 edition, 2008. ISBN 9780914098911. URL: https://www.amazon.com/Calculus-4th-Michael-Spivak/dp/0914098918.

[Sto24] Andrew Stone. Improved upper bounds for odd perfect numbers — part i. *Integers: Electronic Journal of Combinatorial Number Theory*, 2024. URL: https://math.colgate.edu/~integers/y114/y114.pdf.

[Ten15] Gérald Tenenbaum. *Introduction to Analytic and Probabilistic Number Theory*. Volume 163 of Graduate Studies in Mathematics. American Mathematical Society, 3 edition, 2015. ISBN 978-1-4704-7821-6.

[Tit86] E. C. Titchmarsh. *The Theory of the Riemann Zeta-Function*. Oxford University Press, 2 edition, 1986. ISBN 9780198533696.

[Voi98] John Voight. Perfect numbers: an elementary introduction. 1998. Lecture notes / survey. Date: May 31, 1998; updated January 27, 2024. URL: https://jvoight.github.io/notes/perfelem-051015.pdf (visited on 2025-12-25).

[Was97] Lawrence C. Washington. *Introduction to Cyclotomic Fields*. Volume 83 of Graduate Texts in Mathematics. Springer, New York, 2 edition, 1997. ISBN 978-0-387-94762-4. URL: https://link.springer.com/book/10.1007/978-1-4612-1934-7 (visited on 2025-12-29), doi:10.1007/978-1-4612-1934-7.

[Wei03] Eric W. Weisstein. Perfect number. 2003. MathWorld—A Wolfram Web Resource. URL: https://mathworld.wolfram.com/PerfectNumber.html (visited on 2025-12-25).

[Wie09] Arthur Wieferich. Zum letzten fermatschen theorem. *Journal für die reine und angewandte Mathematik*, 136:293–302, 1909.

[WK09] George Woltman and Scott Kurowski. On the discovery of the 45th and 46th known mersenne primes. *The Fibonacci Quarterly*, 46/47(3):194–197, 2009. Abstract PDF. Describes GIMPS methods and reports the discoveries of M37156667 and M43112609. URL: https://www.fq.math.ca/Abstracts/46_47-3/woltman.pdf (visited on 2025-12-27).

[Zha14] Yitang Zhang. Bounded gaps between primes. *Annals of Mathematics*, 179(3):1121–1174, 2014. URL: https://annals.math.princeton.edu/2014/179-3/p07 (visited on 2025-12-27), doi:10.4007/annals.2014.179.3.7.

[DHJPolymath14] D. H. J. Polymath. Variants of the selberg sieve, and bounded intervals containing many primes. *Research in the Mathematical Sciences*, 1:12, 2014. URL: https://link.springer.com/article/10.1186/s40687-014-0019-3 (visited on 2025-12-27), doi:10.1186/s40687-014-0019-3.

[MersenneResearchIncGIMPS24] Mersenne Research, Inc. (GIMPS). Mersenne prime discovery: 2^136279841-1 is prime! 2024. GIMPS press release page (52nd known Mersenne prime). URL: https://www.mersenne.org/primes/?press=M136279841 (visited on 2025-12-25).

[MersenneResearchIncGIMPS25] Mersenne Research, Inc. (GIMPS). Gimps milestones report. 2025. URL: https://www.mersenne.org/report\TU\textbackslash{}_milestones/ (visited on 2025-12-25).

[OEISFInc25a] OEIS Foundation Inc. A000396: perfect numbers. 2025. The On-Line Encyclopedia of Integer Sequences (OEIS). URL: https://oeis.org/A000396 (visited on 2025-12-25).

[OEISFInc25b] OEIS Foundation Inc. A001097: twin primes. Online, 2025. The On-Line Encyclopedia of Integer Sequences (OEIS). URL: https://oeis.org/A001097 (visited on 2025-12-27).

[OEISFInc25c] OEIS Foundation Inc. A001223: prime gaps p_n+1-p_n. Online, 2025. The On-Line Encyclopedia of Integer Sequences (OEIS). URL: https://oeis.org/A001223 (visited on 2025-12-27).

[OEISFInc25d] OEIS Foundation Inc. A001359: lesser of twin primes. Online, 2025. The On-Line Encyclopedia of Integer Sequences (OEIS). URL: https://oeis.org/A001359 (visited on 2025-12-27).

[OEISFInc25e] OEIS Foundation Inc. A023201: primes p such that $p+6$ is also prime (sexy primes). Online, 2025. The On-Line Encyclopedia of Integer Sequences (OEIS). URL: https://oeis.org/A023201 (visited on 2025-12-27).

[OEISFInc25f] OEIS Foundation Inc. A046132: primes p such that $p-4$ is also prime (cousin prime pairs). Online, 2025. The On-Line Encyclopedia of Integer Sequences (OEIS). URL: https://oeis.org/A046132 (visited on 2025-12-27).

[PrimePUTM25a] Prime Pages (UTM). Carmichael number (prime glossary). https://t5k.org/glossary/page.php?sort=CarmichaelNumber, 2025. Online; accessed 2025-12-28.

[PrimePUTM25b] Prime Pages (UTM). Primorial (prime glossary). https://t5k.org/glossary/page.php?sort=Primorial, 2025. Online; accessed 2025-12-28.

[PrimePUTM25c] Prime Pages (UTM). Wieferich prime (prime glossary). https://t5k.org/glossary/page.php?sort=WieferichPrime, 2025. Online; accessed 2025-12-28.

[TheOFInc25a] The OEIS Foundation Inc. A000215: Fermat numbers F(n) = 2^(2^n) + 1. https://oeis.org/A000215, 2025. Online; accessed 2025-12-28.

[TheOFInc25b] The OEIS Foundation Inc. A001220: Wieferich primes (base 2). https://oeis.org/A001220, 2025. Online; accessed 2025-12-28.

[TheOFInc25c] The OEIS Foundation Inc. A001358: Semiprimes. https://oeis.org/A001358, 2025. Online; accessed 2025-12-28.

[TheOFInc25d] The OEIS Foundation Inc. A002110: Primorial numbers n#. https://oeis.org/A002110, 2025. Online; accessed 2025-12-28.

[TheOFInc25e] The OEIS Foundation Inc. A002997: Carmichael numbers. https://oeis.org/A002997, 2025. Online; accessed 2025-12-28.

[Wikipediacontributors25a] Wikipedia contributors. Carmichael number — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?oldid=1328580842&title=Carmichael_number, 2025. Online; accessed 2025-12-28.

[Wikipediacontributors25b] Wikipedia contributors. Fermat number — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?oldid=1329504472&title=Fermat_number, 2025. Online; accessed 2025-12-28.

[Wikipediacontributors25c] Wikipedia contributors. Harmonic number. Wikipedia, 2025. Permanent revision as of 20:03, 12 December 2025 (UTC). URL: https://en.wikipedia.org/w/index.php?oldid=1327129204\TU\textbackslash{}&title=Harmonic\TU\textbackslash{}_number (visited on 2025-12-25).

[Wikipediacontributors25d] Wikipedia contributors. Perfect number. Wikipedia, 2025. Permanent revision as of 13:46, 22 December 2025 (UTC). URL: https://en.wikipedia.org/w/index.php?oldid=1328905011\TU\textbackslash{}&title=Perfect\TU\textbackslash{}_number (visited on 2025-12-25).

[Wikipediacontributors25e] Wikipedia contributors. Prime number. Wikipedia, 2025. Permanent revision as of 20:47, 2 December 2025 (UTC). URL: https://en.wikipedia.org/w/index.php?oldid

=1325385945\TU\textbackslash{}&title=Prime\TU\textbackslash{}_number (visited on 2025-12-25).

[Wikipediacontributors25f] Wikipedia contributors. Prime-counting function — Wikipedia, the free encyclopedia. 2025. Accessed: 2025-12-29. URL: https://en.wikipedia.org/w/index.php?title=Prime-counting_function&oldid=1328904428 (visited on 2025-12-29).

[Wikipediacontributors25g] Wikipedia contributors. Primorial — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?oldid=1328293471&title=Primorial, 2025. Online; accessed 2025-12-28.

[Wikipediacontributors25h] Wikipedia contributors. Riemann zeta function — Wikipedia, the free encyclopedia. 2025. Accessed: 2025-12-29. URL: https://en.wikipedia.org/w/index.php?title=Riemann_zeta_function&oldid=1326434355 (visited on 2025-12-29).

[Wikipediacontributors25i] Wikipedia contributors. Riemann–von mangoldt formula — Wikipedia, the free encyclopedia. 2025. Accessed: 2025-12-29. URL: https://en.wikipedia.org/w/index.php?title=Riemann%E2%80%93von_Mangoldt_formula&oldid=1322838097 (visited on 2025-12-29).

[Wikipediacontributors25j] Wikipedia contributors. Taylor series. Wikipedia, 2025. Permanent revision as of 04:10, 24 December 2025 (UTC). URL: https://en.wikipedia.org/w/index.php?oldid=1329166943\TU\textbackslash{}&title=Taylor\TU\textbackslash{}_series (visited on 2025-12-25).

[Wikipediacontributors25k] Wikipedia contributors. Wieferich prime — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?oldid=1329186350&title=Wieferich_prime, 2025. Online; accessed 2025-12-28.

[Wikipediacontributors25l] Wikipedia contributors. Z function — Wikipedia, the free encyclopedia. 2025. Accessed: 2025-12-29. URL: https://en.wikipedia.org/w/index.php?title=Z_function&oldid=1313086592 (visited on 2025-12-29).